

Quantum Framework Documentation

Version 1.4.2-SNAPSHOT, 2026-09-21T22:14:39Z

Table of Contents

1. Getting Started	2
2. Overview: Building SaaS with Quantum	3
2.1. SaaS and Multi-Tenancy First	3
3. Getting Started: Your First Quantum Application	4
3.1. Prerequisites	4
3.2. Project Setup	4
3.3. Configuration	4
3.4. Your First Model	5
3.5. Repository	6
3.6. REST Resource	6
3.7. Running the Application	7
3.8. Testing Your API	7
3.9. What You Get Automatically	7
3.10. Next Steps	8
3.11. Optional: Enable Ontology Modules	8
4. Core Concepts	9
5. Multi-Tenancy Models	10
5.1. DEDICATED_REALM : One Tenant per Database	10
5.2. POOLED_REALM : Many Tenants in One Database	10
5.3. Freemium and Trial Tenants	11
5.4. Tenant Onboarding and Access Invites	11
5.4.1. 1. Tenant provisioning: create the tenant itself	11
5.4.2. 2. Tenant decommissioning: archive before topology-specific cleanup	14
5.4.3. 2. Tenant onboarding workflow: define the journey after invite/registration	15
5.4.4. 3. Registration requests: capture onboarding data for approval	16
5.4.5. 4. Access invites: grant a user into an existing tenant	17
5.4.6. Invite extension points	19
5.4.7. How these pieces fit together	19
5.5. Choosing a Tenancy Model: Three Options	20
5.6. The GitHub-Like Identity Model, Mapped	20
5.7. User-Definable Facet Segmentation & Coordinate SPI in Multi-Tenant Environments	21
5.7.1. Physical Realm Isolation vs. Logical Facet Segmentation	21
5.7.2. Single Server Instance with Per-Tenant Heterogeneous Policies	22
5.7.3. Configuring User-Definable Facets	22
5.7.4. Coordinate Resolution Precedence and Defaults	23
5.7.5. Multi-Hop Traversal Isolation	24
DomainContext, RuleContext, and DataDomain	25
6. DataDomain	26

7. DomainContext	27
8. RuleContext	28
9. End-to-End Flow	29
10. Resolvers and Variables in Rule Filters	30
10.1. Business Scenario: Multi-Tenant Franchise & Dynamic Partner Segmentation	30
10.2. How the Feature Supports the Business Solution	30
11. Concrete example: building and using a resolver	32
11.1. 1) Implement the SPI	32
11.2. 2) How RuleContext uses resolvers	33
11.3. 3) Author a rule that consumes the variable	33
11.4. 4) End-to-end behavior	34
11.5. String literals vs. typed values in resolver variables	34
11.5.1. Example A: Resolver returns ObjectIds (typed)	34
11.5.2. Example B: Resolver returns String literals (force raw strings)	35
11.5.3. Other types supported	35
11.6. Using AccessListResolver with Ontology (optional)	36
11.7. Admitted vocabulary for rule predicates	36
12. Modeling with Functional Areas, Domains, and Actions	38
12.1. Prefer Annotations for Functional Mapping (recommended)	38
12.2. DataDomain on Models	40
12.3. Persistence Repositories	40
12.4. Exposing REST Resources	41
12.5. Lombok in Models	41
12.6. Validation with Jakarta Bean Validation	41
12.7. Jackson vs Jakarta Validation Annotations	42
13. Jackson ObjectMapper in Quarkus and in Quantum	43
14. Validation Lifecycle and Morphia Interceptors	45
15. Functional Area/Domain in RuleContext Permission Language	47
16. StateGraphs on Models	50
17. State Graph (DOT)	51
18. CompletionTasks and CompletionTaskGroups	53
18.1. When to use them	53
18.2. Core entities	53
18.2.1. CompletionTask	53
18.2.2. CompletionTaskGroup	54
18.3. What the framework manages vs. what your application manages	55
18.4. State model	55
18.5. REST API surface	56
18.5.1. Task group endpoints	56
18.5.2. Task endpoints	57
18.6. Request and response payloads	57

18.6.1. Creating a group	58
18.6.2. Creating a task in a group	58
18.6.3. Completing a task	59
18.7. Completion event stream	59
18.8. Typical orchestration flow	60
18.9. Best practices	61
19. References and EntityReference	62
20. Tracking References with @TrackReferences and Delete Semantics	63
20.1. Ontologies in Quantum: Modeling Relationships That Are Resilient and Fast	64
20.1.1. What is an Ontology?	64
20.1.2. Ontology vs. Object Model	64
20.1.3. Why prefer Ontology-driven relationships over @Reference/EntityReference	65
20.1.4. How Quantum supports Ontologies	65
20.1.5. Modeling guidance: from object fields to predicates	66
20.1.6. Querying with ontology edges vs direct references	66
20.1.7. Migration: from @Reference to ontology edges	67
20.1.8. Performance and operational notes	67
20.1.9. How this integrates with Functional Areas/Domains	67
20.1.10. Summary	67
20.1.11. Concrete example: Sales Orders, Shipments, and evolving to Fulfillment>Returns	67
20.2. Integrating Ontology with Morphia, Permissions, and Multi-tenancy	72
20.2.1. Big picture: where ontology fits	72
20.2.2. Rule language: add hasEdge()	72
20.2.3. Passing tenantId correctly	73
20.2.4. Morphia repository integration patterns	73
20.2.5. Where to hook materialization	74
20.2.6. Security and multi-tenant considerations	74
20.2.7. Developer workflow and DX checklist	74
20.2.8. Cookbook: end-to-end example with Orders + Org	75
20.2.9. Migration notes for teams using @Reference	75
21. Building APIs	76
22. REST: Find, Get, List, Save, Update, Delete	77
22.1. Base Concepts	77
22.2. Example Resource	77
22.3. Authorization Layers in REST CRUD	77
22.4. Querying	78
22.5. Responses and Schemas	79
22.6. Error Handling	79
22.6.1. Query Language (ANTLR-based)	79
22.7. Simple filters (equals)	80
22.8. Advanced filters: grouping and AND/OR/NOT	80

22.9. IN lists	80
22.10. Sorting	81
22.11. Projections	81
22.12. End-to-end examples	81
23. CSV Export and Import	82
23.1. Export: GET /csv	82
23.2. Import: POST /csv (multipart)	84
23.3. Import with preview sessions	85
23.4. Enhanced Import with ImportProfile	86
23.4.1. Key Features	86
23.4.2. Using a Profile	86
23.4.3. Profile with Session Flow	86
23.4.4. Example ImportProfile (JSON)	86
23.4.5. Intent Column	87
23.5. RowValueResolver: Custom Per-Row Logic	87
23.5.1. Key Features	88
23.5.2. Example Implementation	88
23.5.3. Configuration in ImportProfile	88
23.5.4. Result Types	89
23.6. Dynamic Attribute Import	89
23.6.1. Import Strategies	89
23.6.2. DOT_NOTATION Example	89
23.6.3. JSON_COLUMN Example	90
23.6.4. Configuration in ImportProfile	90
23.6.5. Merge Strategies	90
Authentication and Authorization	91
24. JWT Provider	92
25. Pluggable Authentication	93
26. Creating an Auth Plugin (using the Custom JWT provider as a reference)	94
27. AuthProvider interface (what a provider must implement)	95
28. UserManagement interface (operations your plugin must support)	96
29. Leveraging BaseAuthProvider in your plugin	97
30. Implementing your own provider	98
31. CredentialUserIdPassword model and DomainContext	99
32. Quarkus OIDC out-of-the-box and integrating with common IdPs	101
33. Authorization via RuleContext	103
33.1. Using Ontology Edges in List Endpoints (optional)	103
33.1.1. OntologyAwareResource	103
34. Query Language Reference	105
34.1. Basic Syntax	105
34.1.1. Operators	105

34.1.2. Value Types	105
34.1.3. Logical Operators	106
34.2. Common Patterns	106
34.2.1. String Matching	106
34.2.2. Text Search	106
34.2.3. Numeric Ranges	107
34.2.4. Date and Time Queries	108
34.2.5. List Membership	108
34.3. Querying Dynamic Attributes	109
34.3.1. Dynamic Attribute Structure	109
34.3.2. Querying by Attribute ID and Value	109
34.3.3. Querying by Attribute Name and Value	110
34.3.4. Combining Multiple Dynamic Attribute Conditions	110
34.3.5. Querying by Set Name	110
34.3.6. Advanced Tags (Simple Key-Value)	110
34.3.7. Advanced Examples	111
34.3.8. Variables in Filters	111
34.3.9. Performance Tips	112
34.3.10. Integration with REST APIs	113
34.3.11. Error Handling	113
34.3.12. See Also	114
34.3.13. Execution engines and listeners	114
34.3.14. Expressing Ontology Constraints in Queries (optional)	120
34.3.15. Ontology operators: hasEdge, hasOutgoingEdge, and hasIncomingEdge	120
34.3.16. Root projection recap: fields:[+..., -...]	124
35. Relationship hydration with expand(path)	125
35.1. How it works	125
35.2. What is expand(path)?	125
35.3. Filtering related entities (inline)	125
35.4. Projection with fields:[+/-]	126
35.5. Examples	126
35.6. Semantics and limits	126
35.7. Aggregation examples: what expand(...) compiles to	127
35.7.1. Single reference: expand(customer)	127
35.7.2. Array of references: expand(items[*].product)	128
35.7.3. Nested arrays: expand(patient.visits[].diagnoses[])	129
35.7.4. Multi-Hop Reference Traversal Chains: expand(hop1) && expand(hop1.hop2)	129
35.7.5. When is AGGREGATION used vs FILTER?	130
36. Planner and QueryGateway	132
36.1. Planner decisions	132
36.2. Public facade: QueryGateway	132

36.3. Examples end-to-end	133
36.4. REST endpoints (v1)	133
36.5. Row- and Field-Level Governance	133
36.6. Governed Aggregation Execution & Per-Stage Security Pushdown	134
36.6.1. Per-Stage Pushdown Mechanics	135
36.6.2. Configuration and Defaults	135
36.6.3. Request/Response: /api/query/plan	135
36.6.4. Request/Response: /api/query/find (Multi-Hop Aggregation)	136
36.6.5. cURL Examples	136
37. Security	138
Authentication and Authorization	139
38. JWT Provider	140
39. Pluggable Authentication	141
40. Creating an Auth Plugin (using the Custom JWT provider as a reference)	142
41. AuthProvider interface (what a provider must implement)	143
42. UserManagement interface (operations your plugin must support)	144
43. Leveraging BaseAuthProvider in your plugin	145
44. Implementing your own provider	146
45. CredentialUserIdPassword model and DomainContext	147
46. Quarkus OIDC out-of-the-box and integrating with common IdPs	150
47. Authorization via RuleContext	153
48. Generating JWT Tokens for API Access	154
48.1. Token Anatomy	154
48.2. Generating User Tokens via Login	154
48.3. Service Tokens for System-to-System Integration	155
48.3.1. Generating a Service Token	155
48.3.2. Request Parameters	156
48.3.3. What Happens on the Backend	157
48.3.4. Using a Service Token	157
48.3.5. Example: Configuring an MCP Server with a Service Token	157
48.4. How Tokens Scope to Security Policies	158
48.4.1. Token-to-Policy Flow	158
48.4.2. Role Merging	159
48.4.3. Data Scoping via DataDomain	159
48.4.4. Realm Override with X-Realm	160
48.4.5. Limiting Service Token Scope	160
48.5. Generating RSA Key Pairs	160
48.5.1. Where to Place the Keys	161
48.6. Configuration Reference	162
Multiple Authentication Providers	163
49. How It Works	164

49.1. Provider Resolution	164
49.2. Canonical Identity Service	164
50. Configuring <code>application.properties</code>	166
50.1. Minimal Custom JWT Provider	166
50.2. Adding an OIDC Provider (e.g., AWS Cognito)	166
50.3. Using Keycloak or Auth0	167
51. Issuer Resolution in Detail	168
51.1. Role Provenance Tracking	168
52. Creating a New Auth Provider	170
53. Key Pair Management	172
53.1. Default Keys Module (<code>quantum-default-keys</code>)	172
53.2. Configurable Key Locations	172
53.2.1. Typical Production Setup	173
53.3. Full Per-Tenant Design	173
Permissions: Rule Bases, SecurityURIHeader, and SecurityURIBody	174
54. Introduction: Layered Enforcement Overview	175
55. Key Concepts	178
56. Rule Structure (Illustrative)	179
57. Matching Algorithm	181
58. Priorities	183
59. Grant-based vs Deny-based Rule Sets	184
60. Row Filter Computation (Data Scoping)	186
60.1. The pipeline	186
60.2. What is scoped by default	186
60.3. Available substitution variables (static)	186
60.4. Dynamic access lists (the <code>AccessListResolver</code> SPI)	187
60.5. Summary	188
61. Feature Flags, Variants, and Target Rules	189
62. Scoped Action Enablement and Dependency Tracking	192
62.1. Core idea: the same scoped action tuple stays canonical	192
62.2. The three answers returned by the framework	193
62.3. Runtime models	193
62.4. REST API	194
62.5. Manifest-driven dependency relationships	196
62.6. Resolver SPI	196
62.7. Built-in OSS dependency types	197
62.8. Missing manifests and unsupported dependency types	198
62.9. Example contributor	198
62.10. Relationship to policies, feature flags, and UI actions	199
62.11. Enterprise extensions	199
62.12. Guidance	200

63. Multiple Matching RuleBases	201
64. Identity and Role Matching	202
64.1. How roles are defined for an identity (role sources and resolution)	202
64.1.1. Populating <code>RoleSource</code> and <code>roleAssignments</code>	204
64.1.2. AuthProviders: what you need to do	204
64.1.3. Security Check responses: enriching <code>roleAssignments</code>	206
65. Example Scenarios	208
66. Operational Tips	209
67. Field-Level Policy (Property Exclusions)	210
67.1. Declaring exclusions on a rule	210
67.2. Layered enforcement — one policy source, every egress guarded	210
67.3. Limitations (v1)	211
68. How UIActions and DefaultUIActions are calculated	212
69. How This Integrates End-to-End	214
70. Administering Policies via REST (PolicyResource)	215
70.1. Model shape (Policy)	215
70.2. How rule-contributed filters work (<code>andFilterString</code> , <code>orFilterString</code> , <code>joinOp</code>)	215
70.3. <code>AccessListResolvers</code> (SPI) for list-based access	219
70.4. <code>PrincipalContextPropertiesResolver</code> (SPI) for custom principal properties	221
70.5. Endpoints	227
70.6. Examples	228
70.7. How changes affect rule bases and enforcement	229
71. Realm override (X-Realm) and Impersonation (X-Impersonate)	231
71.1. What they do (at a glance)	231
71.2. How the headers integrate with permission evaluation	232
71.3. Required credential configuration (<code>CredentialUserIdPassword</code>)	232
71.4. End-to-end behavior from <code>SecurityFilter</code> (reference)	233
71.5. Practical differences and use cases	233
71.6. Examples	234
72. Data domain assignment on create: <code>DomainContext</code> and <code>DataDomainPolicy</code>	236
72.1. The problem this solves (and why it matters)	236
72.2. Key concepts recap: <code>DomainContext</code> and <code>DataDomain</code>	236
72.3. The default policy (do nothing and it works)	236
72.4. Policy scopes: principal-attached vs. global	237
72.5. The policy map and matching	237
72.6. How the resolver works	238
72.7. When would you want a non-global policy?	239
72.8. Relation to tenancy models	239
72.9. Authoring tips	239
72.10. API pointers	240
72.11. Ontology in Permission Rules (optional)	240

72.11.1. Rule language: add hasEdge()	240
72.12. Business Scenarios & Policy Enforcement Solutions	242
72.12.1. Business Scenario 1: Healthcare Temporary On-Call Staffing & Regulatory Audits	242
72.12.2. Business Scenario 2: Aerospace Defense Project Classification Guard	242
72.13. Label resolution SPI and hasLabel() in rules	245
73. Script Helpers reference (when enabled)	248
74. Scoped Action Enablement	251
74.1. Core principle	251
74.2. What the framework returns	252
74.3. Runtime models	252
74.4. REST API	253
74.4.1. Check request	253
74.4.2. Check response	254
74.4.3. Manifest response	255
74.5. Manifest model and contributor SPI	255
74.5.1. Contributor precedence	255
74.6. Resolver SPI	256
74.7. Built-in OSS dependency types	256
74.8. Evaluation behavior	257
74.9. Example contributor	257
74.10. Relationship to permissions and UI actions	258
74.11. Enterprise extensions	259
74.12. Guidance	259
74.12.1. @FunctionalMapping	260
74.12.2. @FunctionalAction	260
74.12.3. Default Action Mapping	261
74.12.4. How the Framework Uses These Annotations	261
74.12.5. Migration from Legacy Methods	262
74.12.6. Best Practices	262
74.12.7. Bypassing Data Scoping for System Operations	263
74.12.8. Integration with Permission Rules	265
74.12.9. See Also	265
74.12.10. Concepts	265
74.12.11. ShareLink Model	266
74.12.12. Link Lifecycle	266
74.12.13. Repository Operations	267
74.12.14. Access Logging	267
74.12.15. Link Signing with HMAC	267
74.12.16. Security Considerations	268
74.12.17. See Also	268
75. Media references	270

75.1. OAuth Server Module	270
75.1.1. When to Use	270
75.1.2. Adding the Dependency	271
75.1.3. Prerequisites	271
75.1.4. Endpoints	271
75.1.5. Supported Grant Types	272
75.1.6. Registering OAuth Clients	273
75.1.7. Integration Examples	274
75.1.8. Architecture	275
75.1.9. Data Model	276
75.1.10. Security Considerations	276
75.1.11. See Also	276
76. AI Integration	277
77. MCP Server and Client	278
77.1. Module Overview	278
77.2. MCP Server	278
77.2.1. Security	278
77.2.2. Tools	279
77.2.3. Resources	280
77.2.4. Connecting an MCP Client	280
77.3. MCP Client	281
77.3.1. Configuration	281
77.3.2. Using MCP Tools in AI Services	282
77.3.3. Programmatic Access to MCP Clients	282
77.3.4. Dev UI: Testing MCP Clients	283
77.3.5. Optional: Resources as Tools	283
77.3.6. Optional: Headers and Authorization	284
77.4. Agent REST API	284
77.4.1. Endpoints	284
77.4.2. Execute Request	284
77.5. Per-Tenant Agent Configuration	285
77.5.1. Run-As Principal	286
77.6. Agent Configuration API	286
77.6.1. Agent Entity	286
77.6.2. CRUD Endpoints	287
77.6.3. Example: Creating an Agent	287
77.7. Architecture	288
77.8. Ontology Tools	288
77.9. Integrating with Claude	289
77.9.1. Claude Desktop	289
77.9.2. Claude Code (CLI)	289

77.9.3. Claude.ai (Browser) with Custom Connectors	290
77.9.4. Cursor IDE	290
77.10. How MCP Maps to Framework Agent and Tool Concepts	291
77.10.1. Concept Mapping	291
77.10.2. Execution Flow Comparison	291
77.11. End-to-End Example: Claude Querying a Supply Chain Application	292
77.11.1. Prerequisites	292
77.11.2. Step 1: Add the MCP Server Module	292
77.11.3. Step 2: Configure Agent Properties	292
77.11.4. Step 3: Start the Application	293
77.11.5. Step 4: Configure Claude	293
77.11.6. Step 5: Interact with Claude	294
77.11.7. What Happened Under the Hood.	295
77.11.8. Adding Ontology Context for Richer Queries	296
77.12. MCP Server Configuration Reference.	296
78. Advanced Features	297
78.1. Email Templates.	297
78.1.1. What the framework supports	297
78.1.2. Architecture.	297
78.1.3. Template sources and resolution order	298
78.1.4. Rendering templates	299
78.1.5. Tenant-scoped REST API.	300
78.1.6. Onboarding workflow integration.	301
78.1.7. Delivering email	302
78.1.8. When to use email templates	303
78.1.9. See also	303
78.1.10. What a BootstrapPack is.	304
78.1.11. Why BootstrapPacks exist	304
78.1.12. BootstrapPacks vs seed packs	305
78.1.13. When to use one vs the other	305
78.1.14. Rule of thumb	306
78.1.15. Step kinds.	306
78.1.16. Apply modes and policies	306
78.1.17. SPI model	306
78.1.18. Admin REST API	307
78.1.19. Startup application	308
78.1.20. Run history	309
78.1.21. Examples	309
78.1.22. Choosing the right abstraction	309
78.1.23. See also	310
79. Overview	311

80. Semantic Versioning	312
81. Configuration	313
82. How change sets are discovered and executed	314
83. Authoring a change set	315
84. Example change sets in the framework	316
85. REST APIs to trigger migrations (MigrationResource)	317
86. Per-entity index management (BaseResource)	319
87. Global index management (MigrationService)	320
88. Validating versions at startup	321
89. Notes and best practices	322
89.1. Checksum vs. from/to versions	322
89.2. Testing Framework	323
89.3. Prerequisites and glossary	323
89.4. Pattern 1 — Extend BaseRepoTest	323
89.5. Pattern 2 — Try-with-resources SecuritySession	324
89.6. Pattern 3 — Scoped-call wrapper (ScopedCallScope)	325
89.7. Pattern 4 — @TestSecurity annotation (Quarkus)	326
89.8. Testing REST APIs vs. Repository Logic	327
89.9. Useful Quarkus Test features	327
89.10. Real-world tips	327
89.11. Summary	328
REST Usage Governance and Metering Boundaries	329
90. Runtime modes	330
91. Stable identity	331
92. Property manifest	332
93. Extension contracts	333
94. Storage and scaling boundary	334
95. Network guard boundary	335
96. Tutorials	336
Supply Chain Collaboration SaaS: A Business-First Guide	337
97. What a supply-chain SaaS needs (and how Quantum helps)	338
98. Why multi-tenancy is a natural fit for supply chains	339
99. Who uses the system (organizations and roles)	340
100. Identity and access: meet partners where they are	341
101. Modeling without jargon: Areas, Domains, and Actions	342
102. Policies that say “who can do what” (Rule Language)	343
103. Domain contexts and domain policies (placing and sharing data)	344
104. Bootstrap tenants with Seed Packs (zero-touch baseline data)	345
105. Security policies: practical protections and proofs	346
106. A small, powerful API surface: List + Query	347
107. Delegated Administration (tenant-level user management)	348

108. Integrations and data management	349
109. End-to-end examples	350
110. What you don't have to build from scratch	351
111. Other helpful capabilities you get out of the box	352
112. Next steps	353
113. A day in the life: From Purchase Order to Delivery	354
114. Appendix	356
114.1. Configuration Reference	356
114.1.1. Configuration Basics	356
114.1.2. Database Configuration	356
114.1.3. Authentication Configuration	357
114.1.4. Quantum Framework Configuration	359
114.1.5. See Also.	361
114.1.6. Ontology Configuration (optional)	361
114.1.7. Query Gateway Configuration	362
114.1.8. Control-plane split (deployment mode)	363
114.2. Quarkus Foundation	364
114.2.1. Key Quarkus Capabilities	364
114.2.2. Arc (Quarkus CDI)	364
114.2.3. Native Compilation with GraalVM	365
114.2.4. Extension Ecosystem	366
114.2.5. Quantum's Quarkus Integration	366
114.2.6. Getting Started with Quarkus	366
114.2.7. See Also.	366
114.2.8. Core Concepts	366
114.2.9. Multi-Tenancy	367
114.2.10. Security	367
114.2.11. Data and Persistence	368
114.2.12. REST and APIs	368
114.2.13. Framework Components	368
114.2.14. Annotations	369
114.2.15. External Integrations	369
114.2.16. Ontology (optional feature)	370

Quantum is a Quarkus-based framework for building multi-tenant SaaS platforms with MongoDB. This guide helps mid-level Java developers build secure, scalable applications with consistent REST APIs, policy-driven authorization, and flexible data sharing.

Chapter 1. Getting Started

Chapter 2. Overview: Building SaaS with Quantum

Quantum is a Quarkus-based framework that accelerates building multi-tenant SaaS platforms on MongoDB. It provides:

- Multi-tenancy primitives for tenant creation, isolation, and data sharing
- A domain-first programming model with Functional Areas, Functional Domains, and Actions
- Data security and contextual evaluation via `DataDomain`, `DomainContext`, and `RuleContext`
- Consistent REST resources for find/get/list/save/update/delete operations
- Pluggable authentication with a provided JWT module and extension points

This guide targets mid-level Java developers and follows a structure similar to Spring's reference docs. Use Maven to generate HTML/PDF: see docs module README for commands.

2.1. SaaS and Multi-Tenancy First

SaaS solutions require:

- Onboarding automation: programmatic tenant creation, freemium/trial flows
- Isolation with selective sharing
- Policy-driven access that adapts to user, org, tenant, and action
- Operational efficiency (observability, cost control, upgradeability)

Quantum's building blocks address these needs out-of-the-box while remaining flexible to fit your architecture.

Chapter 3. Getting Started: Your First Quantum Application

This section walks through creating a simple multi-tenant Product catalog to demonstrate core Quantum concepts.

3.1. Prerequisites

- Java 17+
- Maven 3.8+
- MongoDB (local or cloud)
- Basic Quarkus knowledge (see [Quarkus Foundation](#))

3.2. Project Setup

Create a new Quarkus project with Quantum dependencies:

```
mvn io.quarkus:quarkus-maven-plugin:create \
    -DprojectGroupId=com.example \
    -DprojectArtifactId=product-catalog \
    -DclassName="com.example.ProductResource" \
    -Dpath="/products"
```

Add Quantum dependencies to `pom.xml`:

```
<dependency>
  <groupId>com.e2eq.framework</groupId>
  <artifactId>quantum-framework</artifactId>
  <version>${quantum.version}</version>
</dependency>
```

`quantum-framework` is the compatibility bundle and includes the framework's server-side REST adapters. For a service with an explicit endpoint boundary, select the narrower modules instead. For example, add `com.end2endlogic:quantum-query-rest` only when that backend is intended to host the existing `/api/query` API. SDK-only consumers should omit REST and repository modules so they do not expose or persist server-owned APIs.

3.3. Configuration

Create `.env` file (copy from template):

```
MONGODB_USERNAME=your-username
MONGODB_PASSWORD=your-password
```

```
MONGODB_DATABASE=product-catalog
MONGODB_HOST=localhost:27017
JWT_SECRET=your-secret-key
```

Basic `application.properties`:

```
# MongoDB
quarkus.mongodb.connection-
string=${MONGODB_CONNECTION_STRING:mongodb://localhost:27017}
quarkus.mongodb.database=${MONGODB_DATABASE:product-catalog}

# JWT Authentication
auth.provider=custom
auth.jwt.secret=${JWT_SECRET:change-me-in-production}
auth.jwt.expiration=60

# CORS for development
quarkus.http.cors=true
quarkus.http.cors.origins=http://localhost:3000
```

3.4. Your First Model

Create a Product model with multi-tenancy built-in:

```
package com.example.model;

import com.e2eq.framework.annotations.FunctionalMapping;
import com.e2eq.framework.model.persistent.base.BaseModel;
import dev.morphia.annotations.Entity;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;
import lombok.experimental.SuperBuilder;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import java.math.BigDecimal;

@Entity
@Data
@NoArgsConstructor
@SuperBuilder
@EqualsAndHashCode(callSuper = true)
@FunctionalMapping(area = "catalog", domain = "product")
public class Product extends BaseModel {

    @NotBlank
    @Size(max = 50)
```

```

    private String sku;

    @NotBlank
    @Size(max = 200)
    private String name;

    private String description;
    private BigDecimal price;
    private boolean active = true;
}

```

Key points: - Extends `BaseModel` for automatic `DataDomain`, audit fields, and ID management - `@FunctionalMapping` declares this model's business area and domain for security rules - Standard Jakarta validation annotations - Lombok reduces boilerplate

3.5. Repository

Create a repository interface:

```

package com.example.repository;

import com.e2eq.framework.model.persistent.morphia.MorphiaRepo;
import com.example.model.Product;

public interface ProductRepo extends MorphiaRepo<Product> {
    // Custom queries can be added here
}

```

3.6. REST Resource

Create a REST endpoint:

```

package com.example.resource;

import com.e2eq.framework.rest.resources.BaseResource;
import com.example.model.Product;
import com.example.repository.ProductRepo;
import jakarta.ws.rs.Path;

@Path("/products")
public class ProductResource extends BaseResource<Product, ProductRepo> {
    // Inherits all CRUD endpoints: GET, POST, PUT, DELETE
    // GET /products/list - paginated list with filtering
    // GET /products/id/{id} - get by ID
    // POST /products - create new product
    // PUT /products/set?id={id}&pairs=field:value - update fields
    // DELETE /products/id/{id} - delete product
}

```

```
}
```

3.7. Running the Application

Start your application:

```
./mvnw quarkus:dev
```

The application provides: - Swagger UI at <http://localhost:8080/q/swagger-ui/> - Dev UI at <http://localhost:8080/q/dev/>

3.8. Testing Your API

Create a product:

```
curl -X POST http://localhost:8080/products \
-H "Content-Type: application/json" \
-d '{
  "sku": "WIDGET-001",
  "name": "Super Widget",
  "description": "The best widget ever",
  "price": 29.99,
  "active": true
}'
```

List products:

```
curl "http://localhost:8080/products/list?limit=10&sort=+name"
```

Filter products:

```
curl "http://localhost:8080/products/list?filter=active:true&price:>##20"
```

3.9. What You Get Automatically

With this minimal setup, Quantum provides:

Multi-tenancy: Each product is automatically tagged with the creator's DataDomain (tenant, org, owner)

Security: DataDomain filtering ensures users only see their own data by default

Validation: Jakarta Bean Validation runs before persistence

Audit Trail: Automatic createdBy, createdAt, lastUpdatedBy, lastUpdatedDate fields

Consistent APIs: Standard REST patterns across all resources

Query Language: Powerful filtering with the ANTLR-based query syntax

OpenAPI: Automatic API documentation

3.10. Next Steps

- Add authentication: [Authentication Guide](#)
- Create sharing rules: [Permissions Guide](#)
- Learn the query language: [Query Language](#)
- See real-world example: [Supply Chain Tutorial](#)

3.11. Optional: Enable Ontology Modules

You can adopt ontology incrementally. If you don't enable it, everything works as before.

Quick checklist

1) Add dependencies (app-level) - quantum-ontology-core, quantum-ontology-mongo, quantum-ontology-policy-bridge - Do not add quantum-ontology-rest unless this application intentionally hosts the ontology control-plane API. Normal consumers call quantum-ontology-service through its generated SDK.

2) Turn it on via config

```
e2eq.ontology.enabled=true
```

3) Provide an ontology registry (TBox) - Start with an in-memory registry (recommended initially). See [Ontologies in Quantum](#).

4) Wire data and indexes - Inject EdgeDao as a CDI bean (@Inject); indexes are ensured automatically at startup. - Indices: (tenantId, p, dst) and (tenantId, src, p)

5) Materialize edges on entity changes - Use OntologyMaterializer when sources or intermediates change (e.g., Order/Customer/Org/Address/Shipment in the e-commerce example).

6) Use edges in queries/policies - Wrap BSON via ListQueryRewriter or constrain by _id sets. See [Integrating Ontology](#).

Notes

- Keep it optional: only wire beans and create collections when e2eq.ontology.enabled=true.
- Multi-tenant: always pass tenantId from RuleContext.

Chapter 4. Core Concepts

Chapter 5. Multi-Tenancy Models

Quantum supports multiple multi-tenant models for MongoDB deployments:

Every persisted `Realm` has a `deploymentType`:

- `DEDICATED` is the default. Realm records created before this field existed, and any record whose value is absent or null, resolve to `DEDICATED`.
- `SHARED` must be set explicitly before the realm can accept pooled tenant admissions.

The system-management realm catalog exposes this value so operators can see the database topology directly. Tenant lifecycle requests use `DEDICATED_REALM` and `POOLED_REALM`; those request values must agree with the realm's persisted `DEDICATED` or `SHARED` type.

Database migration `1.0.6` backfills existing system-realm catalog records. It updates only Realm documents where `deploymentType` is missing or null, setting them to `DEDICATED`; an explicitly configured `SHARED` value is preserved. Deployments should advance `quantum.database.version` to `1.0.6` and run the normal Quantum migration workflow rather than patching Realm documents individually.

5.1. `DEDICATED_REALM`: One Tenant per Database

- Each tenant is mapped to a dedicated MongoDB database within a cluster.
- Strong isolation at the database level; operational controls via MongoDB roles.
- Pros: Simplified backup/restore per tenant; reduced risk of data bleed.
- Cons: More databases to manage (indexes, connections), higher operational overhead.

How Quantum helps:

- `DataDomain` carries tenant identifiers (e.g., `tenantId`, `ownerId`, `orgRefName`) on each model.
- Repositories can resolve connections/DB selection per tenant, enabling routing to the appropriate database.

5.2. `POOLED_REALM`: Many Tenants in One Database

- Multiple tenants share a single database and collections.
- Isolation is enforced at the application layer using `DataDomain` filters.
- Pros: Fewer databases to manage; efficient index utilization and connection pooling.
- Cons: Strict discipline required to enforce filtering and access rules.

How Quantum helps:

- `DataDomain` is part of every persisted model, enabling programmatic, rule-based filtering.
- `RuleContext` and `DomainContext` inject tenant-aware filters into governed repositories and resources.

- Cross-tenant sharing can be modeled by specific DataDomain fields and RuleContext logic granting read access across tenants on a per-functional-area basis.

In both modes, realm means MongoDB placement and schema lifecycle while DataDomain means row visibility. `POOLED_REALM` is therefore safe only for models and repository paths that participate in governed DataDomain filtering. A shared database does not weaken or replace the policy boundary.

5.3. Freemium and Trial Tenants

- Programmatically create tenants to support self-service onboarding.
- Attach time-bound or capability-bound policies.
- Use scheduled jobs to convert/expire trials.

Quantum patterns:

- Tenant onboarding service creates a DataDomain scope and any default records.
- Policies are encoded in RuleContext checks to allow or restrict actions based on time, plan, or feature flags.

5.4. Tenant Onboarding and Access Invites

Quantum separates realm creation, tenant admission, user access, and the user-facing onboarding journey. Those are related, but they are not the same thing:

- Dedicated provisioning creates the tenant realm, runs realm migrations, and establishes the initial admin user.
- Pooled admission adds a tenant to an existing realm and does not create or initialize a database.
- Tenant onboarding workflow defines the steps a tenant user should complete before access is considered fully activated.
- Access invites bring an individual user into an existing tenant and can attach scoped access metadata.

This split helps teams support both admin-driven setup and self-service onboarding without forcing every tenant into a single path.

5.4.1. 1. Tenant provisioning: create the tenant itself

The admin tenant provisioning API is exposed at:

- `POST /admin/tenants`
- `DELETE /admin/tenants/{realmId}`

Provisioning is intended for `admin` and `system` roles. The same endpoint supports both deployment topologies. Omitting `deploymentTopology` preserves the historical `DEDICATED_REALM` behavior. `POOLED_REALM` admission succeeds only when `placementRealmId` identifies an existing Realm explicitly marked `SHARED`; legacy and untyped realms are treated as `DEDICATED`.

Request fields:

Field	Purpose
<code>tenantDisplayName</code>	Human-readable label for the tenant
<code>tenantEmailDomain</code>	Source for the realm id; dots are converted to dashes
<code>orgRefName</code>	Organization reference used in the tenant <code>DomainContext</code> and <code>DataDomain</code>
<code>accountId</code>	Initial account number for the tenant
<code>adminUserId</code>	User id for the initial tenant admin
<code>adminSubject</code>	Optional stable subject override; defaults to <code>adminUserId</code>
<code>adminPassword</code>	Initial password for the admin user
<code>applicationId</code>	Application that owns the realm placement; may use the configured default
<code>deploymentTopology</code>	<code>DEDICATED_REALM</code> or <code>POOLED_REALM</code> ; when omitted, uses <code>quantum.tenant.provisioning.deployment-topology</code> (default <code>DEDICATED_REALM</code>)
<code>placementRealmId</code>	Existing target realm for <code>POOLED_REALM</code> ; when omitted, uses <code>quantum.tenant.provisioning.pooled-realm</code>
<code>seedArchetypes</code> or <code>archetypes</code>	Optional list of seed archetypes for dedicated realm creation. Pooled admission rejects archetypes until the seed contract can prove all writes are tenant-scoped.

Representative request:

```
{
  "tenantDisplayName": "Acme Logistics",
  "tenantEmailDomain": "acme.example",
  "orgRefName": "acme.example",
  "accountId": "1000000001",
  "adminUserId": "admin@acme.example",
  "adminSubject": "admin@acme.example",
  "adminPassword": "change-me-now",
  "seedArchetypes": ["DemoArchetype"]
}
```

Representative response:

```
{
  "realmId": "acme-example",
  "realmCreated": true,
  "userCreated": true,
  "appliedSeedArchetypes": ["DemoArchetype"],
  "warnings": []
}
```

What provisioning does:

- derives `realmId` from `tenantEmailDomain`
- writes a `Realm` catalog record in the system realm
- builds tenant `DomainContext` and `DataDomain`
- runs migrations in the new tenant realm
- creates or validates the initial admin user
- applies applicable base seed packs
- optionally applies requested seed archetypes
- applies indexes and verifies initialization

For `POOLED_REALM`, provisioning instead:

- derives a distinct tenant id from `tenantEmailDomain`
- resolves `placementRealmId` to an existing `Realm` catalog record
- creates a PARTICIPANT `RealmTenantMembership`
- assigns the tenant's users to the shared realm with the tenant's own `DomainContext` and `DataDomain`
- creates only the requested administrator identity and tenant-scoped realm assignment; it does not create baseline or demo identities
- skips realm migrations, base seed packs, archetypes, and index initialization
- never creates or drops the shared realm/database

The pooled realm must therefore be provisioned and migrated before admitting tenants. Every persisted model reachable by a pooled tenant must participate in governed `DataDomain` filtering for reads and writes. A user currently has one active tenant `DataDomain` assignment per realm; admitting the same identity into a second tenant in that realm fails closed until an explicit tenant-selection claim is available.

Example pooled admission:

```
{
  "tenantDisplayName": "Acme Logistics",
  "tenantEmailDomain": "acme.example",
  "orgRefName": "acme.example",
  "accountId": "1000000001",
  "adminUserId": "admin@acme.example",
  "adminPassword": "change-me-now",
  "deploymentTopology": "POOLED_REALM",
  "placementRealmId": "shared-logistics"
}
```

Provisioning is designed to be largely idempotent. If the same tenant or admin user already exists with matching attributes, the service proceeds and returns warnings instead of blindly recreating

state. If existing state conflicts with the request, the service rejects the operation.

5.4.2. 2. Tenant decommissioning: archive before topology-specific cleanup

Tenant deletion is a governed lifecycle operation, not a direct CRUD delete. The placement topology determines the cleanup boundary:

- **DEDICATED_REALM**: revoke tenant access, create and verify a database archive, then drop the tenant database and retire its realm catalog state.
- **POOLED_REALM**: revoke tenant access, export only rows matching the canonical tenant **DataDomain** identity (**orgRefName**, **accountNum**, **tenantId**, and **dataSegment**), then stamp every matching row with one immutable purge batch reference and one **purgeAfter** timestamp. The shared database and realm are never dropped.

MongoDB TTL cleanup is asynchronous and may remove documents from different collections at different times. Revoking tenant access before expiration is therefore the logical atomic barrier: users cannot observe a partially drained tenant. Before the workflow reports the pooled purge as scheduled, it must reconcile archive and expiration counts for every collection declared tenant-scoped.

Revocation verification includes already-issued tokens or sessions, not only future logins. Suspending a system-plane membership by itself is not a safe barrier for a stateless tenant plane. A deployment must install a revocation provider that invalidates active access (or waits behind an equivalent verified write/read fence) before archive and expiration proceed.

Before a pooled purge is planned, the persisted collection inventory must be reconciled with the collection ownership catalog. Every existing user collection must have exactly one scope:

- **TENANT_SCOPED**: selectable by the canonical tenant identity and eligible for tenant archive and expiration.
- **REALM_SHARED**: shared within the realm and excluded from every individual tenant purge.
- **SYSTEM_GLOBAL**: owned by the system management plane and excluded from tenant purge.

An existing collection without a scope declaration blocks the operation. Declaring a collection as shared or global also forbids tenant-selection and expiration field mappings for that collection.

The default Mongo registry is configured with comma-separated collection names:

```
quantum.tenant-storage.tenant-scoped-  
collections=orders,orderEvents,tenantFiles.files,tenantFiles.chunks  
quantum.tenant-storage.realm-shared-collections=realmConfiguration  
quantum.tenant-storage.system-global-collections=
```

Only MongoDB internal **system.*** namespaces are excluded automatically. GridFS and other blob collections are ordinary user collections for this purpose and must be classified explicitly. Reading the inventory does not create a Morphia datastore, collections, or indexes.

TTL indexes must exist on the configured **purgeAfter** field for every tenant-scoped collection. Realm

bootstrap/migrations should install them; the default Mongo expiration provider also creates the absolute-time TTL index idempotently before it stamps rows. Large collections may be processed in resumable batches, but every batch uses the same purge reference and timestamp. Blob payloads owned by tenant rows are part of both archive and expiration coverage.

`ownerId` is not part of tenant deletion identity. It can narrow policy access, but using it for archive or purge selection can leave tenant-owned data behind. Direct `DELETE /admin/tenants/{realmId}` remains disabled until a governed control-plane workflow supplies archive verification, collection-scope metadata, and purge reconciliation.

Direct tenant deletion is disabled. Decommissioning must use the governed control-plane lifecycle. In particular, pooled tenant removal is a scoped DataDomain operation and must never drop its shared realm/database.

5.4.3. 2. Tenant onboarding workflow: define the journey after invite/registration

Tenant onboarding workflow configuration is exposed at:

- `GET /onboarding/workflow/current`
- `POST /onboarding/workflow/current`

This is tenant-scoped configuration. The active realm is derived from the current security context, so the workflow is saved and read per tenant realm.

The persisted model is `TenantOnboardingWorkflow`, which includes:

- `workflowEnabled`
- `inviteRequired`
- `registrationRequired`
- `surveyRequired`
- `adminApprovalRequired`
- `autoAssignSurveyOnInvite`
- `defaultSurveyRefName`
- `defaultInviteEmailTemplateKey`
- `workflowDefinitionJson`
- `completionMessage`

`defaultInviteEmailTemplateKey` is the email-template key the onboarding flow should use for invite messages. The framework default is `tenant-onboarding-invite`, which can be satisfied by:

- a classpath template under `templates/email/tenant-onboarding-invite/`
- a realm-scoped override in `email_templates`
- a workflow update that points to a different key

By default, the workflow steps are generated in this order:

- `invite`
- `registration`
- `survey`
- `approval`
- `activation`

Each step can be enabled or omitted by toggling the corresponding booleans. When no custom workflow JSON is supplied, the framework generates a default `workflowDefinitionJson` using those flags.

Representative response:

```
{
  "refName": "default-tenant-onboarding",
  "displayName": "Tenant User Onboarding",
  "activeStatus": true,
  "inviteRequired": true,
  "registrationRequired": true,
  "surveyRequired": true,
  "adminApprovalRequired": true,
  "autoAssignSurveyOnInvite": true,
  "defaultSurveyRefName": "tenant-user-onboarding",
  "defaultInviteEmailTemplateKey": "tenant-onboarding-invite",
  "completionMessage": "Your tenant onboarding is complete. You can now sign in and begin using the workspace.",
  "steps": [
    { "key": "invite", "type": "access_invite", "required": true },
    { "key": "registration", "type": "registration_request", "required": true },
    { "key": "survey", "type": "survey", "required": true, "surveyRefName": "tenant-user-onboarding" },
    { "key": "approval", "type": "admin_approval", "required": true },
    { "key": "activation", "type": "account_activation", "required": true }
  ]
}
```

This workflow config answers "what steps should happen?" It does not by itself send invites or create tenant users.

5.4.4. 3. Registration requests: capture onboarding data for approval

The registration-request endpoints are exposed at:

- `POST /onboarding/registrationRequest/create`
- `POST /onboarding/registrationRequest/approve`

This API is the registration step that the onboarding workflow can reference through the

`registration_request` step type. It is useful when a tenant wants users or organizations to submit onboarding information before access is approved.

At a high level:

- `create` persists an `ApplicationRegistration`
- `approve` transitions an existing registration request through the repository approval logic

This is the structured "please review my onboarding submission" side of the journey, whereas access invites are the "you have been invited into this tenant" side.

5.4.5. 4. Access invites: grant a user into an existing tenant

Access invites are exposed at:

- `GET /access/invites`
- `GET /access/invites/{refName}`
- `POST /access/invites`
- `POST /access/invites/{refName}/revoke`
- `POST /access/invites/accept`

The first four require an authenticated caller in the target tenant. Invite acceptance is `@PermitAll` because the recipient may not yet have tenant access.

The persisted `AccessInvite` model includes:

- `email` or `targetUserId`
- `invitedByUserId`
- `scopeRefs`
- `grantedRoles`
- `allowedFunctionalAreas`
- `allowedFunctionalDomains`
- `allowedActions`
- `inviteMessage`
- `expiresAt`
- `acceptedAt`
- `acceptedUserId`
- `status` of `PENDING`, `ACCEPTED`, `REVOKED`, or `EXPIRED`

Representative invite creation request:

```
{  
  "email": "jane.doe@partner.example",  
}
```

```

"scopeRefNames": ["north-america"],
"grantedRoles": ["user"],
"allowedFunctionalAreas": ["integration"],
"allowedFunctionalDomains": ["exchange", "workflow"],
"allowedActions": ["view", "create"],
"expiresInDays": 14,
"inviteMessage": "Join the tenant workspace for partner onboarding."
}

```

Representative creation response:

```

{
  "refName": "invite-1775100000000",
  "email": "jane.doe@partner.example",
  "scopeRefNames": ["north-america"],
  "grantedRoles": ["user"],
  "allowedFunctionalAreas": ["integration"],
  "allowedFunctionalDomains": ["exchange", "workflow"],
  "allowedActions": ["view", "create"],
  "status": "PENDING",
  "inviteToken": "raw-token-returned-at-create-time-only"
}

```

Important invite behavior:

- either `email` or `targetUserId` is required
- only one active pending invite is allowed for the same email or target user
- tokens are stored as a hash; the raw token is only returned at invite creation time
- invites expire automatically based on `expiresAt`
- invites can be revoked before acceptance

Representative accept request:

```

{
  "realm": "acme-example",
  "token": "raw-token-returned-at-create-time-only",
  "email": "jane.doe@partner.example",
  "firstName": "Jane",
  "lastName": "Doe",
  "password": "choose-a-password"
}

```

Representative accept response:

```

{
  "userId": "jane.doe@partner.example",

```



```
"email": "jane.doe@partner.example",
"defaultRealm": "acme-example",
"inviteRefName": "invite-1775100000000",
"grantedScopes": ["north-america"]
}
```

What happens when an invite is accepted:

- the invite token is validated and checked for expiry
- the user identity is resolved from the authenticated user, request payload, invite target user, or invite email
- if the user does not already exist, a new credential is created and a password is required
- if the user already exists, the service ensures the user is authorized for the tenant realm
- a `UserProfile` is created if needed
- the invite status is changed to `ACCEPTED`

5.4.6. Invite extension points

The invite flow is intentionally extensible through two SPIs:

- `AccessInviteProvisioner`
- `AccessInviteNotificationService`

`AccessInviteProvisioner` can:

- validate requested scopes when the invite is created
- perform application-specific provisioning when the invite is accepted

`AccessInviteNotificationService` can:

- deliver the invite token through email, messaging, or another notification mechanism

The framework ships with no-op default beans for both, so applications can opt in to deeper behavior without rewriting the invite flow.

5.4.7. How these pieces fit together

A common pattern looks like this:

1. An admin provisions the tenant with `POST /admin/tenants`.
2. The tenant configures or accepts the default onboarding workflow with `GET/POST /onboarding/workflow/current`.
3. A tenant admin creates an access invite for a user with `POST /access/invites`.
4. The invited user accepts the invite with `POST /access/invites/accept`.
5. If the workflow requires registration, survey, or approval, those steps are completed before the

tenant considers onboarding complete.

This separation gives you flexibility:

- use tenant provisioning for admin-created tenants
- use registration requests for self-service intake and approval
- use access invites for controlled user admission into an existing tenant
- use workflow configuration to decide which of those steps are mandatory in each tenant

5.5. Choosing a Tenancy Model: Three Options

With realm membership (see [design/realm-membership-adr.adoc](#)) the platform supports three deliberate configurations:

Option	Shape	When to use
One tenant, one realm (default)	<code>POST /admin/tenants</code> provisions a realm with a single OWNER membership.	The standard SaaS tenant: full physical isolation, independent lifecycle, own backups/residency. This is what a multi-tenant platform provisions per customer.
Shared collaboration realm	One OWNER org plus N PARTICIPANT memberships (<code>RealmTenantMembership.membershipRole</code>). All parties' data lives in one realm; visibility is shaped per org by DataDomain stamps plus relationship policies (<code>hasEdge</code>) and field-level exclusions.	Multi-party operations against shared entities — e.g. a fulfillment realm where carrier, supplier, and customer orgs collaborate on the same sales orders and fulfillment plans, each seeing their own slice.
GitHub-like multi-realm identity	Users hold ONE global identity and N per-realm role assignments (<code>UserRealmRole</code>): ADMIN in realm A, VIEWER in realm B.	Consultants, partners, and platform operators who work across several tenants without duplicate accounts.

5.6. The GitHub-Like Identity Model, Mapped

GitHub concept	Quantum primitive	Notes
Account	Credential (system realm)	One global identity; <code>defaultRealm</code> is the personal/default context.
Organization	Organization + Account entities	Carry DataDomain identity inside realms.

GitHub concept	Quantum primitive	Notes
Org membership + role	<code>UserRealmRole</code> (userId, realmRefName, roles[], sponsoringOrgRefName)	ADMIN in A / VIEWER in B = two documents. <code>status</code> : active/invited/suspended.
Repository	Realm	The unit of placement and lifecycle (own database, migrations, seeds).
Repo transfer / ownership	<code>RealmTenantMembership</code> . <code>membershipRole = owner</code>	Exactly one owner per realm; owner holds lifecycle and billing authority.
Invitation	<code>AccessInvite</code> (/access-invites)	Existing invite flow grants a user into a realm; on acceptance a <code>UserRealmRole</code> records the granted roles.
Token scopes	JWT per-realm role claims	At issuance the IdP resolves <code>RealmMembershipService.rolesForUser(user, realm)</code> ; the tenant plane validates via JWKS and routes by claims — no callback to the control plane per request.

Resolution APIs live in `RealmMembershipService` (quantum-system): `membershipsForOrg`, `membersOfRealm`, `ownerOfRealm`, `realmsForUser`, `rolesForUser`. Embedded deployments resolve locally; remote (tier-2) deployments get the same surface backed by the control-plane client (Phase C).

5.7. User-Definable Facet Segmentation & Coordinate SPI in Multi-Tenant Environments

Traditional multi-tenancy models stop at isolating database collections by a single tenant ID. However, enterprise SaaS platforms often require multidimensional logical segmentation within and across tenants — such as isolating data by geographic region, operating subsidiary, business unit, partner channel, or sensitivity facet.

Quantum delivers this capability through the **Store-Free Coordinate SPI** (`DataDomainResolver`) and **User-Definable Facet Segmentation**, allowing a single server instance to support heterogeneous segmentation policies for different tenants.

5.7.1. Physical Realm Isolation vs. Logical Facet Segmentation

Quantum maintains a strict distinction between physical database placement and logical row-level segmentation:

- **Physical Realm Isolation:** Dictates MongoDB cluster and database routing (`DEDICATED` vs `SHARED / POOLED`). It acts as an inviolate physical admission gate.
- **Logical Facet Segmentation:** Dictates which rows, relationships, and fields within a realm are visible to a given principal based on multidimensional coordinates (e.g. `tenantId`, `orgRefName`, `department`, `region`, `classificationTier`).

5.7.2. Single Server Instance with Per-Tenant Heterogeneous Policies

A single Quantum server instance natively supports distinct, user-defined segmentation policies for different tenants without requiring separate runtime processes or re-deployments:

- **Tenant Isolation:** In dedicated realms, policies reside in the tenant's own database; in pooled realms, policies are isolated by `tenantId`.
- **Custom Dimension Modeling:** Tenant A can segment data by `department` and `complianceTier`, while Tenant B concurrently segments data by `region` and `partnerId`.
- **Dynamic Variable Injection:** Policies automatically bind contextual coordinates into `RuleContext` (`${pTenantId}`, `${orgRefName}`, `${ownerId}`, `${policyFilter}`, and custom facet variables), which are evaluated transparently across both single-collection queries and multi-hop aggregation pipelines.

5.7.3. Configuring User-Definable Facets

Facet definitions and policy rules are declared via `DataDomainComponentBinding` and `DataDomainPolicyEntry`:

Configuring Facet Bindings (`DataDomainComponentBinding`)

```
{
  "tenantId": "tenant-acme",
  "name": "OrderSegmentation",
  "facets": {
    "region": {
      "type": "FIELD",
      "targetPath": "shippingAddress.region"
    },
    "department": {
      "type": "FIELD",
      "targetPath": "departmentCode"
    },
    "classification": {
      "type": "VARIABLE",
      "targetPath": "principal.securityClearance"
    }
  }
}
```

Configuring Policy Constraints (`DataDomainPolicyEntry`)

```
{
  "tenantId": "tenant-acme",
  "targetClass": "com.e2eq.sample.Order",
  "action": "find",
  "filter": "status:!\\"ARCHIVED\\" && region:${userRegion}",
  "facetFilters": {
    "department": "${userDepartment}",
  }
}
```

```

    "classification": "CONFIDENTIAL"
  }
}

```

5.7.4. Coordinate Resolution Precedence and Defaults

When a request arrives at `QueryGatewayResource` or a governed `MorphiaRepo`, `DataDomainResolver` resolves coordinates using a strict three-tier precedence hierarchy:

```

flowchart TD
    Req["Incoming Query / Traversal Request"] --> T1["Tier 1: Tenant-Specific Policy?<br/>(systemDataDomainPolicies for Class + Action)"]
    T1 -- Yes --> ApplyTenant["Apply Tenant Policy & Bind Custom Facets"]
    T1 -- No --> T2["Tier 2: Global / System Policy Provider?"]
    T2 -- Yes --> ApplyGlobal["Apply Global Policy & Bind Standard Facets"]
    T2 -- No --> T3["Tier 3: Caller PrincipalContext Present?<br/>(tenantId, ownerId, orgRefName)"]
    T3 -- Yes --> ApplyPrincipal["Apply Principal Default Coordinates"]
    T3 -- No --> FailClosed["DataDomainResolution.Unresolvable<br/>(FAIL CLOSED - HTTP 500 / 403)"]

    ApplyTenant --> Execute["Execute Query / Traversal Pipeline"]
    ApplyGlobal --> Execute
    ApplyPrincipal --> Execute

```

Resolution Defaults Table

Resolution Stage	Resolution Source	Default / Fallback Behavior
Tenant Policy Configured	<code>systemDataDomainPolicies</code> collection for tenant.	Employs tenant's custom <code>filter</code> and <code>facetFilters</code> . Variables are injected into <code>RuleContext</code> .
No Tenant Policy Configured	System Global Policy Provider.	Employs platform-wide default data domain rules if registered; otherwise falls back to Tier 3.
Fallback (Zero Policy Records)	Caller's <code>PrincipalContext</code> .	Resolves default coordinates directly from user credentials: <code>pTenantId = principal.tenantId</code> , <code>ownerId = principal.userId</code> , <code>orgRefName = principal.orgRefName</code> .
Missing Security Credentials	None.	Yields <code>DataDomainResolution.Unresolvable</code> . Fails closed immediately with HTTP 500 (<code>SecurityContextMissing</code>) or HTTP 403.

5.7.5. Multi-Hop Traversal Isolation

During multi-hop traversals (e.g. `Customer` → `Order` → `Supplier` via `expand(...)`):

1. `DataDomainResolver.resolveForHop(targetClass, viaPath, context)` is invoked at each relationship boundary.
2. Target entity policies and row-level security constraints are applied to that hop's MongoDB `$lookup` pipeline.
3. Access constraints **accumulate monotonically**: a caller cannot hop across a foreign reference to bypass tenant boundaries or see records they are not authorized to view in the target domain.
4. References pointing to external tenants or unauthorized records are pruned silently, returning `null` or empty arrays to eliminate information leakage.

DomainContext, RuleContext, and DataDomain

Quantum enforces multi-tenant isolation and sharing through contextual data carried on models and evaluated at runtime.

Chapter 6. DataDomain

Every persisted model includes a DataDomain that describes ownership and scope, commonly including fields such as:

- `tenantId`: Identifies the tenant
- `orgRefName`: Organization unit reference within a tenant
- `ownerId`: Owning user or system entity
- `realm`: Optional runtime override for partitioning

These fields enable filtering, authorization, and controlled sharing of data between tenants or org units.

Chapter 7. DomainContext

DomainContext represents the current execution context for a request or operation, typically capturing:

- current tenant/org/user identity
- functional area / functional domain
- the action being executed (e.g., CREATE, UPDATE, VIEW, DELETE, ARCHIVE)

It feeds downstream components (repositories, resources) to consistently apply filtering and policy decisions.

Chapter 8. RuleContext

RuleContext encapsulates policy evaluation. It can:

- Enforce whether an action is allowed for a given model and DataDomain
- Produce additional filters and projections used by repositories
- Grant cross-tenant read access for specific functional areas (e.g., shared catalogs) while keeping others strictly isolated

Chapter 9. End-to-End Flow

1. A REST request enters a BaseResource-derived endpoint.
2. The resource builds a DomainContext from the security principal and request parameters.
3. RuleContext evaluates permissions and returns effective filters.
4. Repository applies filters (DataDomain-aware) to find/get/list/update/delete.
5. The model's UIActionList can be computed to reflect what the caller can do next.

This pattern ensures consistent enforcement across all CRUD operations, independent of the specific model or repository.

Chapter 10. Resolvers and Variables in Rule Filters

RuleContext can attach FILTERs (not only ALLOW/DENY) to repository queries using rule fields and filter strings. Variables inside those filter strings are populated from:

- PrincipalContext and ResourceContext standard variables: `principalId`, `pAccountId`, `pTenantId`, `ownerId`, `orgRefName`, `resourceId`, `action`, `functionalDomain`, `area`
- Dotted coordinate namespaces:
 - `${facet.<key>}` & `${coord.<key>}`: Resolved dynamically from tenant `DataDomainPolicyEntry.facetFilters` or contextual coordinate maps
 - `${principal.<attr>}`: Extracted from authenticated principal claims (`userId`, `defaultRealm`, `tenantId`, custom attributes)
 - `${dd.<attr>}`: Extracted from target `DataDomain` coordinates (`tenantId`, `orgRefName`, `accountNum`, `dataSegment`, `ownerId`)
- AccessListResolver SPI implementations: per-request computed Collections (e.g., customer IDs the caller can access)

Implementation highlights: - `AccessListResolver` has methods `key()`, `supports(...)`, `resolve(...)`. Resolvers are injected and invoked for each request; results are published as variables by key. - `MorphiaUtils.VariableBundle` carries both string variables and typed object variables (including Collections and arrays) to the query listener. - The `QueryToFilterListener` and in-memory `QueryToPredicateJsonListener` support single-variable IN clauses with or without brackets (`field:^[${var}]` or `field:^[${var}]`), expanding Collections/arrays into native `$in` filters and coercing types (`ObjectId`, numbers, booleans, dates).

10.1. Business Scenario: Multi-Tenant Franchise & Dynamic Partner Segmentation

A national retail brand operates diverse franchised store chains sharing a centralized inventory platform. Access is governed not by static roles, but by custom dynamic coordinates: store tiers (`facet.tier: ["FLAGSHIP", "REGIONAL"]`) and sales zones (`coord.salesZone: "NORTHEAST"`).

10.2. How the Feature Supports the Business Solution

Administrators define a single generic security policy:

```
tier:^[${facet.allowedTiers}] && zone:${coord.salesZone} && status:AVAILABLE
```

The `DataDomainResolver` SPI binds the caller's credentials into typed facet collections and zone variables at request time. Bracketless IN (`tier:^[${facet.allowedTiers}]`) automatically unpacks collections into native MongoDB `$in` operators. Unauthenticated or foreign-tenant requests fail closed.

Authoring examples: - Constrain by principal domain:

+

```
orgRefName:${orgRefName} && dataDomain.tenantId:${pTenantId}
```

- Bracketless single-variable IN for customer visibility or facet tiers:

```
customerId:^${accessibleCustomerIds} && tier:^${facet.allowedTiers}
```

For the complete query language reference, see [Query Language](#).

Chapter 11. Concrete example: building and using a resolver

This section shows how to implement a resolver that restricts access to orders by the set of customerIds the current user is allowed to see.

11.1. 1) Implement the SPI

Create a CDI bean that implements `AccessListResolver`. It decides when it applies and returns a Collection of values. The collection can be `ObjectId`, `String`, numbers, etc.

```
import com.e2eq.framework.securityrules.AccessListResolver;
import com.e2eq.framework.model.persistent.base.UnversionedBaseModel;
import com.e2eq.framework.model.securityrules.PrincipalContext;
import com.e2eq.framework.model.securityrules.ResourceContext;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;
import org.bson.types.ObjectId;
import java.util.*;

@ApplicationScoped
public class CustomerAccessResolver implements AccessListResolver {

    @Inject CustomerAccessService service; // your app-specific service

    @Override
    public String key() {
        // This becomes the variable name available to rules: ${accessibleCustomerIds}
        return "accessibleCustomerIds";
    }

    @Override
    public boolean supports(PrincipalContext pctx, ResourceContext rctx,
                           Class<? extends UnversionedBaseModel> modelClass) {
        // Optionally narrow by area/domain/action/model
        return rctx != null && "sales".equalsIgnoreCase(rctx.getArea())
            && "order".equalsIgnoreCase(rctx.getFunctionalDomain());
    }

    @Override
    public Collection<?> resolve(PrincipalContext pctx, ResourceContext rctx,
                                Class<? extends UnversionedBaseModel> modelClass) {
        // Return the set of customer ids for this user; could be ObjectId or String.
        // Example returns strings; the query listener will coerce 24-hex to ObjectId.
        return service.findCustomerIdsForUser(pctx.getUserId());
    }
}
```

Notes: - You can return `List<ObjectId>` directly if you prefer; no coercion needed then. - The resolver runs per request. Cache internally if the computation is expensive.

11.2. 2) How RuleContext uses resolvers

At query time, `RuleContext` discovers all `AccessListResolver` beans and calls `supports(...)`. For those that apply, it invokes `resolve(...)` and publishes the result into the variable bundle under the provided key(). Variables are available to the BI-API query via `${...}`.

Internally this uses `MorphiaUtils.VariableBundle` and `QueryToFilterListener` to carry both strings and typed objects/collections. Starting with this release you can reuse the exact same variable bundle for custom evaluation paths. The `RuleContext.resolveVariableBundle(...)` helper runs all matching `AccessListResolver` beans and returns the maps you can feed directly into `QueryPredicates` (for in-memory rules) or any other component that understands `${var}` tokens.

```
PrincipalContext pc = ...; // build from request
ResourceContext rc = ...;

MorphiaUtils.VariableBundle vars = ruleContext.resolveVariableBundle(pc, rc,
    UserProfile.class);

Predicate<JsonNode> predicate = QueryPredicates.compilePredicate(
    "customerId^[${accessibleCustomerIds}]",
    vars.strings,
    vars.objects
);

JsonNode order = QueryPredicates.toJsonNode(Map.of("customerId",
    "5f1e1a5e5e5e5e5e5e5e5e5e51"));
boolean allowed = predicate.test(order); // true when resolver returned that ObjectId
```

This allows the same resolver outputs to drive Morphia queries and client-side predicate checks without duplicating resolver logic.

11.3. 3) Author a rule that consumes the variable

Given the resolver above, a rule can attach an IN filter to constrain queries:

```
// andFilterString (example)
customerId^[${accessibleCustomerIds}]
```

When executed: - If `accessibleCustomerIds` is a `Collection/array`, each element is type-coerced (`ObjectId`, `number`, `date`, `boolean`, or `string`) and used in `$in`. - If `accessibleCustomerIds` is a comma-separated string, it is split and each token is coerced similarly. - An empty collection results in an empty `$in` (matches none), effectively denying access via filtering, not via `ALLOW/DENY`.

11.4. 4) End-to-end behavior

- SecurityFilter sets ResourceContext (area/domain/action) per request.
- RuleContext evaluates rules for the principal and resource and gathers resolvers.
- The repository composes filters including the rule-provided IN clause with the access list.
- Only documents whose customerId is in the caller's resolved set are returned.

11.5. String literals vs. typed values in resolver variables

When an `AccessListResolver` returns a list of values that will be used in an `IN` clause (for example, `field:^[${var}]`), the engine attempts to coerce each element to an appropriate type so Mongo/Morphia filters are typed correctly:

- 24-hex string → `ObjectId`
- `true/false` → `Boolean`
- integer → `Long`
- decimal → `Double`
- ISO-8601 datetime → `java.util.Date`
- `yyyy-MM-dd` → `java.time.LocalDate`
- otherwise → `String`

This works well when your target field is an `ObjectId`, number, or date. However, string fields can contain values that look like other types (for example, a 24-hex string that resembles an `ObjectId`). In those cases you must force "treat as plain string" so no coercion occurs.

To do this, the framework provides a small wrapper type `StringLiteral`. If a resolver returns `StringLiteral` instances, the listener unwraps them to plain `String` values and skips coercion entirely.

11.5.1. Example A: Resolver returns ObjectIds (typed)

```
@ApplicationScoped
public class CustomerAccessResolver implements AccessListResolver {
    public static final ObjectId ID1 = new ObjectId("5f1e1a5e5e5e5e5e5e51");
    public static final ObjectId ID2 = new ObjectId("5f1e1a5e5e5e5e5e5e52");

    @Override public String key() { return "accessibleCustomerIds"; }
    @Override public boolean supports(PrincipalContext p, ResourceContext r, Class<?
extends UnversionedBaseModel> m) {
        return r != null && "sales".equalsIgnoreCase(r.getArea()) && "order"
.equalsIgnoreCase(r.getFunctionalDomain()) && "view".equalsIgnoreCase(r.getAction());
    }
    @Override public Collection<?> resolve(PrincipalContext p, ResourceContext r,
```



```

Class<? extends UnversionedBaseModel> m) {
    return java.util.List.of(ID1, ID2); // typed values pass through as-is
}

```

Rule:

```
customerId:^(${accessibleCustomerIds})
```

Result: \$in with List<ObjectId> on customerId.

11.5.2. Example B: Resolver returns String literals (force raw strings)

```

@ApplicationScoped
public class CustomerCodeResolver implements AccessListResolver {
    @Override public String key() { return "accessibleCustomerCodes"; }
    @Override public boolean supports(PrincipalContext p, ResourceContext r, Class<?
extends UnversionedBaseModel> m) {
        return r != null && "sales".equalsIgnoreCase(r.getArea()) && "order"
.equalsIgnoreCase(r.getFunctionalDomain()) && "view".equalsIgnoreCase(r.getAction());
    }
    @Override public Collection<?> resolve(PrincipalContext p, ResourceContext r,
Class<? extends UnversionedBaseModel> m) {
        return java.util.List.of(
            com.e2eq.framework.model.persistent.morphia.StringLiteral.of(
                "5f1e1a5e5e5e5e5e5e5e5e51"),
            com.e2eq.framework.model.persistent.morphia.StringLiteral.of("CUST-42")
        );
    }
}

```

Rule:

```
customerCode:^(${accessibleCustomerCodes})
```

Result: \$in with List<String> on customerCode (even for hex-like strings).

11.5.3. Other types supported

Resolvers can also return numbers, booleans, and dates/datetimes. Already-typed elements (Number, Boolean, java.util.Date, java.time.LocalDate, ObjectId) are preserved. String elements are heuristically parsed into those types unless wrapped with `StringLiteral`.

Authoring tips:

- Prefer returning already-typed values when you know the target field type.

- Use `StringLiteral` when a value might be misinterpreted (for example, 24-hex or numeric-looking strings).
- For CSV strings published under a variable, the engine splits by comma and applies the same per-element coercion.

11.6. Using AccessListResolver with Ontology (optional)

When ontology is enabled, an `AccessListResolver` can compute ID lists from semantic edges (materialized in Mongo) and publish them as variables for use in rule filters.

Example resolver (conceptual)

```
@ApplicationScoped
public class OrdersByOrgResolver implements AccessListResolver {
    @Inject EdgeDao edgeDao; // from quantum-ontology-mongo
    @Override public String key() { return "idsByPlacedInOrg"; }
    @Override public boolean supports(PrincipalContext p, ResourceContext r, Class<?
extends UnversionedBaseModel> model) {
        return model.getSimpleName().equals("Order");
    }
    @Override public Collection<?> resolve(PrincipalContext p, ResourceContext r, Class
<? extends UnversionedBaseModel> model) {
        String tenantId = p.getDataDomain().getTenantId();
        String org = p.getDataDomain().getOrgRefName();
        return edgeDao.srcIdsByDst(tenantId, "placedInOrg", org);
    }
}
```

Rule filter usage

```
id:^{idsByPlacedInOrg}
```

Notes

- Always scope by `tenantId` from `RuleContext/PrincipalContext`.
- This is optional and only active if you wire ontology components. For a deeper integration path, see [Integrating Ontology](#).

11.7. Admitted vocabulary for rule predicates

When a rule references an ontology predicate through a query helper such as `hasEdge("placedInOrg", ...)`, the predicate must be **admitted** for policy use, not merely declared in the TBox. As `RuleContext` registers a rule it fires a `RuleVocabularyCheck`, and `PolicyVocabularyGuard` (`quantum-ontology-policy-bridge`) validates the referenced predicates against the realm's admitted

set. A predicate that is undeclared (**ABSENT**) or declared-but-not-admitted (**PROVISIONAL**) fails rule registration, naming the rule and the offending predicate.

Enforcement is fail-closed: if the realm's admitted set cannot be read it is treated as **UNAVAILABLE** and the guard refuses to downgrade a governed realm to "all admitted." A legacy realm (`admittedPredicates == null`) admits all declared predicates for back-compatibility. See [Ontology Modeling: provisional vs. admitted vocabulary tier](#) for the tier model, and the Quantum Enterprise **Ontology governance control plane** chapter for the admin promote/demote endpoints.

Chapter 12. Modeling with Functional Areas, Domains, and Actions

Quantum organizes your system around three core constructs:

- Functional Area: A broad capability area (e.g., Identity, Catalog, Orders, Collaboration).
- Functional Domain: A cohesive sub-area within an area (e.g., in Collaboration: Partners, Shipments, Tasks).
- Actions: The set of operations applicable to a domain (CREATE, UPDATE, VIEW, DELETE, ARCHIVE, plus domain-specific actions).

These constructs allow:

- Fine-grained sharing: Point specific functional areas to shared databases while others remain strictly segmented.
- Policy composition: Apply RuleContext decisions at the level of area/domain/action.

12.1. Prefer Annotations for Functional Mapping (recommended)

Starting with this version, you should use annotations to declare a model or resource's Functional Area/Domain and the Action being performed. The legacy `bmFunctionalArea()` and `bmFunctionalDomain()` methods are still supported for backward compatibility in this release, but they will be phased out soon.

- Class-level mapping: use `@FunctionalMapping(area = "<area>", domain = "<domain>")` on your model or resource class.
- Method-level action: use `@FunctionalAction("<ACTION>")` on your JAX-RS resource methods when the action is not implied by the HTTP verb.

Example: model annotated with `FunctionalMapping`

```
import dev.morphia.annotations.Entity;
import lombok.*;
import lombok.experimental.SuperBuilder;
import com.e2eq.framework.model.persistent.base.BaseModel;
import com.e2eq.framework.annotations.FunctionalMapping;

@Entity
@Data
@NoArgsConstructor
@SuperBuilder
@EqualsAndHashCode(callSuper = true)
@FunctionalMapping(area = "catalog", domain = "product")
public class Product extends BaseModel {
```

```

private String sku;
private String name;
// No need to override bmFunctionalArea/bmFunctionalDomain when using
@FunctionalMapping
}

```

Example: resource method annotated with FunctionalAction

```

import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import com.e2eq.framework.annotations.FunctionalAction;

@Path("/products")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class ProductResource {

    // Action will default from HTTP verb (POST -> CREATE), but you can be explicit:
    @POST
    @FunctionalAction("CREATE")
    public Product create(Product payload) { /* ... */ return payload; }

    // GET will infer VIEW automatically when building the ResourceContext
    @GET
    @Path("/{id}")
    public Product get(@PathParam("id") String id) { /* ... */ return new Product(); }
}

```

How the framework uses these annotations

- **SecurityFilter:** If the matched resource class has `@FunctionalMapping`, it uses area/domain from the annotation. If the method has `@FunctionalAction`, it uses that value; otherwise, it infers the action from the HTTP method (GET=VIEW, POST=CREATE, PUT/PATCH=UPDATE, DELETE=DELETE). If annotations are absent, it falls back to the existing path- and convention-based logic.
- **MorphiaRepo.fillUIActions:** If the model class has `@FunctionalMapping`, its area/domain are used to resolve allowed UI actions; otherwise, it falls back to the legacy `bmFunctionalArea()/bmFunctionalDomain()` methods.
- **PermissionResource:** When listing functional domains, it prefers `@FunctionalMapping` on entity classes and falls back to `bmFunctionalArea()/bmFunctionalDomain()` when missing.

Migration notes

- **Preferred:** add `@FunctionalMapping` to each model class (or resource class) and remove the `bmFunctionalArea()/bmFunctionalDomain` overrides.
- **Transitional:** you can keep the legacy methods; they will be used only if the annotation is not present.

- Future: `bmFunctionalArea` and `bmFunctionalDomain` will be removed in a future release; plan to migrate now.

See also: [Security Annotations: FunctionalMapping and FunctionalAction](#)

12.2. DataDomain on Models

All persisted models carry `DataDomain` (`tenantId`, `orgRefName`, `ownerId`, etc.) for rule-based filtering and cross-tenant sharing.

Example model:

```
import dev.morphia.annotations.Entity;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;
import lombok.experimental.SuperBuilder;
import com.e2eq.framework.model.persistent.base.BaseModel;

@Entity
@Data
@NoArgsConstructor
@SuperBuilder
@EqualsAndHashCode(callSuper = true)
public class Product extends BaseModel {
    private String sku;
    private String name;

    @Override
    public String bmFunctionalArea() { return "Catalog"; }

    @Override
    public String bmFunctionalDomain() { return "Product"; }
}
```

12.3. Persistence Repositories

Define a repository to persist and query your model. With Morphia:

```
import com.e2eq.framework.model.persistent.morphia.MorphiaRepo;

public interface ProductRepo extends MorphiaRepo<Product> {
    // custom queries can be added here
}
```

12.4. Exposing REST Resources

Expose consistent CRUD endpoints by extending `BaseResource`.

```
import com.e2eq.framework.rest.resources.BaseResource;
import jakarta.ws.rs.Path;

@Path("/products")
public class ProductResource extends BaseResource<Product, ProductRepo> {
    // Inherit find, get, list, save, update, delete endpoints
}
```

With this minimal setup, you get standard REST APIs guarded by `RuleContext/DataDomain` and enriched with `UIAction` metadata.

12.5. Lombok in Models

Lombok reduces boilerplate in Quantum models and supports inheritance-friendly builders.

Common annotations you will see:

- `@Data`: Generates getters, setters, `toString`, `equals`, and `hashCode`.
- `@NoArgsConstructor`: Required by frameworks that need a no-arg constructor (e.g., Jackson, Morphia).
- `@EqualsAndHashCode(callSuper = true)`: Includes superclass fields in equality and hash.
- `@SuperBuilder`: Provides a builder that cooperates with parent classes (useful for `BaseModel` subclasses).

Example:

```
@Data
@NoArgsConstructor
@SuperBuilder
@EqualsAndHashCode(callSuper = true)
public class Product extends BaseModel {
    private String sku;
    private String name;
}
```

Notes: - Prefer `@SuperBuilder` over `@Builder` when extending `BaseModel/UnversionedBaseModel`. - Keep `equals/hashCode` stable for collections and caches; include `callSuper` when needed.

12.6. Validation with Jakarta Bean Validation

Quantum uses Jakarta Bean Validation to enforce invariants on models at persist time (and

optionally at REST boundaries).

Typical annotations:

- `@Size(min=3)`: String/collection length constraints.
- `@Valid`: Cascade validation to nested objects (e.g., `DataDomain` on models).
- `@NotNull`, `@Email`, `@Pattern`, etc., as needed.

Where validation runs:

- Repository layer via Morphia `ValidationInterceptor` (`prePersist`):
- Executes `validator.validate(entity)` before the document is written.
- If there are violations and the entity does not implement `InvalidSavable` with `canSaveInvalid=true`, an `E2eqValidationException` is thrown.
- If `DataDomain` is null and `SecurityContext` has a principal, `ValidationInterceptor` will default the `DataDomain` from the principal context.
- Optionally at REST boundaries: You may also annotate resource DTOs/parameters with Jakarta validation; Quarkus can validate them before the method executes.

12.7. Jackson vs Jakarta Validation Annotations

These two families of annotations serve different purposes and complement each other:

- Jackson annotations (`com.fasterxml.jackson.annotation.*`) control JSON serialization/deserialization.
- Examples: `@JsonIgnore`, `@JsonIgnoreProperties`, `@JsonProperty`, `@JsonInclude`.
- They do not enforce business constraints; they affect how JSON is produced/consumed.
- Jakarta Validation annotations (`jakarta.validation.*`) declare constraints that are evaluated at runtime.
- Examples: `@NotNull`, `@Size`, `@Valid`, `@Pattern`.

Correspondence and interplay:

- Use Jackson to hide or rename fields in API responses/requests (e.g., `@JsonIgnore` on transient/calculated fields such as `UIActionList`).
- Use Jakarta Validation to ensure incoming/outgoing models satisfy required constraints; `ValidationInterceptor` runs before persistence to enforce them.
- It's common to annotate the same field with both families when you both constrain values and want specific JSON behavior.

Chapter 13. Jackson ObjectMapper in Quarkus and in Quantum

How Quarkus creates ObjectMapper:

- Quarkus produces a CDI-managed ObjectMapper. You can customize it by providing a bean that implements `io.quarkus.jackson.ObjectMapperCustomizer`.
- You can also tweak common features via `application.properties` using `quarkus.jackson.*` properties.

Quantum defaults:

- The framework provides a `QuarkusJacksonCustomizer` that:
- Sets `DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES = true` (reject unknown JSON fields).
- Registers custom serializers/deserializers for `org.bson.types.ObjectId` so it can be used as `String` in APIs.

Snippet from the framework:

```
@Singleton
public class QuarkusJacksonCustomizer implements ObjectMapperCustomizer {
    @Override
    public void customize(ObjectMapper objectMapper) {
        objectMapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, true);
        SimpleModule module = new SimpleModule();
        module.addSerializer(ObjectId.class, new ObjectIdJsonSerializer());
        module.addDeserializer(ObjectId.class, new ObjectIdJsonDeserializer());
        objectMapper.registerModule(module);
    }
}
```

Customize in your app:

- Add another `ObjectMapperCustomizer` bean (order is not guaranteed; make changes idempotent):

```
@Singleton
public class MyJacksonCustomizer implements ObjectMapperCustomizer {
    @Override
    public void customize(ObjectMapper mapper) {
        mapper.findAndRegisterModules();
        mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
        mapper.setSerializationInclusion(JsonInclude.Include.NON_NULL);
    }
}
```

```
}
```

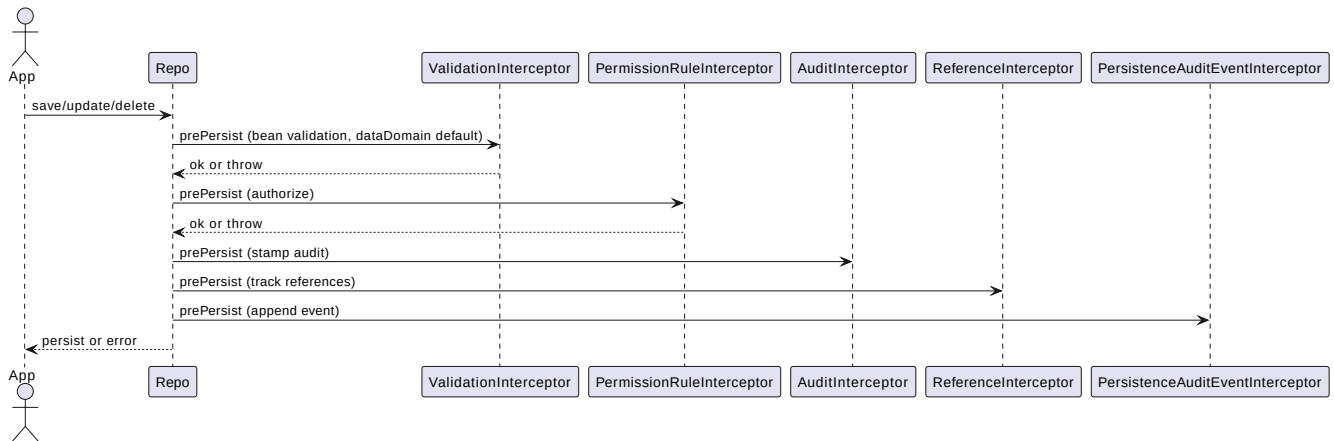
- Or set properties in `application.properties`:

```
# Fail if extraneous fields are present
quarkus.jackson.fail-on-unknown-properties=true
# Example date format and inclusion
quarkus.jackson.write-dates-as-timestamps=false
quarkus.jackson.serialization-inclusion=NON_NULL
```

When to adjust:

- Relax fail-on-unknown only for backward-compatibility scenarios; strictness helps catch client mistakes.
- Register modules (JavaTime, etc.) if your models include those types.

Chapter 14. Validation Lifecycle and Morphia Interceptors



Morphia interceptors enhance and enforce behavior during persistence. Quantum registers the following for each realm-specific datastore:

Order of registration (see MorphiaDataStore): 1) ValidationInterceptor 2) PermissionRuleInterceptor 3) AuditInterceptor 4) ReferenceInterceptor 5) PersistenceAuditEventInterceptor

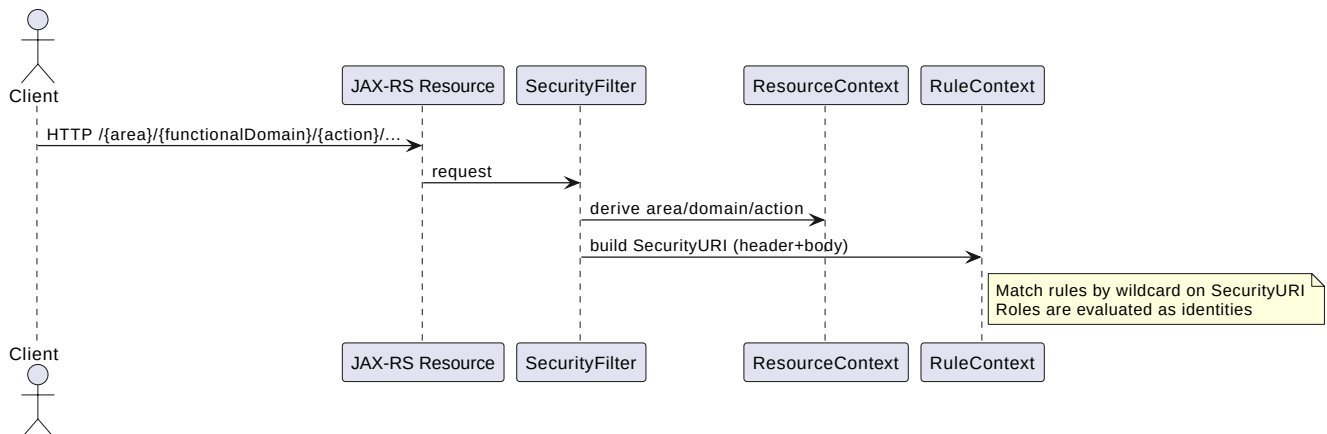
High-level responsibilities:

- ValidationInterceptor (prePersist):
 - Defaults DataDomain from SecurityContext if missing.
 - Runs bean validation and throws E2eqValidationException on violations unless the entity supports saving invalid states (InvalidSavable).
- PermissionRuleInterceptor (prePersist):
 - Evaluates RuleContext with PrincipalContext and ResourceContext from SecurityContext.
 - Throws SecurityCheckException if the rule decision is not ALLOW (enforcing write permissions for save/update/delete).
- AuditInterceptor (prePersist):
 - Sets AuditInfo on creation and updates lastUpdate fields on modification; captures impersonation details if present.
- ReferenceInterceptor (prePersist):
 - For @Reference fields annotated with @TrackReferences, maintains back-references on the parent entities via ReferenceEntry and persists the parent when needed.
- PersistenceAuditEventInterceptor (prePersist when @AuditPersistence is present):
 - Appends a PersistentEvent with type PERSIST, date, userId, and version to the model's persistentEvents before saving.

When does validation occur?

- On every save/update path that hits persistence, prePersist triggers validation (and permission/audit/reference processing) before the document is written to MongoDB, guaranteeing constraints and policies are enforced consistently across all repositories.

Chapter 15. Functional Area/Domain in RuleContext Permission Language



Models express their placement in the business model via: - `bmFunctionalArea()`: returns a broad capability area (e.g., Catalog, Collaboration, Identity) - `bmFunctionalDomain()`: returns the specific domain within that area (e.g., Product, Shipment, Partner)

How these map into authorization and rules:

- **ResourceContext/DomainContext**: When a request operates on a model, the framework derives the functional area and domain from the model type (or resource) and places them on the current context alongside the action (CREATE, UPDATE, VIEW, DELETE, ARCHIVE). **RuleContext** consumes these to evaluate policies.
- **Permission language (SecurityURI-based matching)**: The framework derives area and functionalDomain from REST path segments using the convention: `/[area]/[functionalDomain]/[action]/...`. These values, together with action and identity, form the SecurityURI (header + body) used by the rule engine. Rules match on SecurityURI fields using wildcard string comparison; there is no HTTP method/URL pattern matching.
- **Permission language (query variables)**: The ANTLR-based query language exposes variables that can be referenced in filters:
- `${area}` corresponds to `bmFunctionalArea()`
- `${functionalDomain}` corresponds to `bmFunctionalDomain()` These can be used to author reusable filters or to record audit decisions by area/domain.
- **Repository filters**: **RuleContext** can contribute additional predicates that are area/domain-specific, enabling fine-grained sharing. For example, a shared Catalog area may allow cross-tenant VIEW, while a Collaboration.Shipment domain remains tenant-strict.

Examples

1) SecurityURI-based rule matching (Permissions)

```
- name: allow-catalog-product-reads
description: Allow USER and ADMIN to view products in the Catalog area
securityURI:
```

```

header:
  identity: USER           # role treated as identity; author a second rule for
ADMIN if needed
  area: Catalog
  functionalDomain: Product
  action: view
body:
  realm: system-com
  accountNumber: '*'
  tenantId: '*'
  dataSegment: '*'
  ownerId: '*'
  resourceId: '*'
effect: ALLOW
priority: 300
finalRule: false

```

2) Query variable usage (Filters)

You can reference the active area/domain in filter expressions (e.g., for auditing or conditional branching in custom rule evaluators):

```

# Constrain reads differently when operating in the Catalog area
(${area}:"Catalog" && dataDomain.orgRefName:"PUBLIC") ||
(${area}:"!Catalog" && dataDomain.tenantId:${pTenantId})

```

3) Model-driven mapping

Given a model like:

```

@Override public String bmFunctionalArea() { return "Collaboration"; }
@Override public String bmFunctionalDomain(){ return "Shipment"; }

```

- Incoming REST requests that operate on Shipment resources set area=Collaboration and functionalDomain=Shipment in the ResourceContext.
- RuleContext evaluates policies considering action + area + domain, e.g., deny cross-tenant UPDATE in Collaboration.Shipment, but allow cross-tenant VIEW in Collaboration.Partner if marked shared.

Notes

- Path convention: Use leading segments `/area/functionalDomain/action/...` so the framework can derive ResourceContext reliably. Extra segments after the first three are allowed; only the first three are used to compute area, domain, and action.
- Nonconformant paths: If the path has fewer than three segments, the framework sets an anonymous/default ResourceContext. In practice, rules will typically evaluate to DENY unless there is an explicit allowance for anonymous contexts.

- See also: the Permissions section for rule-base matching and priorities, and the DomainContext/RuleContext section for end-to-end flow.

Chapter 16. StateGraphs on Models

StateGraphs let you restrict valid values and transitions of String state fields. They are declared on model fields with `@StateGraph` and enforced during save/update when the model class is annotated with `@Stateful`.

Key pieces: - `@StateGraph(graphName="...")`: mark a String field as governed by a named state graph. - `@Stateful`: mark the entity type as participating in state validation. - `StateGraphManager`: runtime registry that holds graphs and validates transitions. - `StringState` and `StateNode`: define the graph (states, initial/final flags, transitions).

Defining a state graph at startup:

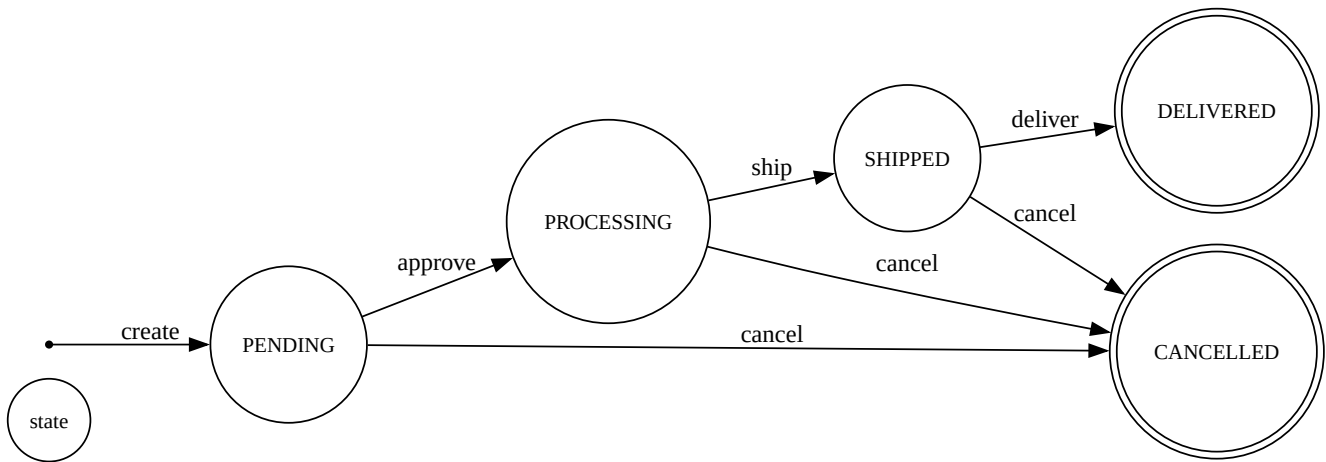
```
@Startup
@ApplicationScoped
public class StateGraphInitializer {
    @Inject StateGraphManager stateGraphManager;
    @PostConstruct void init() {
        StringState order = new StringState();
        order.setFieldName("orderStringState");

        Map<String, StateNode> states = new HashMap<>();
        states.put("PENDING", StateNode.builder().state("PENDING").initialState(true)
            .finalState(false).build());
        states.put("PROCESSING", StateNode.builder().state("PROCESSING").initialState(
            false).finalState(false).build());
        states.put("SHIPPED", StateNode.builder().state("SHIPPED").initialState(false)
            .finalState(false).build());
        states.put("DELIVERED", StateNode.builder().state("DELIVERED").initialState(
            false).finalState(true).build());
        states.put("CANCELLED", StateNode.builder().state("CANCELLED").initialState(
            false).finalState(true).build());
        order.setStates(states);

        Map<String, List<StateNode>> transitions = new HashMap<>();
        transitions.put("PENDING", List.of(states.get("PROCESSING"), states.get(
            "CANCELLED")));
        transitions.put("PROCESSING", List.of(states.get("SHIPPED"), states.get(
            "CANCELLED")));
        transitions.put("SHIPPED", List.of(states.get("DELIVERED"), states.get(
            "CANCELLED")));
        transitions.put("DELIVERED", null);
        transitions.put("CANCELLED", null);
        order.setTransitions(transitions);

        stateGraphManager.defineStateGraph(order);
    }
}
```

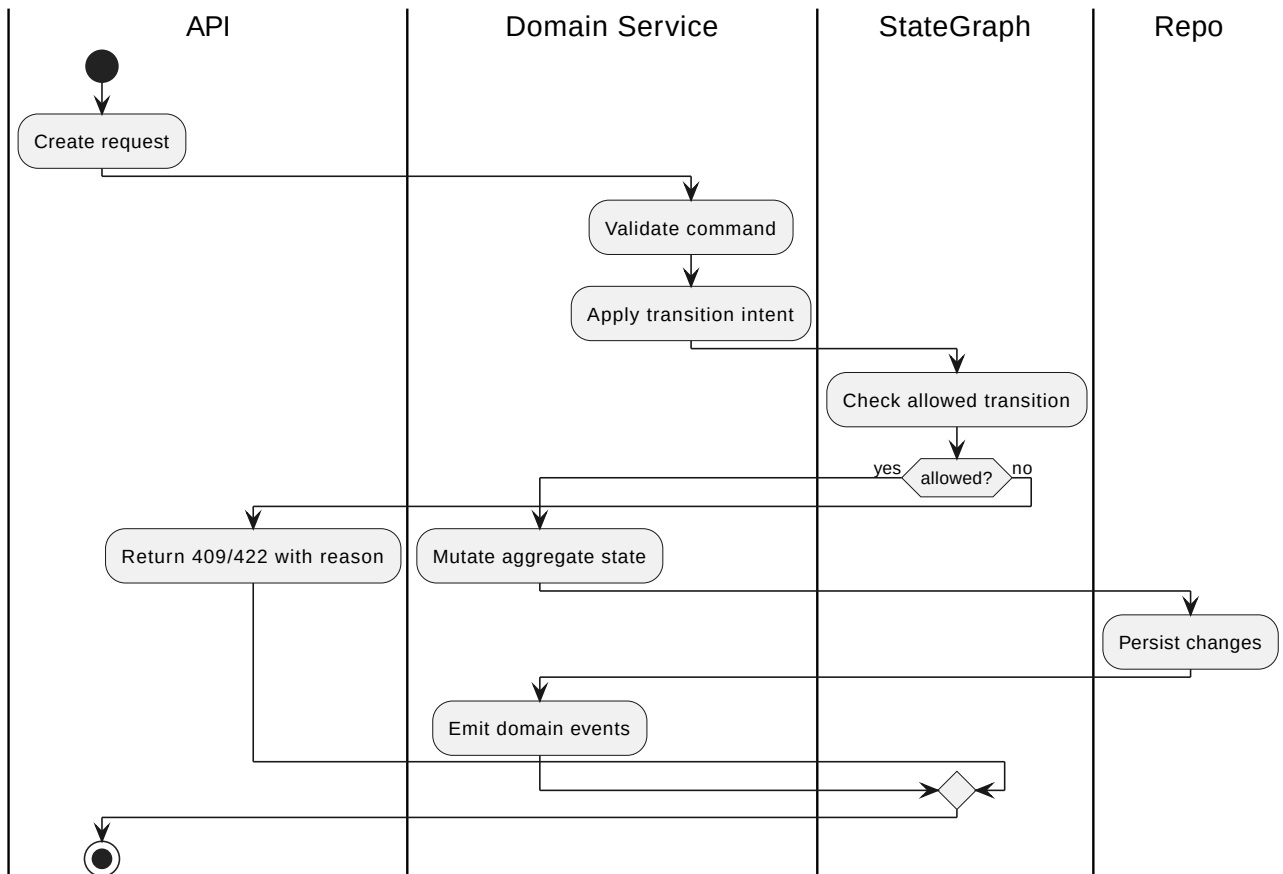

Chapter 17. State Graph (DOT)



Using the graph in a model:

```
@Stateful
@Entity
@EqualsAndHashCode(callSuper = true)
public class Order extends BaseModel {
    @StateGraph(graphName = "orderStringState")
    private String status;

    @Override public String bmFunctionalArea() { return "Orders"; }
    @Override public String bmFunctionalDomain(){ return "Order"; }
}
```



How it affects save/update:

- On create: `validateInitialStates` ensures the field value is one of the configured initial states. Otherwise, `InvalidStateTransitionException` is thrown.
- On update: `validateStateTransitions` checks each `@StateGraph` field's old → new transition against the graph via `StateGraphManager.validateTransition()`. If invalid, save/update fails with `InvalidStateTransitionException`. This applies to full-entity saves and to partial updates via `repo.update(...pairs)` on that field.
- Utilities: `StateGraphManager.getNextPossibleStates(graphName, current)` and `printStateGraph(...)` can aid UIs.

Chapter 18. CompletionTasks and CompletionTaskGroups

CompletionTasks and CompletionTaskGroups provide a small persistence-backed contract for tracking work that finishes later. They are useful when a request starts work immediately but the actual completion happens in a worker, another service, or a human-in-the-loop integration.

This section documents the concrete structures and runtime behavior, because other guides reference "completion tasks" as a pattern but do not define the payloads and lifecycle precisely.

18.1. When to use them

Use CompletionTasks when you need:

- A durable record of work that may outlive the initiating HTTP request
- A simple per-item status model with timestamps and outcome text
- A way to group related tasks under one business operation
- A lightweight event stream so clients can react when a task completes

Do not treat them as a workflow engine. The framework does not infer dependencies, retries, compensation rules, or aggregate group state transitions for you.

18.2. Core entities

Both entities extend `BaseModel`, so they inherit the common Quantum fields such as `id`, `refName`, `displayName`, `dataDomain`, `activeStatus`, `auditInfo`, and `version`.

The inherited fields matter here in two ways:

- `id` is the record identifier used by the REST endpoints and emitted in completion events.
- `refName`, `displayName`, and `dataDomain` are still available for your application-level correlation and multi-tenant scoping, even though they are not task-specific.

18.2.1. CompletionTask

```
@Entity("completionTask")
public class CompletionTask extends BaseModel {
    public enum Status { PENDING, RUNNING, SUCCESS, FAILED }

    @Reference
    CompletionTaskGroup group;

    String details;
    Status status;
    Date createdAt;
```

```

    Date completedDate;
    String result;
}

```

Field	Meaning	Set by
group	Optional Morphia reference to the parent <code>CompletionTaskGroup</code>	Client on create, or the <code>create/{groupId}</code> endpoint
details	Human-readable explanation of the unit of work, usually "what this task is doing"	Client/application
status	Task execution state: <code>PENDING</code> , <code>RUNNING</code> , <code>SUCCESS</code> , <code>FAILED</code>	Server sets <code>PENDING</code> on create; application or API updates afterward
createdDate	Timestamp when the task record was created	Server
completedDate	Timestamp written when the task is completed through the completion endpoint	Server
result	Outcome text, response summary, or error message	Application or completion endpoint caller

18.2.2. CompletionTaskGroup

```

@Entity("completionTaskGroup")
public class CompletionTaskGroup extends BaseModel {
    public enum Status { NEW, RUNNING, COMPLETE }

    String description;
    Status status;
    Date createdDate;
    Date completedDate;
}

```

Field	Meaning	Set by
description	Human-readable summary of the batch, workflow, or request that owns the tasks	Client/application
status	Coarse aggregate state: <code>NEW</code> , <code>RUNNING</code> , <code>COMPLETE</code>	Server sets <code>NEW</code> on create; application owns later transitions
createdDate	Timestamp when the group was created	Server

Field	Meaning	Set by
<code>completedDate</code>	Timestamp for when your application considers the group finished	Application

18.3. What the framework manages vs. what your application manages

The code-level behavior is intentionally small:

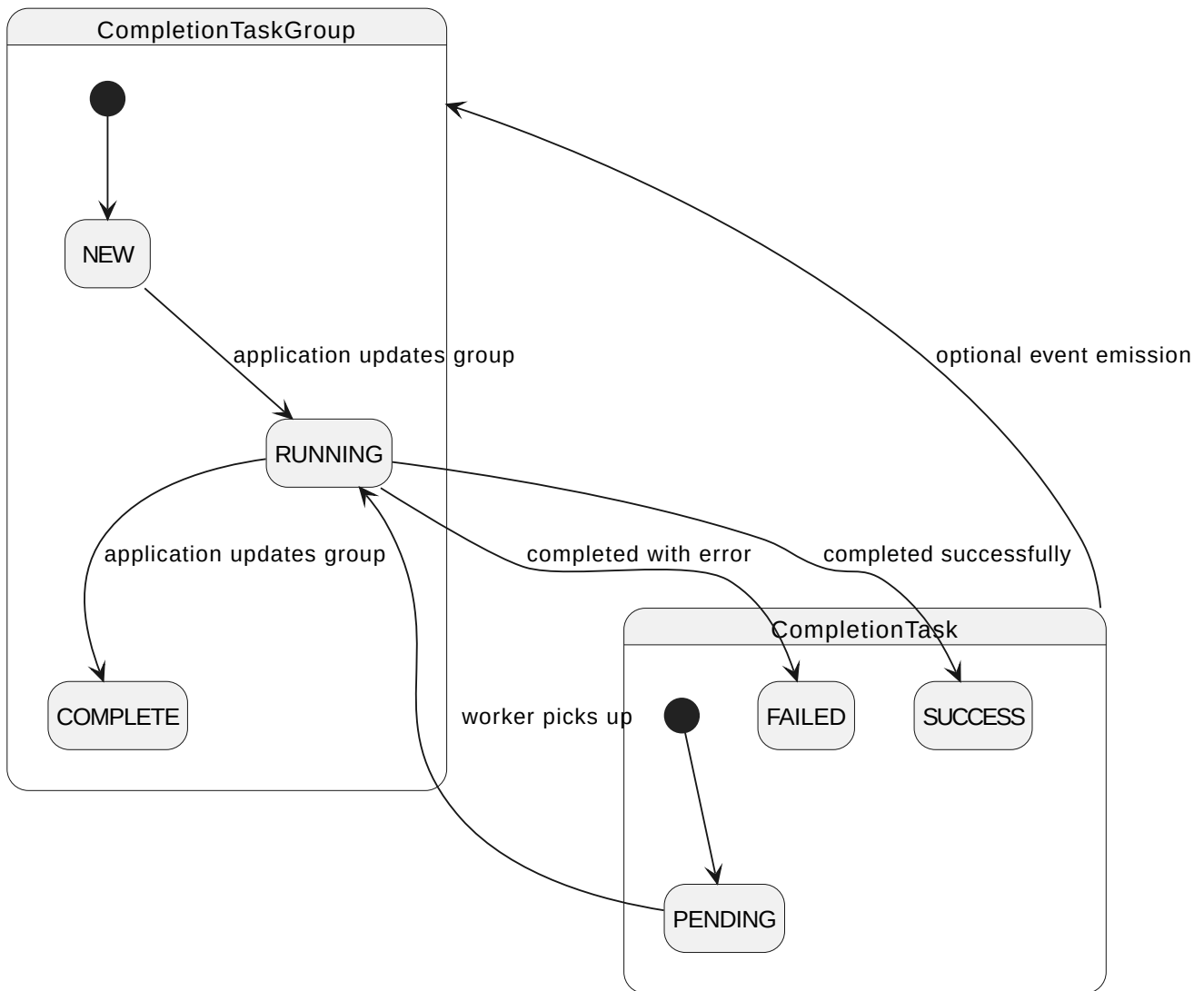
- `CompletionTaskGroupRepo.createGroup(...)` always sets `status=NEW` and `createdDate=now`.
- `CompletionTaskRepo.createTask(...)` always sets `status=PENDING` and `createdDate=now`.
- `CompletionTaskRepo.completeTask(...)` sets `status`, `completedDate=now`, and `result`.
- If the task belongs to a group, task completion emits an SSE message for that group.

The framework does not automatically:

- Change a group from `NEW` to `RUNNING`
- Mark a group `COMPLETE` when all child tasks finish
- Count completed tasks or calculate percentages
- Retry failed tasks
- Validate that only terminal states are used with the completion endpoint

That means group rollup is application-owned. If you want `CompletionTaskGroup.status` to represent aggregate progress, update it explicitly from your worker or orchestration layer.

18.4. State model



Recommended conventions:

- Set a task to **RUNNING** when a worker begins processing it.
- Use **SUCCESS** and **FAILED** as the terminal task states.
- Set a group to **RUNNING** once at least one child task has begun.
- Set a group to **COMPLETE** only when your own aggregation logic decides the batch is finished.

18.5. REST API surface

The completion-task resources are small wrappers on top of the generic **BaseResource**, so they expose both task-specific endpoints and inherited read endpoints.

18.5.1. Task group endpoints

Method and path	Purpose	Notes
POST /integration/completionTaskGroup/create	Create a new task group	Server forces <code>status=NEW</code> and <code>createdDate</code>
GET /integration/completionTaskGroup/id/{id}	Fetch a group by Mongo <code>id</code>	Inherited from <code>BaseResource</code>
GET /integration/completionTaskGroup/refName/{refName}	Fetch a group by <code>refName</code>	Inherited from <code>BaseResource</code>
GET /integration/completionTaskGroup/subscribe/{id}	Subscribe to completion events for tasks in the group	Returns server-sent events

18.5.2. Task endpoints

Method and path	Purpose	Notes
POST /integration/completionTask/create/{groupId}	Create a task and attempt to attach it to the given group	Server forces <code>status=PENDING</code> and <code>createdDate</code>
PUT /integration/completionTask/complete/{id}?status=SUCCESS&result=...	Update a task status, set <code>completedDate</code> , and store <code>result</code>	Despite the method name, the code accepts any <code>CompletionTask.Status</code>
GET /integration/completionTask/id/{id}	Fetch a task by Mongo <code>id</code>	Inherited from <code>BaseResource</code>
GET /integration/completionTask/refName/{refName}	Fetch a task by <code>refName</code>	Inherited from <code>BaseResource</code>

18.6. Request and response payloads

18.6.1. Creating a group

Minimal request:

```
{
  "refName": "catalog-import-20260327",
  "displayName": "Catalog Import 2026-03-27",
  "description": "Nightly catalog import for 250 SKUs"
}
```

Representative response:

```
{
  "id": "67e51d7c9d0f1d6df2f8ab12",
  "refName": "catalog-import-20260327",
  "displayName": "Catalog Import 2026-03-27",
  "description": "Nightly catalog import for 250 SKUs",
  "status": "NEW",
  "createdDate": "2026-03-27T04:12:14.118Z",
  "completedDate": null
}
```

18.6.2. Creating a task in a group

Minimal request:

```
{
  "refName": "sku-100045",
  "displayName": "Import SKU 100045",
  "details": "Import SKU 100045 from supplier feed"
}
```

Representative response:

```
{
  "id": "67e51df49d0f1d6df2f8ab13",
  "refName": "sku-100045",
  "displayName": "Import SKU 100045",
  "details": "Import SKU 100045 from supplier feed",
  "status": "PENDING",
  "createdDate": "2026-03-27T04:14:12.091Z",
  "completedDate": null,
  "result": null,
  "group": {
    "id": "67e51d7c9d0f1d6df2f8ab12"
  }
}
```



```
}
```

Notes:

- The endpoint path requires a `groupId`.
- If the `groupId` does not resolve to an existing group, the current repository implementation still creates the task, but leaves `group` unset.
- The server ignores any client-supplied `status` or `createdDate` during creation and overwrites them.

18.6.3. Completing a task

Example:

```
curl -X PUT \  
  
"http://localhost:8080/integration/completionTask/complete/67e51df49d0f1d6df2f8ab13?status=SUCCESS&result=Imported%20successfully"
```

Representative response:

```
{  
  "id": "67e51df49d0f1d6df2f8ab13",  
  "status": "SUCCESS",  
  "completedDate": "2026-03-27T04:16:08.501Z",  
  "result": "Imported successfully"  
}
```

Important behavior:

- `completedDate` is always set by the completion endpoint.
- The endpoint name says "complete", but the implementation accepts any value in `CompletionTask.Status`.
- Updating the task does not automatically update the parent group status.

18.7. Completion event stream

`GET /integration/completionTaskGroup/subscribe/{groupId}` returns server-sent events. Each event is emitted when `CompletionTaskRepo.completeTask(...)` is called for a task in that group.

The event payload is plain text, not JSON. The current format is:

```
task:{taskId}:{status}
```

Example event data:

```
task:67e51df49d0f1d6df2f8ab13:SUCCESS
```

If you need richer event payloads, build that in your own orchestration layer or extend the resource.

18.8. Typical orchestration flow

Example creation flow:

```
CompletionTaskGroup group = completionTaskGroupRepo.createGroup(
    CompletionTaskGroup.builder()
        .refName("catalog-import-20260327")
        .displayName("Catalog Import 2026-03-27")
        .description("Nightly catalog import for 250 SKUs")
        .build()
);

for (Sku s : skus) {
    completionTaskRepo.createTask(
        CompletionTask.builder()
            .refName("sku-" + s.code())
            .displayName("Import SKU " + s.code())
            .details("Import SKU " + s.code() + " from supplier feed")
            .build(),
        group.getId().toString()
    );
}
```

Example worker progression:

```
CompletionTask task = completionTaskRepo.findOneByStatus(CompletionTask.Status.
PENDING);
if (task != null) {
    completionTaskRepo.update(task.getId(), "status", CompletionTask.Status.RUNNING);
    completionTaskGroupRepo.update(task.getGroup().getId(), "status",
CompletionTaskGroup.Status.RUNNING);

    try {
        // ... do work ...
        completionTaskRepo.completeTask(
            task.getId().toString(),
            CompletionTask.Status.SUCCESS,
            "Imported successfully"
        );
    } catch (Exception e) {
```

```

        completionTaskRepo.completeTask(
            task.getId().toString(),
            CompletionTask.Status.FAILED,
            e.getMessage()
        );
    }
}

```

Example aggregate rollup owned by the application:

```

List<CompletionTask> tasks = completionTaskRepo.findByGroup(group);

boolean allTerminal = tasks.stream().allMatch(t ->
    t.getStatus() == CompletionTask.Status.SUCCESS ||
    t.getStatus() == CompletionTask.Status.FAILED
);
boolean anyRunning = tasks.stream().anyMatch(t ->
    t.getStatus() == CompletionTask.Status.RUNNING
);

if (allTerminal) {
    completionTaskGroupRepo.update(
        group.getId(),
        "status", CompletionTaskGroup.Status.COMPLETE,
        "completedDate", new Date()
    );
} else if (anyRunning) {
    completionTaskGroupRepo.update(
        group.getId(),
        "status", CompletionTaskGroup.Status.RUNNING
    );
}

```

18.9. Best practices

- Keep **details** short and stable so operators can quickly scan task lists.
- Use **result** for outcome summaries, error messages, or external correlation identifiers.
- Populate **refName** with a meaningful business key so tasks can be found without raw Mongo ids.
- Use **dataDomain** consistently so task and group records stay tenant-scoped.
- Treat group status as a denormalized summary owned by your orchestration code.
- Purge or archive old completed groups to avoid unbounded growth.

Chapter 19. References and EntityReference

Morphia `@Reference` establishes relationships between entities: - One-to-one: a `BaseModel` field annotated with `@Reference`. - One-to-many: a `Collection<BaseModel>` field annotated with `@Reference`.

Example:

```
@Entity
public class Shipment extends BaseModel {
    @Reference(ignoreMissing = false)
    @TrackReferences
    private Partner partner;    // parent entity
}
```

`EntityReference` is a lightweight reference object used across the framework to avoid `DBRef` loading when only identity info is needed. Any model can produce one:

```
EntityReference ref = shipment.createEntityReference();
// contains: entityId, entityType, entityRefName, entityDisplayName (and optional
realm)
```

REST convenience:

- `BaseResource` exposes `GET /entityref` to list `EntityReference` for a model with optional filter/sort.
- `Repositories` expose `getEntityReferenceListByQuery(...)`, and utilities exist to convert lists of `EntityReference` back to entities when needed.

When to use which:

- Use `@Reference` for strong persistence-level links where Morphia should maintain foreign references.
- Use `EntityReference` for UI lists, foreign-key-like pointers in other documents, events/audit logs, or cross-module decoupling without `DBRef` behavior.

Chapter 20. Tracking References with @TrackReferences and Delete Semantics

@TrackReferences on a @Reference field tells the framework to maintain a back-reference set on the parent entity. The back-reference field is `UnversionedBaseModel.references` (a `Set<ReferenceEntry>`), which is calculated/maintained by the framework and should not be set by clients.

What references contains:

- Each `ReferenceEntry` holds: `referencedId` (`ObjectId` of the child), `type` (fully-qualified class name of the child's entity), and `refName` (child's stable reference name).
- It indicates that the parent is being referenced by the given child entity. The set is used for fast checks and to enforce referential integrity.

How tracking works (save/update):

- `ReferenceInterceptor` inspects @Reference fields annotated with @TrackReferences during `prePersist`.
- When a child references a parent, a `ReferenceEntry` for the child is added to the parent's references set and the parent is saved to persist the back-reference.
- For @Reference collections, entries are added for each child-parent pair.
- If a @Reference is null but `ignoreMissing=false`, a save will fail with an `IllegalStateException` since the parent is required.

How it affects delete:

- During delete in `MorphiaRepo.delete(...)`:
- If `obj.references` is empty, the object can be deleted directly (after removing any references it holds to parents).
- If `obj.references` is not empty, the repo checks each `ReferenceEntry`. If any referring parent still exists, a `ReferentialIntegrityViolationException` is thrown to prevent breaking relationships.
- If all references are stale (referring objects no longer exist), the repo removes stale entries, removes this object's own reference constraints from parents, and performs the delete within a transaction.
- `removeReferenceConstraint(...)` ensures that, when deleting a child, its `ReferenceEntry` is removed from `parent.references` and the parent is saved, keeping back-references consistent.

Practical guidance:

- Annotate parent links with both @Reference and @TrackReferences when you need strong integrity guarantees and easy “who references me?” queries.
- Use `ignoreMissing=true` only for optional references; you still get back-reference tracking when not null.

- Expect HTTP delete to fail with a meaningful error if there are live references; remove or update those references first, or design cascading behavior explicitly in your domain logic.

20.1. Ontologies in Quantum: Modeling Relationships That Are Resilient and Fast



Looking for the short implementation plan? See PROPOSAL.md at the repository root for a concise module-by-module checklist.

This section explains what an ontology is, how it differs from a traditional object model, and how the Quantum Ontology modules make it practical to apply ontology ideas to your domain models and queries. It also contrasts ontology-driven relationships with direct object references (for example, using `@Reference` or `EntityReference`).

20.1.1. What is an Ontology?

In software terms, an ontology is a formal, explicit specification of concepts and their relationships.

- Concepts (Classes): Named categories/types in your domain. Concepts can form taxonomies (is-a hierarchies), be declared disjoint, or be equivalent.
- Relationships (Properties): Named relationships between entities. Properties can have a domain (applies to X) and a range (points to Y). They may be inverse or transitive.
- Axioms (Rules): Constraints and entailment rules, including property chains such as: if $(A \xrightarrow{p} B)$ and $(B \xrightarrow{q} C)$ then we infer $(A \xrightarrow{r} C)$.
- Inference: The process of deriving new facts (types, labels, edges) that were not explicitly stored but follow from axioms and known facts.

An ontology is not the data; it is the schema plus logic that gives your data additional meaning and enables consistent, automated inferences.

20.1.2. Ontology vs. Object Model

A conventional object model focuses on concrete classes, fields, and direct references between objects at implementation time. An ontology focuses on semantic types and relationships, with explicit rules that can derive new knowledge independent of how objects are instantiated.

Key differences:

- Purpose - Object model: Encapsulate data and behavior for application code generation and persistence.
- Ontology: Encode shared meaning, constraints, and inference rules that remain stable as implementation details change.
- Relationship handling - Object model: Typically uses direct references or foreign keys; traversals are hard-coded and fragile to change.
- Ontology: Uses named predicates (properties) and can infer additional relationships by rules (property chains, inverses, transitivity).
- Polymorphism and evolution - Object model: Polymorphism requires class inheritance in code; cross-cutting categories are awkward to add later.
- Ontology: Entities can have multiple types/labels at once. New concepts and properties can be introduced without breaking existing data.
- Querying - Object model: Queries couple to concrete classes and field paths; changes force query rewrites.
- Ontology: Queries target semantic

relationships; reasoners can materialize edges that queries reuse, decoupling queries from implementation details.

20.1.3. Why prefer Ontology-driven relationships over @Reference/EntityReference

Direct references (@Reference or custom EntityReference) are simple to start but become restrictive as domains grow: - Tight coupling: Code and queries couple to concrete field paths (customer.primaryAddress.id), making refactors risky. - Limited expressivity: Hard to encode and reuse higher-order relationships (e.g., "partners of my supplier's parent org"). - Poor polymorphism: References point to one collection/type; accommodating multiple target types requires extra code. - Performance pitfalls: Deep traversals cause extra queries, N+1 selects, or complex \$lookup joins.

Ontology-driven edges address these issues: - Decoupling via predicates: Use named predicates (e.g., hasAddress, memberOf, supplies) that remain stable while internal object fields change. - Inference for reachability: Property chains can materialize implied links ($A \xrightarrow{p} B \ \& \ B \xrightarrow{q} C \Rightarrow A \xrightarrow{r} C$), avoiding runtime multi-hop traversals. - Polymorphism-first: A predicate can connect heterogeneous types; type inferences (domain/range) remain consistent. - Query performance: Pre-materialized edges allow single-hop, index-friendly queries (in or eq filters) instead of ad-hoc multi-collection traversals. - Resilience to change: You can add or modify rules without rewriting data structures or touching referencing fields across models.

20.1.4. How Quantum supports Ontologies

Quantum provides three cooperating modules that make ontology modeling practical and fast:

- quantum-ontology-core (package com.e2eq.ontology.core)
- OntologyRegistry: Holds the TBox (terminology) of your ontology.
- ClassDef: Concept names and relationships (parents, disjointWith, sameAs).
- PropertyDef: Property names with optional domain, range, inverse flags, and transitivity.
- PropertyChainDef: Rules that define multi-hop implications (chains $\rightarrow$ implied property).
- TBox: Container for classes, properties, and property chains.
- Reasoner interface and ForwardChainingReasoner: Given an entity snapshot and the registry, computes inferences:
- New types/labels to assert on entities.
- New edges to add (implied by property chains, inverses, or other rules).
- quantum-ontology-mongo (package com.e2eq.ontology.mongo)
- EdgeDao: A thin DAO around an edges collection in Mongo. Each edge contains tenantId, src, predicate p, dst, inferred flag, provenance, and timestamp.
- OntologyMaterializer: Runs the Reasoner for an entity snapshot and upserts the inferred edges, so queries can be rewritten to simple in/in eq filters.
- quantum-ontology-policy-bridge (package com.e2eq.ontology.policy)
- ListQueryRewriter: Takes a base query and rewrites it using the EdgeDao to filter by the set of

source entity ids that have a specific predicate to a given destination.

- This integrates ontology edges with RuleContext or policy decisions: policy asks for entities related by a predicate; the rewriter converts that into an efficient Mongo query.

These modules let you define your ontology (core), materialize derived relations (mongo), and leverage them in access and list queries (policy bridge).

20.1.5. Modeling guidance: from object fields to predicates

- Name relationships explicitly
- Define clear predicate names (hasAddress, memberOf, supplies, owns, assignedTo). Avoid encoding relationship semantics in field names only.
- Keep object model minimal and flexible
- Store lightweight identifiers (ids) as needed, but avoid deeply nested reference graphs that encode traversals in code.
- Model polymorphic relationships
- Prefer predicates that naturally connect multiple possible types (e.g., assignedTo can target User, Team, Bot) and rely on ontology type assertions to constrain where needed.
- Use property chains for common paths
- If business logic often traverses $A \rightarrow B \rightarrow C$, define a chain $p \sqcap q \Rightarrow r$ and materialize r for faster queries and simpler policies.
- Capture inverses and transitivity
- For natural inverses (parentOf $\sqcap$ childOf) or transitive relations (partOf, locatedIn), define them in the ontology so edges and queries stay consistent.
- Keep provenance
- Record why an edge exists (prov.rule, prov.inputs) so you can recompute, audit, or retract when inputs change.

20.1.6. Querying with ontology edges vs direct references

- Direct reference example (fragile/slow)
- Query: "Find Orders whose buyer belongs to Org X or its parents."
- With @Reference: requires joining $\text{Order} \rightarrow \text{User} \rightarrow \text{Org}$ and recursing org.parent; costly and tightly coupled to fields.
- Ontology edge example (resilient/fast)
- Define predicates: placedBy(order, user), memberOf(user, org), ancestorOf(org, org). Define chain placedBy $\sqcap$ memberOf $\Rightarrow$ placedInOrg.
- Materialize edges: (order --placedInOrg--> org). Also make ancestorOf transitive.
- Query becomes: where order._id in EdgeDao.srcIdsByDst(tenantId, "placedInOrg", orgX).
- With transitivity, you can precompute ancestor closure or add a chain placedInOrg $\sqcap$ ancestorOf $\Rightarrow$ placedInOrg to include parents automatically.

20.1.7. Migration: from @Reference to ontology edges

- Start by introducing predicates alongside existing references; do not remove references immediately.
- Materialize edges for hot read paths; keep provenance so you can reconstruct.
- Gradually update queries (list screens, policy filters) to use ListQueryRewriter with EdgeDao instead of deep traversals or \$lookup.
- Once stable, you can simplify models by removing rigid reference fields where unnecessary and rely on edges for read-side composition.

20.1.8. Performance and operational notes

- Indexing: Create compound indexes on edges: (tenantId, p, dst) and (tenantId, src, p) to support both reverse and forward lookups.
- Write amplification vs read wins: Materialization adds write work, but dramatically improves read latency and simplifies queries.
- Consistency: Re-materialize edges on relevant entity changes (source, destination, or intermediate) using OntologyMaterializer.
- Multi-tenancy: Keep tenantId in the edge key and filters; the provided EdgeDao methods include tenant scoping.

20.1.9. How this integrates with Functional Areas/Domains

- Functional domains often map to concept clusters in the ontology. Use @FunctionalMapping to aid discovery and apply policies per area/domain.
- Policies can refer to relationships semantically ("hasEdge placedInOrg OrgX") and rely on the policy bridge to turn this into efficient data filters.

20.1.10. Summary

- Ontology-powered relationships provide a stable, semantic layer over your object model.
- The Quantum Ontology modules let you define, infer, and query these relationships efficiently on MongoDB.
- Compared with direct @Reference/EntityReference, ontology edges are more expressive, resilient to change, and typically faster for complex list/policy queries once materialized.

20.1.11. Concrete example: Sales Orders, Shipments, and evolving to Fulfillment/Returns

This example shows how to use an ontology to model relationships around Orders, Customers, and Shipments, and how the model can evolve to include Fulfillment and Returns without breaking existing queries. We will:

- Define core concepts and predicates.
- Add property chains that materialize implied relationships for fast queries.

- Show how queries are rewritten using edges instead of deep object traversals.
- Evolve the model to support Fulfillment and Returns with minimal changes.

Core concepts (classes)

- Order, Customer, Organization, Shipment, Address, Region
- Later evolution: FulfillmentTask, FulfillmentUnit, ReturnRequest, ReturnItem, RMA

Key predicates (relationships)

- placedBy(order, customer): who placed the order
- memberOf(customer, org): a customer belongs to an organization (or account)
- orderHasShipment(order, shipment): outbound shipment for the order
- shipsTo(shipment, address): shipment destination
- locatedIn(address, region): address is located in a Region
- ancestorOf(org, org): organizational ancestry (transitive)

Property chains (implied relationships)

- placedBy $\square$ memberOf $\Rightarrow$ placedInOrg
- If (order --placedBy- $\rightarrow$ customer) and (customer --memberOf- $\rightarrow$ org), then infer (order --placedInOrg- $\rightarrow$ org)
- orderHasShipment $\square$ shipsTo $\Rightarrow$ orderShipsTo
- If (order --orderHasShipment- $\rightarrow$ shipment) and (shipment --shipsTo- $\rightarrow$ address), infer (order --orderShipsTo- $\rightarrow$ address)
- orderShipsTo $\square$ locatedIn $\Rightarrow$ orderShipsToRegion
- If (order --orderShipsTo- $\rightarrow$ address) and (address --locatedIn- $\rightarrow$ region), infer (order --orderShipsToRegion- $\rightarrow$ region)
- placedInOrg $\square$ ancestorOf $\Rightarrow$ placedInOrg
- Makes placedInOrg resilient to org hierarchy changes (ancestorOf is transitive). This is a common “closure” trick: re-assert the same predicate via chain to absorb hierarchy.

A minimal Java-style snippet to define this TBox

```
import java.util.*;
import com.e2eq.ontology.core.OntologyRegistry;
import com.e2eq.ontology.core.OntologyRegistry.*;

Map<String, ClassDef> classes = Map.of(
    "Order", new ClassDef("Order", Set.of(), Set.of(), Set.of()),
    "Customer", new ClassDef("Customer", Set.of(), Set.of(), Set.of()),
    "Organization", new ClassDef("Organization", Set.of(), Set.of(), Set.of()),
    "Shipment", new ClassDef("Shipment", Set.of(), Set.of(), Set.of()),
    "Address", new ClassDef("Address", Set.of(), Set.of(), Set.of()),
```

```

    "Region", new ClassDef("Region", Set.of(), Set.of(), Set.of())
);

Map<String, PropertyDef> props = Map.of(
    "placedBy", new PropertyDef("placedBy", Optional.of("Order"), Optional.of("Customer"), false, Optional.empty(), false),
    "memberOf", new PropertyDef("memberOf", Optional.of("Customer"), Optional.of("Organization"), false, Optional.empty(), false),
    "orderHasShipment", new PropertyDef("orderHasShipment", Optional.of("Order"), Optional.of("Shipment"), false, Optional.empty(), false),
    "shipsTo", new PropertyDef("shipsTo", Optional.of("Shipment"), Optional.of("Address"), false, Optional.empty(), false),
    "locatedIn", new PropertyDef("locatedIn", Optional.of("Address"), Optional.of("Region"), false, Optional.empty(), false),
    "ancestorOf", new PropertyDef("ancestorOf", Optional.of("Organization"), Optional.of("Organization"), false, Optional.empty(), true), // transitive
    // implied predicates (no domain/range required, but you may add them for validation)
    "placedInOrg", new PropertyDef("placedInOrg", Optional.of("Order"), Optional.of("Organization"), false, Optional.empty(), false),
    "orderShipsTo", new PropertyDef("orderShipsTo", Optional.of("Order"), Optional.of("Address"), false, Optional.empty(), false),
    "orderShipsToRegion", new PropertyDef("orderShipsToRegion", Optional.of("Order"), Optional.of("Region"), false, Optional.empty(), false)
);

List<PropertyChainDef> chains = List.of(
    new PropertyChainDef(List.of("placedBy", "memberOf"), "placedInOrg"),
    new PropertyChainDef(List.of("orderHasShipment", "shipsTo"), "orderShipsTo"),
    new PropertyChainDef(List.of("orderShipsTo", "locatedIn"), "orderShipsToRegion"),
    new PropertyChainDef(List.of("placedInOrg", "ancestorOf"), "placedInOrg")
);

OntologyRegistry.TBox tbox = new OntologyRegistry.TBox(classes, props, chains);
OntologyRegistry registry = OntologyRegistry.inMemory(tbox);

```

Materializing edges for an Order

- Explicit facts for order O1:
- O1 placedBy C9
- C9 memberOf OrgA
- O1 orderHasShipment S17
- S17 shipsTo Addr42
- Addr42 locatedIn RegionWest
- OrgA ancestorOf OrgParent
- Inferred edges after running the reasoner for O1's snapshot:
- O1 placedInOrg OrgA

- O1 placedInOrg OrgParent (via closure with ancestorOf)
- O1 orderShipsTo Addr42
- O1 orderShipsToRegion RegionWest

How queries become simple and fast

- List Orders for Organization OrgParent (including children):
- Instead of joining Order → Customer → Org and recursing org.parent, run a single filter using materialized edges.

```
import com.mongodb.client.model.Filters;
import org.bson.conversions.Bson;
import com.e2eq.ontology.policy.ListQueryRewriter;

Bson base = Filters.eq("status", "OPEN");
Bson rewritten = rewriter.rewriteForHasEdge(base, tenantId, "placedInOrg", "OrgParent");
// Use rewritten in your Mongo find
```

- List Orders shipping to RegionWest:

```
Bson rewritten2 = rewriter.rewriteForHasEdge(Filters.empty(), tenantId,
"orderShipsToRegion", "RegionWest");
```

Why this is resilient

- If tomorrow Customer becomes AccountContact and the organization model gains Divisions and multi-parent org graphs, you only adjust predicates and chains.
- Queries that rely on placedInOrg or orderShipsToRegion remain unchanged and fast, because edges are re-materialized by OntologyMaterializer.

Evolving the model: add Fulfillment

New concepts

- FulfillmentTask: a unit of work to pick/pack/ship order lines
- FulfillmentUnit: a logical grouping (e.g., wave, tote, parcel)

New predicates

- fulfills(task, order)
- realizedBy(order, fulfillmentUnit)
- taskProduces(task, shipment)

New chains (implied)

- fulfills $\Rightarrow$ derived edge from task to order; combine with taskProduces to connect order to shipment without touching Order fields:
- fulfills $\square$ taskProduces $\Rightarrow$ orderHasShipment
- realizedBy $\square$ orderHasShipment $\Rightarrow$ fulfilledByUnit
- If (order --realizedBy- $\rightarrow$ fu) and (order --orderHasShipment- $\rightarrow$ s) $\Rightarrow$ (fu --fulfillsShipment- $\rightarrow$ s) or simply (order --fulfilledByUnit- $\rightarrow$ fu)

These chains let you introduce warehouse concepts without changing how UI filters orders by organization or ship-to region. Existing queries still operate via placedInOrg and orderShipsToRegion.

Evolving further: add Returns

New concepts

- ReturnRequest, ReturnItem, RMA

New predicates

- hasReturn(order, returnRequest)
- returnFor(returnItem, order)
- returnRma(returnRequest, rma)

New chains (implied)

- hasReturn $\Rightarrow$ openReturnOnOrg via placedInOrg:
- hasReturn $\square$ placedInOrg $\Rightarrow$ returnPlacedInOrg
- returnFor $\square$ orderShipsToRegion $\Rightarrow$ returnShipsToRegion

Example queries with new capabilities

- List Orders with open returns in OrgParent:

```
Bson r = rewriter.rewriteForHasEdge(Filters.empty(), tenantId, "returnPlacedInOrg",
"OrgParent");
```

- List Returns associated to Orders shipping to RegionWest:

```
Bson r2 = rewriter.rewriteForHasEdge(Filters.empty(), tenantId, "returnShipsToRegion",
"RegionWest");
```

Comparison with direct references (@Reference/EntityReference)

- With direct references you would encode fields like Order.customer, Order.shipments, Shipment.address, Address.region and then implement multi-hop traversals in code or \$lookup pipelines, rewriting them whenever you add Fulfillment or Returns.

- With ontology edges, you keep predicates stable and add property chains. Existing list and policy queries keep working and typically become faster due to single-hop filters on an indexed edges collection.

Operational tips for this scenario

- Ensure EdgeDao has indexes on (tenantId, p, dst) and (tenantId, src, p).
- Use OntologyMaterializer when Order, Shipment, Customer, Address, or org hierarchy changes to keep edges fresh.
- Keep provenance in edge.prov (rule, inputs) so you can recompute or retract edges when source data changes.

20.2. Integrating Ontology with Morphia, Permissions, and Multi-tenancy

This section focuses on integration and developer experience: how ontology edges flow into Morphia-based repositories and the permission rule language, while remaining fully multi-tenant and secure.

20.2.1. Big picture: where ontology fits

- Write path (materialization):
- Your domain code persists entities with minimal direct references.
- An OntologyMaterializer runs when entities change to derive and upsert edges into the edges collection (per tenant).
- Policy path (authorization and list filters):
- The permission rule language evaluates the caller's SecurityContext/RuleContext and produces logical filters.
- When a rule asks for a semantic relationship (hasEdge), we use ListQueryRewriter + EdgeDao to translate that into efficient Mongo filters over ids.
- Read path (queries):
- Morphia repos apply the base data-domain filters and the rewritten ontology constraint to queries, producing fast lists without deep joins.

20.2.2. Rule language: add hasEdge()

We introduce a policy function/operator to reference ontology edges directly from rules:

- Signature: hasEdge(predicate, dstIdOrVar)
- predicate: String name of the ontology predicate (e.g., "placedInOrg", "orderShipsToRegion").
- dstIdOrVar: Either a concrete id/refName or a variable resolved from RuleContext (e.g., principal.orgRefName, request.region).
- Semantics: The rule grants/filters entities for which an edge (tenantId, src = entity_id, p =

predicate, dst = resolvedDst) exists.

- Composition: hasEdge can be combined with existing rule clauses (and/or/not) and other filters (states, tags, ownerId, etc.).

Example rule snippets (illustrative):

- Allow viewing Orders in the caller's org (including ancestors via ontology closure):
- allow VIEW Order when hasEdge("placedInOrg", principal.orgRefName)
- Restrict list to Orders shipping to a region chosen in request:
- allow LIST Order when hasEdge("orderShipsToRegion", request.region)

Under the hood, policy evaluation uses `ListQueryRewriter.rewriteForHasEdge(...)`, which converts hasEdge into a set of source ids and merges that with the base query.

20.2.3. Passing tenantId correctly

- Always resolve tenantId from RuleContext/SecurityContext (the same source your repos use for realm/database selection).
- EdgeDao and ListQueryRewriter already accept tenantId; never cross tenant boundaries when reading edges.
- Index recommendation (per tenant):
- (tenantId, p, dst)
- (tenantId, src, p)

20.2.4. Morphia repository integration patterns

The goal is zero-friction usage in existing repos without invasive changes.

Option A: Apply ontology constraints in code paths that already construct BSON filters.

- If your repo method builds a Bson filter before calling find(), wrap it through rewriter:

```
Bson base = Filters.and(existingFilters...);
Bson rewritten = hasEdgeRequested
    ? rewriter.rewriteForHasEdge(base, tenantId, predicate, dst)
    : base;
var cursor = datastore.getDatabase().getCollection(coll).find(rewritten);
```

Option B: Apply ontology constraints to Morphia Filter/Query via ids.

- When the repo uses Morphia's typed query API instead of BSON, pre-compute the id set and constrain by `_id`:

```
Set<String> ids = edgeDao.srcIdsByDst(tenantId, predicate, dst);
if (ids.isEmpty()) {
```

```

    return List.of(); // short-circuit
}
query.filter(Filters.in("_id", ids));

```

Option C: Centralize in a tiny helper for developer ergonomics.

- Provide one helper in your application layer, invoked wherever policies inject additional constraints:

```

public final class OntologyFilterHelper {
    private final ListQueryRewriter rewriter;
    public OntologyFilterHelper(ListQueryRewriter r) { this.rewriter = r; }

    public Bson ensureHasEdge(Bson base, String tenantId, String predicate, String dst)
    {
        return rewriter.rewriteForHasEdge(base, tenantId, predicate, dst);
    }
}

```

20.2.5. Where to hook materialization

- On entity changes that are sources or intermediates for chains:
- Order (placedBy, orderHasShipment), Customer (memberOf), Shipment (shipsTo), Address (locatedIn), Organization (ancestorOf/parent), and any Fulfillment/Returns entities.
- Recommended patterns:
- On-save/on-update hooks in your service layer call `OntologyMaterializer.apply(...)` with the explicit edges known from the entity snapshot.
- For intermediates (e.g., `Address.region` changed), enqueue affected sources for recomputation; use provenance to locate impacted edges.
- Provide nightly/backfill jobs for recomputing edges across a tenant when ontology rules evolve.

20.2.6. Security and multi-tenant considerations

- Edge rows include `tenantId` and should be validated/filtered by tenant on every operation.
- Never trust a client-supplied predicate or destination id blindly; combine with rule evaluation and whitelist allowed predicates per domain if needed.
- For shared resources across tenants (rare), model cross-tenant permissions at the policy layer; don't reuse edges across tenants unless explicitly designed.

20.2.7. Developer workflow and DX checklist

- When writing a rule: use `hasEdge("<predicate>", <rhs>)` and rely on `RuleContext` variables for the destination when possible.
- When writing a list endpoint: read optional ontology filter hints from the policy layer; if

present, apply `ensureHasEdge(...)` before `find()`.

- When changing domain relationships: update predicates/chains and re-materialize; list/policy code stays unchanged.
- When indexing a new tenant: include the edges indexes early and validate via a smoke test query using `ListQueryRewriter`.

20.2.8. Cookbook: end-to-end example with Orders + Org

- Policy: allow LIST Order when `hasEdge("placedInOrg", principal.orgRefName)`
- Request lifecycle: 1) Security filter builds `SecurityContext` and `RuleContext` with `tenantId` and `principal`. 2) Policy evaluation returns a directive to constrain by `hasEdge("placedInOrg", orgRefName)`. 3) Repo builds base filter (`state != ARCHIVED`, etc.). 4) Repo calls `OntologyFilterHelper.ensureHasEdge(base, tenantId, "placedInOrg", orgRefName)`. 5) Mongo executes a single-hop query using materialized edges; results respect both policy and multi-tenancy.

20.2.9. Migration notes for teams using @Reference

- Keep existing references for write-side integrity and local joins where simple.
- Introduce ontology edges on hot read paths first; update policy rules to `hasEdge` and verify results.
- Gradually replace deep `$lookup` traversals with `hasEdge`-based rewrites.
- Ensure materialization hooks are deployed before removing data fields used as inputs to the ontology.

Chapter 21. Building APIs

Chapter 22. REST: Find, Get, List, Save, Update, Delete

Quantum provides consistent REST resources backed by repositories. Extend `BaseResource` to expose CRUD quickly and consistently.

22.1. Base Concepts

- `BaseResource<T, R extends Repo<T>>` provides endpoints for:
- `find`: query by criteria (filters, pagination)
- `get`: fetch by id or refName
- `list`: list all within scope with paging
- `save`: create
- `update`: modify existing
- `delete`: delete or soft-delete/archival depending on model
- `UIActionList`: derive available actions based on current model state.
- DataDomain filtering is applied across all operations to enforce multi-tenancy.

22.2. Example Resource

```
import com.e2eq.framework.rest.resources.BaseResource;
import jakarta.ws.rs.Path;

@Path("/products")
public class ProductResource extends BaseResource<Product, ProductRepo> {
}
```

22.3. Authorization Layers in REST CRUD

Quantum combines static, identity-based checks with dynamic, domain-aware policy evaluation. In practice you will often use both:

1) Hard-coded permissions via annotations

- Use standard Jakarta annotations like `@RolesAllowed` (or the framework's `@RoleAllow` if present) on resource classes or methods to declare role-based checks that must pass before executing an endpoint.
- These checks are fast and decisive. They rely on the caller's roles as established by the current `SecurityIdentity`.

Example:

```
import jakarta.annotation.security.RolesAllowed;

@RolesAllowed({"ADMIN", "CATALOG_EDITOR"})
@Path("/products")
public class ProductResource extends BaseResource<Product, ProductRepo> {
    // Only ADMIN or CATALOG_EDITOR can access all inherited CRUD endpoints
}
```

2) JWT groups and role mapping

- When using the JWT provider, the token's groups/roles claims are mapped into the Quarkus `SecurityIdentity` (see the Authentication guide).
- Groups in JWT typically become roles on `SecurityIdentity`; these roles are what `@RolesAllowed/@RoleAllow` checks evaluate.
- You can augment or transform roles using a `SecurityIdentityAugmentor` (see `RolesAugmentor` in the framework) to add derived roles based on claims or external lookups.

3) RuleContext layered authorization (dynamic policies)

- After annotation checks pass, `RuleContext` evaluates domain-aware permissions. This layer can:
- Enforce `DataDomain` scoping (tenant/org/owner)
- Allow cross-tenant reads for specific functional areas when policy permits
- Contribute query predicates and projections to repositories
- Think of `@RolesAllowed/@RoleAllow` as the coarse-grained gate, and `RuleContext` as the fine-grained, context-sensitive policy engine.

4) Quarkus SecurityIdentity and SecurityFilter

- Quarkus produces a `SecurityIdentity` for each request containing principal name and roles.
- The framework's `SecurityFilter` inspects the incoming request (e.g., JWT) and populates/augments the `SecurityIdentity` and the derived `DomainContext` used by `RuleContext` and repositories.
- `BaseResource` and underlying repos (e.g., `MorphiaRepo`) consume `SecurityIdentity/DomainContext` to apply permissions and filters consistently.

For detailed rule-base matching (URL, headers, body predicates, priorities), see the [Permissions](#) section.

22.4. Querying

- Use query parameters or a request body (depending on your API convention) to express filters.
- `RuleContext` contributes tenant-aware filters and projections automatically.
- See [Query Language](#) for the full `BIAPIQuery` syntax, including array filtering with `elemMatch` and `IN`-clause enhancements that accept resolver-provided lists.

22.5. Responses and Schemas

- Models are returned with calculated fields (e.g., `actionList`) when appropriate.
- OpenAPI annotations in your models/resources integrate with MicroProfile OpenAPI for schema docs.

22.6. Error Handling

- Validation errors (e.g., `ImportRequiredField`, `Size`) return helpful messages.
- Rule-based denials return appropriate HTTP statuses (403/404) without leaking cross-tenant metadata.

22.6.1. Query Language (ANTLR-based)

The `find`/`list` endpoints accept a filter string parsed by an ANTLR grammar (`BIAPIQuery.g4`). Use the filter query parameter to express predicates; combine them with logical operators and grouping. Sorting and projection are separate query parameters.

- Operators:
- Equals: `'='`
- Not equals: `'!='`
- Less than/Greater than: `'<' / '>'`
- Less-than-or-equal/Greater-than-or-equal: `'<=' / '>='`
- Exists (field present): `'~'` (no value)
- In list: `'^'` followed by `[v1,v2,...]`
- Boolean literals: `true/false`
- Null literal: `null`
- Logical:
- AND: `'&&'`
- OR: `'||'`
- NOT: `'!'` (applies to a single allowed expression)
- Grouping: parentheses `'('` and `')'`
- Values by type:
- Strings: unquoted or quoted with `"..."`; quotes allow spaces and punctuation
- Whole numbers: prefix with `'#'` (e.g., `#10`)
- Decimals: prefix with `'.'` (e.g., `19.99`)
- Date: `yyyy-MM-dd` (e.g., `2025-09-10`)
- DateTime (ISO-8601): `2025-09-10T12:30:00Z` (timezone supported)
- ObjectId (Mongo 24-hex): `5f1e9b9c8a0b0c0d1e2f3a4b`

- Reference by ObjectId: @@5f1e9b9c8a0b0c0d1e2f3a4b
- Variables:
 \${ownerId|principalId|resourceId|action|functionalDomain|pTenantId|pAccountId|rTenantId|rAccountId|realm|area}

22.7. Simple filters (equals)

```
# string equality
name:"Acme Widget"
# whole number
quantity:#10
# decimal number
price:##19.99
# date and datetime
shipDate:2025-09-12
updatedAt:2025-09-12T10:15:00Z
# boolean
active:true
# null checks
description:null
# field exists
lastLogin:~
# object id equality
id:5f1e9b9c8a0b0c0d1e2f3a4b
# variable usage (e.g., tenant scoping)
dataDomain.tenantId:${pTenantId}
```

22.8. Advanced filters: grouping and AND/OR/NOT

```
# Products that are active and (name contains widget OR gizmo), excluding discontinued
active:true && (name:*widget* || name:*gizmo*) && status:! "DISCONTINUED"

# Shipments updated after a date AND (destination NY OR CA)
updatedAt:>=2025-09-01 && (destination:"NY" || destination:"CA")

# NOT example: items where category is not null and not (price < 10)
category:!null && !(price:<##10)
```

Notes: - Wildcard matching uses ": **name:*widget** (prefix/suffix/contains). '?' matches a single character. - Use parentheses to enforce precedence; otherwise AND/OR follow standard left-to-right with explicit operators.

22.9. IN lists

```
status:^[ "OPEN", "CLOSED", "ON_HOLD" ]
```

```
ownerId:^[ "u1","u2","u3"]
referenceId:^[ @5f1e9b9c8a0b0c0d1e2f3a4b, @@6a7b8c9d0e1f2a3b4c5d6e7f]
```

22.10. Sorting

Provide a sort query parameter (comma-separated fields): - '-' prefix = descending, '+' or no prefix = ascending.

Examples:

```
# single field descending
?sort=-createdAt

# multiple fields: createdAt desc, refName asc
?sort=-createdAt,refName
```

22.11. Projections

Limit returned fields with the projection parameter (comma-separated): - '+' prefix = include, '-' prefix = exclude.

Examples:

```
# include only id and refName, exclude heavy fields
?projection=+id,+refName,-auditInfo,-persistentEvents
```

22.12. End-to-end examples

- GET `/products/list?skip=0&limit=50&filter=active:true&&name:*widget*&sort=-updatedAt&projection=+id,+name,-auditInfo`
- GET `/shipments/list?filter=(destination:"NY" | | destination:"CA")&&updatedAt:>=2025-09-01&sort=origin`

These features integrate with RuleContext and DataDomain: your filter runs within the tenant/org scope derived from the security context; RuleContext may add further predicates or projections automatically.

Chapter 23. CSV Export and Import

These endpoints are inherited by every resource that extends `BaseResource`. They are mounted under the resource's base path. For example, `PolicyResource` at `/security/permission/policies` exposes:

- `GET /security/permission/policies/csv`
- `POST /security/permission/policies/csv`
- `POST /security/permission/policies/csv/session`
- `POST /security/permission/policies/csv/session/{sessionId}/commit`
- `DELETE /security/permission/policies/csv/session/{sessionId}`
- `GET /security/permission/policies/csv/session/{sessionId}/rows`

Authorization and scoping:

- All CSV endpoints are protected by the same `@RolesAllowed("user", "admin")` checks as other CRUD operations.
- `RuleContext` filters and `DataDomain` scoping apply the same way as `list/find`; exports stream only what the caller may see, and imports are saved under the same permissions.
- In multi-realm deployments, include your `X-Realm` header as you do for CRUD; underlying repos resolve realm and domain context consistently.

23.1. Export: GET /csv

Produces a streamed CSV download of the current resource collection.

Query parameters and behavior:

fieldSeparator (default ",")

Single character used to separate fields. Typical values: `,`, `;`, `\t`.

requestedColumns (default refName)

Comma-separated list of model field names to include, in output order. If omitted, `BaseResource` defaults to `refName`. Nested list extraction is supported with the `[0]` notation on a single nested property across all requested columns (e.g., `addresses[0].city`, `addresses[0].zip`). Indices other than `[0]` are rejected. If the nested list has multiple items, multiple rows are emitted per record (one per list element), preserving other column values.

quotingStrategy (default QUOTE_WHERE_ESSENTIAL)

- `QUOTE_WHERE_ESSENTIAL`: quote only when needed (when a value contains the separator or `quoteChar`).
- `QUOTE_ALL_COLUMNS`: quote every column in every row.

quoteChar (default ")

The character used to surround quoted values.

decimalSeparator (default .)

Reserved for decimal formatting. Note: current implementation ignores this value; decimals are rendered using the locale-independent dot.

charsetEncoding (default UTF-8-without-BOM)

One of: `US-ASCII`, `UTF-8-without-BOM`, `UTF-8-with-BOM`, `UTF-16-with-BOM`, `UTF-16BE`, `UTF-16LE`. “with-BOM” values write a Byte Order Mark at the beginning of the file (UTF-8: `EF BB BF`; UTF-16: `FE FF`).

filter (optional)

ANTLR DSL filter applied server-side before streaming (see Query Language section). Reduces rows and can improve performance.

filename (default downloaded.csv)

Suggested download filename returned via Content-Disposition header.

offset (default 0)

Zero-based index of the first record to stream.

length (default 1000, use -1 for all)

Maximum number of records to stream from offset. Use `-1` to stream all (be mindful of client memory/time).

prependHeaderRow (optional boolean, default false)

When true, the first row contains column headers. Requires `requestedColumns` to be set (the default `refName` satisfies this requirement).

preferredColumnNames (optional list)

Overrides header names positionally when `prependHeaderRow=true`. The list length must be $\leq$ `requestedColumns`; an empty string entry means “use default field name” for that column.

Response:

- 200 OK with Content-Type: text/csv and Content-Disposition: attachment; filename="..."
- On validation/processing errors, the response status is 400/500 and the body contains a single text line describing the problem (e.g., “Incorrect information supplied: ...”). Unrecognized query parameters are rejected with 400.

Examples:

- Export selected fields with header, custom filename and filter

```
curl -H "Authorization: Bearer $JWT" \
  -H "X-Realm: system-com" \

"https://host/api/products/csv?requestedColumns=id,refName,price&prependHeaderRow=true
&filename=products.csv&filter=active:true&sort=+refName"
```

- Export nested list's first element across columns

```
# emits one row per address entry when more than one is present
curl -H "Authorization: Bearer $JWT" \

"https://host/api/customers/csv?requestedColumns=refName,addresses[0].city,addresses[0].zip&prependHeaderRow=true"
```

23.2. Import: POST /csv (multipart)

Consumes a CSV file (multipart/form-data) and imports records in batches. The form field name for the file is file.

Query parameters and behavior:

fieldSeparator (default ",)

Single character expected between fields.

quotingStrategy (default QUOTE_WHERE_ESSENTIAL)

Same values as export; controls how embedded quotes are recognized.

quoteChar (default ")

The expected quote character in the file.

skipHeaderRow (default true)

When true, the first row is treated as a header and skipped. Mapping is positional, not by header names.

charsetEncoding (default UTF-8-without-BOM)

The file encoding. “with-BOM” variants allow consuming a BOM at the start.

requestedColumns (required)

Comma-separated list of model field names in the same order as the CSV columns. This positional mapping drives parsing and validation. Nested list syntax `[0]` is allowed with the same constraints as export.

Behavior:

- Each row is parsed into a model instance using type-aware processors (ints, longs, decimals, enums, etc.).
- Bean Validation is applied; rows with violations are collected as errors and not saved; valid rows are batched and saved.
- For each saved batch, insert vs update is determined by refName presence in the repository.
- Response entity includes counts (importedCount, failedCount) and per-row results when available.
- Response headers:

- X-Import-Success-Count: number of rows successfully imported.
- X-Import-Failed-Count: number of rows that failed validation or DB write.
- X-Import-Message: summary message.

Example (direct import):

```
curl -X POST \
  -H "Authorization: Bearer $JWT" \
  -H "X-Realm: system-com" \
  -F "file=@policies.csv" \

"https://host/api/security/permission/policies/csv?requestedColumns=refName,principalId,description&skipHeaderRow=true&fieldSeparator=,&quoteChar=\"&quotingStrategy=QUOTE_WHERE_ESSENTIAL&charsetEncoding=UTF-8-without-BOM"
```

23.3. Import with preview sessions

Use a two-step flow to analyze first, then commit only valid rows.

- POST /csv/session (multipart): analyzes the file and creates a session
 - Same parameters as POST /csv (fieldSeparator, quotingStrategy, quoteChar, skipHeaderRow, charsetEncoding, requestedColumns).
 - Returns a preview ImportResult including sessionId, totals (totalRows, validRows, errorRows), and row-level findings. No data is saved yet.
- POST /csv/session/{sessionId}/commit: imports only error-free rows from the analyzed session
 - Returns CommitResult with inserted/updated counts.
 - DELETE /csv/session/{sessionId}: cancels and discards session state (idempotent; always returns 204).
- GET /csv/session/{sessionId}/rows: page through analyzed rows
 - Query params:
 - skip (default 0), limit (default 50)
 - onlyErrors (default false): when true, returns only rows with errors
 - intent (optional): filter rows by intended action: INSERT, UPDATE, or SKIP

Notes and constraints:

- requestedColumns must reference actual model fields. Unknown fields or multiple different nested properties are rejected (only one nested property across requestedColumns is allowed when using [0]).
- Unrecognized query parameters are rejected with HTTP 400 to prevent silent misconfiguration.
- Very large exports should prefer streaming with sensible length settings or server-side filters to reduce memory and time.

- Imports run under the same security rules as POST / (save). Ensure the caller has permission to create/update the target entities in the chosen realm.

23.4. Enhanced Import with ImportProfile

ImportProfile provides reusable, configurable transformation and mapping rules for CSV imports. Profiles are persistent entities that can be created via REST and referenced during imports.

23.4.1. Key Features

- **Value Mapping:** Transform CSV values before type conversion (e.g., "Y" → **true**, "Active" → **ACTIVE**)
- **Lookup Resolution:** Resolve foreign references by looking up values in other collections
- **String Transformations:** Trim, case conversion, regex replacement
- **Intent Control:** Specify INSERT vs UPDATE per row via an intent column
- **Header Modifiers:** Mark fields as required (*), optional (?), calculated (~), or key (#)

23.4.2. Using a Profile

Create a profile first, then reference it during import:

```
# Create profile
curl -X POST \
  -H "Authorization: Bearer $JWT" \
  -H "Content-Type: application/json" \
  "https://host/api/integration/import-profiles" \
  -d @product-import-profile.json

# Import using profile
curl -X POST \
  -H "Authorization: Bearer $JWT" \
  -F "file=@products.csv" \
  "https://host/api/products/csv/session/profile?profileRefName=product-import-v1"
```

23.4.3. Profile with Session Flow

- POST /{entity}/csv/session/profile (multipart): analyzes the file using the specified profile
 - Query param: **profileRefName** - reference to an existing ImportProfile
 - Returns preview ImportResult including sessionId, totals, and row-level findings
- POST /{entity}/csv/session/{sessionId}/commit: imports valid rows (same as basic flow)

23.4.4. Example ImportProfile (JSON)

```
{
```

```

"refName": "product-import-v1",
"displayName": "Product Import Profile",
"targetType": "com.example.Product",
"columnMappings": [
  {
    "sourceColumn": "Status",
    "targetField": "status",
    "valueMappings": {
      "A": "ACTIVE",
      "I": "INACTIVE",
      "D": "DISCONTINUED"
    },
    "unmappedValueBehavior": "FAIL"
  },
  {
    "sourceColumn": "Category",
    "targetField": "categoryRefName",
    "lookup": {
      "lookupCollection": "Category",
      "lookupMatchField": "displayName",
      "lookupReturnField": "refName",
      "onNotFound": "FAIL"
    }
  }
],
"intentColumn": "_action",
"defaultIntent": "UPSERT"
}

```

23.4.5. Intent Column

When `intentColumn` is configured, each row can specify its own intent:

Intent	Behavior
INSERT	Insert new record. Fails if refName already exists.
UPDATE	Update existing record. Fails if refName does not exist.
UPSERT	Auto-detect: INSERT if new, UPDATE if exists.
SKIP	Skip this row entirely.



`MERGE` and `DELETE` are intentionally not supported for safety reasons.

For complete ImportProfile documentation, see [CSV Import Enhancements](#).

23.5. RowValueResolver: Custom Per-Row Logic

The `RowValueResolver` SPI allows arbitrary code execution for each row during import. Unlike static lookups, `RowValueResolvers` have access to all columns in the row and can invoke external

services.

23.5.1. Key Features

- **Arc CDI Beans:** Full dependency injection support
- **Multi-Collection Updates:** Update multiple collections from a single CSV row
- **Full Row Access:** Read any column value, not just the mapped column
- **Context Awareness:** Access to realm, session, row number, and profile

23.5.2. Example Implementation

```
@ApplicationScoped
public class GeocodeResolver implements RowValueResolver {

    @Inject
    GeocodingService geocodingService;

    @Override
    public String getName() {
        return "geocodeResolver";
    }

    @Override
    public ResolveResult resolve(String inputValue, Map<String, Object> rowData,
    ImportContext context) {
        String street = (String) rowData.get("street");
        String city = (String) rowData.get("city");
        String state = (String) rowData.get("state");

        GeocodingResult result = geocodingService.geocode(
            String.format("%s, %s, %s", street, city, state));

        if (result != null && result.isValid()) {
            return ResolveResult.success(new GeoPoint(result.getLatitude(), result
            .getLongitude()));
        }
        return ResolveResult.nullValue();
    }
}
```

23.5.3. Configuration in ImportProfile

```
{
  "columnMappings": [
    {
      "sourceColumn": "street",
      "targetField": "location",
```

```

    "rowValueResolverName": "geocodeResolver"
  }
]
}

```

23.5.4. Result Types

Method	Description
<code>ResolveResult.success(value)</code>	Set the field to the resolved value
<code>ResolveResult.passthrough(original)</code>	Keep the original value unchanged
<code>ResolveResult.nullValue()</code>	Set the field to null
<code>ResolveResult.skip()</code>	Skip this row (not an error)
<code>ResolveResult.error(message)</code>	Mark the row as an error

For complete RowValueResolver documentation, see [CSV Import Enhancements](#).

23.6. Dynamic Attribute Import

For entities with dynamic attributes (schema-less, nested key-value structures), Quantum supports multiple import strategies.

23.6.1. Import Strategies

Strategy	Description
DOT_NOTATION	Use <code>dyn.setName.attrName[:type]</code> column headers. Best for spreadsheet-friendly imports.
JSON_COLUMN	Single JSON column containing all attributes. Best for programmatic imports.
VERTICAL	Multiple rows per entity with one attribute per row. Best for attribute-centric workflows.
HYBRID	Combines DOT_NOTATION and JSON_COLUMN in the same import.

23.6.2. DOT_NOTATION Example

```

name,sku,dyn.logistics.weight:Double,dyn.logistics.hazmat:Boolean,dyn.compliance.ce_mark:Boolean
"Widget Pro","WGT-001",2.5,false,true
"Gadget X","GDG-002",1.2,false,true

```

23.6.3. JSON_COLUMN Example

```
name,sku,dynamicAttributes
"Widget Pro","WGT-
001","{"logistics":{"weight":2.5,"hazmat":false},"compliance":{"ce_mark":true}}
```

23.6.4. Configuration in ImportProfile

```
{
  "refName": "product-import-dynamic",
  "dynamicAttributeStrategy": "DOT_NOTATION",
  "dynamicAttributePrefix": "dyn.",
  "enableDynamicAttributeDiscovery": true,
  "dynamicAttributeMergeStrategy": "MERGE"
}
```

23.6.5. Merge Strategies

Strategy	Behavior
REPLACE	Remove all existing attributes, replace with imported
MERGE	Keep existing attributes, update matching, add new
APPEND	Keep existing attributes, only add new (no updates)

For complete dynamic attribute import documentation, see [CSV Dynamic Attributes Design](#).

Authentication and Authorization

Quantum integrates with Quarkus security while providing a pluggable approach to authentication. The repository includes a JWT provider module to get started quickly and an extension surface to replace or complement it.

Chapter 24. JWT Provider

- Module: quantum-jwt-provider
- Purpose: Validate JWTs on incoming requests, populate the security principal, and surface tenant/org/user claims that feed DomainContext.
- Configuration: Standard Quarkus/MicroProfile JWT properties plus custom claim mappings as needed for DataDomain.

Chapter 25. Pluggable Authentication

You can introduce alternative authentication mechanisms (e.g., API keys, SAML/OIDC front-channel tokens exchanged for back-end JWTs, HMAC signatures) by providing CDI beans that integrate with the security layer and emit the same normalized context consumed by `DomainContext`/`RuleContext`.

Typical steps:

1. Implement a request filter or identity provider that validates the token/credential.
2. Map identity and tenant claims into a principal model (`tenantId`, `orgRefName`, `userId`, `roles`).
3. Ensure `BaseResource` (and other entry points) can derive `DomainContext` from that principal.

Chapter 26. Creating an Auth Plugin (using the Custom JWT provider as a reference)

An auth plugin is typically a CDI bean that:

- Extends `BaseAuthProvider` to inherit user-management helpers and persistence utilities.
- Implements `AuthProvider` to integrate with request-time authentication flows.
- Implements `UserManagement` to expose CRUD-style operations for users, passwords, and roles.

A concrete provider should:

- Be annotated as a CDI bean (e.g., `@ApplicationScoped`).
- Provide a stable `getName()` identifier (e.g., "custom", "oidc", "apikey").
- Use config properties for secrets, issuers, token durations, and any external identity provider details.
- Build a Quarkus `SecurityIdentity` with the authenticated principal and roles.

Chapter 27. AuthProvider interface (what a provider must implement)

Core methods:

- `SecurityIdentity validateAccessToken(String token)`
 - Parse and validate the incoming credential (JWT, API key, signature).
 - Return a `SecurityIdentity` with principal name and roles; throw a security exception for invalid tokens.
- `String getName()`
 - A short identifier for the provider; persisted alongside credentials and used in logs/metrics.
- `LoginResponse login(String userId, String password)`
 - Credential-based login. Return a structured response:
 - `positiveResponse`: includes `SecurityIdentity`, `roles`, `accessToken`, `refreshToken`, `expirationTime`, and `realm/mongodbUrl` if applicable.
 - `negativeResponse`: includes error codes/reason/message for clients to act on (e.g., password change required).
- `LoginResponse refreshTokens(String refreshToken)`
 - Validate the refresh token, mint a new access token (and optionally a new refresh token), and return a positive response.

Notes:

- Login flow should check `force-change-password` or equivalent flags and return a negative response when user interaction is required before issuing tokens.
- `validateAccessToken` should only accept valid, non-expired tokens and construct `SecurityIdentity` consistently with role mappings used across the platform.

Chapter 28. UserManagement interface (operations your plugin must support)

Typical responsibilities include:

- User lifecycle
 - `String createUser(String userId, String password, Set<String> roles, DomainContext domainContext, [optional] DataDomain)`
 - `void changePassword(String userId, String oldPassword, String newPassword, Boolean forceChangePassword)`
 - `boolean removeUserWithUserId(String userId)`
 - `boolean removeUserWithSubject(String subject)`
- Role management
 - `void assignRolesForUserId(String userId, Set<String> roles)`
 - `void assignRolesForSubject(String subject, Set<String> roles)`
 - `void removeRolesForUserId(String userId, Set<String> roles)`
 - `void removeRolesForSubject(String subject, Set<String> roles)`
 - `Set<String> getUserRolesForUserId(String userId)`
 - `Set<String> getUserRolesForSubject(String subject)`
- Lookups and existence checks
 - `Optional<String> getSubjectForUserId(String userId)`
 - `Optional<String> getUserIdForSubject(String subject)`
 - `boolean userIdExists(String userId)`
 - `boolean subjectExists(String subject)`

Return values and exceptions:

- Throw `SecurityException` or domain-specific exceptions for invalid states (duplicate users, bad password, unsupported hashing).
- Return `Optional` for lookups that may not find a result.
- For removals, return `boolean` to communicate whether a record was deleted.

Chapter 29. Leveraging BaseAuthProvider in your plugin

When you extend BaseAuthProvider, you inherit ready-to-use capabilities that reduce boilerplate:

- Impersonation controls
 - enableImpersonationWithUserId / enableImpersonationWithSubject
 - disableImpersonationWithUserId / disableImpersonationWithSubject
 - These set or clear an impersonation filter script and realm regex that downstream services can honor to act on behalf of another identity under controlled scope.
- Realm override helpers
 - enableRealmOverrideWithUserId / enableRealmOverrideWithSubject
 - disableRealmOverrideWithUserId / disableRealmOverrideWithSubject
 - Useful for multi-realm/tenant scenarios, enabling scoped cross-realm behavior.
- Persistence utilities
 - Built-in use of the credential repository to save, update, and delete credentials.
 - Consistent validation of inputs (non-null checks, non-blank checks).
 - Hashing algorithm guardrails to ensure only supported algorithms are used.

Best practices when deriving:

- Record the auth provider name in stored credentials as informational provenance. It tells callers which provider resolved the credential or token; it is not an authentication routing or authorization binding.
- Reuse the role merge/remove patterns to avoid accidental role loss.
- Prefer emitting precise exceptions (e.g., NotFound for missing users, SecurityException for access violations).

Chapter 30. Implementing your own provider

Checklist:

- Class design
 - `@ApplicationScoped` bean
 - extends `BaseAuthProvider`
 - implements `AuthProvider` and `UserManagement`
 - return a stable `getName()`
- Configuration
 - Externalize secrets (signing keys), issuers, token durations, and realm details via `MicroProfile Config`.
- `SecurityIdentity`
 - Consistently build identities with principal and roles; include useful attributes for auditing/telemetry.
- Tokens/credentials
 - For JWT-like tokens, implement robust parsing, signature verification, expiration checks, and claim validation.
 - For non-JWT credentials (API keys, HMAC), ensure replay protection and scope binding.
- Responses and errors
 - Use structured `LoginResponse` for both success and error paths.
 - Prefer idempotent user/role operations; validate inputs and surface actionable messages.

Chapter 31. CredentialUserIdPassword model and DomainContext

This section explains how user credentials are represented, how those records tie to tenancy and realms, and how the server chooses the database (“realm”) for REST calls.

What the credential model represents

userId

The human-friendly login handle that users type. Must be unique within the applicable tenancy/realm scope.

subject

A stable, system-generated identifier for the principal. Tokens and internal references favor subject over userId because subjects do not change.

description, emailOfResponsibleParty

Optional metadata to describe the credential and provide an owner contact.

domainContext

The tenancy and organization placement of the principal. It contains:

- **tenantId**: Logical tenant partition.
- **orgRefName**: Organization/business unit within the tenant.
- **accountId**: Account or billing identifier.
- **defaultRealm**: The default database/realm used for this identity’s operations.
- **dataSegment**: Optional partitioning segment for advanced sharding or data slicing.

roles

The set of authorities granted (e.g., USER, ADMIN). These become groups/roles on the SecurityIdentity.

issuer

An identifier for who issued the credential or tokens (useful for auditing and multi-provider setups).

passwordHash, hashingAlgorithm

The stored password hash and declared algorithm. Not exposed over REST. Providers verify passwords against this.

forceChangePassword

Flag that forces a password reset on next login; the login flow returns a structured negative response instead of tokens.

lastUpdate

Timestamp for auditing and token invalidation strategies.

area2RealmOverrides

Optional map to route specific functional areas to different realms than the default (e.g., “Reporting” → analytics-realm).

realmRegEx

Optional regex to limit or override which realms this identity may act in; also used by impersonation/override flows.

impersonateFilterScript

Optional script indicating the filter/scope applied during impersonation so actions are constrained.

authProviderName

Informational provenance naming the provider that resolved this credential or token (e.g., “custom”, “oidc”). Provider selection comes from an explicit request override or the ordered `auth.provider` configuration, never from this field.

How DomainContext selects the realm for REST calls

- For each authenticated request, the server derives or retrieves a DomainContext associated with the principal.
- The DomainContext.defaultRealm indicates which backing MongoDB database (“realm”) should be used by repositories for that request.
- If realm override features are enabled (e.g., through provider helpers or per-credential overrides), the system may route certain functional areas to alternate realms using area2RealmOverrides or validated by realmRegEx.
- The remainder of DomainContext (tenantId, orgRefName, accountId, dataSegment) is applied as scope constraints through permission rules and repository filters so reads and writes are automatically restricted to the correct tenant/org segment.

Chapter 32. Quarkus OIDC out-of-the-box and integrating with common IdPs

Quarkus ships with first-class OpenID Connect (OIDC) support, enabling both service-to-service and browser-based logins.

What the Quarkus OIDC extension provides

- OIDC client and server-side adapters:
 - Authorization Code flow with PKCE for browser sign-in.
 - Bearer token authentication for APIs (validating access tokens on incoming requests).
 - Token propagation for downstream calls (forwarding or exchanging tokens).
- Token verification and claim mapping:
 - Validates issuer, audience, signature, expiration, and scopes.
 - Maps standard claims (sub, email, groups/roles) into the security identity.
- Multi-tenancy and configuration:
 - Supports multiple OIDC tenants via configuration, each with its own issuer, client id/secret, and flows.
- Logout and session support:
 - Front-channel and back-channel logout hooks depending on provider capabilities.

Integrating with common providers

- Works with providers like Keycloak, Auth0, Okta, Azure AD, Cognito, and enterprise IdPs exposing OIDC.
- Configure the issuer URL and client credentials. Quarkus discovers endpoints via the provider's .well-known/openid-configuration.
- For roles/permissions, map provider groups/roles claims to your platform roles in the identity.

OIDC vs OAuth vs OpenID (terminology and evolution)

OAuth 2.0

Authorization framework for delegated access (scopes), not authentication. Defines flows to obtain access tokens for APIs.

OpenID (OpenID 1.x/2.0)

Older federated identity protocol that preceded OIDC. It has been superseded by OpenID Connect.

OpenID Connect (OIDC)

An identity layer on top of OAuth 2.0. Adds standardized authentication, user info endpoints, ID tokens (JWT) with subject and profile claims, and discovery metadata. In practice, OIDC is the modern standard for SSO and user authentication; OAuth remains the authorization substrate

underneath.

Summary

- OpenID → historical, replaced by OIDC.
- OAuth 2.0 → authorization framework.
- OIDC → authentication (identity) layer built on OAuth 2.0.

OIDC and SAML in relation to SSO

SAML (Security Assertion Markup Language)

XML-based federation protocol widely used in enterprises for browser SSO; uses signed XML assertions transported through browser redirects/posts.

OIDC

JSON/REST-oriented, uses JWTs, and is well-suited for modern SPAs and APIs.

Relationship: * Both enable SSO and federation across identity providers and service providers. * Many enterprise IdPs support both; OIDC is generally simpler for APIs and modern web stacks, while SAML is entrenched in legacy/enterprise SSO.

Bridging: * Gateways or identity brokers can translate SAML assertions to OIDC tokens and vice versa, allowing gradual migration.

Common customer IdP models and OIDC integration patterns

- Centralized IdP (single-tenant)
 - One organization-wide IdP issues tokens for all users.
 - Configure a single OIDC tenant in Quarkus; map groups/roles to application roles.
- Multi-tenant SaaS with per-tenant IdP (BYOID)
 - Each customer brings their own IdP.
 - Configure Quarkus OIDC multitenancy with per-tenant issuer discovery and client credentials.
 - Tenant selection can be based on domain, request header, or path; the selected OIDC tenant performs login and token validation.
- Brokered identity
 - Use a broker that federates to multiple upstream IdPs (OIDC, SAML).
 - Quarkus integrates with the broker as a single OIDC client; the broker handles IdP routing and protocol translation.
- Hybrid API and web flows
 - Browser apps use Authorization Code flow with sessions; APIs use bearer token authentication.
 - The OIDC extension can handle both in the same application when properly configured.

Chapter 33. Authorization via RuleContext

Authentication establishes identity; RuleContext enforces what the identity can do. For each action (CREATE, UPDATE, VIEW, DELETE, ARCHIVE), RuleContext can:

- Allow or deny the action
- Contribute additional filters (e.g., org scoping, functional-area specific sharing)
- Adjust UIActionList to reflect permitted next steps

This division of responsibilities keeps providers focused on identity while policies remain centralized in RuleContext.

33.1. Using Ontology Edges in List Endpoints (optional)

When ontology is enabled and edges are materialized, list endpoints can avoid deep joins or multi-collection traversals by rewriting queries based on semantic relationships.

Pattern A: Wrap BSON with ListQueryRewriter

```
Bson base = Filters.and(existingFilters...);
Bson rewritten = rewriter.rewriteForHasEdge(base, tenantId, "placedInOrg", orgRefName);
collection.find(rewritten).iterator();
```

Pattern B: Constrain Morphia query by IDs

```
Set<String> ids = edgeDao.srcIdsByDst(tenantId, "orderShipsToRegion", region);
if (!ids.isEmpty()) {
    query.filter(dev.morphia.query.filters.Filters.in("_id", ids));
}
```

Notes

- Always scope by tenantId from DomainContext/RuleContext.
- Index edges on (tenantId, p, dst) and (tenantId, src, p) to keep queries fast.
- See [Integrating Ontology](#) for more integration options.

33.1.1. OntologyAwareResource

`OntologyAwareResource` is an abstract base class in the framework (package `com.e2eq.ontology.policy.rest`, quantum-ontology-policy-bridge) that extends `BaseResource` to add ontology-aware list and detail endpoints. Resources extending it expose standard CRUD plus:

- **Ontology list endpoint:** `GET /{basePath}/ontology` — lists entities with optional ontology-based filtering (e.g. filter by related customer, project, or timesheet IDs via the ontology graph)

- **Ontology context endpoint:** `GET /{basePath}/id/{id}/ontology` — returns ontology edges for a single entity
- **Optional aggregation and expand:** when `feature.ontologyList.aggregation.enabled=true`, supports the `expand` query param to return combined attributes spanning collections (via MongoDB `$lookup`)

Extending OntologyAwareResource

1. Extend `OntologyAwareResource<T, R>` instead of `BaseResource<T, R>`.
2. Add `GET @Path("ontology")` — call `getOntologyList(headers, skip, limit, filter, sort, projection, expand?, constraints...)`.
3. Add `GET @Path("id/{id}/ontology")` — call `buildOntologyContextResponse(headers, id)`.
4. For ontology filters, pass `constraint(predicate, ids)` where `predicate` is the ontology property (e.g. `invoicesCustomer`) and `ids` are destination entity IDs.
5. Override `supportsExpandInOntologyList()` to return `true` if the entity has expandable references (e.g. `customerReference`, `projectReference`).

Configuration

Property	Default	Description
<code>feature.ontologyList.aggregation.enabled</code>	<code>false</code>	When <code>true</code> , resources with <code>supportsExpandInOntologyList()=true</code> can use aggregation for the <code>expand</code> query param

Dependencies

`OntologyAwareResource` uses `ListQueryRewriter`, `OntologyContextEnricherMongo` (quantum-ontology-mongo), and `MorphiaDataStoreWrapper` (quantum-morphia-repos). It is provided by quantum-ontology-policy-bridge; applications must include that module (and thus quantum-ontology-mongo transitively) when using this base class.

Chapter 34. Query Language Reference

Quantum uses an ANTLR-based query language (BIAPIQuery.g4) for filtering, searching, and constraining data across all REST endpoints. This single, consistent syntax works everywhere: list APIs, permission rules, and access resolvers.

34.1. Basic Syntax

34.1.1. Operators

Operator	Symbol	Example
Equals	:	name:"Widget"
Not equals	:!	status:! "DELETED"
Less than	:<	price:<##100
Greater than	:>	quantity:>#50
Less than or equal	:≤	createdAt:≤ 2024-12-31
Greater than or equal	:>=	updatedAt:>=2024-01-01T00:00:00Z
Field exists	:~	description:~
In list	:^	status:^["ACTIVE", "PENDING"]
Not in list	:!^	status:!^["DELETED", "ARCHIVED"]

34.1.2. Value Types

Type	Prefix	Example
String	none or "..."	name:widget or name:"Super Widget"
Number (integer)	#	quantity:#42
Number (decimal)	##	price:##19.99
Date	none	shipDate:2024-12-25
DateTime	none	createdAt:2024-12-25T10:30:00Z
Boolean	none	active:true
Null	none	description:null
ObjectId	none	id:507f1f77bcf86cd799439011
Reference	@@	parentId:@@507f1f77bcf86cd799439011

Type	Prefix	Example
Variable	<code>\${...}</code>	<code>ownerId:\${principalId}, tier:^\${facet.allowedTiers}, zone:\${coord.salesZone}</code>



Dotted Variable Identifiers & Single-Variable IN: The query grammar admits dotted variable paths (`${facet.<key>}`, `${coord.<key>}`, `${principal.<attr>}`, `${dd.<attr>}`). When paired with the list membership operator (`:^`), a single variable resolving to a Java `Collection` or array (e.g. `allowedTiers = ["TIER_1", "TIER_2"]`) can be passed without brackets (`tier:^${facet.allowedTiers}`), and the query compiler will automatically unpack the elements into a native `$in` evaluation.

34.1.3. Logical Operators

Operator	Symbol	Example
AND	<code>&&</code>	<code>active:true && price:>##10</code>
OR	<code> </code>	<code>status:"ACTIVE" status:"PENDING"</code>
NOT	<code>!!</code>	<code>!!(price:<##5)</code>
Grouping	<code>()</code>	<code>(active:true featured:true) && price:>##0</code>

34.2. Common Patterns

34.2.1. String Matching

```
# Exact match
name:"Super Widget"

# Wildcard matching
name:*widget*      # contains "widget"
name:widget*       # starts with "widget"
name:*widget       # ends with "widget"
name:w?dget        # single character wildcard

# Case sensitivity (depends on database collation)
name:"WIDGET"       # may or may not match "widget"
```

34.2.2. Text Search

```
# Full-text search (requires MongoDB text index)
text("priority escalation")

# Combine text search with other filters
text("priority escalation") && status:"OPEN"
```



```
# Valid: top-level text AND a grouped OR of non-text predicates
text("priority") && (status:"OPEN" || status:"PENDING")
```

Notes:

- `text(...)` maps to MongoDB `$text` queries and is case-insensitive by default.
- MongoDB allows only one `text(...)` clause per query.
- Text search must be backed by a text index on the target collection. If the collection has no text index, MongoDB returns an error (Quantum does not pre-check index existence).
- The lexer keyword `text` is also accepted as a normal field name or unquoted value, so queries such as `type:text` and `text:active` remain valid.
- In-memory evaluation (policy/tests) approximates `$text` with case-insensitive **whole-word** matching (tokenized contains-any). It does not implement stemming or stop words. Example: `text("cat")` matches `"the cat sat"` but not `"category"`.



MongoDB requires `$text` to be a top-level query operator. The following usages are **not allowed** and will be rejected at parse time:

Invalid Usage	Example	Reason
Inside NOT	<code>!!text("foo")</code>	MongoDB does not support <code>\$text</code> inside <code>\$not</code> or <code>\$nor</code>
Inside OR	<code>text("foo") status:"OPEN"</code>	MongoDB does not allow <code>\$text</code> inside <code>\$or</code>
Inside elemMatch	<code>tags:{text("foo")}</code>	MongoDB does not support <code>\$text</code> inside <code>\$elemMatch</code>
Multiple text clauses	<code>text("foo") && text("bar")</code>	Only one <code>\$text</code> operator is allowed per query

Valid usage: `text(...)` combined with `&&` (AND) at the top level is supported because MongoDB allows `$text` alongside other top-level conditions.

34.2.3. Numeric Ranges

```
# Price between 10 and 100
price:>=#10 && price:<=#100
```

```
# Quantity greater than 0
quantity:>#0
```

```
# Exact count
itemCount:#5
```

34.2.4. Date and Time Queries

```
# Orders from today
createdAt:>=2024-12-25

# Orders from last week
createdAt:>=2024-12-18 && createdAt:<2024-12-25

# Specific timestamp
updatedAt:2024-12-25T14:30:00Z

# Orders modified this year
updatedAt:>=2024-01-01T00:00:00Z
```

34.2.5. List Membership

```
# Status in specific values (IN)
status:^[ "ACTIVE", "PENDING", "PROCESSING" ]

# Exclude statuses (NOT IN)
status:!^[ "DELETED", "ARCHIVED" ]

# User IDs from a list (IN)
ownerId:^[ "user1", "user2", "user3" ]

# Exclude specific users (NOT IN)
ownerId:!^[ "user1", "user2" ]

# ObjectId list (IN)
categoryId:^[ @507f1f77bcf86cd799439011, @507f1f77bcf86cd799439012 ]

# ObjectId list (NOT IN)
categoryId:!^[ @507f1f77bcf86cd799439011, @507f1f77bcf86cd799439012 ]

# Mixed types (coerced automatically)
priority:^[ #1, #2, #3 ]

# Using variables (CSV expansion supported by access resolvers)
customerId:!^[ ${accessibleCustomerIds} ]
```

Null and Existence Checks

```
# Field has any value (not null)
description:~

# Field is null
description:null
```

```
# Field is not null
description:!null

# Field exists and is not empty string
description:~ && description:!""
```

34.3. Querying Dynamic Attributes

Models that implement `DynamicAttributeSupport` can store flexible, user-defined attributes organized into named sets. These can be queried using the `elemMatch` syntax for array fields.

34.3.1. Dynamic Attribute Structure

Dynamic attributes are stored as:

```
{
  "dynamicAttributeSets": [
    {
      "name": "logistics",
      "attributes": [
        { "id": "uuid-1", "name": "weight", "value": 25.5, "type": "NUMBER" },
        { "id": "uuid-2", "name": "hazmat", "value": false, "type": "BOOLEAN" }
      ]
    },
    {
      "name": "compliance",
      "attributes": [
        { "id": "uuid-3", "name": "certNumber", "value": "CERT-123", "type": "STRING" }
      ]
    }
  ]
}
```

34.3.2. Querying by Attribute ID and Value

Use `elemMatch` syntax `{...}` to query attributes within the array:

```
# Find records where a specific attribute (by ID) has a specific value
dynamicAttributeSets.attributes:{id:21f63b90-08b4-4280-a28d-f003f9c114b3 &&
value:"Conference hall"}

# Find records with a numeric attribute value
dynamicAttributeSets.attributes:{id:94138c39-2115-4681-8c18-aa0c0596b065 && value:#99}

# Find records with a decimal attribute value
```

```
dynamicAttributeSets.attributes:{id:uuid-1 && value:##25.5}
```

34.3.3. Querying by Attribute Name and Value

```
# Find records where an attribute named "weight" has value > 10
dynamicAttributeSets.attributes:{name:"weight" && value:>##10}

# Find records with a specific attribute name and string value
dynamicAttributeSets.attributes:{name:"certNumber" && value:"CERT-123"}

# Boolean attribute query
dynamicAttributeSets.attributes:{name:"hazmat" && value:true}
```

34.3.4. Combining Multiple Dynamic Attribute Conditions

```
# Multiple attributes must match (separate elemMatch conditions)
dynamicAttributeSets.attributes:{name:"weight" && value:>##10} &&
dynamicAttributeSets.attributes:{name:"hazmat" && value:false}

# OR within a single attribute query
dynamicAttributeSets.attributes:{value:"A" || value:"B"}

# Combined with regular field queries
status:"ACTIVE" && dynamicAttributeSets.attributes:{name:"priority" && value:"HIGH"}
```

34.3.5. Querying by Set Name

To filter by the set name itself:

```
# Records that have a "logistics" attribute set
dynamicAttributeSets:{name:"logistics"}

# Combine set name with attribute value
dynamicAttributeSets:{name:"logistics"} &&
dynamicAttributeSets.attributes:{name:"weight" && value:>##50}
```

34.3.6. Advanced Tags (Simple Key-Value)

For simpler tag-based filtering, use **advancedTags**:

```
# Find by tag name and value
advancedTags:{name:"priority" && value:"urgent"}

# Multiple tag conditions
advancedTags:{name:"region" && value:"US"} && advancedTags:{name:"tier" &&
```

```
value:"premium"}
```

34.3.7. Advanced Examples

Complex Business Logic

```
# Active products under $50 OR featured products at any price
(active:true && price:<##50) || featured:true

# Orders needing attention: overdue OR high-value pending
(dueDate:<2024-12-25 && status!="COMPLETED") ||
(status:"PENDING" && totalAmount:>##1000)

# Products with inventory issues
(quantity:<=#5 && reorderPoint:>#5) || stockStatus:"OUT_OF_STOCK"
```

Multi-tenant Filtering

```
# User's own records
dataDomain.ownerId:${principalId}

# Organization-wide access
dataDomain.orgRefName:${orgRefName}

# Tenant-scoped with public sharing
dataDomain.tenantId:${pTenantId} || dataDomain.orgRefName:"PUBLIC"
```

Audit and Compliance

```
# Records modified by specific user
auditInfo.lastUpdatedBy:"john.doe"

# Changes in date range
auditInfo.lastUpdatedDate:>=2024-12-01 &&
auditInfo.lastUpdatedDate:<2024-12-31

# Created vs modified
auditInfo.createdDate:auditInfo.lastUpdatedDate # never modified
auditInfo.createdDate:!auditInfo.lastUpdatedDate # has been modified
```

34.3.8. Variables in Filters

Variables are resolved from the current security context and can be used in permission rules and access resolvers.

Standard Variables

Variable	Description
<code>\${principalId}</code>	Current user's ID
<code>\${pTenantId}</code>	Principal's tenant ID
<code>\${pAccountId}</code>	Principal's account ID
<code>\${pOrgRefName}</code>	Principal's organization
<code>\${realm}</code>	Current realm/database
<code>\${area}</code>	Current functional area
<code>\${functionalDomain}</code>	Current functional domain
<code>\${action}</code>	Current action (CREATE, UPDATE, etc.)

Custom Variables from Access Resolvers

```
// In your AccessListResolver
@Override
public String key() {
    return "accessibleCustomerIds"; // becomes ${accessibleCustomerIds}
}

@Override
public Collection<?> resolve(...) {
    return Arrays.asList("CUST001", "CUST002", "CUST003");
}
```

```
# Use in filter
customerId:^(${accessibleCustomerIds})
```

34.3.9. Performance Tips

Efficient Queries

```
# Good: Use indexed fields first
status:"ACTIVE" && createdAt:>=2024-01-01

# Better: Combine with specific values
status:"ACTIVE" && ownerId:${principalId} && createdAt:>=2024-01-01

# Avoid: Leading wildcards on large collections
name:*widget # can be slow on millions of records
```

Projection for Large Objects

```
# In REST calls, limit returned fields
GET /products/list?filter=active:true&projection=+id,+name,+price,-description
```

34.3.10. Integration with REST APIs

List Endpoints

```
# Basic filtering
GET /products/list?filter=active:true

# With sorting and pagination
GET /products/list?filter=price:>##10&sort=-createdDate&skip=20&limit=10

# Complex filter with projection
GET
/orders/list?filter=(status:"PENDING"||status:"PROCESSING")&&totalAmount:>##100&projection=+id,+status,+totalAmount,+customerName
```

Permission Rules

```
- name: user-own-records
  priority: 300
  match:
    method: [GET]
    url: /api/**
  effect: ALLOW
  andFilterString: "dataDomain.ownerId:${principalId}"
```

Access Resolvers

```
// Resolver returns customer IDs user can access
public Collection<?> resolve(...) {
    return customerService.getAccessibleIds(principalId);
}

// Used in permission rule
andFilterString: "customerId:[${accessibleCustomerIds}]"
```

34.3.11. Error Handling

Common syntax errors and solutions:

```
# Wrong: Missing quotes for multi-word strings
```

```

name:Super Widget
# Right:
name:"Super Widget"

# Wrong: Incorrect number prefix
price:19.99
# Right:
price:##19.99

# Wrong: Invalid date format
createdDate:12/25/2024
# Right:
createdDate:2024-12-25

# Wrong: Unbalanced parentheses
(active:true && price:>##10
# Right:
(active:true && price:>##10)

# Wrong: text() inside NOT
!!text("search")
# Right: Use text() at top level only
text("search") && status:!"DELETED"

# Wrong: text() inside OR
text("search") || status:"ACTIVE"
# Right: Use text() with AND only
text("search") && (status:"ACTIVE" || status:"PENDING")

# Wrong: Multiple text() clauses
text("foo") && text("bar")
# Right: Combine search terms in a single text()
text("foo bar")

```

34.3.12. See Also

- [REST CRUD Querying](#)
- [Permission Rules](#)
- [Access Resolvers](#)

34.3.13. Execution engines and listeners

The BIAPI query syntax is parsed once (via ANTLR) and can be executed by different "listeners" depending on the use case. Quantum ships with two primary implementations that share the same grammar and semantics:

- Morphia listener: converts a query into Mongo/Morphia Filters for database-side execution
- In-memory listener: converts a query into a Java Predicate over JSON data for

Morphia: QueryToFilterListener

Use this when you want the database to perform the filtering. The listener walks the parse tree and produces a `dev.morphia.query.filters.Filter` which you can apply to Morphia queries. This is ideal for repository APIs and any endpoint where you want to leverage MongoDB indexes and avoid loading large data sets into memory.

Key characteristics: - Output type: Morphia Filter - Execution: database-side (MongoDB) - Semantics: identical to grammar (comparisons, IN/NIN, exists, null, regex with wildcards, elemMatch, boolean &&/||/!!) - Text search: `text("...")` is supported when a collection has a text index; only one text clause is allowed per query, and it must be at the top level (cannot be inside OR, NOT, or elemMatch). - Variable expansion: supports `${vars}` and single-variable IN list expansion (e.g., `[$ids]` can expand to a collection/array or a comma-separated string)

Example:

```
import com.e2eq.framework.grammar.*;
import com.e2eq.framework.model.persistent.morphia.QueryToFilterListener;
import dev.morphia.query.filters.Filter;
import dev.morphia.query.filters.Filters;
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.ParseTreeWalker;
import org.apache.commons.text.StringSubstitutor;

String query = "(status:Assigned||status:Pending)&&displayName:*Route*";
var vars = java.util.Map.<String,String>of();

// Parse
CharStream cs = CharStreams.fromString(query);
BIAPIQueryLexer lexer = new BIAPIQueryLexer(cs);
CommonTokenStream tokens = new CommonTokenStream(lexer);
BIAPIQueryParser parser = new BIAPIQueryParser(tokens);
BIAPIQueryParser.QueryContext tree = parser.query();

// Build Morphia filter
QueryToFilterListener listener = new QueryToFilterListener(vars, new
StringSubstitutor(vars), /* modelClass */ null);
ParseTreeWalker.DEFAULT.walk(listener, tree);
Filter morphiaFilter = listener.getFilter();

// Use with Morphia query (example)
// datastore.find(MyEntity.class).filter(morphiaFilter).iterator().toList();
```

A few query examples (taken from `testQueryStrings.txt`):

- Equality: `field:"quotedString"`
- Comparisons: `field:>##12.56`, `field:≤#123`

- IN/NIN: field:^[value1,value2], field:!^[value1,value2]
- Exists/Null: field:~, field:null
- elemMatch: arrayField:{{subField:<#12} | {(subField:>#15)}}

In-memory (JsonNode): QueryToPredicateJsonListener

Use this when you need to evaluate queries in memory without reflection on POJOs. This implementation compiles a query into a `java.util.function.Predicate` over a Jackson `JsonNode`. It is Quarkus/GraalVM friendly, useful for: - Unit tests where you want to validate query behavior without a database - Post-filtering or pre-filtering of already-fetched data - Evaluating access rules or business logic against transient objects

Key characteristics: - Output type: `Predicate<JsonNode>` - Execution: in-memory - No runtime reflection: operates on `JsonNode` - Semantics and variable expansion match the Morphia listener

Convenience helpers exist in `QueryPredicates`:

```
import com.e2eq.framework.query.QueryPredicates;
import com.fasterxml.jackson.databind.JsonNode;
import java.util.function.Predicate;
import java.util.Map;

String query = "(status:Assigned||status:Pending)&&displayName:*Route*";
Predicate<JsonNode> p = QueryPredicates.compilePredicate(query, Map.of(), Map.of());

// Example data as a POJO or Map -> convert to JsonNode
record Ticket(String status, String displayName) {}
Ticket ticket = new Ticket("Assigned", "Route Exception in
Route:To[http://com.xxx/update]");
JsonNode node = QueryPredicates.toJsonNode(ticket);

boolean include = p.test(node); // true
```

Additional examples

- Equality and comparisons

```
var vars = Map.<String,String>of();
var objVars = Map.<String,Object>of();
Predicate<JsonNode> eq = QueryPredicates.compilePredicate("quantity:#42", vars,
objVars);
Predicate<JsonNode> gt = QueryPredicates.compilePredicate("price:>##19.99", vars,
objVars);

JsonNode product = QueryPredicates.toJsonNode(Map.of("quantity", 42, "price", 25.00));
assert eq.test(product);
assert gt.test(product);
```

- IN / NIN with variable expansion

```
var vars = Map.of("principalId", "66d1f1ab452b94674bbd934a");
Predicate<JsonNode> in =
QueryPredicates.compilePredicate("ownerId:^([${principalId}],value2]", vars, Map.of());
JsonNode doc = QueryPredicates.toJsonNode(Map.of("ownerId",
"66d1f1ab452b94674bbd934a"));
assert in.test(doc);
```

- elemMatch over arrays of objects

```
String q = "items:{(sku:abc||qty:>#10)&&price:<=##9.99}";
Predicate<JsonNode> em = QueryPredicates.compilePredicate(q, Map.of(), Map.of());
JsonNode order = QueryPredicates.toJsonNode(Map.of(
    "items", java.util.List.of(
        Map.of("sku","abc","qty", 5, "price", 9.99),
        Map.of("sku","xyz","qty", 12, "price", 8.50)
    ));
// Matches: first item by sku OR second item by qty with price cap
assert em.test(order);
```

- Regex with wildcards

```
Predicate<JsonNode> rx = QueryPredicates.compilePredicate("displayName:*Route*",
Map.of(), Map.of());
JsonNode ticket = QueryPredicates.toJsonNode(Map.of("displayName", "Route Exception in
Route:To[...]"));
assert rx.test(ticket);
```

Choosing the right listener

- Use Morphia (QueryToFilterListener) when:
 - You are filtering MongoDB collections and want the DB to do the work (indexing, pagination, scalability)
 - You need server-side performance and minimal memory footprint
- Use In-memory (QueryToPredicateJsonListener) when:
 - You run in Quarkus native image and want to avoid reflection on POJOs
 - You are writing unit tests or applying rules to transient/aggregated data
 - You need to evaluate a query over already materialized objects without another database round-trip

Both listeners aim to maintain parity with the grammar. If you observe mismatches, please file an issue with the query string, the evaluated data sample, and the expected vs actual results.

Query Field Validation

Quantum provides validation capabilities to detect references to non-existent fields before query execution, preventing runtime errors and providing early feedback.

Validating Listeners

Two validating listener implementations extend the standard listeners:

- `ValidatingQueryToFilterListener`: extends `QueryToFilterListener` for Morphia queries
- `ValidatingQueryToPredicateJsonListener`: extends `QueryToPredicateJsonListener` for in-memory predicates

Both validate field references against a model class during parsing and accumulate errors.

Example usage:

```
import com.e2eq.framework.model.persistent.morphia.ValidatingQueryToFilterListener;
import com.e2eq.framework.grammar.*;
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.ParseTreeWalker;

String query = "name:John AND invalidField:test";

BIAPIQueryLexer lexer = new BIAPIQueryLexer(CharStreams.fromString(query));
CommonTokenStream tokens = new CommonTokenStream(lexer);
BIAPIQueryParser parser = new BIAPIQueryParser(tokens);

ValidatingQueryToFilterListener listener =
    new ValidatingQueryToFilterListener(UserProfile.class);
ParseTreeWalker.DEFAULT.walk(listener, parser.query());

if (listener.hasValidationErrors()) {
    List<String> errors = listener.getValidationErrors();
    throw new IllegalArgumentException("Invalid query: " + errors);
}

Filter filter = listener.getFilter();
```

@ValidQueryFilter Annotation

For automatic validation in DTOs and models, use the `@ValidQueryFilter` annotation:

```
import com.e2eq.framework.annotations.ValidQueryFilter;
import jakarta.validation.constraints.NotNull;

public class SearchRequest {
    @NotNull
    @ValidQueryFilter(modelClass = UserProfile.class)
    private String filterQuery;
```

```
// getters/setters
}
```

The annotation integrates with Jakarta Bean Validation and is automatically enforced by the `ValidationInterceptor` during entity persistence.

REST endpoint example:

```
@Path("/users")
public class UserResource {

    @GET
    public Response searchUsers(@Valid @BeanParam SearchParams params) {
        // filterQuery is validated before this method executes
        return Response.ok(userService.search(params.getFilterQuery())).build();
    }
}

public class SearchParams {
    @QueryParam("filter")
    @ValidQueryFilter(modelClass = UserProfile.class)
    private String filterQuery;
}
```

Persisted filter example:

```
public class SavedSearch extends BaseModel {
    private String name;

    @ValidQueryFilter(modelClass = Order.class)
    private String filterExpression;

    // When saved, ValidationInterceptor validates the filter
}
```

QueryFieldValidator Utility

For programmatic validation without listeners:

```
import com.e2eq.framework.query.QueryFieldValidator;

// Validate against a model class
QueryFieldValidator validator = QueryFieldValidator.forModelClass(UserProfile.class);

boolean isValid = validator.validateField("email"); // true
boolean invalid = validator.validateField("nonExistentField"); // false
```

```

if (validator.hasErrors()) {
    List<String> errors = validator.getErrors();
    // Handle validation errors
}

// Validate against aggregation pipeline schema
Map<String, Class<?>> schema = Map.of(
    "totalAmount", Double.class,
    "orderCount", Long.class,
    "customerName", String.class
);

QueryFieldValidator aggValidator = QueryFieldValidator.forAggregationSchema(schema);
aggValidator.validateField("totalAmount"); // true
aggValidator.validateField("invalidField"); // false

```

Benefits

- Early error detection: catch field reference errors before query execution
- Better error messages: specific feedback about which fields are invalid
- Development safety: prevent typos and refactoring issues
- API validation: validate user-provided query strings in REST endpoints
- Data integrity: prevent saving invalid filter expressions to the database

34.3.14. Expressing Ontology Constraints in Queries (optional)

The BI-API query language primarily targets fields on documents. When using the ontology modules, there are two ways to incorporate ontology relationships into queries:

Option A: Variable from an AccessListResolver

- A resolver computes the set of IDs using EdgeDao (e.g., orders related by placedInOrg to the caller's org).
- Use an IN clause over id in your filter string:

```

# idsByPlacedInOrg is published by an AccessListResolver
id: ^${idsByPlacedInOrg}

```

34.3.15. Ontology operators: hasEdge, hasOutgoingEdge, and hasIncomingEdge

When the ontology module is enabled and edges are materialized in MongoDB, you can constrain results by semantic graph relationships using `hasEdge`, `hasOutgoingEdge`, and `hasIncomingEdge`.

Syntax

```
# Standard 2-argument form (relationship existence)
hasEdge(predicate, dst)
hasOutgoingEdge(predicate, dst)
hasIncomingEdge(predicate, src)

# Qualified 3-argument form (relationship existence WITH edge property constraints)
hasEdge(predicate, dst, { edgeFilter })
hasOutgoingEdge(predicate, dst, { edgeFilter })
hasIncomingEdge(predicate, src, { edgeFilter })
```

Arguments

- **predicate**: Relationship identifier. Can be an unquoted identifier (**placedInOrg**), a quoted string ("**placedInOrg**"), or a variable expression (**\${orgPredicate}**).
- **dst / src**: Target or source entity coordinate. Can be a string literal ("**OrgA**"), an **ObjectId**, a reference literal (**@507f1f77bcf86cd799439011**), or a variable placeholder (**\${orgRefName}**, **\${principalId}**).
- **{ edgeFilter }**: (Optional) A bracketed sub-query containing filtering expressions evaluated directly against the properties of the matching edge in the **edges** collection.

Edge Property Scoping and Mechanics

When querying an edge with property constraints:

1. **Automatic Field Scoping**: Any attribute referenced inside the **{ ... }** block that is not a reserved root edge attribute (**src**, **dst**, **p**, **dstType**, **dataDomain**, **_id**) is automatically rewritten to target dynamic edge properties stored under **props.<field>**. For example, **{ role:"PRIMARY" && active:true }** is compiled to **props.role == "PRIMARY" AND props.active == true**.
2. **MongoDB Compound Index Utilization**: Edge property filters are pushed down directly into MongoDB queries on the **edges** collection. They take advantage of compound indexes:
 - **idx_domain_p_dst_props**: **{"dataDomain": 1, "p": 1, "dst": 1, "props": 1}** (for **hasEdge / hasOutgoingEdge**)
 - **idx_domain_src_p_props**: **{"dataDomain": 1, "src": 1, "p": 1, "props": 1}** (for **hasIncomingEdge**)
3. **Supported Operators**: All standard BIAPI equality (**:**), inequality (**!:**), comparisons (**>**, **<**, **>=**, **<=**), list membership (**^**, **!^**), existence (**~**), and logical compositions (**&&**, **||**, **!!**) are supported on edge properties.
4. **Prohibited Operators**: Full-text search **text(...)** is prohibited inside **{ edgeFilter }** blocks because MongoDB does not permit **\$text** inside embedded document conditions or sub-filters; queries containing **text(...)** within edge filters are rejected at query validation time.

Defaults and Fallback Behavior

Element	Default Behavior	Rationale
Omitted { edgeFilter }	Matches any edge with the specified predicate and target/source, regardless of properties.	Preserves 100% backward compatibility for standard 2-argument calls.
Missing Edge Properties	Evaluates to false (no match). If an edge does not contain the queried property in its props map, the filter fails closed.	Predictable fail-closed semantics for dynamic schemas.
Tenant Scoping & Missing Security Context	Tenant ID is resolved from the active SecurityContext / RuleContext (\${pTenantId}). If the query is executed without an active tenant or security context, the edge lookup returns empty results (_id: "none").	Inviolate multi-tenant data isolation; queries never leak edge relationships across realms or unauthenticated callers.
Dynamic Model Schema Validation	When query schema validation is enabled (ValidatingQueryToFilterListener), edge property keys bypass static entity model field validation.	Edge properties are open and dynamic; static entity models do not need predefined getters or schema mappings for edge property names.

Examples

Basic edge existence check (2-argument default)

```
# Find orders placed in an organization (finds source Order entities)
hasEdge("placedInOrg", ${orgRefName})

# Find locations accessible by a specific associate (finds target Location entities)
hasIncomingEdge(canAccessLocation, ${principalId})
```

Qualified edge filtering with exact property match

```
# Find accounts where user has a PRIMARY role
hasIncomingEdge("memberOf", ${principalId}, { role:"PRIMARY" })

# Find orders routed to a facility with an expedited service level
hasEdge("routedToFacility", "FAC-901", { serviceLevel:"EXPEDITED" })
```

Qualified edge filtering with multi-attribute constraints

```
# Find suppliers with valid certified relationships and confidence >= 0.90
hasEdge("suppliedBy", @@507f1f77bcf86cd799439011, { certified:true &&
```



```
confidence:>=#0.90 })
```

```
# Find assets with active status and priority in high tiers  
hasEdge("assignedToTeam", "TEAM-ALPHA", { active:true && tier:^[#1, #2] })
```

Combining edge traversal filters with root entity filters

```
# Orders in pending state placed in ORG-1 with a high-priority edge tag  
hasEdge("placedInOrg", "ORG-1", { priority:"HIGH" }) && status:"PENDING" &&  
total:>##500.00
```

Query Macros for Temporal & AI Confidence Governance

To simplify complex temporal and probabilistic filtering, the query preprocessor recognizes two high-level macros:

1. `@asOf(timestamp)`:

- Compiles to bitemporal range matching: `validFrom:≤ts || validFrom:null` && `(validTo:≥ts || validTo:null` (or prefixed with `edges.` inside expansion/traversal contexts).
- Backed by compound index `idx_domain_temporal`.
- Strips surrounding quotes so ISO-8601 timestamps parse cleanly as ANTLR `DATE` / `DATETIME` tokens.
- **Business Scenario & Solution:** Point-in-time compliance audits (e.g. healthcare HIPAA audits or financial regulatory inquiries) can reproduce past authorization states without maintaining separate historical snapshot tables or running deletion cron jobs.

2. `@minConfidence(threshold)`:

- Compiles to confidence threshold filtering: `confidence:≥threshold` (or `edges.confidence:≥threshold`).
- Backed by compound index `idx_domain_confidence`.
- **Business Scenario & Solution:** AI and LLM graph derivation governance (e.g. anti-money laundering beneficial ownership links). Mission-critical operations require human-certified links (`confidence: 1.0`), while exploratory analysis accepts AI-inferred relationships (`@minConfidence(0.80)`).

Query Macro Examples

```
# Temporal Point-in-Time Audit: Did associate have active role on September 1st, 2026?  
hasEdge("assignedToBranch", "BRANCH-NY", { @asOf(2026-09-01T00:00:00Z) &&  
role:"MANAGER" })
```

```
# Probabilistic AI Governance: Only traverse high-confidence vendor relationships  
hasEdge("subsidiaryOf", "PARENT-CORP", { @minConfidence(0.85) &&  
assertionMethod:"LLM_INFERENCE" })
```

```
# Combining temporal validity, AI confidence, and security classification  
hasEdge("operatesInFacility", "FAC-901", {
```

```
@asOf(2026-09-20T00:00:00Z) &&  
@minConfidence(0.90) &&  
securityLabel:"CONFIDENTIAL"  
})
```

34.3.16. Root projection recap: `fields:[+...,-...]`

To include or exclude fields from the root documents:

- Syntax: `fields:[+f1,+f2,-f3]`
- Include mode: if any `+` is present, only the listed fields are included (with explicit `-` carving out exceptions).
- Exclude mode: if only `-` entries exist, all fields are included except those.
- Default `_id`: preserved unless explicitly specified (`+_id/-_id`).

Examples

```
# Include a few fields from the root and drop one  
fields:[+_id,+total,-internalNotes]  
  
# Combine with expansion (planner chooses aggregation mode)  
expand(customer) && fields:[+_id,+customer,+total]
```

For per-expansion projections and advanced traversal filters, see [Query Expansion](#).

Chapter 35. Relationship hydration with `expand(path)`

This section documents the new relationship traversal and hydration capability added to the BIAPI query language. It preserves backward compatibility and introduces no SQL-like syntax.

35.1. How it works

- You add one or more `expand(path)` directives to your query string to request hydration of related entities at specific paths.
- The planner inspects the query. If any `expand(...)` directives are present, it selects AGGREGATION mode; otherwise, FILTER mode.
- In FILTER mode, the system converts the query to a Morphia Filter and queries a single MongoDB collection.
- In AGGREGATION mode, the system constructs an aggregation pipeline (with `$lookup`, `$unwind`, etc.) to materialize related documents. Execution may be guarded by a feature flag in the `QueryGatewayResource`.
- Projection can be applied at the root and per expansion using `fields:[+/-]` syntax. Unknown projection paths are hard errors.

See also:

- [Planner and QueryGateway](#) for mode selection and Java APIs
- [QueryGateway REST endpoints](#) to plan or execute queries over HTTP

35.2. What is `expand(path)`?

Use `expand(path)` to hydrate properties that are modeled as `EntityReference` or ontology-annotated relations. The path is dot-based and may include array wildcards `[*]` between segments.

- Single reference: `expand(customer)`
- Array of references: `expand(items[*].product)`
- Nested arrays: `expand(patient.visits[.diagnoses[]])`

Expansion is a query-time operation: the related document(s) are materialized and embedded at the specified path in the result.

35.3. Filtering related entities (inline)

Continue to use your existing filter primitives in the same query string to filter the root collection. Filtering on related entities will be supported via either:

- Inline path blocks: `path { key:value, key2:^[v1,v2] }`

- Or filters inside `expand(...)`: `expand(customer, status:active)`

Note: In v1, parsing of `expand(path)` is enabled; related-entity filters inside `expand` and inline path blocks will be introduced with the planner/compiler work.

35.4. Projection with fields:[+/-]

A unified projection syntax can be used both at the root and per expansion.

- Inclusion mode: `fields:[+_id,+total,+customer.name]`
- Exclusion mode: `fields:[-internalNotes,-secret]`
- Inside `expand`: `expand(customer, fields:[+name,+tier,-ssn])`

Rules: - If any `+` appears → include only those paths, then apply `-` as carve-outs. - If only `-` appears → include all by default and remove listed paths. - `_id` keeps current default behavior unless explicitly set.

35.5. Examples

- Hydrate a single reference and keep a few fields

```
q = "realm:acme && expand(customer, fields:[+name,+tier]) &&
fields:[+_id,+total,+customer.name]"
```

- Hydrate array of product references inside line items, filter to active products, project a couple of fields

```
q = "expand(items[*].product, active:true, fields:[+sku,+title]) &&
fields:[+_id,+items.product.sku]"
```

- Filter by a related product without projecting it (inline block form)

```
q = "items[*].product{ active:true, sku:^(A123,B456) } && fields:[+_id,+total]"
```

35.6. Semantics and limits

- Backward compatible: if no `expand(...)` or related-path filters are present, queries run as they do today (single-collection).
- Depth for nested paths is bounded; on mixed array/object hops, traversal stops at the first non-reference boundary.
- Unknown projection paths are hard errors (the request fails with a clear message).

See also: [Planner and QueryGateway](#)

35.7. Aggregation examples: what expand(...) compiles to

This section shows typical MongoDB aggregation stages the planner emits when `expand(path)` is present. Exact collection names and field names depend on your model mapping. If the collection cannot be resolved at plan-time, the planner will emit a placeholder from: `"unknown"`; the REST gateway returns HTTP 422 on execution in that case.

Tip: You can inspect the chosen mode and expand paths with `POST /api/query/plan`. See [QueryGateway REST endpoints](#).

35.7.1. Single reference: expand(customer)

Query

```
q = "status:active && expand(customer, fields:[+name,+tier]) &&
fields:[+_id,+total,+customer]"
```

Typical pipeline skeleton

```
[
  { "$match": { "status": "active" } },
  { "$lookup": {
    "from": "customers",
    "localField": "customerId",
    "foreignField": "_id",
    "as": "__exp_customer"
  }},
  { "$unwind": { "path": "$__exp_customer", "preserveNullAndEmptyArrays": true }},
  { "$set": { "customer": "$__exp_customer" }},
  { "$project": { "_id": 1, "total": 1, "customer": { "name": 1, "tier": 1 } } }
]
```

Result shape (conceptual)

```
{
  "_id": "0123",
  "total": 99.50,
  "customer": { "_id": "C1", "name": "Acme", "tier": "GOLD" }
}
```

Notes - The intermediate field `__exp_customer` is an internal staging alias; the planner may omit it in future. - Per-expansion projection `fields:[+name,+tier]` narrows the embedded document via `$project`.

35.7.2. Array of references: `expand(items[*].product)`

Query

```
q = "expand(items[*].product, fields:[+sku,+title]) && fields:[+_id,+items.product]"
```

Typical pipeline patterns (two common strategies)

1) Unwind-and-regroup (easy to read)

```
[
  { "$unwind": { "path": "$items", "preserveNullAndEmptyArrays": true } },
  { "$lookup": {
    "from": "products",
    "localField": "items.productId",
    "foreignField": "_id",
    "as": "__exp_product"
  } },
  { "$unwind": { "path": "$__exp_product", "preserveNullAndEmptyArrays": true } },
  { "$set": { "items.product": "$__exp_product" } },
  { "$group": {
    "_id": "$_id",
    "doc": { "$first": "$$ROOT" },
    "items": { "$push": "$items" }
  } },
  { "$replaceRoot": { "newRoot": { "$mergeObjects": [ "$doc", { "items": "$items" } ] } } },
  { "$project": { "_id": 1, "items.product": { "sku": 1, "title": 1 } } }
]
```

2) Map-with-lookup (keeps array order without grouping; requires `$lookup` with pipeline + `$map`)

```
[
  { "$set": {
    "items": {
      "$map": {
        "input": "$items",
        "as": "it",
        "in": {
          "$let": {
            "vars": {
              "p": { "$first": {
                "$lookup": {
                  "from": "products",
                  "let": { "pid": "$$it.productId" },
                  "pipeline": [ { "$match": { "$expr": { "$eq": [ "$_id", "$$pid" ] } } } ],
                  "as": "p"
                }
              }
            }
          }
        }
      }
    }
  } } ],
```

```

        }
      }
    },
    "in": { "$mergeObjects": [ "$$it", { "product": "$p" } ] }
  }
}
}
}
},
{ "$project": { "_id": 1, "items.product": { "sku": 1, "title": 1 } } }
]

```

Notes - The actual compiler may choose either strategy based on simplicity; behavior is equivalent for most cases. - Null or missing productId yields product: null due to preserveNullAndEmptyArrays.

35.7.3. Nested arrays: expand(patient.visits[].diagnoses[])

Query

```

q = "expand(patient.visits[*].diagnoses[*]) &&
fields:[+_id,+patient.visits.diagnoses]"

```

Typical staged pipeline (high level)

1. \$lookup patient -> __exp_patient; \$set patient
2. \$unwind visits (preserveNullAndEmptyArrays)
3. \$unwind visits.diagnoses
4. \$lookup diagnosis -> set visits.diagnoses[*]
5. \$group back by root _id while reconstructing visits and diagnoses arrays in original order
6. optional \$project to constrain fields

Caveats - Depth and mixed object/array hops are bounded; traversal stops at a non-reference boundary. - For very large arrays, prefer targeted filtering before expansion to reduce fan-out.

35.7.4. Multi-Hop Reference Traversal Chains: expand(hop1) && expand(hop1.hop2)

Multiple relationship hops can be chained together using standard logical conjunction (&&). For example, traversing from a Customer to their Orders, and from each Order to its Supplier:

Query

```

q = "customerName:Cust-1 && expand(orderRef) && expand(orderRef.supplierRef)"

```

Instead of simple uncorrelated lookups, the compiler generates correlated `$lookup` stages with dedicated inner pipelines:

```
[
  { "$match": { "dataDomain.tenantId": "acme", "customerName": "Cust-1" } },
  { "$lookup": {
    "from": "orders",
    "let": { "ref_id": "$orderRef._id" },
    "pipeline": [
      { "$match": { "$expr": { "$eq": ["$_id", "$$ref_id"] } } },
      { "$match": { "dataDomain.tenantId": "acme" } },
      { "$project": { "secretDiscount": 0 } }
    ],
    "as": "orderRef"
  } },
  { "$unwind": { "path": "$orderRef", "preserveNullAndEmptyArrays": true } },
  { "$lookup": {
    "from": "suppliers",
    "let": { "ref_id": "$orderRef.supplierRef._id" },
    "pipeline": [
      { "$match": { "$expr": { "$eq": ["$_id", "$$ref_id"] } } },
      { "$match": { "dataDomain.tenantId": "acme" } },
      { "$project": { "costMargin": 0 } }
    ],
    "as": "orderRef.supplierRef"
  } },
  { "$unwind": { "path": "$orderRef.supplierRef", "preserveNullAndEmptyArrays": true } }
]
```

Key Security Defaults for Multi-Hop Traversals:

- **Per-Stage Row Security:** Each inner pipeline injects a `$match` stage enforcing target collection `DataDomain` constraints and rule filters.
- **Monotonic Restriction Accumulation:** Security restrictions strictly accumulate. Upstream constraints are preserved and compounded down the traversal tree, preventing privilege escalation.
- **Cross-Tenant Pruning:** If an entity reference points across tenant boundaries or to a document forbidden by target security rules, the inner pipeline produces 0 matches, evaluating to `null` or `[]` without leaking document existence.
- **Field Exclusions (`$project`):** Target entity field exclusions defined in `RuleContext` are pushed down into each inner pipeline so sensitive fields never leave MongoDB.

35.7.5. When is AGGREGATION used vs FILTER?

- **FILTER:** no `expand(...)`. Planner returns a Morphia Filter and the gateway executes `find` with

dev.morphia.query.Query.

- AGGREGATION: one or more `expand(...)`. Planner returns an aggregation pipeline with per-stage `$lookup` inner pipelines, `$unwind`, and `$project` stages. Execution requires `feature.queryGateway.execution.enabled=true`; otherwise, POST `/api/query/find` returns HTTP 501. If any `$lookup.from` is *"unknown"*, execution returns HTTP 422.

Chapter 36. Planner and QueryGateway

The planner decides whether a query runs as a simple single-collection filter (FILTER mode) or as a MongoDB aggregation (AGGREGATION mode). This is transparent to callers.

In v1 the planner recognizes `expand(path)` and selects AGGREGATION mode when present. Otherwise, FILTER mode is used and the existing Morphia path is taken.

36.1. Planner decisions

- FILTER mode: No `expand(...)` in the query. The system generates a Morphia Filter and queries a single collection.
- AGGREGATION mode: One or more `expand(...)` directives are present. The system will build an aggregation pipeline (with `$lookup`, etc.). In v1, a placeholder pipeline is returned to establish the contract.

36.2. Public facade: QueryGateway

A new collection-agnostic facade exposes plan/inspect capabilities and will later execute aggregation pipelines when enabled.

Java

```
import com.e2eq.framework.model.persistent.morphia.planner.PlannerResult;
import com.e2eq.framework.model.persistent.morphia.query.QueryGateway;
import com.e2eq.framework.model.persistent.morphia.query.QueryGatewayImpl;

public class PlannerSnippet1 {
    static class Order {}
    public static void main(String[] args) {
        QueryGateway gateway = new QueryGatewayImpl();
        PlannerResult pr = gateway.plan("expand(customer) && status:active", Order.class);
        System.out.println(pr.getMode());           // AGGREGATION
        System.out.println(pr.getExpandPaths());    // [customer]
    }
}
```

If you prefer a lower-level API:

Java

```
import com.e2eq.framework.model.persistent.morphia.MorphiaUtils;
import com.e2eq.framework.model.persistent.morphia.planner.PlannedQuery;

PlannedQuery planned = MorphiaUtils.convertToPlannedQuery(
    "expand(items[*].product) && status:active",
    Order.class
);
```

```
switch (planned.getMode()) {
  case FILTER:
    // Use planned.getFilter() with Morphia datastore
    break;
  case AGGREGATION:
    // Use planned.getAggregation() to inspect the pipeline (placeholder in v1)
    break;
}
```

36.3. Examples end-to-end

- No expansion (FILTER mode):

```
q = "realm:acme && status:active && fields:[+_id,+total]"
```

- With expansion (AGGREGATION mode):

```
q = "expand(customer, fields:[+name,+tier]) && status:active &&
fields:[+_id,+customer.name]"
```

See also: [Relationship hydration with expand\(path\)](#). For concrete pipelines, see [Aggregation examples: what expand\(...\) compiles to](#).

36.4. REST endpoints (v1)

The gateway (`QueryGatewayResource`, `@Path("/api/query")`, `@FunctionalMapping(area="integration", domain="query")`) exposes a planner endpoint plus generic governed query operations:

- `POST /api/query/plan`—returns planner mode (`FILTER` vs `AGGREGATION`) and expand paths detected
- `POST /api/query/find`—executes governed `FILTER` and `AGGREGATION` queries, returning the standard `Collection<T>` envelope
- `POST /api/query/count`—returns the governed count for a `FILTER`-mode query
- `POST /api/query/delete`—deletes a single entity by id (governed; 404 across `DataDomain` boundaries)
- `POST /api/query/deleteMany`—deletes the matching `FILTER`-mode set
- `POST /api/query/export`—streams a governed CSV export

36.5. Row- and Field-Level Governance

All query gateway operations execute through multi-layered row- and field-level security filters derived from `RuleContext` and `DataDomainResolver`, failing closed when no valid security context is

established:

1. Root Entity Governance:

- `securedFilterArray(...)` ANDs the query filter with the caller's active `DataDomain` scope and rule `filterConstraints` resolved from `RuleContext`.
- Field-level exclusions (`applyExcludedFieldProjection(...)`) exclude sensitive fields at the MongoDB datastore level, preventing them from leaving MongoDB.

2. Fail-Closed Guarantee:

- Unless `SecurityContext.isIgnoringRules()` is explicitly set, requests lacking an established `PrincipalContext` and `ResourceContext` are immediately refused with HTTP 500 (`{"error": "SecurityContextMissing"}`).

3. Information Leakage Prevention:

- `delete` ANDs the target `_id` with the secured row filter; records outside the caller's `DataDomain` return HTTP 404 (`deletedCount=0, success=false`) without disclosing whether the entity exists in another tenant.

36.6. Governed Aggregation Execution & Per-Stage Security Pushdown

When a query contains `expand(...)` directives, the planner selects `AGGREGATION` mode and compiles the query into a multi-stage MongoDB aggregation pipeline via `MongoAggregationCompiler`:

flowchart TD

```
subgraph Root ["Root Document Stage"]
    RMatch["$match: dataDomain + Root Rule Constraints"]
    RFilter["$match: User Root Filter"]
end

subgraph Hop1 ["Hop 1: $lookup inner pipeline (e.g. orderRef)"]
    H1Match["$match: $expr eq [_id, $$ref_id]"]
    H1Sec["$match: Order DataDomain + Order Rule Constraints"]
    H1Proj["$project: Exclude Order Sensitive Fields"]
end

subgraph Hop2 ["Hop 2: $lookup inner pipeline (e.g. supplierRef)"]
    H2Match["$match: $expr eq [_id, $$ref_id]"]
    H2Sec["$match: Supplier DataDomain + Supplier Rule Constraints (Accumulated)"]
    H2Proj["$project: Exclude Supplier Sensitive Fields"]
end

Root --> Hop1
Hop1 --> Hop2
Hop2 --> FinalProj["Final $project / field filtering"]
```

36.6.1. Per-Stage Pushdown Mechanics

1. **Root Pre-Filtering:** The root collection pipeline begins with a leading `$match` enforcing the root model's `DataDomain` policy and security rules.
2. **Inner Pipeline `$lookup` Security:** For each expansion hop, target entity row-level constraints are pushed directly into the inner `pipeline: [ ... ]` of MongoDB's `$lookup` stage.
3. **Monotonic Restriction Accumulation:** Security restrictions accumulate along multi-hop paths (e.g. Customer → Order → Supplier). Downstream hops inherit and compound upstream access constraints, ensuring that traversal can never be exploited to elevate privileges or bypass intermediate isolation boundaries.
4. **Target Field Exclusion Pushdown:** Target entity field exclusions (`excludedFieldPaths`) are pushed into the inner pipeline as a `$project` stage, redacting sensitive fields directly within the MongoDB engine.
5. **Cross-Tenant Reference Pruning:** If a foreign reference points to an entity in a different tenant or an entity that fails target row security, the inner `$match` stage filters it out. The populated property resolves to an empty array or `null`, completely preventing cross-tenant data leakage.

36.6.2. Configuration and Defaults

Property	Type	Default	Description
<code>feature.queryGateway.execution.enabled</code>	boolean	<code>false</code>	Guards execution of <code>POST /api/query/find</code> for aggregation queries. When <code>false</code> , aggregation queries return HTTP 501 (<code>ExecutionDisabled</code>). Must be set to <code>true</code> to enable governed aggregation pipeline execution.
<code>feature.queryGateway.maxPageSize</code>	integer	<code>1000</code>	Maximum allowed page size (<code>limit</code>) for <code>/api/query/find</code> requests. Requests exceeding this limit are capped.



`count` and `deleteMany` explicitly reject `expand(...)` queries with HTTP 400 (`InvalidQuery`) because relational hydration is invalid for scalar counting and bulk document deletion.

36.6.3. Request/Response: `/api/query/plan`

Request body

```
{
  "rootType": "com.e2eq.framework.api.query.MultiHopCustomer",
  "query": "expand(orderRef) && expand(orderRef.supplierRef) && status:active"
}
```

Response body

```
{
  "mode": "AGGREGATION",
  "expandPaths": [
    "orderRef",
    "orderRef.supplierRef"
  ]
}
```

36.6.4. Request/Response: /api/query/find (Multi-Hop Aggregation)

Request body (AGGREGATION mode with multi-hop hydration)

```
{
  "rootType": "com.e2eq.framework.api.query.MultiHopCustomer",
  "query": "expand(orderRef) && expand(orderRef.supplierRef) && customerName:Cust-1",
  "page": { "limit": 10, "skip": 0 },
  "sort": [ { "field": "createdDate", "dir": "DESC" } ],
  "realm": "acme"
}
```

Response body (Collection envelope with hydrated nested entities)

```
{
  "offset": 0,
  "limit": 10,
  "rows": [
    {
      "id": "607f1f77bcf86cd799439001",
      "customerName": "Cust-1",
      "orderRef": {
        "id": "607f1f77bcf86cd799439002",
        "orderNumber": "ORD-1001",
        "supplierRef": {
          "id": "607f1f77bcf86cd799439003",
          "supplierName": "Acme Industrial Supplies"
        }
      }
    }
  ],
  "rowCount": 1,
  "filter": "expand(orderRef) && expand(orderRef.supplierRef) && customerName:Cust-1"
}
```

36.6.5. cURL Examples

Plan multi-hop expansion

```
curl -sS -X POST \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <jwt-token>' \
  localhost:8080/api/query/plan \
  -d '{
    "rootType": "com.e2eq.framework.api.query.MultiHopCustomer",
    "query": "expand(orderRef) && expand(orderRef.supplierRef) && status:active"
  }'
```

Execute governed multi-hop find

```
curl -sS -X POST \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <jwt-token>' \
  localhost:8080/api/query/find \
  -d '{
    "rootType": "com.e2eq.framework.api.query.MultiHopCustomer",
    "query": "expand(orderRef) && expand(orderRef.supplierRef) && customerName:Cust-1",
    "page": { "limit": 10, "skip": 0 },
    "realm": "acme"
  }'
```

See also: [Relationship hydration with expand\(path\)](#) and [Query Language Reference](#).

Chapter 37. Security

Authentication and Authorization

Quantum integrates with Quarkus security while providing a pluggable approach to authentication. The repository includes a JWT provider module to get started quickly and an extension surface to replace or complement it.

Chapter 38. JWT Provider

- Module: quantum-jwt-provider
- Purpose: Validate JWTs on incoming requests, populate the security principal, and surface tenant/org/user claims that feed DomainContext.
- Configuration: Standard Quarkus/MicroProfile JWT properties plus custom claim mappings as needed for DataDomain.

Chapter 39. Pluggable Authentication

You can introduce alternative authentication mechanisms (e.g., API keys, SAML/OIDC front-channel tokens exchanged for back-end JWTs, HMAC signatures) by providing CDI beans that integrate with the security layer and emit the same normalized context consumed by `DomainContext`/`RuleContext`.

Typical steps:

1. Implement a request filter or identity provider that validates the token/credential.
2. Map identity and tenant claims into a principal model (`tenantId`, `orgRefName`, `userId`, `roles`).
3. Ensure `BaseResource` (and other entry points) can derive `DomainContext` from that principal.

Chapter 40. Creating an Auth Plugin (using the Custom JWT provider as a reference)

An auth plugin is typically a CDI bean that:

- Extends `BaseAuthProvider` to inherit user-management helpers and persistence utilities.
- Implements `AuthProvider` to integrate with request-time authentication flows.
- Implements `UserManagement` to expose CRUD-style operations for users, passwords, and roles.

A concrete provider should:

- Be annotated as a CDI bean (e.g., `@ApplicationScoped`).
- Provide a stable `getName()` identifier (e.g., "custom", "oidc", "apikey").
- Use config properties for secrets, issuers, token durations, and any external identity provider details.
- Build a Quarkus `SecurityIdentity` with the authenticated principal and roles.

Chapter 41. AuthProvider interface (what a provider must implement)

Core methods:

- `SecurityIdentity validateAccessToken(String token)`
 - Parse and validate the incoming credential (JWT, API key, signature).
 - Return a `SecurityIdentity` with principal name and roles. Throw a security exception for invalid tokens.
- `String getName()`
 - A short identifier for the provider. Persisted alongside credentials and used in logs/metrics.
- `LoginResponse login(String userId, String password)`
 - Credential-based login. Return a structured response:
 - `positiveResponse`: includes `SecurityIdentity`, `roles`, `accessToken`, `refreshToken`, `expirationTime`, and `realm/mongodbUrl` if applicable.
 - `negativeResponse`: includes error codes/reason/message for clients to act on (e.g., password change required).
- `LoginResponse refreshTokens(String refreshToken)`
 - Validate the refresh token, mint a new access token (and optionally a new refresh token), and return a positive response.

Notes:

- Login flow should check `force-change-password` or equivalent flags and return a negative response when user interaction is required before issuing tokens.
- `validateAccessToken` should only accept valid, non-expired tokens and construct `SecurityIdentity` consistently with role mappings used across the platform.

Chapter 42. UserManagement interface (operations your plugin must support)

Typical responsibilities include:

- User lifecycle
 - `String createUser(String userId, String password, Set<String> roles, DomainContext domainContext, [optional] DataDomain)`
 - `void changePassword(String userId, String oldPassword, String newPassword, Boolean forceChangePassword)`
 - `boolean removeUserWithUserId(String userId)`
 - `boolean removeUserWithSubject(String subject)`
- Role management
 - `void assignRolesForUserId(String userId, Set<String> roles)`
 - `void assignRolesForSubject(String subject, Set<String> roles)`
 - `void removeRolesForUserId(String userId, Set<String> roles)`
 - `void removeRolesForSubject(String subject, Set<String> roles)`
 - `Set<String> getUserRolesForUserId(String userId)`
 - `Set<String> getUserRolesForSubject(String subject)`
- Lookups and existence checks
 - `Optional<String> getSubjectForUserId(String userId)`
 - `Optional<String> getUserIdForSubject(String subject)`
 - `boolean userIdExists(String userId)`
 - `boolean subjectExists(String subject)`

Return values and exceptions:

- Throw `SecurityException` or domain-specific exceptions for invalid states (duplicate users, bad password, unsupported hashing).
- Return `Optional` for lookups that may not find a result.
- For removals, return `boolean` to communicate whether a record was deleted.

Chapter 43. Leveraging BaseAuthProvider in your plugin

When you extend BaseAuthProvider, you inherit ready-to-use capabilities that reduce boilerplate:

- Impersonation controls
 - enableImpersonationWithUserId / enableImpersonationWithSubject
 - disableImpersonationWithUserId / disableImpersonationWithSubject
 - These set or clear an impersonation filter script and realm regex that downstream services can honor to act on behalf of another identity under controlled scope.
- Realm override helpers
 - enableRealmOverrideWithUserId / enableRealmOverrideWithSubject
 - disableRealmOverrideWithUserId / disableRealmOverrideWithSubject
 - Useful for multi-realm/tenant scenarios, enabling scoped cross-realm behavior.
- Persistence utilities
 - Built-in use of the credential repository to save, update, and delete credentials.
 - Consistent validation of inputs (non-null checks, non-blank checks).
 - Hashing algorithm guardrails to ensure only supported algorithms are used.

Best practices when deriving:

- Record the auth provider name in stored credentials as informational provenance. It tells callers which provider resolved the credential or token; it is not an authentication routing or authorization binding.
- Reuse the role merge/remove patterns to avoid accidental role loss.
- Prefer emitting precise exceptions (e.g., NotFound for missing users, SecurityException for access violations).

Chapter 44. Implementing your own provider

Checklist:

- Class design
 - `@ApplicationScoped` bean
 - extends `BaseAuthProvider`
 - implements `AuthProvider` and `UserManagement`
 - return a stable `getName()`
- Configuration
 - Externalize secrets (signing keys), issuers, token durations, and realm details via `MicroProfile Config`.
- `SecurityIdentity`
 - Consistently build identities with principal and roles; include useful attributes for auditing/telemetry.
- Tokens/credentials
 - For JWT-like tokens, implement robust parsing, signature verification, expiration checks, and claim validation.
 - For non-JWT credentials (API keys, HMAC), ensure replay protection and scope binding.
- Responses and errors
 - Use structured `LoginResponse` for both success and error paths.
 - Prefer idempotent user/role operations; validate inputs and surface actionable messages.

Chapter 45. CredentialUserIdPassword model and DomainContext

This section explains how user credentials are represented, how those records tie to tenancy and realms, and how the server chooses the database (“realm”) for REST calls.

What the credential model represents

userId

The human-friendly login handle that users type. Must be unique within the applicable tenancy/realm scope.

subject

A stable, system-generated identifier for the principal. Tokens and internal references favor subject over userId because subjects do not change.

description, emailOfResponsibleParty

Optional metadata to describe the credential and provide an owner contact.

domainContext

The tenancy and organization placement of the principal. It contains:

- **tenantId**: Logical tenant partition.
- **orgRefName**: Organization/business unit within the tenant.
- **accountId**: Account or billing identifier.
- **defaultRealm**: The default database/realm used for this identity’s operations.
- **dataSegment**: Optional partitioning segment for advanced sharding or data slicing.

roles

The set of authorities granted (e.g., USER, ADMIN). These become groups/roles on the SecurityIdentity.

issuer

An identifier for who issued the credential or tokens (useful for auditing and multi-provider setups).

passwordHash, hashingAlgorithm

The stored password hash and declared algorithm. Not exposed over REST. Providers verify passwords against this.

forceChangePassword

Flag that forces a password reset on next login; the login flow returns a structured negative response instead of tokens.

lastUpdate

Timestamp for auditing and token invalidation strategies.

area2RealmOverrides

Optional map to route specific functional areas to different realms than the default (e.g., “Reporting” → analytics-realm).

realmRegEx

Optional regex to limit or override which realms this identity may act in; also used by impersonation/override flows.

impersonateFilterScript

Optional script indicating the filter/scope applied during impersonation so actions are constrained.

authProviderName

Informational provenance naming the provider that resolved this credential or token (e.g., “custom”, “oidc”). Provider selection comes from an explicit request override or the ordered `auth.provider` configuration, never from this field.

How DomainContext selects the realm for REST calls

- For each authenticated request, the server derives or retrieves a DomainContext associated with the principal.
- The DomainContext.defaultRealm indicates which backing MongoDB database (“realm”) should be used by repositories for that request.
- If realm override features are enabled (e.g., through provider helpers or per-credential overrides), the system may route certain functional areas to alternate realms using area2RealmOverrides or validated by realmRegEx.
- The remainder of DomainContext (tenantId, orgRefName, accountId, dataSegment) is applied as scope constraints through permission rules and repository filters so reads and writes are automatically restricted to the correct tenant/org segment.

Typical flow

1. Login

- A user authenticates with userId/password (or other mechanism).
- On success, a token is returned alongside role information; the principal is associated with a DomainContext that includes the defaultRealm.

2. Subsequent REST calls

- The token is validated; the server reconstructs SecurityIdentity and DomainContext.
- Repositories choose the datastore for defaultRealm and enforce tenant/org filters using the DomainContext values.
- If the request targets a functional area with a defined override, the operation may route to a different realm for that area alone.

3. UI implications

- The client does not need to know which realm is selected; it simply calls the API. The server

ensures the correct database is used based on DomainContext and any configured overrides.

Best practices

- Keep `userId` immutable once established; use `subject` for internal joins and token subjects.
- Always attach the correct `DomainContext` when creating users to avoid cross-tenant leakage.
- Use realm overrides deliberately for well-isolated areas (e.g., analytics, archiving) and document them for operators.

Chapter 46. Quarkus OIDC out-of-the-box and integrating with common IdPs

Quarkus ships with first-class OpenID Connect (OIDC) support, enabling both service-to-service and browser-based logins.

What the Quarkus OIDC extension provides

- OIDC client and server-side adapters:
 - Authorization Code flow with PKCE for browser sign-in.
 - Bearer token authentication for APIs (validating access tokens on incoming requests).
 - Token propagation for downstream calls (forwarding or exchanging tokens).
- Token verification and claim mapping:
 - Validates issuer, audience, signature, expiration, and scopes.
 - Maps standard claims (sub, email, groups/roles) into the security identity.
- Multi-tenancy and configuration:
 - Supports multiple OIDC tenants via configuration, each with its own issuer, client id/secret, and flows.
- Logout and session support:
 - Front-channel and back-channel logout hooks depending on provider capabilities.

Integrating with common providers

- Works with providers like Keycloak, Auth0, Okta, Azure AD, Cognito, and enterprise IdPs exposing OIDC.
- Configure the issuer URL and client credentials. Quarkus discovers endpoints via the provider's .well-known/openid-configuration.
- For roles/permissions, map provider groups/roles claims to your platform roles in the identity.

OIDC vs OAuth vs OpenID (terminology and evolution)

OAuth 2.0

- Authorization framework for delegated access (scopes), not authentication. Defines flows to obtain access tokens for APIs.

OpenID (OpenID 1.x/2.0)

- Older federated identity protocol that preceded OIDC. It has been superseded by OpenID Connect.

OpenID Connect (OIDC)

- An identity layer on top of OAuth 2.0. Adds standardized authentication, user info endpoints, ID tokens (JWT) with subject and profile claims, and discovery metadata.
- In practice, OIDC is the modern standard for SSO and user authentication; OAuth remains

the authorization substrate underneath.

Summary

- OpenID → historical, replaced by OIDC.
- OAuth 2.0 → authorization framework.
- OIDC → authentication (identity) layer built on OAuth 2.0.

OIDC and SAML in relation to SSO SAML (Security Assertion Markup Language)::

- XML-based federation protocol widely used in enterprises for browser SSO.
- Uses signed XML assertions transported through browser redirects/posts.

OIDC

- JSON/REST-oriented, uses JWTs, and is well-suited for modern SPAs and APIs.
- Relationship:
 - Both enable SSO and federation across identity providers and service providers.
 - Many enterprise IdPs support both; OIDC is generally simpler for APIs and modern web stacks, while SAML is entrenched in legacy/enterprise SSO.
- Bridging:
 - Gateways or identity brokers can translate SAML assertions to OIDC tokens and vice versa, allowing gradual migration.

Common customer IdP models and OIDC integration patterns

- Centralized IdP (single-tenant):
 - One organization-wide IdP issues tokens for all users.
 - Configure a single OIDC tenant in Quarkus; map groups/roles to application roles.
- Multi-tenant SaaS with per-tenant IdP:
 - Each customer brings their own IdP (BYOID).
 - Configure Quarkus OIDC multitenancy with per-tenant issuer discovery and client credentials.
 - Tenant selection can be based on domain, request header, or path; the selected OIDC tenant performs login and token validation.
- Brokered identity:
 - Use a broker (e.g., a central identity layer) that federates to multiple upstream IdPs (OIDC, SAML).
 - Quarkus integrates with the broker as a single OIDC client; the broker handles IdP routing and protocol translation.
- Hybrid API and web flows:
 - Browser apps use Authorization Code flow with sessions; APIs use bearer token authentication.
 - Quarkus OIDC extension can handle both in the same application when properly configured.

Best practices

- Prefer OIDC for new integrations; use SAML through a broker if enterprise constraints require it.
- Normalize roles/claims server-side so downstream authorization (RuleContext, repositories) sees consistent group names regardless of IdP.
- Use token exchange or client credentials for service-to-service calls; do not reuse end-user tokens where not appropriate.
- For multi-tenant OIDC, secure tenant resolution logic and validate issuer/tenant binding to prevent mix-ups.

Chapter 47. Authorization via RuleContext

Authentication establishes identity; RuleContext enforces what the identity can do. For each action (CREATE, UPDATE, VIEW, DELETE, ARCHIVE), RuleContext can:

- Allow or deny the action
- Contribute additional filters (e.g., org scoping, functional-area specific sharing)
- Adjust UIActionList to reflect permitted next steps

This division of responsibilities keeps providers focused on identity while policies remain centralized in RuleContext.

Chapter 48. Generating JWT Tokens for API Access

Applications frequently need machine-to-machine access that does not go through an interactive login flow. The framework provides two mechanisms: **short-lived user tokens** generated via the login endpoint, and **long-lived service tokens** generated via the dedicated service-token endpoint. Both produce standard RS256-signed JWTs that carry identity, roles, and scope claims consumed by the `SecurityFilter` and the permission system.

48.1. Token Anatomy

Every JWT produced by the framework contains the following claims:

Claim	Example	Purpose
<code>sub</code>	<code>a1b2c3d4-...</code> (UUID)	The subject — a stable identifier for the principal. For user tokens, this is the user's <code>subject</code> from the credential store. For service tokens, a newly generated UUID.
<code>iss</code>	<code>https://myapp.example.com/issuer</code>	Issuer — must match <code>mp.jwt.verify.issuer</code> in the consuming application's config for the token to be accepted.
<code>aud</code>	<code>b2bi-api-client</code>	Audience — the intended recipient. Auth tokens use <code>b2bi-api-client</code> ; refresh tokens use <code>b2bi-api-client-refresh</code> .
<code>groups</code>	<code>["admin", "user"]</code>	Roles embedded in the token. These are merged with roles from the credential store and user group membership at validation time.
<code>scope</code>	<code>authToken</code>	Distinguishes auth tokens (<code>authToken</code>) from refresh tokens (<code>refreshToken</code>).
<code>iat</code>	<code>1707849600</code>	Issued-at timestamp (seconds since epoch).
<code>exp</code>	<code>1707853200</code>	Expiration timestamp (seconds since epoch).

The token is signed with the application's RSA private key using the RS256 algorithm. By default, the key is loaded from `privateKey.pem` on the classpath (provided by the `quantum-default-keys` module during development). In production, configure `quantum.jwt.private-key-location` to point at an externally managed key. Consumers validate the signature using the corresponding public key (`publicKey.pem` or the configured `quantum.jwt.public-key-location`).

48.2. Generating User Tokens via Login

The standard way to obtain a token is through the login endpoint:

```
curl -sS -X POST \  
  'http://localhost:8080/auth/login?userId=admin@example.com&password=secret'
```


Response:

```
{
  "accessToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6...\"",
  "refreshToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6...\""
}
```

The access token expires after the configured duration (`com.b2bi.jwt.duration` in seconds). Use the refresh token to obtain a new access token without re-authenticating:

```
curl -sS -X POST \
  -H 'Authorization: Bearer <refresh-token>' \
  'http://localhost:8080/auth/refresh'
```

This flow is appropriate for interactive clients (UIs, CLI tools) that can handle periodic token refresh.

48.3. Service Tokens for System-to-System Integration

For integrations that cannot perform interactive login — MCP servers, cron jobs, webhook handlers, CI/CD pipelines, partner APIs — the framework provides **service tokens**: long-lived JWTs with a dedicated credential record.

48.3.1. Generating a Service Token

```
curl -sS -X POST \
  -H 'Authorization: Bearer <admin-access-token>' \
  -H 'Content-Type: application/json' \
  http://localhost:8080/auth/service-token \
  -d '{
    "roles": ["user"],
    "expirationSeconds": 31536000,
    "description": "MCP server integration token"
  }'
```

Alternatively, you can use the **Quantum CLI** tool to generate service tokens interactively or via command-line arguments.

Using the Quantum CLI

The Quantum CLI (`quantum-cli` module) provides a `generate-token` command that handles authentication and token generation in a single step.

Build the CLI:

```
mvn clean package -pl quantum-cli -am
```

The executable JAR is located at `quantum-cli/target/quarkus-app/quarkus-run.jar`.

Usage:

```
java -jar quantum-cli/target/quarkus-app/quarkus-run.jar \
  --user admin@company.com \
  --roles admin,system \
  --expires -1 \
  --description "MCP Server Token" \
  --url http://localhost:8080
```

Command Options:

- `-u, --user`: (Required) The User ID to authenticate as.
- `-p, --password`: (Required) The password for authentication. You will be prompted for it if not provided.
- `-r, --roles`: Comma-separated list of roles to embed in the token.
- `-e, --expires`: Expiration in seconds. Use `-1` for a non-expiring token.
- `-d, --description`: Optional description for the service token.
- `--url`: The base URL of the Quantum API (defaults to `http://localhost:8080`).

Response:

```
{
  "accessToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6I..",
  "issuer": "https://myapp.example.com/issuer",
  "subject": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "credentialType": "SERVICE_TOKEN"
}
```

48.3.2. Request Parameters

Field	Type	Required	Description
<code>roles</code>	string array	Yes	Roles to embed in the token and store on the credential (e.g., <code>["user"]</code> , <code>["admin"]</code> , <code>["agent-read"]</code>).
<code>expirationSeconds</code>	long	No	Seconds until expiry. When <code>null</code> , the token is issued for approximately 100 years (effectively non-expiring).
<code>description</code>	string	No	Human-readable label for the credential (e.g., <code>"Nightly sync job"</code> , <code>"Claude MCP integration"</code>).

48.3.3. What Happens on the Backend

When you call `POST /auth/service-token`, the framework:

1. **Validates the caller**—the endpoint requires `admin` or `system` role (`@RolesAllowed({"admin", "system"})`). Only administrators can create service tokens.
2. **Looks up the caller's credential**—from the JWT `sub` claim, the framework finds the caller's `CredentialUserIdPassword` in the system realm.
3. **Creates a new `SERVICE_TOKEN` credential** with:
 - `userId: svc:<uuid>` (auto-generated, not user-controlled)
 - `subject`: A new UUID
 - `credentialType: SERVICE_TOKEN`
 - `parentCredentialSubject`: The caller's subject (for audit trail)
 - `roles`: From the request
 - `domainContext`: Inherited from the caller's credential (same realm, tenant, org)
 - `issuer`: The configured `mp.jwt.verify.issuer`
4. **Saves the credential** to MongoDB.
5. **Links it to the caller's UserProfile** via `additionalCredentialRefs` so the token can be traced to its creator.
6. **Signs and returns the JWT**.

48.3.4. Using a Service Token

The service token is used like any other JWT—pass it in the `Authorization` header:

```
curl -sS -X GET \  
  -H 'Authorization: Bearer <service-token>' \  
  http://localhost:8080/api/query/rootTypes
```

The `SecurityFilter` validates the token, looks up the `SERVICE_TOKEN` credential by its `sub` claim, and builds a `PrincipalContext` with the credential's `DomainContext`, roles, and realm—exactly as it would for a user token.

48.3.5. Example: Configuring an MCP Server with a Service Token

Generate a token for your MCP integration:

```
# Login as admin first  
ACCESS_TOKEN=$(curl -sS -X POST \  
  'http://localhost:8080/auth/login?userId=admin@mycompany.com&password=admin-secret'  
  \  
  | jq -r '.accessToken')
```

```
# Generate a service token with read-only roles
SERVICE_TOKEN=$(curl -sS -X POST \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H 'Content-Type: application/json' \
  http://localhost:8080/auth/service-token \
  -d '{
    "roles": ["user"],
    "expirationSeconds": 31536000,
    "description": "Claude MCP integration - read-only"
  }' | jq -r '.accessToken')

echo "Service token: $SERVICE_TOKEN"
```

Then configure Claude Desktop to use it via a bridge that includes the token:

```
{
  "mcpServers": {
    "quantum": {
      "command": "node",
      "args": ["/path/to/quantum-mcp-bridge/dist/index.js"],
      "env": {
        "QUANTUM_BASE_URL": "https://api.mycompany.com",
        "QUANTUM_AUTH_TOKEN": "<paste-service-token-here>"
      }
    }
  }
}
```

For the native MCP server (when the Quantum app itself serves `/mcp`), authentication depends on your deployment. In development, the MCP endpoint may be accessible without a token if you configure it appropriately. In production, configure your MCP client or reverse proxy to include the service token as a Bearer header.

48.4. How Tokens Scope to Security Policies

A token does not grant unlimited access. Every API call passes through the same security pipeline regardless of whether it was made by a human user, an MCP client, or a service integration.

48.4.1. Token-to-Policy Flow

```
JWT arrives in Authorization header
|
v
SecurityFilter parses JWT
|
v
Extract "sub" claim -> look up CredentialUserIdPassword in system realm
```

```

|
v
Build PrincipalContext:
- userId (from credential)
- roles (merged from TOKEN + CREDENTIAL + USERGROUP)
- dataDomain (tenantId, orgRefName, accountNum, ownerId)
- defaultRealm (from credential's DomainContext)
- area2RealmOverrides (optional per-area realm routing)
|
v
Determine ResourceContext:
- area (from @FunctionalMapping, e.g., "integration")
- functionalDomain (e.g., "query")
- action (from @FunctionalAction or HTTP method, e.g., "find")
|
v
RuleContext.checkRules(principalContext, resourceContext):
- Match rules by identity (role or userId)
- Match by area, functionalDomain, action
- Apply DataDomain constraints (tenantId, orgRefName, etc.)
- Return ALLOW / DENY with winning rule name
|
v
If DENY -> HTTP 403 with decision details
If ALLOW (EXACT) -> proceed, no data filters
If ALLOW (SCOPED) -> proceed with data-level filters applied at repo layer

```

48.4.2. Role Merging

When a token is validated, the effective roles come from three sources:

1. **TOKEN:** The **groups** claim in the JWT itself.
2. **CREDENTIAL:** The **roles** array stored on the **CredentialUserIdPassword** document in MongoDB.
3. **USERGROUP:** Roles inherited from **UserGroup** membership, resolved via the user's **UserProfile** in the target realm.

The union of all three sources becomes the effective role set for the request.

48.4.3. Data Scoping via DataDomain

The credential's **DomainContext** determines which tenant, organization, and account the token operates within. This translates to a **DataDomain** that is applied as query filters at the repository layer:

- **tenantId:** Isolates data by tenant. A service token created by an **acme-corp** admin can only access **acme-corp** data.
- **orgRefName:** Scopes to an organization within the tenant.
- **accountNum:** Scopes to an account.

- **ownerId**: Set to the credential's `userId`; used for owner-based access rules.

These constraints are enforced automatically by Morphia repositories and cannot be bypassed by the token holder.

48.4.4. Realm Override with `X-Realm`

If the service token's credential has `authorizedRealms` or a matching `realmRegEx`, the caller can pass `X-Realm: <target-realm>` to operate in a different realm. The `SecurityFilter` validates that the credential is authorized for the target realm, then rebuilds the `PrincipalContext` with the target realm's `DomainContext`.

```
curl -sS -X POST \  
  -H 'Authorization: Bearer <service-token>' \  
  -H 'X-Realm: acme-corp-staging' \  
  -H 'Content-Type: application/json' \  
  http://localhost:8080/api/agent/execute \  
  -d '{"tool": "query_find", "arguments": {"rootType": "Order", "query":  
  "status:OPEN"}}'
```

48.4.5. Limiting Service Token Scope

To follow the principle of least privilege:

- **Assign minimal roles** — give the service token only the roles it needs. If it only reads data, do not include `admin`.
- **Use per-tenant agent configuration** — configure `quantum.agent.tenant.<realm>.enabledTools` to restrict which gateway tools the agent can call.
- **Set `maxFindLimit`** — cap query results with `quantum.agent.tenant.<realm>.maxFindLimit`.
- **Use `expirationSeconds`** — even for long-lived tokens, set an expiration. A 1-year token (31536000 seconds) is preferable to an effectively infinite one.
- **Monitor and audit** — the `parentCredentialSubject` on `SERVICE_TOKEN` credentials links every service token to the admin who created it. Token validation logs include the `userId` and `realm` for each request.

48.5. Generating RSA Key Pairs

The framework signs tokens with an RSA key pair. To generate a new key pair:

```
# Generate a 2048-bit RSA private key in PKCS#8 format  
openssl genpkey -algorithm RSA -out privateKey.pem -pkeyopt rsa_keygen_bits:2048  
  
# Extract the public key  
openssl rsa -pubout -in privateKey.pem -out publicKey.pem
```



The private key must be in PKCS#8 PEM format (begins with -----BEGIN PRIVATE KEY-----). PKCS#1 format (-----BEGIN RSA PRIVATE KEY-----) is not supported. If you have a PKCS#1 key, convert it:

```
openssl pkcs8 -topk8 -inform PEM -outform PEM -nocrypt -in rsa_private.pem -out privateKey.pem
```

48.5.1. Where to Place the Keys

There are three options, from simplest to most production-ready:

Option 1: Use the default keys (development only)

Add the `quantum-default-keys` module to your application POM. This places a pre-generated key pair on the classpath automatically — no files to copy.

```
<dependency>
  <groupId>com.end2endlogic</groupId>
  <artifactId>quantum-default-keys</artifactId>
  <version>${quantum.version}</version>
</dependency>
```



The default keys are published in the open-source repository. Never use them in production.

Option 2: Classpath keys (simple deployments)

Place `privateKey.pem` and `publicKey.pem` in `src/main/resources/` of your application module. The framework loads them from the classpath at startup and caches them in memory.

Option 3: External keys via configuration (recommended for production)

Store the keys outside the application artifact and configure their locations in `application.properties`:

```
quantum.jwt.private-key-location=file:/etc/quantum/keys/privateKey.pem
quantum.jwt.public-key-location=file:/etc/quantum/keys/publicKey.pem

# SmallRye JWT validation also needs the public key path
mp.jwt.verify.publickey.location=/etc/quantum/keys/publicKey.pem
```

This approach allows key rotation without rebuilding or redeploying the application. See [Key Pair Management](#) for the full configuration reference.

48.6. Configuration Reference

Property	Default	Description
<code>auth.provider</code>	(required)	Comma-separated list of provider names (e.g., <code>custom</code> , <code>custom,oidc</code>). See Multiple Auth Providers .
<code>mp.jwt.verify.publickey.location</code>	(required)	Classpath location of the RSA public key PEM file (e.g., <code>publicKey.pem</code>).
<code>mp.jwt.verify.issuer</code>	(required)	Expected issuer claim in JWTs. Must match the issuer used during token generation.
<code>mp.jwt.verify.audiences</code>	(required)	Comma-separated list of accepted audience values (e.g., <code>b2bi-api-client</code> , <code>b2bi-api-client-refresh</code>).
<code>com.b2bi.jwt.duration</code>	<code>10000</code>	Access token duration in seconds for the custom JWT provider.
<code>auth.jwt.secret</code>	(required)	Secret for HMAC-based operations. Use an environment variable: <code>\${JWT_SECRET}</code> .
<code>auth.jwt.expiration</code>	<code>15</code>	Fallback access token expiration in minutes.
<code>auth.jwt.refresh-expiration</code>	<code>30</code>	Refresh token expiration in minutes.
<code>quantum.jwt.private-key-location</code>	<i>(unset — defaults to classpath <code>privateKey.pem</code>)</i>	Path to the RSA private key for token signing. Supports <code>file:</code> , <code>classpath:</code> , or bare classpath resource names. See Key Pair Management .
<code>quantum.jwt.public-key-location</code>	<i>(unset — defaults to classpath <code>publicKey.pem</code>)</i>	Path to the RSA public key for token verification. Same prefix conventions as the private key property. NOTE: also update <code>mp.jwt.verify.publickey.location</code> to match.

Multiple Authentication Providers

Quantum supports running multiple authentication providers simultaneously, enabling scenarios such as an internal JWT provider alongside an external OIDC provider (for example Cognito, Keycloak, or Auth0). Providers are registered via a single configuration property, and the framework automatically routes token validation to the correct provider based on the JWT `iss` (issuer) claim.

Chapter 49. How It Works

The `AuthProviderFactory` is the central registry for all configured authentication providers. At startup it reads the `auth.provider` property, which accepts a comma-separated list of provider names, instantiates each one via CDI, and holds them in an ordered list.

```
# Single provider (default)
auth.provider=custom

# Multiple providers
auth.provider=custom,oidc
```

49.1. Provider Resolution

When a request arrives with a JWT, the framework resolves which provider should validate it:

1. **By issuer** — `AuthProviderFactory.getProviderForIssuer(String issuer)` iterates through all configured providers and calls `getIssuer()` on each. The first provider whose issuer matches the JWT `iss` claim wins.
2. **By name** — `AuthProviderFactory.getProviderByName(String name)` performs a case-insensitive lookup (for example `"custom"` or `"oidc"`).
3. **Default** — `AuthProviderFactory.getAuthProvider()` returns the first configured provider. Unknown or missing issuer routing fails closed; callers that intentionally want the default provider must request it explicitly.

For user-facing login, an explicit `provider` request may contain a comma-separated ordered list. When omitted, the ordered `auth.provider` configuration is used. Every provider named by either list must be available; configuration errors are not silently skipped.

49.2. Canonical Identity Service

`CanonicalIdentityService` is the single entry point for token validation across all providers. It has two signatures:

```
// Use the default provider
SecurityIdentity validateAccessTokenCanonical(String token);

// Route to the provider matching this issuer
SecurityIdentity validateAccessTokenCanonical(String token, String issuer);
```

Internally, the service supports two provider interfaces:

- **ClaimsAuthProvider** — a modern SPI where the provider returns a `ProviderClaims` object (subject, roles, attributes). The service then passes those claims to `IdentityAssembler`, which resolves the `userId` from the credential store, merges roles from the token, credential, and user groups, and

builds a canonical `QuarkusSecurityIdentity`.

- **Legacy `AuthProvider`** —the provider returns a `SecurityIdentity` directly, and the service normalizes it via `SecurityIdentityNormalizer` (ensuring the principal is always the `userId`, not the IdP subject).

This means you can mix modern claims-based providers with legacy ones in the same application.

Chapter 50. Configuring `application.properties`

50.1. Minimal Custom JWT Provider

```
# Select the custom JWT provider
auth.provider=custom

# Enable SmallRye JWT validation
quarkus.smallrye-jwt.enabled=true

# Public key for token signature verification (classpath resource)
mp.jwt.verify.publickey.location=publicKey.pem

# Issuer claim that tokens must contain
mp.jwt.verify.issuer=https://myapp.example.com/issuer

# Audiences the token must target
mp.jwt.verify.audiences=my-api-client,my-api-client-refresh

# Signing secret for the custom provider (used for HMAC-based flows)
auth.jwt.secret=${JWT_SECRET:change-me-in-production}

# Token lifetimes (minutes)
auth.jwt.expiration=15
auth.jwt.refresh-expiration=30
```

50.2. Adding an OIDC Provider (e.g., AWS Cognito)

To add an external OIDC provider alongside the custom provider:

```
# Enable both providers
auth.provider=custom,oidc

# --- Custom JWT provider (unchanged) ---
quarkus.smallrye-jwt.enabled=true
mp.jwt.verify.publickey.location=publicKey.pem
mp.jwt.verify.issuer=https://myapp.example.com/issuer
mp.jwt.verify.audiences=my-api-client
auth.jwt.secret=${JWT_SECRET}
auth.jwt.expiration=15
auth.jwt.refresh-expiration=30

# --- OIDC provider ---
quarkus.oidc.enabled=true
quarkus.oidc.auth-server-url=https://cognito-
```

```

idp.${aws.cognito.region}.amazonaws.com/${aws.cognito.user-pool-id}
quarkus.oidc.client-id=${aws.cognito.clientId}
quarkus.oidc.token.issuer=https://cognito-
idp.${aws.cognito.region}.amazonaws.com/${aws.cognito.user-pool-id}
quarkus.oidc.roles.role-claim-path=cognito:groups
auth.oidc.provider-name=oidc
auth.oidc.role-claim-path=cognito:groups
auth.oidc.username-claim=preferred_username

# Cognito environment variables
aws.cognito.user-pool-id=${USER_POOL_ID}
aws.cognito.client-id=${APPLICATION_CLIENT_ID}
aws.cognito.region=us-east-1
aws.cognito.jwks.url=https://cognito-
idp.${aws.cognito.region}.amazonaws.com/${aws.cognito.user-pool-id}/.well-
known/jwks.json

```

50.3. Using Keycloak or Auth0

The pattern is the same — configure `quarkus.oidc.*` to point at the provider's OIDC discovery endpoint:

```

# Keycloak
quarkus.oidc.auth-server-url=https://keycloak.example.com/realms/my-realm
quarkus.oidc.client-id=my-client
quarkus.oidc.credentials.secret=${KEYCLOAK_CLIENT_SECRET}
quarkus.oidc.roles.role-claim-path=realm_access/roles

# Auth0
quarkus.oidc.auth-server-url=https://my-tenant.auth0.com
quarkus.oidc.client-id=my-client-id
quarkus.oidc.credentials.secret=${AUTH0_CLIENT_SECRET}
quarkus.oidc.roles.role-claim-path=https://my-app.example.com/roles

```

Quarkus OIDC automatically discovers the `.well-known/openid-configuration` endpoint from the `auth-server-url`, so you do not need to specify individual endpoints for token validation, JWKS, or user info.

Chapter 51. Issuer Resolution in Detail

When a JWT arrives at the `SecurityFilter`, the following resolution path executes:

```
Incoming request with Authorization: Bearer <token>
|
v
SecurityFilter extracts JWT
|
v
Parse the "iss" claim from the token
|
v
CanonicalIdentityService.validateAccessTokenCanonical(token, issuer)
|
v
AuthProviderFactory.getProviderForIssuer(issuer)
|
+-- Iterate configured providers:
|   provider.getIssuer().equals(issuer) ?
|   -> yes: use this provider
|   -> no: try next
|
+-- No match: fail provider resolution
|
v
Provider validates token and returns ProviderClaims or SecurityIdentity
|
v
IdentityAssembler merges roles from:
- TOKEN (from JWT claims)
- CREDENTIAL (from credential store)
- USERGROUP (from UserProfile -> UserGroup membership)
|
v
Canonical SecurityIdentity with userId as principal
```

Each provider's `getIssuer()` method returns its expected issuer string. For the custom JWT provider, this is the value of `mp.jwt.verify.issuer`. For an OIDC provider, it is typically the `auth-server-url` (for example, <https://cognito-idp.us-east-1.amazonaws.com/{pool-id}>).

51.1. Role Provenance Tracking

After token validation, the framework tracks the source of every role via `RoleAssignment`:

Source	Meaning
TOKEN	Role came from the JWT claims (e.g., Cognito <code>cognito:groups</code> or Keycloak <code>realm_access/roles</code>)
CREDENTIAL	Role stored in the credential record in MongoDB
USERGROUP	Role inherited from <code>UserGroup</code> membership via the user's <code>UserProfile</code>

The final `SecurityIdentity` contains the union of all three sources. Login and token-refresh responses include a `List<RoleAssignment>` so clients and audit logs can trace exactly where each role originated.

Chapter 52. Creating a New Auth Provider

To add a new authentication provider:

1. Create an `@ApplicationScoped` CDI bean that extends `BaseAuthProvider` and implements `AuthProvider` and optionally `ClaimsAuthProvider` or `UserManagement` as needed.
2. Return a stable name from `getName()` (e.g., "saml", "apikey", "azure-ad").
3. Implement `getIssuer()` to return the issuer string that your tokens will carry.
4. Implement `validateAccessToken(token)` to parse, verify, and build a `SecurityIdentity`.
5. Optionally implement `ClaimsAuthProvider.validateTokenToClaims(token)` for the modern claims-based flow.
6. Add your provider name to `auth.provider` in `application.properties`.

```
@ApplicationScoped
public class MyOidcProvider extends BaseAuthProvider
    implements AuthProvider, ClaimsAuthProvider, UserManagement {

    @ConfigProperty(name = "my.oidc.issuer")
    String issuer;

    @Override
    public String getName() {
        return "my-oidc";
    }

    @Override
    public String getIssuer() {
        return issuer;
    }

    @Override
    public ProviderClaims validateTokenToClaims(String token) {
        // Validate token against the OIDC provider's JWKS
        // Extract subject, roles, attributes
        // Return ProviderClaims
    }

    // ... implement remaining AuthProvider and UserManagement methods
}
```

Then activate it:

```
auth.provider=custom,my-oidc
my.oidc.issuer=https://my-idp.example.com
```


See [Authentication and Authorization](#) for the full `AuthProvider` and `UserManagement` interface contracts and `BaseAuthProvider` capabilities.

Chapter 53. Key Pair Management

53.1. Default Keys Module (**quantum-default-keys**)

The framework ships a convenience module, **quantum-default-keys**, that places a default RSA key pair (**privateKey.pem** / **publicKey.pem**) on the classpath. This module is declared as an **<optional>true</optional>** dependency of **quantum-jwt-provider**, so it is available during framework development but does **not** propagate transitively to consuming applications.

To use the default keys in your application during development or testing, add the dependency explicitly:

```
<dependency>
  <groupId>com.end2endlogic</groupId>
  <artifactId>quantum-default-keys</artifactId>
  <version>${quantum.version}</version>
</dependency>
```

In production, either:

- **Remove** the **quantum-default-keys** dependency from your POM and supply your own keys externally, or
- **Override** the key locations via **application.properties** (see below) — the configured paths take precedence over any classpath defaults.



The default keys are published in the open-source framework repository and are intended only for development and testing. Never use the default keys in production. Always generate and deploy your own key pair.

53.2. Configurable Key Locations

By default, **TokenUtils** loads keys from classpath resources named **privateKey.pem** and **publicKey.pem**. You can override these locations in **application.properties** to point at external key files:

```
# Load signing key from the filesystem
quantum.jwt.private-key-location=file:/opt/keys/signing.pem

# Load verification key from the filesystem
quantum.jwt.public-key-location=file:/opt/keys/verify.pem
```

Supported path prefixes:

Prefix	Behavior
<code>file:</code>	Load from the filesystem (e.g., <code>file:/opt/keys/signing.pem</code>)
<code>classpath:</code>	Load from the classpath (e.g., <code>classpath:mykeys/signing.pem</code>)
<i>(no prefix)</i>	Treated as a classpath resource name (backward compatible with <code>privateKey.pem</code>)

When these properties are set, `TokenUtilsConfigurer` applies them at application startup and invalidates any previously cached keys.

53.2.1. Typical Production Setup

```
# Point at externally managed keys – no quantum-default-keys JAR needed
quantum.jwt.private-key-location=file:/etc/quantum/keys/privateKey.pem
quantum.jwt.public-key-location=file:/etc/quantum/keys/publicKey.pem

# SmallRye JWT validation must also reference the public key
mp.jwt.verify.publickey.location=/etc/quantum/keys/publicKey.pem
```



`TokenUtils` handles signing (private key); SmallRye JWT handles verification (public key via `mp.jwt.verify.publickey.location`). Both must reference the same key pair. When you move to external keys, update both the `quantum.jwt.*` properties and `mp.jwt.verify.publickey.location`.

53.3. Full Per-Tenant Design

For the complete multi-phase design including per-tenant MongoDB key storage, key rotation, JWKS endpoints, and vault integration, see the [Per-Tenant Key Pairs Design Document](#).

Permissions: Rule Bases, SecurityURIHeader, and SecurityURIBody

This section explains how Quantum evaluates permissions for REST requests using rule bases that match on a SecurityURI composed of a header and a body. The header includes identity, area, functionalDomain, and action. The body includes realm, accountNumber, tenantId, dataSegment, ownerId, and resourceId. It also covers how identities and roles (as found on userProfile or credentialUserIdPassword) are matched, how priority works, and how multiple matching rule bases are evaluated.

Chapter 54. Introduction: Layered Enforcement Overview

Quantum evaluates "can this identity do X?" through three complementary layers. Understanding them in order helps you pick the right tool for the job and combine them safely:

1. REST API annotations (top layer, code-level)

- What: JAX-RS/Jakarta Security annotations on resource methods, for example, `@RolesAllowed("ADMIN")`, `@PermitAll`, `@DenyAll`, `@Authenticated`.
- Purpose: Coarse-grained, immediate gates right at the endpoint. Ideal for baseline protections (for example, only ADMIN may call `/admin/**`) and for non-dynamic constraints that rarely change.
- Pros: Simple, fast, visible in code reviews.
- Cons: Hard-coded; changing access requires a code change, build, and deploy. No data-aware scoping (for example, cannot express tenant/domain filters).

2. Feature flags (exposure and variants)

- What: Turn capabilities on/off per environment or cohort and select variants (A/B, multivariate). See "Feature Flags, Variants, and Target Rules" below.
- Purpose: Control who even sees or can reach a capability during rollout (by tenant, role, geography, plan), independently of authorization. Flags answer "is the feature ON and which variant?"; they do not by themselves prove the caller is authorized.
- Pros: Reversible, environment-aware, safe rollout and experimentation.
- Cons: Not a substitute for authorization; must be paired with roles/permission rules for enforcement.

3. Permission Rules with SecurityURI (fine-grained, dynamic)

- What: Declarative rule bases authored against a SecurityURI composed of header (identity, area, functionalDomain, action) and body (realm, accountNumber, tenantId, dataSegment, ownerId, resourceId). Rules decide ALLOW or DENY and may use an optional postconditionScript for additional checks. See sections "Key Concepts", "Matching Algorithm", and examples below.
- Purpose: Express least-privilege, data-aware policies that evolve without code changes (data-driven authoring).
- Pros: Dynamic, auditable, supports role-based matching and simple attribute scoping via the SecurityURI body.
- Cons: Requires governance of rulebases and careful priority management.

How roles, Functional Areas, and Functional Domains fit in

- Roles: Used by both layers (1) and (3). Annotations directly reference roles. In the rule engine, roles are evaluated by treating each role as an identity: rules authored for a role name (for example, "ADMIN") are considered alongside rules authored for the user's own userId. Effective

roles are resolved by merging IdP roles with roles on the user record; see "How roles are defined for an identity" below.

- **Functional Area/Domain:** Derived from the URL convention `/ {area} / {functionalDomain} / {action}` as parsed by `SecurityFilter.determineResourceContext`. Author policies using these fields to target business capabilities rather than raw URLs or ad-hoc headers.
- **DataDomain:** When rules ALLOW an action, they can attach data-scope filters (tenant/org/owner) so downstream reads/writes are constrained to the caller's domain.

Choosing the right approach

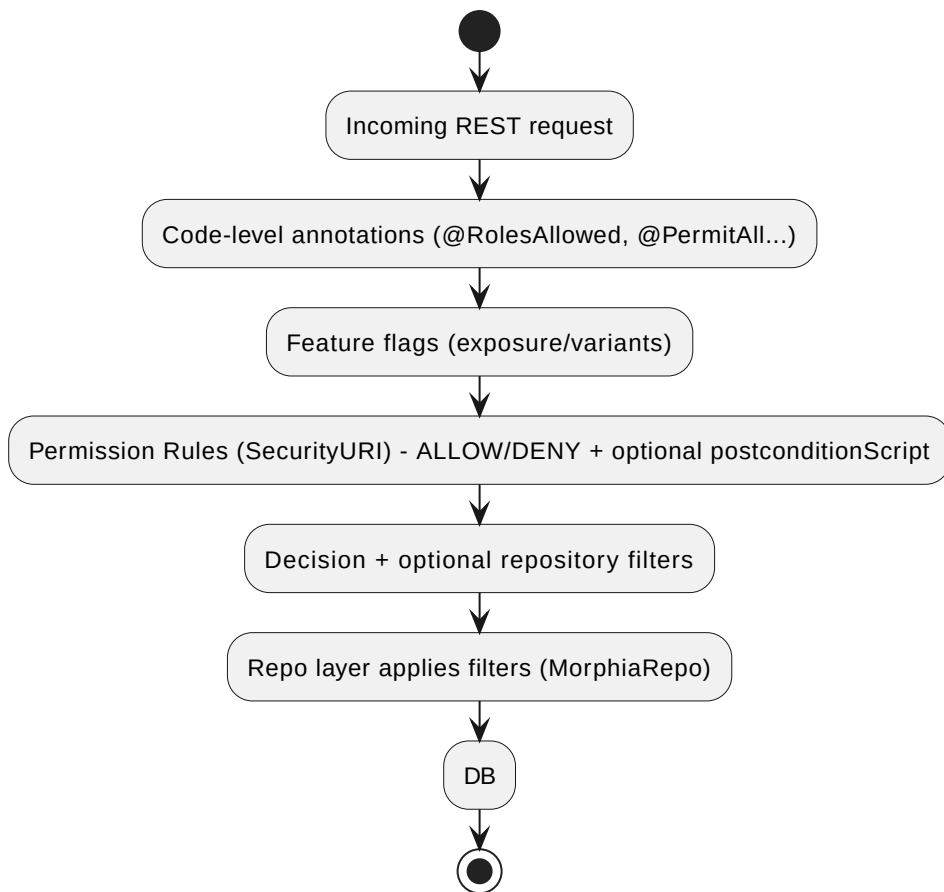
- Use annotations for stable, coarse gates you want visible in code (for example, admin-only endpoints, health endpoints with `@PermitAll`).
- Use feature flags to manage rollout/exposure and variants across environments and cohorts.
- Use permission rules to encode fine-grained, data-aware authorization and to evolve policy without redeploying.

Compare and contrast

- Annotation-based controls are compile-time and hard-code policy into the service; changing them requires code changes.
- Permission Rules and the Rule Language are data-driven and user-changeable (with proper governance), enabling rapid, auditable policy changes and DataDomain scoping.
- In practice: apply annotations as the first gate, evaluate feature flags to determine exposure/variant, then evaluate permission rules to decide ALLOW/DENY and attach scopes. This layered approach yields both safety and agility.



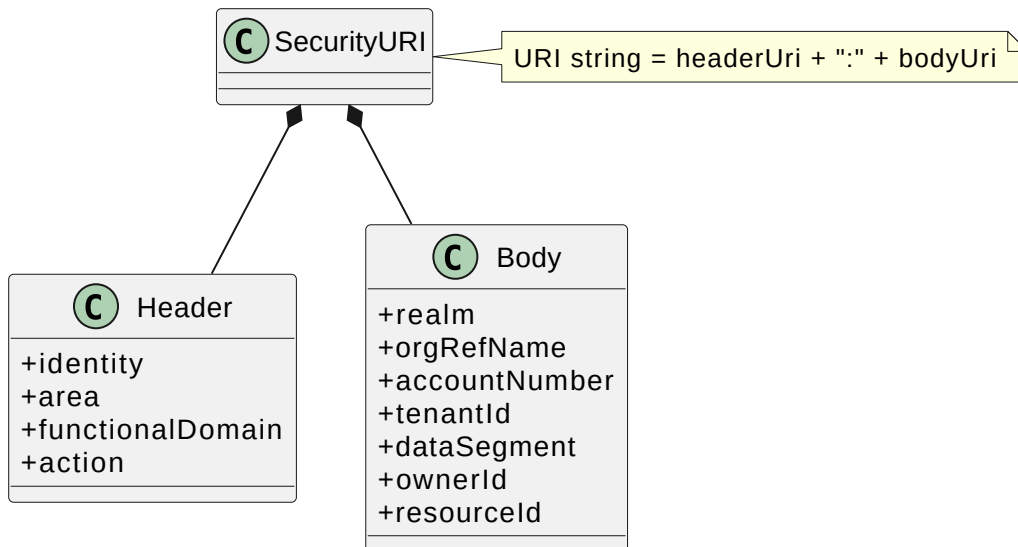
Ontology- and label-aware policy helpers are available to `postconditionScript` when the optional ontology module is enabled. Helpers include `hasEdge`, `hasAnyEdge`, `hasAllEdges`, `relatedIds` for graph checks; `hasLabel` for label checks; and `isA/noViolations` for type/validation contexts. See the "Script Helpers reference" section for details. When data access must be restricted to lists resolved outside the rule engine (for example, from an external ACL service), use `AccessListResolvers` (SPI) and reference their outputs from `andFilterString/orFilterString`.



Chapter 55. Key Concepts

- Identity: The authenticated principal, typically originating from JWT or another provider. It includes:
 - `userId` (or `credentialUserIdPassword` username)
 - roles (authorities/groups)
 - `tenantId`, `orgRefName`, optional realm, and other claims that contribute to `DomainContext`
- `userProfile`: A domain representation of the user that adds human information such as first name, last name, email address, phone number and provides a linkage back to identity, roles, and policy decorations (feature flags, plans, expiration, etc.).
- Rule Base (Permission Rule): A declarative rule with matching criteria and an effect (ALLOW or DENY). Criteria are authored against `SecurityURI` and may include:
 - `SecurityURIHeader` fields: identity (`userId` or role name), area, `functionalDomain`, action
 - `SecurityURIBody` fields: realm, `accountNumber`, `tenantId`, `dataSegment`, `ownerId`, `resourceId`
 - Optional `postconditionScript` evaluated with `pcontext/rcontext` for additional checks
 - Priority: integer used to sort rule evaluation (lower numbers evaluated first)
 - Effect: ALLOW or DENY; ALLOWs may be paired with repository-level scoping using the `SecurityURI` body (e.g., `DataDomain` constraints)

Chapter 56. Rule Structure (Illustrative)



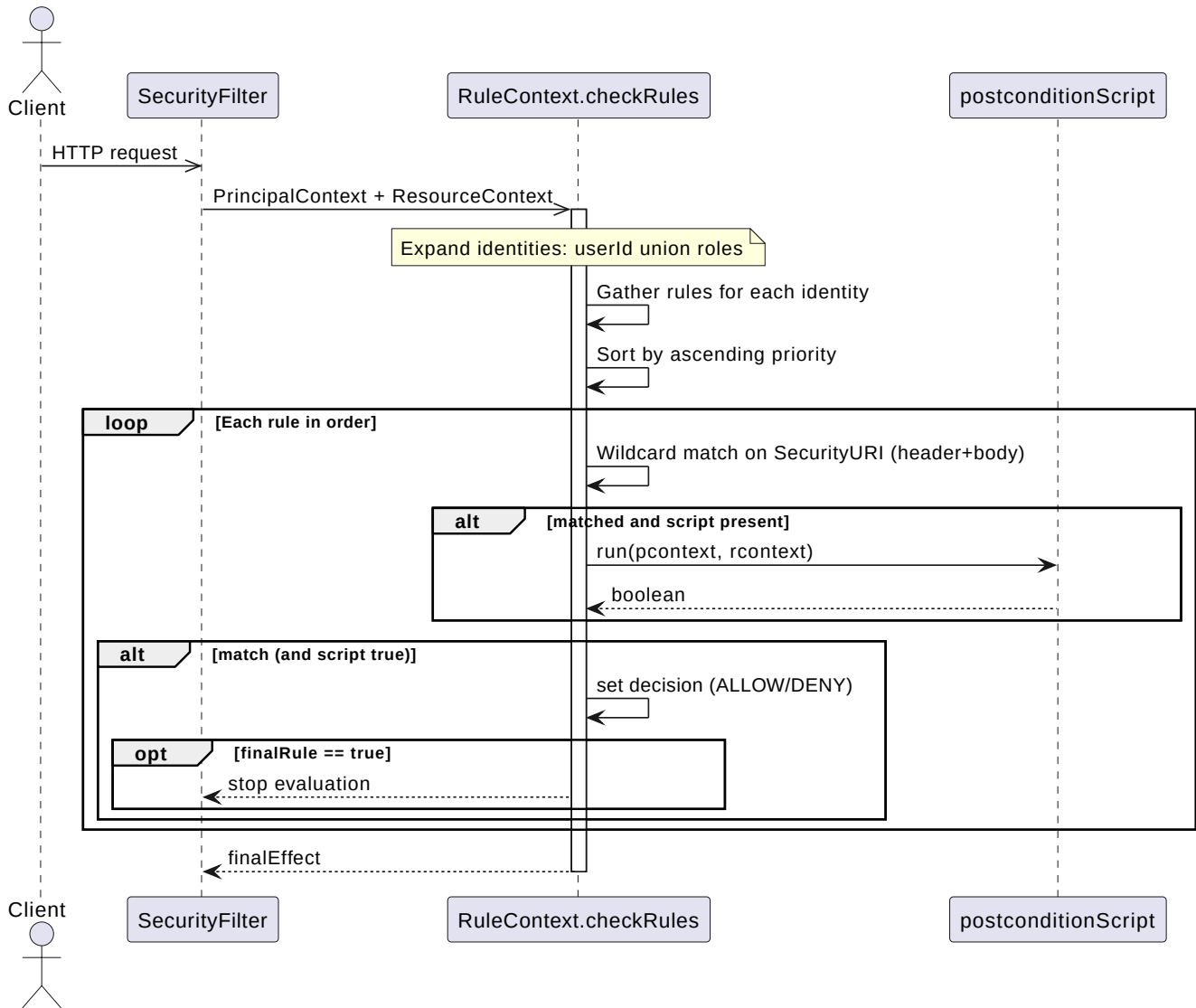
```
- name: allow-catalog-product-reads
description: Allow USER and ADMIN to view products in the Catalog area
securityURI:
  header:
    identity: USER          # or a specific userId; roles are treated as
identities
    area: Catalog
    functionalDomain: Product
    action: view
  body:
    realm: system-com
    accountNumber: '*'
    tenantId: '*'
    dataSegment: '*'
    ownerId: '*'
    resourceId: '*'
  postconditionScript:
  effect: ALLOW
  priority: 300
  finalRule: false

- name: default-deny
description: Fallback deny when nothing else matches
securityURI:
  header:
    identity: '*'
    area: '*'
    functionalDomain: '*'
    action: '*'
  body:
    realm: '*'
    accountNumber: '*'
```

```
tenantId: '*'
dataSegment: '*'
ownerId: '*'
resourceId: '*'
effect: DENY
priority: 10000
finalRule: true
```

- The `securityURI.header` section corresponds to `SecurityURIHeader`: identity (userId or role name), area, functionalDomain, and action. Functional area/domain/action are typically derived from the URL convention `/{{area}}/{{functionalDomain}}/{{action}}` by `SecurityFilter`.
- The `securityURI.body` section corresponds to `SecurityURIBody`: realm, accountNumber, tenantId, dataSegment, ownerId, and resourceId. These values are matched using simple string equality with support for the wildcard `'*'`.
- Optional `postconditionScript` may be provided and is executed as JavaScript with `pcontext` and `rcontext` bindings; the rule only applies if the script evaluates to true.

Chapter 57. Matching Algorithm



The engine evaluates permission rules using SecurityURI wildcard matching. At a high level:

1. Build ResourceContext
 - SecurityFilter derives area, functionalDomain, and action from the request path (or REST annotations if present) and sets the ResourceContext.
2. Expand identities
 - Build a set of identities consisting of the caller's userId plus each effective role. Each of these identities is treated as a potential match target for rules.
3. Gather candidate rules
 - For each identity, collect rules authored for that identity. This forms the candidate set. There is no separate HTTP method/URL or rolesAny/rolesAll matching.
4. Sort by priority
 - Order candidates by ascending priority (lower numbers are evaluated first).
5. URI wildcard comparison

- For each rule in priority order, compare the caller's expanded SecurityURIs to the rule's securityURI using case-insensitive wildcard comparison on the full URI string (header + body). Asterisks (*) in rules match any value.

6. Postcondition script (optional)

- If the rule specifies postconditionScript, execute it as JavaScript with pcontext (principal) and rcontext (resource) variables bound. The rule applies only if the script returns true.

7. Apply effect and finalRule

- When a rule matches (and any script returns true), set the response's finalEffect to the rule's effect (ALLOW or DENY). If finalRule is true, stop evaluating further rules; otherwise continue.

8. Default decision

- If no rule determines a decision, the system returns the default final effect configured by the caller of the check (typically DENY).

Chapter 58. Priorities

- Lower integer = higher priority. Example: priority 1 overrides priority 10.
- Use tight scopes with low priority for critical protections (e.g., denies), and broader ALLOWs with higher numeric priority.
- Recommended ranges:
 - 1–99: global deny rules and emergency blocks
 - 100–499: domain/area-specific critical rules
 - 500–999: standard ALLOW policies
 - 1000+: defaults and catch-alls

Chapter 59. Grant-based vs Deny-based Rule Sets

Grant-based rule sets start with a default decision of DENY and then incrementally add ALLOW scenarios through explicit rules. This model is fail-safe by default: any URL, action, or functional area that does not have a matching ALLOW rule remains inaccessible. As new endpoints or capabilities are added to the system, users will not gain access until an explicit ALLOW is authored. This is the recommended posture for security-sensitive systems and multi-tenant platforms.

Deny-based rule sets start with a default decision of ALLOW and then add DENY scenarios to carve away disallowed cases. In this model, new functionality is exposed by default unless a DENY is added. While convenient during rapid prototyping, this posture risks accidental exposure as the surface area grows.

Practical implications:

- Change management: Grant-based requires adding ALLOWs when shipping new features; Deny-based requires remembering to add new DENYs.
- Auditability: Grant-based policies make it easy to enumerate what is permitted; Deny-based requires proving the absence of permissive gaps.
- Safety: In merge conflicts or partial deployments, Grant-based tends to fail closed (DENY), which is usually safer.

Example defaults:

- Grant-based (recommended):

```
- name: default-deny
  priority: 10000
  securityURI:
    header: { identity: '*', area: '*', functionalDomain: '*', action: '*' }
    body:   { realm: '*', accountNumber: '*', tenantId: '*', dataSegment: '*',
ownerId: '*', resourceId: '*' }
  effect: DENY
  finalRule: true
```

- Deny-based (use with caution):

```
- name: default-allow
  priority: 10000
  securityURI:
    header: { identity: '*', area: '*', functionalDomain: '*', action: '*' }
    body:   { realm: '*', accountNumber: '*', tenantId: '*', dataSegment: '*',
ownerId: '*', resourceId: '*' }
  effect: ALLOW
  finalRule: true
```

Tip: Even in a deny-based set, author low-number DENY rules for critical protections. In most production systems, prefer the grant-based model and layer specific ALLOWs for each capability.

Chapter 60. Row Filter Computation (Data Scoping)

Matching decides *whether* an action is allowed. Once the effect is **ALLOW**, the framework derives the row filter that scopes *which records* a list/query returns. This is the data-scoping half of enforcement, and it is computed per-request from the principal, the matched rules, and (optionally) dynamic access lists.

60.1. The pipeline

1. A **BaseResource** list/query calls **MorphiaRepo.getFilterArray(userFilters, modelClass)**. The caller's own query filters are the starting set; security filters are **AND-ed on top** of them.
2. **RepoSecurityFilterBuilder** short-circuits when **SecurityContext.isIgnoringRules()** is set — returning the filters **unscoped**. This **ignoreRules** path is reserved for system-internal/unauthenticated lookups (e.g. credential resolution at login, infrastructure reads). Otherwise it requires both a **PrincipalContext** and a **ResourceContext** and **fails closed** (throws) if either is missing.
3. **RuleContext.getFilters(filters, pcontext, rcontext, modelClass)** runs **checkRules**; if the final effect is not **ALLOW** the base filters are returned unchanged. Otherwise it resolves a variable bundle and, for each matched **ALLOW** rule, interpolates the rule's **andFilterString/orFilterString** via **MorphiaUtils.convertToFilter(...)**, combining them with the rule's **joinOp** (AND/OR) and deduplicating.
4. The resulting **Filter[]** is AND-ed into the Morphia query.



The **realm** is not a row filter — **getRealmId(principal)** returns the principal's **defaultRealm**, which selects **which database** the query runs against (realm == database). Row filters scope records *within* that database.

60.2. What is scoped by default

There is exactly **one** scope filter baked into the default/system rules:

```
dataDomain.tenantId:${pTenantId}
```

A normal authenticated query therefore gets **tenant isolation only**. Scoping by organization, account, or data segment is **not automatic** — you must author rules that reference the corresponding variables (see below).

60.3. Available substitution variables (static)

MorphiaUtils.createStandardVariableMapFrom registers these scalars from the principal/resource context. They resolve from the principal's **DataDomain** and **DomainContext**, which are populated from the authenticated token's claims.

Variable	Source
<code>\${pTenantId}</code>	<code>DataDomain.tenantId</code> (used by the default rule)
<code>\${pAccountId}</code>	<code>DataDomain.accountNum</code>
<code>\${orgRefName}</code>	<code>DataDomain.orgRefName</code> (note: no <code>p</code> prefix)
<code>\${ownerId}</code>	<code>DataDomain.ownerId</code>
<code>\${defaultRealm}</code>	<code>PrincipalContext.defaultRealm</code> (drives X-Realm scoping)
<code>\${principalId}</code>	<code>PrincipalContext.userId</code>
<code>\${resourceId},</code> <code>\${action},</code> <code>\${functionalDomain},</code> <code>\${area}</code>	<code>ResourceContext</code>
<code>\${dcTenantId},</code> <code>\${dcOrgRefName},</code> <code>\${dcAccountId},</code> <code>\${dcDataSegment}</code>	the <code>DomainContext</code> (X-Realm-aware; may differ from <code>DataDomain</code>)
<code>\${systemTenantId}</code>	<code>DataDomain.tenantId</code> (legacy alias)

To scope by org and account in addition to tenant, author a rule whose filter combines them:

```
- name: scope-by-org-and-account
  priority: 500
  securityURI:
    header: { identity: '*', area: '*', functionalDomain: '*', action: 'view' }
    body: { realm: '*', tenantId: '*', accountNumber: '*', orgRefName: '*' }
  andFilterString: "dataDomain.orgRefName:${orgRefName}"
  orFilterString: "dataDomain.accountId:${pAccountId}"
  joinOp: AND
  effect: ALLOW
```



Because every `${...}` value originates in the token, a token that omits tenant/org/account claims makes the corresponding filters unresolvable — even the default `${pTenantId}` filter. A centralized issuer **MUST** project the principal's `DomainContext` (realm/tenant/org/account) into token claims for this machinery to scope correctly.

60.4. Dynamic access lists (the `AccessListResolver` SPI)

Some scoping is not a static scalar but a **computed collection** — for example, "the set of location IDs this principal may see". These are supplied by `AccessListResolver` implementations rather than by the standard variable map. A resolver whose `key()` returns `accessibleLocationIds` makes the variable `${accessibleLocationIds}` available, referenced with the in-list operator:

```
locationId:^( ${accessibleLocationIds} ]
```

The list can be derived however the resolver chooses — a territory assignment, a `PrincipalContextPropertiesResolver` custom property, or by **walking the ontology graph**. The ontology path is `ListQueryRewriter` (in `quantum-ontology-policy-bridge`), which derives accessible IDs from edges such as `hasEdge("canSeeLocation", ...)`. This is the ontology→rule seam: **ontology edges → access list → `${accessibleLocationIds}` → rule filter**. Predicate references used this way are validated at rule-load time by `RuleVocabularyValidator`.

Two behaviors distinguish dynamic lists from the static scalars:

- **`supports()` gating.** A resolver is only invoked when its `supports(pcontext, rcontext, modelClass)` returns true for the *current request's* context (not the rule's scope). Be permissive, or filter by `modelClass`, or the variable will be unresolvable for valid requests.
- **Skip-on-unresolved.** If a rule's filter references a variable that cannot be resolved for the current model class (e.g. `${accessibleLocationIds}` on a `UserProfile` query), `getFilters` **skips that rule's filter** rather than erroring — so a Location access rule does not wrongly constrain unrelated queries.

60.5. Summary

Scope	Kind	How it applies
Realm	database selection	<code>getRealmId</code> → <code>defaultRealm</code> picks the DB
Tenant	static, default	built-in <code>dataDomain.tenantId:\${pTenantId}</code>
Org / Account / Segment	static, opt-in	author a rule referencing <code>\${orgRefName}</code> / <code>\${pAccountId}</code> / <code>\${dcDataSegment}</code>
Location (and similar)	dynamic, resolver-backed	<code>AccessListResolver</code> (often ontology-edge-backed) supplies <code>\${accessibleLocationIds}</code> ; requires a rule and a registered resolver

Chapter 61. Feature Flags, Variants, and Target Rules

Feature flags complement permission rules by controlling whether a capability is active for a given principal, cohort, or environment. Permissions answer “may this identity perform this action?”; feature flags answer “is this capability turned on, and which variant applies?” Use them together to achieve safe rollouts and fine-grained authorization.

Model reference: `com.e2eq.framework.model.general.FeatureFlag` with key fields:

- `enabled`: master on/off
- `type`: `BOOLEAN` or `MULTIVARIATE`
- `variants`: list of variant keys for multivariate experiments
- `targetRules`: cohort targeting rules
- `environment`: e.g., `dev`, `staging`, `prod`
- `jsonConfiguration`: arbitrary configuration for the feature (e.g., rollout %, UI copy, limits)

Example: Boolean flag for a new export API with environment-specific targeting

```
{
  "refName": "EXPORT_API",
  "description": "Enable CSV export endpoint",
  "enabled": true,
  "type": "BOOLEAN",
  "environment": "prod",
  "targetRules": [
    { "attribute": "role", "operator": "equals", "values": ["BETA"] },
    { "attribute": "tenantId", "operator": "in", "values": ["T100", "T200"] }
  ],
  "jsonConfiguration": { "rateLimitPerMin": 60 }
}
```

Example: Multivariate flag to roll out Search v2 to 10% of users and all members of a beta role

```
{
  "refName": "SEARCH_V2",
  "description": "New search implementation",
  "enabled": true,
  "type": "MULTIVARIATE",
  "variants": ["control", "v2"],
  "environment": "prod",
  "targetRules": [
    { "attribute": "role", "operator": "equals", "values": ["BETA"], "variant": "v2"
  },
    { "attribute": "userId", "operator": "hashMod", "values": ["10"], "variant": "v2"
  }
]
```

```

}
],
"jsonConfiguration": { "defaultVariant": "control" }
}

```

Notes on TargetRules:

- attribute: a property from identity/userProfile (e.g., userId, role, tenantId, location, plan).
- operator: equals, in, contains, startsWith, regex, or domain-specific operators like hashMod for percentage rollouts.
- values: comparison values; semantics depend on operator.
- variant: when type is MULTIVARIATE, selects which variant applies when the rule matches.

How feature flags complement Permission Rule Context:

- The evaluation of a request can enrich the Rule Context (SecurityURI or userProfile) with resolved feature flags and variants (e.g., userProfile.features["SEARCH_V2"] = "v2").
- Permission rules can then require a feature to be present before ALLOWing an action:

```

- name: allow-export-when-flag-on
  description: Allow ADMIN and REPORTER identities to view export when feature flag is
on
  securityURI:
    header:
      identity: ADMIN      # treat roles as identities
      area: Reports
      functionalDomain: Export
      action: view
    body:
      realm: system-com
      accountNumber: '*'
      tenantId: '*'
      dataSegment: '*'
      ownerId: '*'
      resourceId: '*'
  postconditionScript: userProfile?.features?.EXPORT_API === true
  effect: ALLOW
  priority: 300
  finalRule: false

- name: allow-export-when-flag-on-reporter
  description: Same as above but for REPORTER role
  securityURI:
    header:
      identity: REPORTER
      area: Reports
      functionalDomain: Export
      action: view

```

```

body:
  realm: system-com
  accountNumber: '*'
  tenantId: '*'
  dataSegment: '*'
  ownerId: '*'
  resourceId: '*'
postconditionScript: userProfile?.features?.EXPORT_API === true
effect: ALLOW
priority: 300
finalRule: false

```

Alternatively, systems may surface feature decisions via headers (e.g., X-Feature-SEARCH_V2: v2) so that rules or postconditionScript can read them directly from the request context.

Business usage examples for TargetRules and their correlation to Permission Rules:

- Progressive rollout by tenant TargetRule tenantId in [T100, T200] → Permission adds ALLOW for endpoints guarded by that flag so only those tenants can call them during rollout.
 - Role-based beta access: TargetRule role equals BETA → Permission requires both the BETA feature flag and standard role checks (e.g., USER/ADMIN) to ALLOW sensitive actions.
 - Plan/entitlement tiers: TargetRule plan in [Pro, Enterprise] → Permission rules enforce additional data-domain constraints (e.g., export size limits) while the flag simply turns the feature on for eligible plans.

Guidance: Feature Flags vs Permission Rules

- Put into Feature Flags:
 - Gradual, reversible rollouts; A/B or multivariate experiments; UI/behavior switches.
 - Environment gates (dev/staging/prod) and cohort targeting (tenants, beta users, geography).
 - Non-security configuration values in jsonConfiguration (limits, thresholds, copy) that do not change who is authorized.
- Put into Permission Rules:
 - Durable authorization logic: roles, identities, functional area/domain/action, and DataDomain constraints.
 - Compliance and least-privilege decisions where fail-closed behavior is required.
 - Enforcement that remains valid after a feature is fully launched (even when the flag is removed).

Recommendation

Use a grant-based permission posture (default DENY) and let feature flags decide which cohorts even see or can reach new capabilities. Then author explicit ALLOW rules for those capabilities, conditioned on both role and feature presence.

Chapter 62. Scoped Action Enablement and Dependency Tracking

For the dedicated guide covering the runtime API, manifest model, SPI contracts, and built-in resolver types, see [Scoped Action Enablement](#).

Permission checks answer one important question: "may this identity perform `area / functionalDomain / action`?" In practice, teams also need to answer a second question: "why is this action still unavailable?"

Typical reasons include:

- policy denies the action
- a feature flag exists but is disabled
- a required setting has not been configured
- a prerequisite entity or runtime dependency is missing

Quantum now provides a framework-level dependency tracking layer for these cases. The implementation [guide for this work](#) lives in [docs/design/SCOPED_ACTION_ENABLEMENT_IMPLEMENTATION_GUIDE.md](#), and the runtime contracts are implemented in the `quantum-action-enablement-models` and `quantum-action-enablement-quarkus` modules.

62.1. Core idea: the same scoped action tuple stays canonical

The unit of evaluation is still the same tuple already used by the policy engine:

```
area / functionalDomain / action
```

Examples:

- `integration / exchange / create`
- `integration / workflow / create`
- `system / action-enablement / check`

This is intentional. The framework does not introduce a second authorization primitive just to model readiness. Instead, it composes additional dependencies around the same scoped action that policy already understands.

Capabilities are still useful, but at a different level:

- A scoped action is the atomic framework-level unit.
- A capability is an application-defined composition over one or more scoped actions.

- Settings, feature flags, and runtime prerequisites are attached to the scoped action requirement manifest.

That structure is what lets a UI or shell explain "this action is blocked because the policy denies it" versus "this action is allowed, but the required secret or filesystem is not ready yet."

62.2. The three answers returned by the framework

Scoped action enablement separates three concerns that are often blurred together:

Field	Meaning
<code>allowed</code>	Whether the existing permission rule stack allows the scoped action for the evaluated identity and data domain
<code>enabled</code>	Whether rollout and administrative toggles, especially feature flags, say the action should be exposed
<code>ready</code>	Whether operational prerequisites such as settings, entities, or runtime dependencies are satisfied

An action is considered usable only when all three are true:

```
usable = allowed && enabled && ready
```

This distinction matters because the remediation path is different:

- `allowed=false` usually means a policy or role problem
- `enabled=false` usually means a feature flag or administrative rollout problem
- `ready=false` usually means setup, configuration, or runtime dependencies are incomplete

62.3. Runtime models

The core request and response models are intentionally small:

```
public class ScopedActionRef {
    String area;
    String functionalDomain;
    String action;
}

public class DependencyCheckRef {
    String type;
    String refName;
    Map<String, Object> config;
}

public class ScopedActionRequirement {
```

```

    ScopedActionRef scopedAction;
    String displayName;
    String description;
    List<DependencyCheckRef> dependencies;
}

public class ScopedActionEnablementStatus {
    ScopedActionRef scopedAction;
    boolean allowed;
    boolean enabled;
    boolean ready;
    boolean usable;
    List<EnablementBlocker> blockers;
}

```

An **EnablementBlocker** is the machine-readable explanation of why evaluation failed. Each blocker includes:

- **impact**: one of **ALLOWED**, **ENABLED**, or **READY**
- **type**: the dependency type, such as **permission** or **feature-flag**
- **code**: a stable blocker code, such as **permission-denied**
- **message**: a human-readable explanation
- **severity**: typically **warn** or **error**
- **metadata**: additional structured context

62.4. REST API

The runtime API is exposed at:

- **POST** `/system/actions/enablement/check`
- **GET** `/system/actions/enablement/manifest`

This resource itself is annotated as:

- **area=system**
- **functionalDomain=action-enablement**
- **action=check|view**

Example request:

```

{
  "identity": "tenant-admin",
  "realm": "acme",
  "orgRefName": "acme-org",
  "tenantId": "acme-tenant",
  "scope": "api",

```



```

"actions": [
  {
    "area": "integration",
    "functionalDomain": "guided_integration",
    "action": "create"
  }
]
}

```

Representative response:

```

{
  "results": [
    {
      "scopedAction": {
        "area": "integration",
        "functionalDomain": "guided_integration",
        "action": "create"
      },
      "allowed": true,
      "enabled": false,
      "ready": false,
      "usable": false,
      "blockers": [
        {
          "impact": "ENABLED",
          "type": "feature-flag",
          "code": "feature-flag-disabled",
          "message": "Feature flag 'guided-integrations' is disabled.",
          "severity": "warn"
        },
        {
          "impact": "READY",
          "type": "setting-present",
          "code": "setting-missing",
          "message": "Configuration setting 'quantum.integration.shared-storage.root'
is not set.",
          "severity": "error"
        }
      ]
    }
  ]
}

```

This is the key user-facing value: the response tells you whether the action is blocked by authorization, rollout state, or operational readiness, instead of forcing the UI to guess.

62.5. Manifest-driven dependency relationships

The relationship from settings and feature flags to functional area/domain/action lives in a manifest of `ScopedActionRequirement` entries. Each manifest entry says:

- which scoped action is being described
- which dependencies must be satisfied for that action to be usable
- optional display-oriented metadata such as `displayName` and `description`

Applications contribute these manifests via CDI:

```
public interface ScopedActionRequirementContributor {  
  
    default int priority() {  
        return 100;  
    }  
  
    Collection<ScopedActionRequirement> requirements();  
}
```

The framework aggregates contributors through `ScopedActionRequirementRegistry`:

```
public interface ScopedActionRequirementRegistry {  
  
    Optional<ScopedActionRequirement> find(ScopedActionRef ref);  
  
    List<ScopedActionRequirement> list();  
}
```

This is how you express relationships such as:

- action `integration / guided_integration / create` depends on feature flag `guided-integrations`
- action `integration / exchange / create` depends on a tenant filesystem being provisioned
- action `system / some-admin-action / save` depends on a specific config property being present

62.6. Resolver SPI

Each dependency type is implemented by an `ActionDependencyResolver`:

```
public interface ActionDependencyResolver {  
  
    String supportsType();  
  
    DependencyResolutionResult evaluate(  
        DependencyCheckRef dependency,  
        ...  
    )  
}
```

```

        EnablementEvaluationContext context
    );
}

```

`EnablementEvaluationContext` carries the evaluation inputs:

- identity
- realm
- roles
- scope
- `DataDomain`
- the target `ScopedActionRef`

This lets each resolver examine dependencies in the correct tenant, realm, and user context.

62.7. Built-in OSS dependency types

The open-source framework currently provides these dependency resolvers:

Dependency type	Purpose	Typical impact
<code>permission</code>	Evaluate the scoped action through the existing <code>RuleContext</code> and permission model	<code>ALLOWED</code>
<code>feature-flag</code>	Require that a <code>FeatureFlag</code> exists and is enabled in the active realm	<code>ENABLED</code>
<code>setting-present</code>	Require that a named configuration property exists and is non-blank	<code>READY</code>
<code>entity-exists</code>	Require that a referenced entity exists in the current realm	<code>READY</code>

Representative blocker codes include:

- `permission-denied`
- `feature-flag-missing`
- `feature-flag-disabled`
- `setting-name-missing`
- `setting-missing`
- `entity-model-unresolved`
- `entity-missing`

62.8. Missing manifests and unsupported dependency types

The framework intentionally does not fail open here.

If no manifest entry is registered for a scoped action:

- evaluation falls back to a synthetic requirement that checks `permission`
- the response also includes a `manifest-missing` blocker with `impact=READY`

If a manifest references a dependency type for which no resolver is registered:

- the response includes `unsupported-dependency-type`
- the blocker is attached with `impact=READY`

This makes incomplete wiring visible instead of silently treating an action as operationally ready.

62.9. Example contributor

At the framework level, the built-in manifest contributor registers the enablement API itself with a permission dependency. Application modules then add their own manifests.

A typical application contributor looks like this:

```
@ApplicationScoped
public class MyActionEnablementManifestContributor
    implements ScopedActionRequirementContributor {

    @Override
    public Collection<ScopedActionRequirement> requirements() {
        return List.of(
            ScopedActionRequirement.builder()
                .scopedAction(ScopedActionRef.builder()
                    .area("integration")
                    .functionalDomain("guided_integration")
                    .action("create")
                    .build())
                .displayName("Generate Guided Integration")
                .description("Requires policy, feature rollout, and baseline setup.")
                .dependencies(List.of(
                    DependencyCheckRef.builder().type("permission").build(),
                    DependencyCheckRef.builder()
                        .type("feature-flag")
                        .refName("guided-integrations")
                        .build(),
                    DependencyCheckRef.builder()
                        .type("setting-present")
                        .refName("quantum.integration.shared-storage.root")
```

```

        .build()
    ))
    .build()
);
}
}

```

This is the mechanism that turns "settings map to capabilities" into something concrete. The requirement manifest records which settings, feature flags, and prerequisites attach to which **area** / **functionalDomain** / **action** tuple.

62.10. Relationship to policies, feature flags, and UI actions

Think of the layers this way:

1. **@FunctionalMapping** and **@FunctionalAction** define the canonical scoped action tuple in code.
2. Permission rules answer whether that tuple is authorized.
3. Feature flags answer whether the action is turned on for the current cohort or realm.
4. Scoped action requirements answer what else must be configured or provisioned before the action is actually usable.

This complements, rather than replaces, existing UI action derivation:

- **fillUIActions()** still answers what actions are allowed from the permission perspective
- scoped action enablement answers why an action should still be disabled or annotated in the UI

That makes it a better fit for:

- setup dashboards
- onboarding flows
- context-sensitive "why unavailable?" messaging
- shells or MCP clients that need structured blockers

62.11. Enterprise extensions

The framework module is intentionally generic. Enterprise modules can contribute additional dependency resolvers without changing the core contracts.

The current enterprise action-enablement module extends the system with higher-level runtime checks such as:

- **managed-secret-configured**
- **tenant-filesystem-provisioned**
- **standard-mounts-present**

- `workflow-runtime-reachable`

Those resolvers support operational questions such as:

- has the tenant filesystem been provisioned?
- do the standard mounts exist?
- is a required managed secret configured?
- is the workflow runtime reachable?

That deeper operational behavior belongs in the enterprise documentation. The important framework-level point is that enterprise modules plug into the same `ActionDependencyResolver` SPI and return the same `EnablementBlocker` structure.

62.12. Guidance

- Keep permission checks canonical. Do not duplicate authorization logic inside custom readiness resolvers.
- Use `feature-flag` dependencies for rollout state, not for durable authorization.
- Use `setting-present` and `entity-exists` only for generic prerequisites. Domain-heavy operational checks belong in application or enterprise resolvers.
- Keep blocker codes stable because UIs and automation may key off them.
- Treat capabilities as documentation and UX groupings over scoped actions, not as a substitute for `area / functionalDomain / action`.

Chapter 63. Multiple Matching RuleBases

- Ordering: rules are evaluated in ascending priority (lower numbers first).
- Decision: when a rule matches, its effect (ALLOW or DENY) becomes the current decision.
- finalRule: if the matching rule has finalRule: true, evaluation stops immediately; otherwise, evaluation continues and a later rule may overwrite the decision.
- Default: if no rule matches decisively, the default effect supplied by the caller (typically DENY) is returned.

Chapter 64. Identity and Role Matching

- Roles-as-identities: the engine evaluates rules for the caller's `userId` and for each effective role by treating each role name as an identity.
- There are no `rolesAny/rolesAll` fields in rules; author separate rules for specific roles as needed.
- Optional `postconditionScript` can inspect attributes (for example, `tenantId`) via `pcontext` and `rcontext` when additional checks are needed.
- Time or plan-based conditions can be implemented inside `postconditionScript` or via feature flags.

64.1. How roles are defined for an identity (role sources and resolution)

Quantum composes the effective roles for a request by merging:

- Roles from the identity provider (JWT/`SecurityIdentity`)
- Roles configured on the user record (`CredentialUserIdPassword.roles`)

Source details:

- Identity Provider (JWT): roles commonly arrive via standard claims (for example, `groups`, `roles`, or provider-specific fields). Quarkus maps these into `SecurityIdentity.getRoles()`. In multi-realm setups, the realm in `X-Realm` can scope lookups but does not alter what the JWT asserts.
- Quantum user record: `com.e2eq.framework.model.security.CredentialUserIdPassword` has a `String[] roles` field stored per realm. This can be administered by Quantum to grant platform- or tenant-level roles.

Merge semantics (current implementation):

- Union: the effective role set is the union of JWT roles and `CredentialUserIdPassword.roles`. If either source is empty, the other source defines the set.
- Fallback: when neither source yields roles, the framework defaults to `ANONYMOUS`.
- Where implemented: `SecurityFilter.determinePrincipalContext` builds `PrincipalContext` with the merged roles.

Realm considerations:

- The user record is looked up by subject or `userId` in the active realm (default or `X-Realm`). If a realm override is provided, it is validated with `CredentialUserIdPassword.realmRegEx`.
- Roles stored in a user record are realm-specific; JWT roles are whatever the IdP asserts for the token.

Operating models:

- Quantum-managed roles:

- IdP authenticates the user (subject, username). Authorization is primarily driven by roles stored in `CredentialUserIdPassword.roles`.
- Use when you want central, auditable role assignment within Quantum, independent of IdP groups.
- IdP-managed roles:
 - IdP carries authoritative roles/groups in the JWT. Keep `CredentialUserIdPassword.roles` minimal or empty.
 - Use when enterprises require IdP as the source of truth for access groups.
- Hybrid (recommended in many deployments):
 - Effective roles = JWT roles union `CredentialUserIdPassword.roles`.
 - Use JWT for enterprise groups (for example, `DEPT_SALES`, `ORG_ADMIN`) and Quantum roles for app-specific grants (for example, `REPORT_EXPORTER`, `BETA`).
 - This avoids IdP churn for application-local concerns while respecting org policies.

Examples:

- JWT-only:
 - `JWT.groups = [USER, REPORTER]`; user record roles = []
 - Effective roles = `[USER, REPORTER]`
- Quantum-only:
 - `JWT.groups = []`; user record roles = `[USER, ADMIN]`
 - Effective roles = `[USER, ADMIN]`
- Hybrid union:
 - `JWT.groups = [USER]`; user record roles = `[BETA, REPORT_EXPORTER]`
 - Effective roles = `[USER, BETA, REPORT_EXPORTER]`

Guidance and best practices:

- Keep role names stable and environment-agnostic; use realms/permissions to scope where needed.
- Avoid overloading roles for feature rollout; use Feature Flags for rollout and variants, and roles for durable authorization.
- When IdP is authoritative, ensure consistent claim mapping so `SecurityIdentity.getRoles()` contains the expected values; commonly via `groups` claim in JWT.
- Use grant-based permission rules and require the minimal set of roles (`rolesAny/rolesAll`) needed for each capability.

Cross-references:

- User model: `com.e2eq.framework.model.security.CredentialUserIdPassword.roles`
- Context: `com.e2eq.framework.model.securityrules.PrincipalContext.getRoles()`

- Filter logic: `com.e2eq.framework.rest.filters.SecurityFilter.determinePrincipalContext`

64.1.1. Populating `RoleSource` and `roleAssignments`

Quantum exposes structured provenance for each role via:

- `com.e2eq.framework.model.auth.RoleSource` enum with values: `USERGROUP`, `IDP`, `CREDENTIAL`
- `com.e2eq.framework.model.auth.RoleAssignment` record: `role + Set<RoleSource> sources`
- Login response model `com.e2eq.framework.model.auth.AuthProvider.LoginPositiveResponse` now includes a `roleAssignments` field next to plain `roles`.
- `com.e2eq.framework.model.securityrules.SecurityCheckResponse` also includes `roleAssignments`.

Defaults and compatibility:

- If you do nothing, both login and security check responses automatically derive `roleAssignments` from the flat `roles` set and mark each role with `CREDENTIAL`. This preserves backward compatibility.
- To provide accurate provenance, populate `roleAssignments` explicitly where you already know role sources.

JSON example (login success):

```
{
  "userId": "alice",
  "roles": ["user", "admin"],
  "roleAssignments": [
    {"role": "user", "sources": ["idp", "credential"]},
    {"role": "admin", "sources": ["usergroup"]}
  ],
  "accessToken": "...",
  "refreshToken": "...",
  "expirationTime": 1732100000,
  "mongodbUrl": "...",
  "realm": "system"
}
```

64.1.2. AuthProviders: what you need to do

Auth providers (for example, `CustomTokenAuthProvider`) should add provenance at two points:

1. During login (`login(userId, password)`)
2. During token refresh (`refreshTokens(refreshToken)`)

At each point build three role sets and then compute per-role sources:

- IDP roles → from `io.quarkus.security.identity.SecurityIdentity.getRoles()` for the current access/refresh token

- Credential roles → from `com.e2eq.framework.model.security.CredentialUserIdPassword.getRoles()` in the DB
- User group roles → gathered by resolving the user's `UserProfile` and its `UserGroup` memberships

Then build `List<RoleAssignment>` and pass the full constructor of `LoginPositiveResponse`.

Example (inside `CustomTokenAuthProvider.login(...)` after you create `identity` and locate the `credential`):

```
// 1) Collect source role sets
Set<String> idpRoles = (identity != null) ? new java.util.LinkedHashSet<>(identity
.getRoles()) : java.util.Set.of();
Set<String> credentialRoles = new java.util.LinkedHashSet<>(java.util.Arrays.asList
(credential.getRoles()));
Set<String> userGroupRoles = new java.util.LinkedHashSet<>();
try {
    var userProfileOpt = userProfileRepo.getBySubject(credential.getSubject());
    if (userProfileOpt.isPresent()) {
        var groups = userGroupRepo.findByUserProfileRef(userProfileOpt.get()
.createEntityReference());
        if (groups != null) {
            for (com.e2eq.framework.model.security.UserGroup g : groups) {
                if (g != null && g.getRoles() != null) {
                    java.util.Collections.addAll(userGroupRoles, g.getRoles());
                }
            }
        }
    }
} catch (Exception e) {
    io.quarkus.logging.Log.warn("Group role expansion failed; continuing without group
roles", e);
}

// 2) Union of all roles for the response
java.util.Set<String> allRoles = new java.util.LinkedHashSet<>();
allRoles.addAll(idpRoles);
allRoles.addAll(credentialRoles);
allRoles.addAll(userGroupRoles);

// 3) Build per-role source assignments
java.util.List<com.e2eq.framework.model.auth.RoleAssignment> roleAssignments =
allRoles.stream()
    .filter(java.util.Objects::nonNull)
    .map(role -> {
        java.util.EnumSet<com.e2eq.framework.model.auth.RoleSource> src =
            java.util.EnumSet.noneOf(com.e2eq.framework.model.auth.RoleSource.class);
        if (idpRoles.contains(role)) src.add(com.e2eq.framework.model.auth.RoleSource
.IDP);
        if (credentialRoles.contains(role)) src.add(com.e2eq.framework.model.auth
.RoleSource.CREDENTIAL);
```

```

        if (userGroupRoles.contains(role)) src.add(com.e2eq.framework.model.auth
.RoleSource.USERGROUP);
        return new com.e2eq.framework.model.auth.RoleAssignment(role, src);
    })
    .toList();

// 4) Use the full constructor that accepts roleAssignments
return new com.e2eq.framework.model.auth.AuthProvider.LoginResponse(
    true,
    new com.e2eq.framework.model.auth.AuthProvider.LoginPositiveResponse(
        userId,
        identity,
        allRoles,           // Set<String>
        roleAssignments,    // List<RoleAssignment>
        accessToken,
        refreshToken,
        expirationTime,
        mongodbUrl,
        realm
    )
);

```

Notes and tips:

- You may centralize this logic in `IdentityRoleResolver` by adding a method that returns `Map<String, EnumSet<RoleSource>>` (role → sources) and reuse it across login and `/check`.
- If you cannot resolve a user record (for example, token-only identities), populate only `IDP` sources.
- Avoid adding synthetic roles like `ANONYMOUS` to `roleAssignments`—the structure should cover actual grants.
- Consider caching group-derived roles per user/request if lookups are expensive.

64.1.3. Security Check responses: enriching `roleAssignments`

`SecurityCheckResponse` defaults to marking all roles as `CREDENTIAL`. To provide accurate provenance during permission checks:

1. Near `PermissionResource.check(...)`, where roles are resolved, compute the same three source sets as above.
2. Build `List<RoleAssignment>` from the final role set used for `PrincipalContext`.
3. After calling the rule engine, overwrite the response's assignments:

```

SecurityCheckResponse resp = ruleContext.checkRules(pc, rc);
resp.setRoleAssignments(roleAssignments);
return Response.ok(resp).build();

```

This keeps the default behavior (no breaking changes) while enabling clients to see precisely how each role was granted.

Chapter 65. Example Scenarios

1. Public catalog browsing

- Request: GET /Catalog/Products/VIEW?search=widgets
- Identity: anonymous or role USER
- Rules:
 - allow-public-reads (priority 100) ALLOW + readScope orgRefName=PUBLIC
- Outcome: ALLOW; repository applies DataDomain filter orgRefName=PUBLIC

2. Tenant-scoped shipment update

- Request: PUT /Collaboration/Shipments/UPDATE
- Headers: x-tenant-id=T1
- Body: { dataDomain: { tenantId: "T1" }, ... }
- Identity: user in tenant T1 with roles [USER]
- Rules:
 - allow-collab-update (priority 300) requires body.dataDomain.tenantId == identity.tenantId and rolesAny USER, ADMIN ⇒ ALLOW
- Outcome: ALLOW; Rule contributes writeScope tenantId=T1

3. Cross-tenant admin read with higher priority

- Request: GET /api/partners
- Identity: role ADMIN (super-admin)
- Rules:
 - admin-override (priority 50) ALLOW
 - default-tenant-read (priority 600) ALLOW with tenant filter
- Outcome: admin-override wins due to higher precedence (lower number), allowing broader read

4. Conflicting ALLOW and DENY at same priority

- Two rules match with priority 200: one ALLOW, one DENY
- Resolution: DENY wins unless merge strategy configured to handle explicitly; recommended to avoid same-priority conflicts by policy.

Chapter 66. Operational Tips

- Author specific DENY rules with low numbers to prevent accidental exposure.
- Author SecurityURI header (area/domain/action) as narrowly as needed for sensitive domains.
- Prefer SecurityURI body fields and postconditionScript to refine matches without over-broad area/domain/action patterns.
- Log matched rule names and applied scopes for auditability.

Chapter 67. Field-Level Policy (Property Exclusions)

Row filters (`andFilterString` / `orFilterString`) control **which documents** a principal can see. Field-level policy controls **which fields of those documents** — the common "the carrier may see the shipment but not the price" requirement.

67.1. Declaring exclusions on a rule

`Rule.excludedFields` carries dotted property paths. The exclusions apply to any principal/resource the rule matches, composing with the same matching algorithm as row filters:

```
Rule.Builder()
  .withName("carrier-fulfillment-view")
  .withSecurityURI(uri)                // who/what this rule matches
  .withEffect(RuleEffect.ALLOW)
  .withAndFilterString("hasEdge('assignedTo', ${principal.orgRefName})")
  .withExcludedFields(List.of("pricing.unitPrice", "pricing.margin"))
  .build();
```

Semantics are **deny-wins**: the union of `excludedFields` across **all** matched ALLOW rules applies. A field excluded by any applicable rule is excluded, even if another matched rule does not mention it. Paths that do not exist on a queried model are ignored (nothing to protect). Excluded fields are **omitted** from results, not nulled-and-present — clients should treat absence under an active field policy as "restricted", and the REST layer signals this with the `X-Field-Policy` response header listing the applied exclusions.

67.2. Layered enforcement — one policy source, every egress guarded

The resolved exclusion set comes from one place (`RuleContext.getExcludedFieldPaths`) and is enforced wherever data leaves the platform. The query language is a multi-datasource contract, so enforcement is **per egress**, not per database:

Egress	Mechanism	Notes
MongoDB reads	Projection injection in <code>MorphiaRepo.buildFindOptions</code>	Excluded paths never leave the database. Caller INCLUDE projections have policy paths dropped (exclusion wins); caller EXCLUDE projections are unioned with policy paths.

Egress	Mechanism	Notes
MongoDB updates	Preserve-hidden-fields gate in <code>MorphiaRepo.merge</code>	A principal whose policy hides a field cannot blind-overwrite it: the stored value is restored (via <code>FieldPolicyEnforcer.copyPath</code>) before persisting.
In-memory / hierarchical / custom datasources	<code>FieldPolicyEnforcer.mask(result, paths)</code> at result materialization	For non-Mongo backends this is the datastore-level enforcement. Implementing the query contract against a new datasource requires wiring the enforcer — it is part of the adapter contract, exactly like honoring row-filter strings.
REST serialization	<code>FieldPolicyResponseInterceptor</code> (WriterInterceptor)	Defense-in-depth fed by the same resolved set: catches computed getters, reference-expanded fields, and responses assembled outside a secured repo. Adds the <code>X-Field-Policy</code> header when masking occurred.



A serializer-only approach (e.g. Jackson `@JsonView`) was considered and rejected as the primary mechanism: views are compile-time classes while Quantum policies are tenant-authorable runtime data, and serializer enforcement guards only the REST door while the reasoner, MCP surface, exports, and events read through repositories. The serializer layer is kept — as the last lock, driven by the same rule-resolved path set.

67.3. Limitations (v1)

- Wildcard paths (`pricing.*`) are not yet supported — list explicit paths.
- The write gate applies on `merge` (partial update); full-document `save` of a **new** entity is not field-filtered (the creator authored the values).
- Field exclusions are resolved per (principal, resource) pair, like row filters; per-**instance** visibility derives from relationship row filters (`hasEdge`), not from per-document field lists.

Chapter 68. How UIActions and DefaultUIActions are calculated

When the server returns a collection of entities (for example, userProfiles), each entity may expose two action lists:

- **DefaultUIActions:** the full set of actions that conceptually apply to this type of entity (e.g., CREATE, UPDATE, VIEW, DELETE, ARCHIVE). Think of this as the “menu template” for the type.
- **UIActions:** the subset of actions the current user is actually permitted to perform on that specific entity instance right now.

Why they can differ per entity:

- **Entity attributes:** state or flags (e.g., archived, soft-deleted, immutable) can remove or alter available actions at instance level.
- **Permission rule base:** evaluated against the current request, identity, and context to allow or deny actions.
- **DataDomain membership:** tenant/org/owner scoping can further restrict actions if the identity is outside the entity’s domain.

How the server computes them:

1. Start with a default action template for the entity type (DefaultUIActions).
2. Apply simple state-based adjustments (for example, suppress CREATE on already-persisted instances).
3. Evaluate the permission rules with the current identity and context:
 - Consider roles, functional area/domain, action intent, SecurityURI body fields, and any rule-contributed scopes.
 - Resolve DataDomain constraints to ensure the identity is permitted to act within the entity’s domain.
4. Produce UIActions as the allowed subset for that entity instance.
5. Return both lists with each entity in collection responses.

How the client should use the two lists:

- Render the full DefaultUIActions as the visible set of possible actions (icons, buttons, menus) so the UI stays consistent.
- Enable only those actions present in UIActions; gray out or disable the remainder to signal capability but lack of current permission.
- This approach avoids flicker and keeps affordances discoverable while remaining truthful to the user’s current authorization.

Example:

- You fetch 25 userProfiles.
- DefaultUIActions for the type = [CREATE, VIEW, UPDATE, DELETE, ARCHIVE].
- For a specific profile A (owned by your tenant), UIActions may be [VIEW, UPDATE] based on your roles and domain.
- For another profile B (in a different tenant), UIActions may be [VIEW] only.
- The UI renders the same controls for both A and B, but only enables the actions present in each item's UIActions list.

Operational considerations:

- Keep action names stable and documented so front-ends can map to icons and tooltips consistently.
- Prefer small, composable rules that evaluate action permissions explicitly by functional area/domain to avoid surprises.
- Consider server-side caching of action evaluations for list views to reduce latency, respecting identity and scope.

Chapter 69. How This Integrates End-to-End

- BaseResource extracts identity and headers to construct DomainContext.
- Rule evaluation uses SecurityURI (header + body) matching with optional postconditionScript and identity/userProfile to reach a decision and derive scope filters.
- Repositories (e.g., MorphiaRepo) apply the filters to queries and updates, ensuring DataDomain-respecting access.

Chapter 70. Administering Policies via REST (PolicyResource)

The PolicyResource exposes CRUD-style REST APIs for creating and managing policies (rule bases) that drive authorization decisions. Each Policy targets a principalId (either a specific userId or a role name) and contains an ordered list of Rule objects. Rules match requests using SecurityURIHeader and SecurityURIBody and then contribute an effect (ALLOW/DENY) and optional repository filters.

- Base path: /security/permission/policies
- Auth: Bearer JWT (see Authentication); resource methods are guarded by @RolesAllowed("user", "admin") at the BaseResource level and your own realm/role policies.
- Multi-realm: pass X-Realm header to operate within a specific realm; otherwise the default realm is used.

70.1. Model shape (Policy)

A Policy extends FullBaseModel and includes: - id, refName, displayName, dataDomain, archived/expired flags (inherited) - principalId: userId or role name that this policy attaches to - description: human-readable summary - rules: array of Rule entries

Rule fields (key ones):

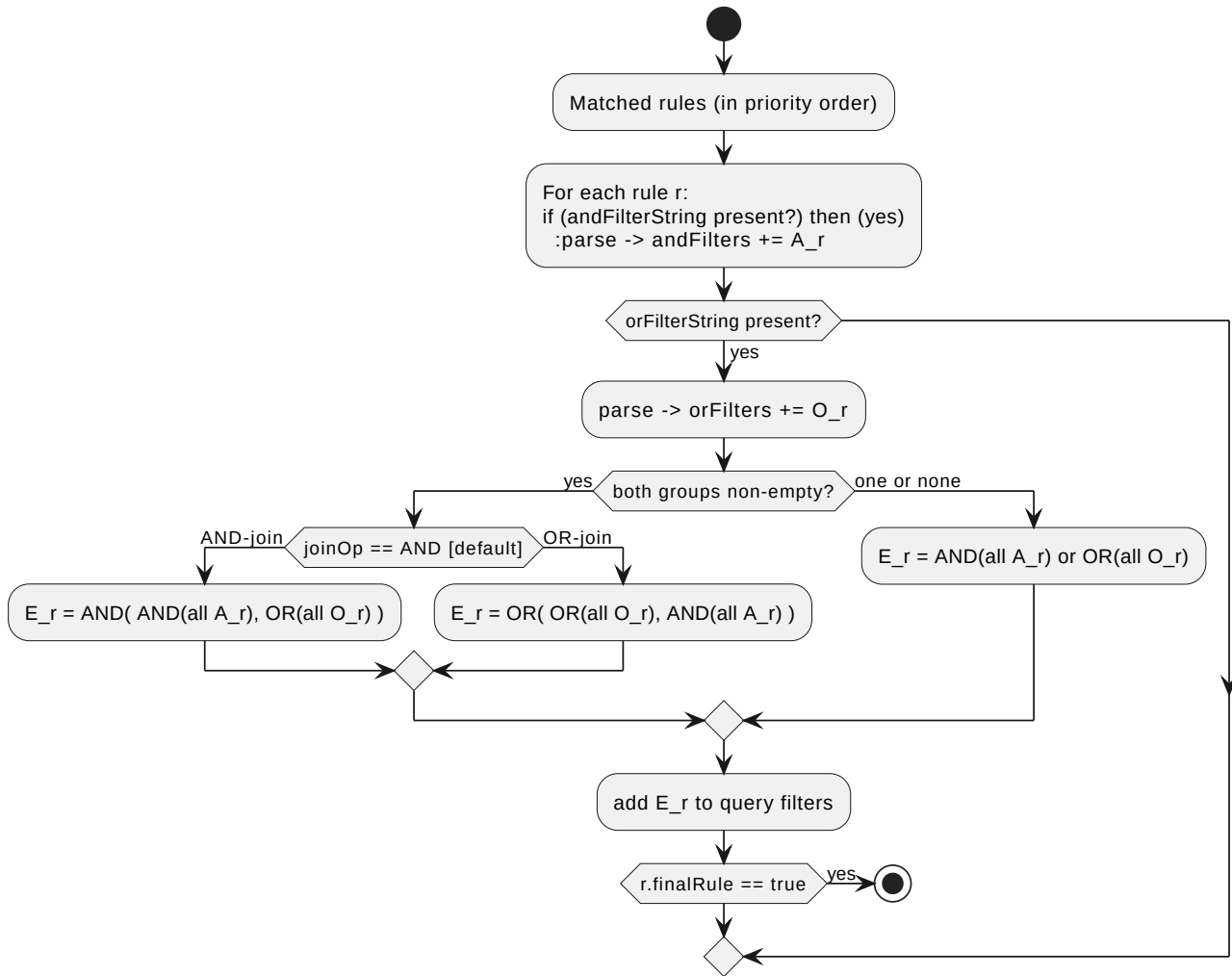
- name, description
- securityURI.header: identity, area, functionalDomain, action (supports wildcard "**")
- securityURI.body: realm, orgRefName, accountNumber, tenantId, ownerId, dataSegment, resourceId (supports wildcard "**")
- effect: ALLOW or DENY
- priority: integer; lower numbers evaluated first
- finalRule: boolean; stop evaluating when this rule applies
- andFilterString / orFilterString: ANTLR filter DSL snippets injected into repository queries (see Query Language section)
- joinOp: how to combine the andFilterString group with the orFilterString group when both are present; defaults to AND

70.2. How rule-contributed filters work (andFilterString, orFilterString, joinOp)

When an ALLOW rule matches, it can contribute repository-level filters that restrict which documents are visible or mutable. The fields are:

- andFilterString: a filter expression added to the AND group.

- `orFilterString`: a filter expression added to the OR group.
- `joinOp`: when both groups are present, specifies how to join them. Values: AND or OR. Default: AND.



Composition algorithm (implemented in `RuleContext.getFilters`):

1. Collect all matched rules (in priority order; skipping NOT_APPLICABLE script results). For each rule:
 - If `andFilterString` is set, parse it into a Morphia Filter and add to the and-group.
 - If `orFilterString` is set, parse it into a Morphia Filter and add to the or-group.
2. If both groups are non-empty for the current rule:
 - If `joinOp == AND` (default): `effective = AND( AND(all and-group), OR(all or-group) )`.
 - If `joinOp == OR`: `effective = OR( OR(all or-group), AND(all and-group) )`.
3. If only one group is non-empty:
 - Use `AND(all and-group)` or `OR(all or-group)` as the effective filter.
4. Append the effective filter(s) to the query's filter list. If the rule has `finalRule: true`, stop accumulating more filters.
5. Deduplicate filters by string form before returning.

Examples:

- AND only

```
- name: tenant-scope
  securityURI:
    header: { identity: USER, area: sales, functionalDomain: order, action: view }
    body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
    andFilterString: "dataDomain.tenantId:${pcontext.dataDomain.tenantId}"
    effect: ALLOW
    priority: 200
```

Resulting Morphia: AND(eq("dataDomain.tenantId", <caller-tenant>))

- OR only

```
- name: visibility-by-segment
  securityURI:
    header: { identity: USER, area: sales, functionalDomain: order, action: view }
    body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
    orFilterString: "dataDomain.dataSegment:^(PUBLIC|INTERNAL)"
    effect: ALLOW
    priority: 210
```

Resulting Morphia: OR(in("dataDomain.dataSegment", [PUBLIC, INTERNAL]))

- AND + OR with joinOp: AND (default)

```
- name: own-or-public
  securityURI:
    header: { identity: USER, area: security, functionalDomain: userProfile, action:
view }
    body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
    andFilterString: "dataDomain.ownerId:${principalId}"
    orFilterString:  "dataDomain.dataSegment:^(PUBLIC)"
    joinOp: AND
    effect: ALLOW
    priority: 220
```

Effective: AND(eq(ownerId, principalId), OR(eq(dataSegment, 'PUBLIC')))

- AND + OR with joinOp: OR

```
- name: owner-or-segment
```

```

securityURI:
  header: { identity: USER, area: files, functionalDomain: document, action: view }
  body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
  andFilterString: "dataDomain.ownerId:${principalId}"
  orFilterString:  "tags:^(shared)"
  joinOp: OR
  effect: ALLOW
  priority: 230

```

Effective: OR(eq(ownerId, principalId), AND(eq(tags, 'shared')))

Placeholders and variables:

The following standard variables are available in andFilterString/orFilterString:

Variable	Source	Description
<code>\${principalId}</code>	PrincipalContext.userId	The authenticated user's ID
<code>\${pAccountId}</code>	PrincipalContext.dataDomain.accountNum	The principal's account number
<code>\${pTenantId}</code>	PrincipalContext.dataDomain.tenantId	The principal's tenant ID
<code>\${ownerId}</code>	PrincipalContext.dataDomain.ownerId	The data owner ID
<code>\${orgRefName}</code>	PrincipalContext.dataDomain.orgRefName	The organization reference name
<code>\${defaultRealm}</code>	PrincipalContext.defaultRealm	The current realm (changes with X-Realm override)
<code>\${resourceId}</code>	ResourceContext.resourceId	The target resource ID
<code>\${action}</code>	ResourceContext.action	The action being performed
<code>\${functionalDomain}</code>	ResourceContext.functionalDomain	The functional domain
<code>\${area}</code>	ResourceContext.area	The functional area
<code>\${dcTenantId}</code>	DomainContext.tenantId	Tenant ID from the current domain context
<code>\${dcOrgRefName}</code>	DomainContext.orgRefName	Organization reference name from domain context
<code>\${dcAccountId}</code>	DomainContext.accountId	Account ID from domain context
<code>\${dcDataSegment}</code>	DomainContext.dataSegment	Data segment number from domain context
<code>\${facet.<key>}</code>	DataDomainPolicyEntry.facetFilters	Tenant-configured facet filters (e.g. <code>\${facet.allowedTiers}</code>)

Variable	Source	Description
<code>\${coord.<key>}</code>	DataDomainResolution coordinates	Custom coordinate variables (e.g. <code>\${coord.salesZone}</code>)
<code>\${principal.<attr>}</code>	PrincipalContext attributes	Authenticated principal attributes and claims
<code>\${dd.<attr>}</code>	DataDomain fields	Target DataDomain partition coordinates



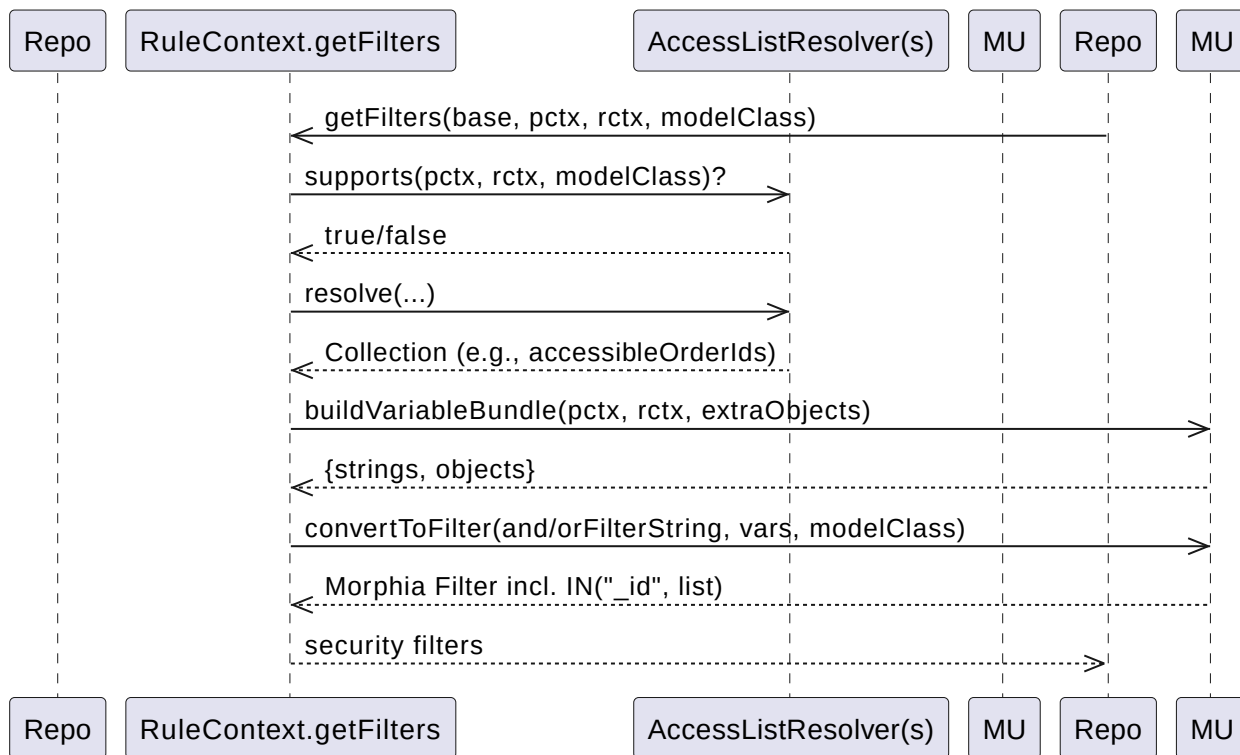
Dotted Variable Identifiers & Single-Variable IN: Variables with dotted paths (`${facet.tier}`, `${coord.salesZone}`) resolve in a strict hierarchy. When evaluated with the list membership operator (`:^`), single variables resolving to a Java `Collection` (e.g. `allowedTiers = ["FLAGSHIP", "REGIONAL"]`) can be passed without brackets (`tier:^${facet.allowedTiers}`), automatically unpacking into native MongoDB `$in` clauses.

X-Realm integration: When `X-Realm` is used, `${defaultRealm}` automatically reflects the target realm. This enables permission filters to dynamically scope data based on the active realm:

```
- name: scope-by-realm
  securityURI:
    header: { identity: USER, area: '*', functionalDomain: '*', action: view }
    body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
    andFilterString: "dataDomain.tenantId:${defaultRealm}"
    effect: ALLOW
    priority: 200
```

- The filter DSL is described in the Query Language guide; it maps to Morphia `dev.morphia.query.filters.Filters` under the hood via `MorphiaUtils.convertToFilter`.

70.3. AccessListResolvers (SPI) for list-based access



AccessListResolvers let you plug in computed collections (IDs, codes, emails, etc.) at request time and reference them from `andFilterString/orFilterString`. This is ideal for ACL-style list checks or integrating with external systems that decide which resources a user may access.

How it works (as implemented):

- SPI: `com.e2eq.framework.securityrules.AccessListResolver`
- `key()`: the variable name published to the filter variable bundle (e.g., "accessibleCustomerIds").
- `supports(pctx, rctx, modelClass)`: return true when this resolver applies for the current request and repository model type.
- `resolve(pctx, rctx, modelClass)`: return a `Collection<?>` which will be available as `${<key>}` in filter strings.
- `RuleContext.getFilters` discovers all `AccessListResolver` beans via CDI, calls `supports(...)`, and for those that apply it puts `key() → resolve(...)` into the "extraObjects" map. `MorphiaUtils.buildVariableBundle` merges these into the variable set used by `convertToFilter`.
- In your filter DSL, use:
 - IN with resolver-provided lists: `field: ^${key}`
 - Equality with resolver-provided scalars: `field: ${key}`
 - You can also reference nested principals: `${pcontext.dataDomain.tenantId}`, `${principalId}`, etc.

Examples:

- Restrict Orders to the set of ids returned by a resolver

```
- name: orders-by-acl
  securityURI:
```

```

header: { identity: USER, area: sales, functionalDomain: order, action: view }
body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
andFilterString: "_id:^(accessibleOrderIds)"
effect: ALLOW
priority: 200

```

- Use resolver that returns customer codes and OR with a public segment

```

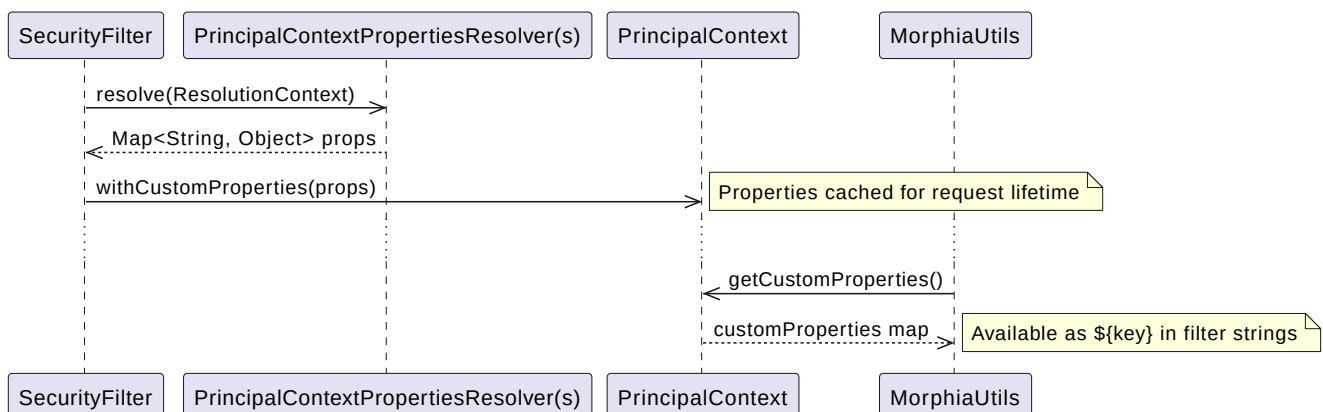
- name: customer-code-or-public
securityURI:
  header: { identity: USER, area: crm, functionalDomain: customer, action: view }
  body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
andFilterString: "code:^(visibleCustomerCodes)"
orFilterString:  "dataDomain.dataSegment:[PUBLIC]"
joinOp: OR
effect: ALLOW
priority: 210

```

Reference implementation (tests/examples): - RuleContext.getFilters collects resolvers and exposes them to the filter StringSubstitutor via MorphiaUtils.buildVariableBundle. - See also: quantum-framework/src/test/java/com/e2eq/framework/securityrules/TestCustomerAccessResolver.java and TestStringAccessResolver.java for sample resolvers.

Guidance: - Keep keys stable and document them; they form your contract between resolver authors and rule authors. - Always scope resolver queries by tenant/realm from PrincipalContext/ResourceContext. - Return empty collections instead of null. An empty IN list yields no matches, which is safe by default.

70.4. PrincipalContextPropertiesResolver (SPI) for custom principal properties



PrincipalContextPropertiesResolver allows applications to enrich PrincipalContext with custom, domain-specific properties during authentication. Unlike AccessListResolver (which is invoked per-query and scoped to specific resources), these properties are resolved once when the principal

context is built and are available throughout the request lifecycle.

Use cases:

- **Associate/Sales Rep binding:** Link a UserProfile to an Associate entity and expose `associateId` for filtering
- **Territory assignments:** Expose `accessibleTerritoryIds` based on the user's role assignments
- **Organizational hierarchies:** Expose `accessibleLocationIds` via ontology relationships
- **Custom attributes:** Any domain-specific property needed for security filtering

How it works:

- SPI: `com.e2eq.framework.model.securityrules.PrincipalContextPropertiesResolver`
- `priority()`: Execution order (lower = earlier). Default: 1000.
- `resolve(ResolutionContext)`: Return a `Map<String, Object>` of properties to add to `PrincipalContext`.
- `ResolutionContext` provides access to:
- `isAnonymous()`, `getPrincipalName()`, `getIdentityRoles()`: Security identity info
- `getUserId()`, `getSubjectId()`: User identifiers
- `getRealm()`, `getDataDomain()`, `getDomainContext()`: Realm and domain context
- `getHeader(name)`: HTTP request headers
- `getJwtClaim(name)`: JWT token claims
- `hasCredentials()`, `getCredentialUserId()`, `getCredentialRoles()`: Database credentials
- `SecurityFilter` discovers all resolvers via CDI, invokes them in priority order, and merges results into `PrincipalContext.customProperties`.
- `MorphiaUtils.createStandardVariableMapFrom` includes custom properties, making them available as `${key}` in filter strings.

Example implementation:

```
package com.myapp.security;

import com.e2eq.framework.model.securityrules.PrincipalContextPropertiesResolver;
import com.e2eq.framework.model.securityrules.ResolutionContext;
import com.e2eq.ontology.repo.OntologyEdgeRepo;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;
import java.util.*;

@ApplicationScoped
public class AssociatePropertiesResolver implements PrincipalContextPropertiesResolver
{

    @Inject
```

```

OntologyEdgeRepo edgeRepo;

@Override
public int priority() {
    return 100; // Run early
}

@Override
public Map<String, Object> resolve(ResolutionContext context) {
    Map<String, Object> properties = new HashMap<>();

    // Skip for anonymous users
    if (context.isAnonymous()) {
        return properties;
    }

    String userProfileId = context.getSubjectId().orElse(context.getUserId());

    // Find Associate linked to this UserProfile via hasUser relationship
    // Associate.hasUser -> UserProfile (functional/1:1)
    Optional<String> associateId = edgeRepo.singleSrcIdByDst(
        context.getDataDomain(), "hasUser", userProfileId);

    if (associateId.isPresent()) {
        properties.put("associateId", associateId.get());

        // Get territories this associate can access
        Set<String> territoryIds = edgeRepo.dstIdsBySrc(
            context.getDataDomain(), "assignedToTerritory", associateId.get());
        properties.put("accessibleTerritoryIds", territoryIds);

        // Get locations via inferred canSeeLocation edge
        Set<String> locationIds = edgeRepo.dstIdsBySrc(
            context.getDataDomain(), "canSeeLocation", associateId.get());
        properties.put("accessibleLocationIds", locationIds);
    }

    return properties;
}
}

```

Using custom properties in rules:

```

# Filter locations to those the associate can see
- name: associate-location-access
  securityURI:
    header: { identity: sales-rep, area: sales, functionalDomain: location, action:
view }
    body: { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }

```

```

andFilterString: "dataDomain.tenantId:${pTenantId}"
orFilterString: "_id:^(accessibleLocationIds)"
effect: ALLOW
priority: 200

```

```

# Filter orders by territory
- name: territory-order-access
  securityURI:
    header: { identity: sales-rep, area: sales, functionalDomain: order, action: view
  }
    body: { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
    '*', dataSegment: '*', resourceId: '*' }
    andFilterString: "dataDomain.tenantId:${pTenantId} AND
territoryId:^(accessibleTerritoryIds)"
    effect: ALLOW
    priority: 200

```

Accessing properties programmatically:

```

// In application code or postconditionScript
PrincipalContext pctx = SecurityContext.getPrincipalContext().get();

// Get a single property
String associateId = (String) pctx.getCustomProperty("associateId");

// Get with type safety
Set<String> locationIds = pctx.getCustomProperty("accessibleLocationIds", Set.class);

// Get all custom properties
Map<String, Object> allProps = pctx.getCustomProperties();

```

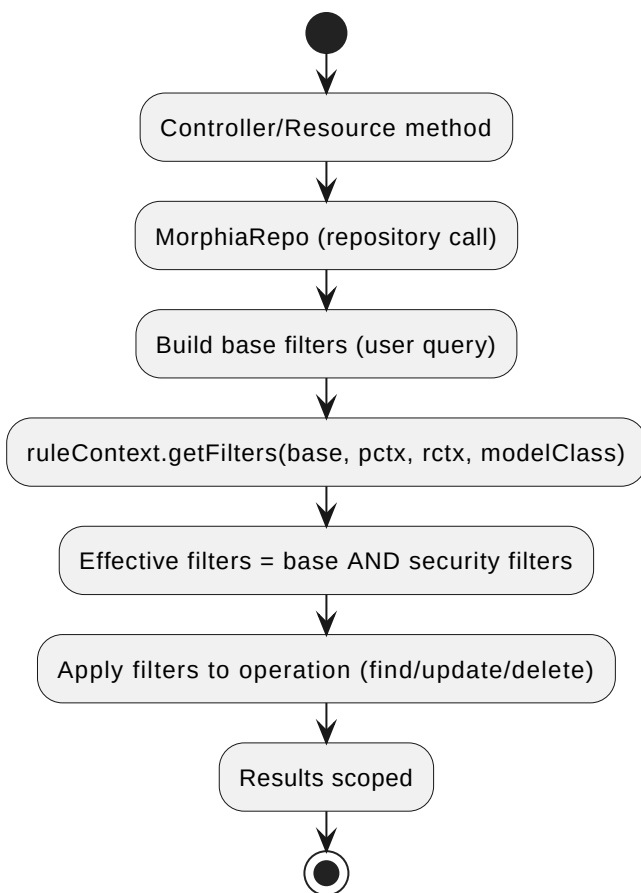
Comparison: PrincipalContextPropertiesResolver vs AccessListResolver

Aspect	PrincipalContextPropertiesResolver	AccessListResolver
When invoked	Once during authentication (eager)	Per query during rule evaluation (lazy)
Context available	SecurityIdentity, JWT claims, HTTP headers, Credentials	PrincipalContext, ResourceContext, ModelClass
Caching	Cached in PrincipalContext for request lifetime	Per-request via RuleContext
Use case	User-level properties constant across all queries	Resource-specific access lists that vary by model/action
Performance	Single invocation per request	Invoked per repository call

Guidance:

- Use `PrincipalContextPropertiesResolver` for properties that apply to the user regardless of what resource they're accessing (e.g., associate ID, territory assignments).
- Use `AccessListResolver` when the access list depends on the specific resource type or action being performed.
- Both mechanisms can coexist and complement each other.
- Return empty maps (not null) from resolvers; resolver failures are logged and skipped, not propagated.
- Properties should not override built-in `PrincipalContext` fields (`userId`, `roles`, etc.).

Where filters are applied (MorphiaRepo methods):



- `getList` / `getListByQuery` / `find`: `RuleContext.getFilters` augments the base filters with the effective security filters before executing the query.
- `count`: same as read paths, ensuring counts reflect scoped visibility.
- `save (create)`: rules typically do not inject filters for inserts; writes are validated by separate preconditions or `postconditionScript`. If your policy encodes write-scope, author `UPDATE/DELETE` rules to guard modifications rather than `CREATE`, unless your domain enforces ownership/tenant on insert.
- `update` / `merge` / `set` / `bulk set`: the security filters are ANDed into the target selection so only documents visible under the policy are affected.
- `delete (by id or by query)`: security filters are applied to the selection to prevent deleting outside the allowed scope.

Implementation reference:

```
// Repositories call into RuleContext to augment filters
List<Filter> filters = ruleContext.getFilters(baseFilters,
    SecurityContext.getPrincipalContext().get(),
    SecurityContext.getResourceContext().get(),
    getPersistentClass());
Query<T> query = datastore.find(getPersistentClass()).filter(filters.toArray(new
Filter[0]));
```

Notes:

- Filters are applied regardless of whether the match decision was ALLOW or DENY; only ALLOW rules contribute filters. DENY rules decide the outcome but do not add filters.
- finalRule: true stops evaluating later rules for both decision and filter contribution.
- If no ALLOW rules match, repositories may still execute with caller-provided filters, but upstream permission checks should have DENIED the action; by convention, most endpoints call checkRules and short-circuit DENY before hitting the database.

Example impact per MorphiaRepo method:

- save: no security filters are injected into the insert operation; enforce ownership/tenant fields via model validation and/or postconditionScript.
- find/findById: adds security filters; if the requested id is outside scope, the result is empty.
- getList/getListByQuery: adds security filters to user-supplied query; scope cannot be broadened by the client.
- update/merge: AND the security filters into the update selector; records outside scope are unaffected.
- delete: AND the security filters into the delete selector; out-of-scope records are not removed.

Example payload:

```
{
  "refName": "defaultUserPolicy",
  "displayName": "Default user policy",
  "principalId": "user",
  "description": "Users can act on their own data; deny dangerous ops in security area",
  "rules": [
    {
      "name": "view-own-resources",
      "description": "Limit reads to owner and default data segment",
      "securityURI": {
        "header": { "identity": "user", "area": "*", "functionalDomain": "*", "action": "*" },
        "body": { "realm": "*", "orgRefName": "*", "accountNumber": "*", "tenantId":
```



```

"\"", "ownerId": "\"", "dataSegment": "\"", "resourceId": "\"" }
    },
    "andFilterString": "
dataDomain.ownerId:${principalId}&&dataDomain.dataSegment:#0",
    "effect": "ALLOW",
    "priority": 300,
    "finalRule": false
  },
  {
    "name": "deny-delete-in-security",
    "securityURI": {
      "header": { "identity": "user", "area": "security", "functionalDomain": "\"",
"action": "delete" },
      "body": { "realm": "\"", "orgRefName": "\"", "accountNumber": "\"", "tenantId":
"\"", "ownerId": "\"", "dataSegment": "\"", "resourceId": "\"" }
    },
    "effect": "DENY",
    "priority": 100,
    "finalRule": true
  }
]
}

```

70.5. Endpoints

All endpoints are relative to `/security/permission/policies`. These are inherited from `BaseResource` and are consistent across entity resources.

- GET `/list`
 - Query params: skip, limit, filter, sort, projection
 - Returns a `Collection<Policy>` with paging metadata; respects X-Realm.
- GET `/id/{id}` and GET `/id?id=...`
 - Fetch a single Policy by id.
- GET `/refName/{refName}` and GET `/refName?refName=...`
 - Fetch a single Policy by refName.
- GET `/count?filter=...`
 - Returns a `CounterResponse` with total matching entities.
- GET `/schema`
 - Returns JSON Schema for Policy.
- POST `/`
 - Create or upsert a Policy (if id is present and matches an existing entity in the selected realm, it is updated).
- PUT `/set?id=...&pairs=field:value`

- Targeted field updates by id. pairs is a repeated query parameter specifying field/value pairs.
- PUT /bulk/setByQuery?filter=...&pairs=...
 - Bulk updates by query. Note: ignoreRules=true is not supported on this endpoint.
- PUT /bulk/setByIds
 - Bulk updates by list of ids posted in the request body.
- PUT /bulk/setByRefAndDomain
 - Bulk updates by a list of (refName, dataDomain) pairs in the request body.
- DELETE /id/{id} (or /id?id=...)
- Delete by id.
- DELETE /refName/{refName} (or /refName?refName=...)
- Delete by refName.
- CSV import/export endpoints for bulk operations:
 - GET /csv – export as CSV (field selection, encoding, etc.)
 - POST /csv – import CSV into Policies
 - POST /csv/session – analyze CSV and create an import session (preview)
 - POST /csv/session/{sessionId}/commit – commit a previously analyzed session
 - DELETE /csv/session/{sessionId} – cancel a session
 - GET /csv/session/{sessionId}/rows – page through analyzed rows
- Index management (admin only):
 - POST /indexes/ensureIndexes/{realm}?collectionName=policy

Headers:

- Authorization: Bearer <token>
- X-Realm: realm identifier (optional but recommended in multi-tenant deployments)

Filtering and sorting:

- filter uses the ANTLR-based DSL (see REST CRUD > Query Language)
- sort uses comma-separated fields with optional +/- prefix; projection accepts a comma-separated field list

70.6. Examples

- Create or update a Policy

```
curl -X POST \
  -H "Authorization: Bearer $JWT" \
  -H "Content-Type: application/json" \
```

```
-H "X-Realm: system-com" \  
https://host/api/security/permission/policies \  
-d @policy.json
```

- List policies for principalId=user

```
curl -H "Authorization: Bearer $JWT" \  
      -H "X-Realm: system-com" \  
      "https://host/api/security/permission/policies/list?filter=principalId:'user'&sort=+refName&limit=50"
```

- Delete a policy by refName

```
curl -X DELETE \  
      -H "Authorization: Bearer $JWT" \  
      -H "X-Realm: system-com" \  
      "https://host/api/security/permission/policies/refName/defaultUserPolicy"
```

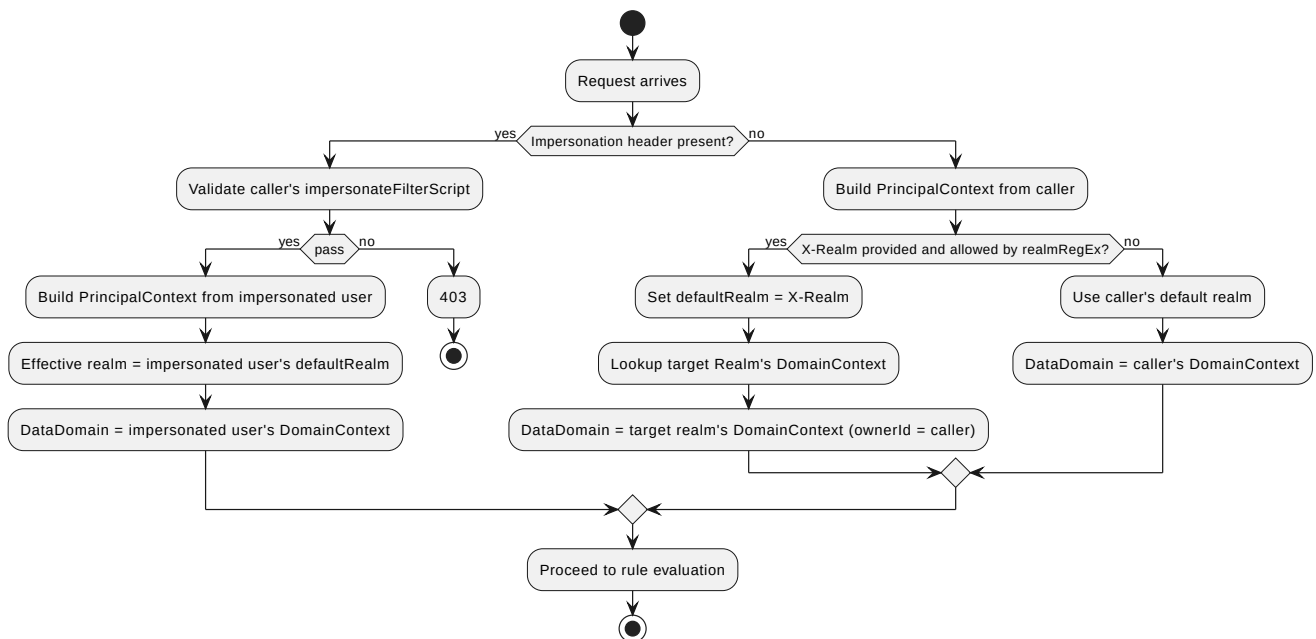
70.7. How changes affect rule bases and enforcement

- Persistence vs. in-memory rules:
- PolicyResource updates the persistent store of policies (one policy per principalId or role with a list of rules).
- RuleContext is the in-memory evaluator used by repositories and resources to enforce permissions. It matches SecurityURIHeader/Body, orders rules by priority, and applies effects and filters.
- Making persisted policy changes effective:
- On startup, migrations (see InitializeDatabase and AddAnonymousSecurityRules) typically seed default policies and/or programmatically add rules to RuleContext.
- When you modify policies via REST, you have two options to apply them at runtime:
 1. Implement a reload step that reads policies from PolicyRepo and rehydrates RuleContext (for example, RuleContext.clear(); then add rules built from current policies).
 2. Restart the service or trigger whatever policy-loader your application uses at boot.
- Tip: If you maintain a background watcher or admin endpoint to refresh policies, keep it tenant/realm-aware and idempotent.
- Evaluation semantics (recap):
- Rules are sorted by ascending priority; the first decisive rule sets the outcome. finalRule=true stops further processing.
- andFilterString/orFilterString contribute repository filters through RuleContext.getFilters(), constraining result sets and write scopes.

- principalId can be a concrete userId or a role; RuleContext considers both the principal and all associated roles.
- Safe rollout:
- Create new policies with a higher numeric priority (lower precedence) first, test with GET /schema and dry-run queries.
- Use realm scoping via X-Realm to stage changes in a non-production realm.
- Prefer DENY with low priority numbers for critical protections.

See also: - Permissions: Matching Algorithm, Priorities, and Multiple Matching RuleBases (sections above) - REST CRUD: Query Language and generic endpoint behaviors

Chapter 71. Realm override (X-Realm) and Impersonation (X-Impersonate)



This section explains how to use the request headers **X-Realm**, **X-Impersonate**-, and **X-Acting-On-Behalf-Of**- alongside permission rule bases. These headers influence which realm (database) a request operates against and, in the case of impersonation, which identity's roles are evaluated by the rule engine.

71.1. What they do (at a glance)

- **X-Realm**: Overrides the target realm (MongoDB database) used by repositories for this request. Your own identity and roles remain the same. **The DataDomain is updated to the target realm's default DomainContext** (while preserving your userId as the ownerId). This ensures that data created via X-Realm is properly scoped to the target realm's tenant/org structure.
- **X-Impersonate-Subject** or **X-Impersonate-UserId**: Causes the request to run as another identity. The effective permissions become those of the impersonated identity (potentially more or less than your own). This is analogous to `sudo` on Unix or to "simulate a user/role" for troubleshooting.
- **X-Acting-On-Behalf-Of-Subject** or **X-Acting-On-Behalf-Of-UserId**: Records that the caller is acting on behalf of another party (for audit/tracking purposes). Unlike impersonation, this does NOT change permissions—the caller retains their own identity and roles.

Only one of **X-Impersonate-Subject** or **X-Impersonate-UserId** may be supplied per request. Supplying both results in a `400/IllegalArgumentException`.

71.2. How the headers integrate with permission evaluation

- Rule matching and effects (ALLOW/DENY) still follow the standard algorithm described earlier.
- With X-Realm (no impersonation):
 - The `PrincipalContext.defaultRealm` is set to the header value (after validation), and repositories operate in that realm.
 - **The DataDomain is updated** from the target realm's `DomainContext` (org, tenant, account, dataSegment), while preserving your `userId` as the `ownerId`.
 - Your own roles and identity remain intact; the rule base is evaluated for your identity and roles but with the target realm's data context.
 - Permission filter strings can use `${defaultRealm}` to dynamically scope queries based on the active realm.
- With impersonation:
 - The `PrincipalContext` is rebuilt from the impersonated user's credential. The effective roles used by the rule engine include the impersonated user's roles; the platform also merges in the caller's security roles from Quarkus `SecurityIdentity`. This means permissions can be a superset; design policy rules accordingly.
 - The effective realm for the request is set to the impersonated user's default realm (not the X-Realm header). If you passed X-Realm, it is still validated (see below) but not used to override the impersonated default realm in the current implementation.
 - The `DataDomain` is set from the impersonated user's `DomainContext`.
- With Acting-On-Behalf-Of:
 - The caller retains their own identity, roles, realm, and `DataDomain`.
 - The `X-Acting-On-Behalf-Of-Subject` or `X-Acting-On-Behalf-Of-UserId` is recorded in the `PrincipalContext` for audit purposes.
 - This is useful for service-to-service calls or when a support agent needs to perform actions that should be attributed to a customer.

71.3. Required credential configuration (CredentialUserIdPassword)

Two fields on `CredentialUserIdPassword` govern whether a user may use these headers:

- `realmRegex` (for X-Realm):
 - A wildcard pattern ("*" matches any sequence; case-insensitive) listing the realms a user is allowed to target with X-Realm.
 - If X-Realm is present but `realmRegex` is null/blank or does not match the requested realm, the server returns 403 Forbidden.
- Examples:

- "*" → allow any realm
- "acme-*" → allow realms that start with acme-
- "dev|stage|prod" is not supported as-is; use wildcards like "dev*" and "stage*" or a combined pattern like "(dev|stage|prod)" only if you store a true regex. The current validator replaces "with "." and matches case-insensitively.
- impersonateFilterScript (for X-Impersonate-*):
- A JavaScript snippet executed by the server (GraalVM) that must return a boolean. It receives three variables: username (the caller's subject), userId (caller's userId), and realm (the requested realm or current DB name).
- If the script evaluates to false, the server returns 403 Forbidden for impersonation.
- If the script is missing (null) and you attempt impersonation, the server rejects the request with 400/IllegalArgumentException.

Example impersonation script (allow only company admins to impersonate in dev realms):

```
// username = caller's subject, userId = caller's userId, realm = requested realm (or
current)
(username.endsWith('@acme.com') && realm.startsWith('dev-'))
```

Tip: Manage these two fields via your auth provider's admin APIs or directly through CredentialRepo in controlled environments.

71.4. End-to-end behavior from SecurityFilter (reference)

The SecurityFilter constructs the PrincipalContext/ResourceContext before rule evaluation:

- X-Realm is read and, if present, validated against the caller's credential.realmRegEx.
- If impersonation headers are present:
 - The caller's credential.impersonateFilterScript is executed. If it returns true, the impersonated user's credential is loaded and used to build the PrincipalContext.
 - The final PrincipalContext carries the impersonated user's defaultRealm and roles (merged with the caller's SecurityIdentity roles), and may copy area2RealmOverrides from the impersonated credential.
- Without impersonation, the PrincipalContext is built from the caller's credential; X-Realm, when valid, sets the defaultRealm for this request.

71.5. Practical differences and use cases

- Realm override (X-Realm):
- Who you are does not change; only where you act changes. Your permissions (as determined by policies attached to your identity/roles) are applied against data in the specified realm.
- **DataDomain changes:** The DataDomain is updated from the target realm's DomainContext (tenant, org, account), ensuring new records are created with the correct scoping.
- Use cases:

- Multi-tenant admin tooling that needs to inspect or repair data in customer realms.
- Reporting or backfills where the same service is pointed at different tenant databases per request.
- Creating data in a target realm with proper DataDomain stamping.
- Impersonation (X-Impersonate-*):
 - Who you are (for authorization purposes) changes. You act with the impersonated identity's permissions; depending on your configuration, additional caller roles may be merged.
 - The DataDomain is set from the impersonated user's credential.
- Use cases:
 - Temporary elevation to an admin identity (sudo-like) for break-glass operations.
 - Simulate what a given role/identity can see/do for troubleshooting or customer support.
 - Testing user-specific permission rules without logging in as that user.
- Acting-On-Behalf-Of (X-Acting-On-Behalf-Of-*):
 - Who you are does NOT change; you retain your own identity, roles, realm, and DataDomain.
 - The "acting on behalf of" information is recorded in the PrincipalContext for audit trails.
- Use cases:
 - Service-to-service calls where the downstream service needs to know the original requestor.
 - Support agents performing actions that should be audited as being done "on behalf of" a customer.
 - Compliance and audit requirements where the chain of delegation must be recorded.

Caveats: - Never set a permissive impersonateFilterScript in production. Keep it restrictive and auditable. - When using both X-Realm and impersonation in one call, be aware that the effective realm will be the impersonated user's default realm; X-Realm is not applied in the impersonation branch in the current implementation. - realmRegex must be populated for any user who needs realm override; leaving it blank effectively disables X-Realm for that user. - Acting-On-Behalf-Of headers do NOT grant any additional permissions; they are purely for audit purposes.

71.6. Examples

- List policies in a different realm using your own identity

```
curl -H "Authorization: Bearer $JWT" \
  -H "X-Realm: acme-prod" \
  "https://host/api/security/permission/policies/list?limit=20&sort=+refName"
```

- Simulate another user by subject while staying in their default realm

```
curl -H "Authorization: Bearer $JWT" \
  -H "X-Impersonate-Subject: 3d8f4e7b-...-idp-subject" \
```



```
"https://host/api/security/permission/policies/list?limit=20"
```

- Attempt impersonation with a realm hint (validated by script; effective realm = impersonated default)

```
curl -H "Authorization: Bearer $JWT" \  
-H "X-Realm: dev-acme" \  
-H "X-Impersonate-UserId: tenant-admin" \  
"https://host/api/security/permission/policies/list?limit=20"
```

Security outcomes in all cases continue to be driven by your rule bases (Policy rules) matched against the effective `PrincipalContext` and `ResourceContext`.

Chapter 72. Data domain assignment on create: DomainContext and DataDomainPolicy

This section explains how Quantum decides which dataDomain is stamped on newly created records, why this decision is necessary in a multi-tenant system, what the default behavior is, and how you can override it globally or per Functional Area / Functional Domain. It also describes the DataDomainResolver interface and the default implementation provided by the framework.

72.1. The problem this solves (and why it matters)

In a multi-tenant platform you must ensure each new record is written to the correct data partition so later reads/updates can be scoped safely. If the dataDomain is wrong or missing, you risk leaking data across tenants or making your own data inaccessible due to mis-scoping.

Historically, Quantum set the dataDomain of new entities to match the creator's credential (i.e., the principal's DomainContext → DataDomain). That default is sensible in many cases, but real systems often need more specific behavior per business area or type. For example: - You may centralize HR records in a single org-level domain regardless of who created them. - Sales invoices for EU customers must live under an EU data segment. - A specific product area might always write into a shared catalog domain separate from the author's tenant.

These needs require a simple, deterministic way to override the default per Functional Area and/or Functional Domain.

72.2. Key concepts recap: DomainContext and DataDomain

DomainContext (on credentials/realms)

captures the principal's scoping defaults (realm, org/account/tenant identifiers, data segment). At request time this is materialized into a DataDomain.

DataDomain

is what gets stamped onto persisted entities and later used by repositories to constrain queries and updates.

If you do nothing, new records inherit the principal's DataDomain.

72.3. The default policy (do nothing and it works)

Out of the box, Quantum preserves the existing behavior: if no policy is configured, the resolver falls back to the authenticated principal's DataDomain. This guarantees compatibility with existing applications.

Concretely: - ValidationInterceptor checks if an entity being persisted lacks a dataDomain. - If

missing, it calls `DataDomainResolver.resolveForCreate(area, domain)`. - The `DefaultDataDomainResolver` first looks for overrides (credential-attached or global); if none match, it returns the principal's `DataDomain` from the current `SecurityContext`.

72.4. Policy scopes: principal-attached vs. global

You can define overrides at two levels: - Principal-attached (per credential): attach a `DataDomainPolicy` to a `CredentialUserIdPassword`. The `SecurityFilter` places this policy into the `PrincipalContext`, so it applies only to records created by that principal. This is useful for VIP service accounts or specific partners. - Global policy: an application-wide `DataDomainPolicy` provided by `GlobalDataDomainPolicyProvider`. If present, this applies when the principal has no specific override for the matching area/domain.

Precedence: principal-attached policy wins over global policy; if neither applies, fall back to the principal's credential domain.

72.5. The policy map and matching

A `DataDomainPolicy` is a small map of rules: `Map<String, DataDomainPolicyEntry> policyEntries`, keyed by "<FunctionalArea>:<FunctionalDomain>" with support for "*" wildcards. The resolver evaluates keys in this order:

1. area:domain (most specific)
2. area:*
3. *:domain
4. : (global catch-all)
5. Fallback to principal's domain if no entry yields a value

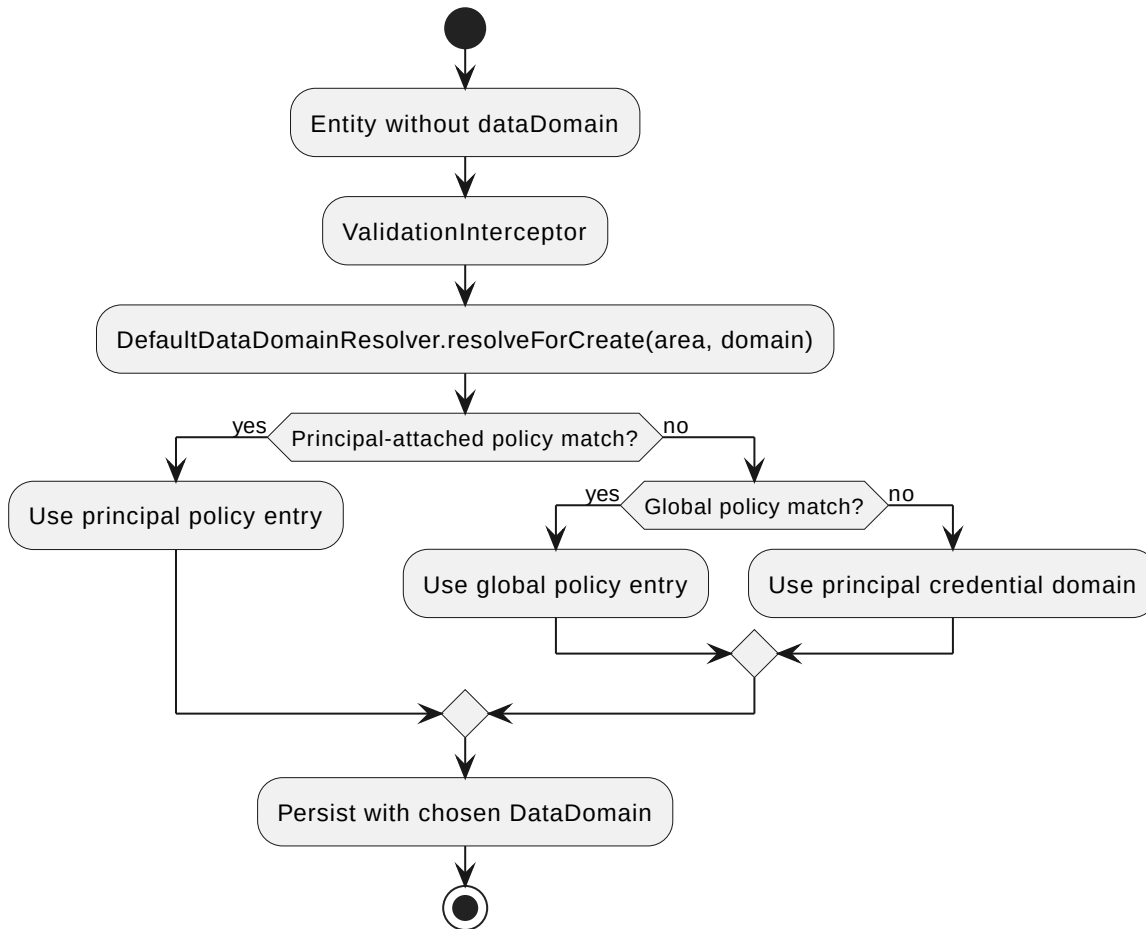
Each `DataDomainPolicyEntry` has a `resolutionMode`: - `FROM_CREDENTIAL` (default): use the principal's credential domain (i.e., the historical behavior). - `FIXED`: use the first `DataDomain` listed in `dataDomains` on the entry.

Example policy definitions (illustrative JSON):

```
{
  "policyEntries": {
    "Sales:Invoice": { "resolutionMode": "FIXED", "dataDomains": [ {"orgRefName":
"ACME", "tenantId": "eu-1", "dataSegment": "INVOICE"} ] },
    "Sales:*":      { "resolutionMode": "FROM_CREDENTIAL" },
    "*:HR":         { "resolutionMode": "FIXED", "dataDomains": [ {"orgRefName":
"GLOBAL", "tenantId": "hr", "dataSegment": "HR"} ] },
    "*:*":          { "resolutionMode": "FROM_CREDENTIAL" }
  }
}
```

Behavior of the above: - Sales:Invoice records always go to the fixed EU invoices domain. - Any

other Sales:* creation uses the creator's credential domain. - All HR records go to a central HR domain. - Otherwise, default to the creator's domain.



72.6. How the resolver works

Interfaces and default implementation:

```

public interface DataDomainResolver {
    DataDomain resolveForCreate(String functionalArea, String functionalDomain);
}

@ApplicationScoped
public class DefaultDataDomainResolver implements DataDomainResolver {
    @Inject GlobalDataDomainPolicyProvider globalPolicyProvider;
    public DataDomain resolveForCreate(String area, String domain) {
        DataDomain principalDD = SecurityContext.getPrincipalDataDomain()
            .orElseThrow(() -> new IllegalStateException("Principal context not providing a
data domain"));
        List<String> keys = List.of(areaOrStar(area)+":"+areaOrStar(domain), areaOrStar
(area)+":*", ".*:"+areaOrStar(domain), ".*.*");
        // 1) principal-attached policy from PrincipalContext
        DataDomain fromPrincipal = resolveFrom(policyFromPrincipal(), keys, principalDD);
        if (fromPrincipal != null) return fromPrincipal;
        // 2) global policy
    }
}
  
```

```

    DataDomain fromGlobal = resolveFrom(globalPolicyProvider.getPolicy().orElse(null),
keys, principalDD);
    if (fromGlobal != null) return fromGlobal;
    // 3) default fallback
    return principalDD;
}
}

```

Integration point: - ValidationInterceptor injects DataDomainResolver and calls it in prePersist when an entity's dataDomain is null. - SecurityFilter propagates a principal's attached DataDomainPolicy (if any) into the PrincipalContext so the resolver can see it.

72.7. When would you want a non-global policy?

Here are a few concrete scenarios: - Centralized HR: All HR Employee records are written to a shared HR domain regardless of the team creating them. This supports a shared-service HR model without duplicating HR data per tenant. - Regulated invoices: In the Sales:Invoice domain for EU, you must write under a specific EU tenantId/dataSegment to satisfy data residency. Other Sales domains can keep default behavior. - Shared catalog: The Catalog:Item domain is a cross-tenant shared catalog maintained by a core team. Writes should go to a canonical catalog domain even when initiated by tenant-specific users. - VIP account override: A particular integration user should always write to a staging domain for testing purposes, while all others use defaults. Attach a small policy to just that credential.

72.8. Relation to tenancy models

The policy mechanism supports both siloed and pooled tenancy: - Siloed tenancy: Most domains default to FROM_CREDENTIAL (each tenant writes to its own partition). Only a few shared services (e.g., HR, catalog) use FIXED to centralize data. - Pooled tenancy: You may lean on FIXED policies more often to route writes into pooled/segment-specific domains (e.g., region, product line), while still enforcing read/write scoping via permissions.

Because the resolver always validates through the principal context and falls back safely, you can introduce overrides gradually without destabilizing existing flows.

72.9. Authoring tips

- Start with no policy and verify your default flows. Add entries only where necessary.
- Prefer specific keys (area:domain) for clarity; use wildcards sparingly.
- Keep FIXED DataDomain objects minimal and valid for your deployment (orgRefName, tenantId, and dataSegment as needed).
- Document any global policy so teams know which areas are centralized.

72.10. API pointers

- `CredentialUserIdPassword.dataDomainPolicy`: optional per-credential overrides (propagated to `PrincipalContext`).
- `GlobalDataDomainPolicyProvider`: holds an optional in-memory global policy (null by default).
- `DataDomainPolicyEntry.resolutionMode`: `FROM_CREDENTIAL` (default) or `FIXED`.
- `DataDomainResolver` / `DefaultDataDomainResolver`: the extension point and default behavior.

72.11. Ontology in Permission Rules (optional)

If you enable the ontology modules, you can author rules that constrain access by semantic relationships, not field paths. This keeps policies stable as your object model evolves.

Key idea

- Materialize edges in Mongo using `OntologyMaterializer` (e.g., `placedInOrg`, `orderShipsToRegion`).
- During rule evaluation, translate a semantic constraint like "has edge `placedInOrg` to `OrgX`" into a set of IDs, and combine that with your query.

How to use

- Preferred: author a rule with a semantic hint and let the application translate it via `ListQueryRewriter`.
- Minimal change path: publish a variable via an `AccessListResolver` and use an IN filter over `_id`.

Example (resolver + IN filter)

- Add an `AccessListResolver` that returns order IDs for which `(tenantId, p="placedInOrg", dst=orgRefName)` exists.
- In your rule's AND filter string (query language), use: `id: ^${idsByPlacedInOrg}`

See also

- Ontology overview and examples: [Ontologies in Quantum](#)
- Integration with Morphia and multi-tenancy: [Integrating Ontology](#)

Operational notes

- Ontology is optional. Enable it per service when the config flag and dependencies are present.
- Always scope edge queries by `tenantId` sourced from `RuleContext`.
- Index edges on `(tenantId, p, dst)` and `(tenantId, src, p)` for performance.

72.11.1. Rule language: add `hasEdge()`

We introduce a policy function/operator to reference ontology edges directly from rules. This lets policies constrain access by semantic relationships instead of field paths.

Signature

- `hasEdge(predicate, dstIdOrVar)`
- `hasIncomingEdge(predicate, srcIdOrVar)`

Parameters

- `predicate`: String name of the ontology predicate (e.g., "placedInOrg", "orderShipsToRegion").
- `dstIdOrVar` / `srcIdOrVar`: Either a concrete id/refName or a variable resolved from RuleContext (e.g., `principal.orgRefName`, `request.region`, `${principalId}`).

Semantics

- **hasEdge**: The rule grants/filters entities (sources) for which an edge exists: (tenantId, src = entity._id, p = predicate, dst = resolvedDst).
- **hasIncomingEdge**: The rule grants/filters entities (targets) for which an edge exists: (tenantId, src = resolvedSrc, p = predicate, dst = entity._id). Useful for finding entities that are destinations of a relationship (e.g., "find locations reachable FROM an associate").
- Multi-tenant safety: `tenantId` is always taken from RuleContext/DomainContext.

Composition

- `hasEdge` and `hasIncomingEdge` can be combined with existing rule clauses (and/or/not) and other filters (states, tags, ownerId, etc.).

Examples

- Allow viewing Orders in the caller's org (including ancestors via ontology closure):
- allow VIEW Order when `hasEdge("placedInOrg", principal.orgRefName)`
- Restrict list to Orders shipping to a region chosen in request:
- allow LIST Order when `hasEdge("orderShipsToRegion", request.region)`
- Allow viewing Locations reachable from the current user:
- allow VIEW Location when `hasIncomingEdge("canAccessLocation", ${principalId})`



In `andFilterString` and `orFilterString`, use standard BIAPI syntax: - 2-argument existence check: `hasEdge(predicate, dst)` and `hasIncomingEdge(predicate, src)`. - 3-argument qualified check: `hasEdge(predicate, dst, { edgeFilter })` and `hasIncomingEdge(predicate, src, { edgeFilter })`. - Query macros: `@asOf(timestamp)` for bitemporal validity and `@minConfidence(threshold)` for probabilistic trust. In `postconditionScript`, use the JavaScript helper counterparts: `hasEdge(predicate, dst, propsOrFn)` and `hasIncomingEdge(predicate, src, propsOrFn)`.

72.12. Business Scenarios & Policy Enforcement Solutions

72.12.1. Business Scenario 1: Healthcare Temporary On-Call Staffing & Regulatory Audits

In a hospital network, physicians hold temporary appointments (e.g. "Acting Chief of Emergency Medicine" from Monday 08:00 to Friday 17:00). Policies must guarantee zero-latency expiration without relying on deletion batch cron jobs, while preserving historical audit trails for HIPAA compliance.

Solution in Permission Rules: Using the `@asOf(ts)` macro inside a qualified edge filter ensures that access expires automatically the moment `now >= validTo`, and enables point-in-time compliance audits:

```
- name: emergency-department-chart-access
  priority: 100
  securityURI:
    header: { identity: PHYSICIAN, area: clinical, functionalDomain: charts, action:
view }
    body: { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
  effect: ALLOW
  andFilterString: >
    hasEdge("actingChiefOf", "DEPT-EMERGENCY", { @asOf(2026-09-20T22:00:00Z) &&
status:"ACTIVE" })
```

72.12.2. Business Scenario 2: Aerospace Defense Project Classification Guard

In defense environments, the association between an engineering contractor and a classified stealth program (`PROJECT-TITAN`) has a security classification of `SECRET` with an `ITAR` handling compartment. An uncleared sub-contractor querying their project dashboard must not discover that `PROJECT-TITAN` exists or is linked to their team.

Solution in Permission Rules: Using Mandatory Access Control (MAC) edge properties pushed into the compound index scan (`idx_domain_security`), the relationship is withheld at the database level:

```
- name: defense-classified-project-view
  priority: 105
  securityURI:
    header: { identity: CONTRACTOR, area: engineering, functionalDomain: projects,
action: view }
    body: { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
  effect: ALLOW
  andFilterString: >
```



```
hasEdge("assignedToProject", "PROJECT-TITAN", {
  securityLabel: "SECRET" &&
  compartments: ^ ["ITAR"]
})
```

Under the hood

- Policy evaluation resolves `dstIdOrVar` / `srcIdOrVar` against `RuleContext` (for example, `principal.orgRefName` → "OrgP").
- The `list/filter` query is rewritten using `ListQueryRewriter.rewriteForHasEdge(...)` or `rewriteForHasIncomingEdge(...)`, which turns the predicate and resolved ID into a set of target/source IDs and merges it with the base query efficiently.
- MongoDB compound indexes `idx_domain_p_dst_props`, `idx_domain_temporal`, `idx_domain_security`, and `idx_domain_confidence` accelerate edge lookups directly at the datastore layer.

Full end-to-end example (implementation pattern)

1) Author a rule (illustrative YAML/pseudocode)

```
- name: list-orders-by-org
  priority: 100
  securityURI:
    header:
      identity: USER
      area: sales
      functionalDomain: order
      action: list
    body:
      realm: '*'
      accountNumber: '*'
      tenantId: '*'
      dataSegment: '*'
      ownerId: '*'
      resourceId: '*'
  effect: ALLOW
  # Semantic constraint expressed via ontology helper in postconditionScript
  postconditionScript: hasEdge("placedInOrg", pcontext?.dataDomain?.orgRefName) ===
true
```

Another example using `hasIncomingEdge`:

```
- name: view-reachable-locations
  priority: 110
  securityURI:
    header:
      identity: USER
      area: core
      functionalDomain: location
```

```

    action: view
  body:
    realm: '*'
    accountNumber: '*'
    tenantId: '*'
    dataSegment: '*'
    ownerId: '*'
    resourceId: '*'
  effect: ALLOW
# Find locations that the current user has a "canAccessLocation" edge to
andFilterString: "hasIncomingEdge(canAccessLocation, ${principalId})"

```

2) Evaluate policy and apply constraint in the repository/list path

```

import com.e2eq.ontology.policy.ListQueryRewriter;
import com.e2eq.ontology.repo.OntologyEdgeRepo;
import com.mongodb.client.model.Filters;
import org.bson.conversions.Bson;

// Injected once per service when ontology is enabled (Quarkus CDI)
@jakarta.inject.Inject OntologyEdgeDao edgeDao;
ListQueryRewriter rewriter = new ListQueryRewriter(edgeDao);

// Inside your list method, after building the base filter
String tenantId = ruleContext.getRealmId(pctx, rctx); // or from DomainContext
String predicate = "placedInOrg";
String orgRefName = pctx.getDataDomain().getOrgRefName(); // resolves
principal.orgRefName

Bson base = Filters.and(existingFilters...);
Bson rewritten = rewriter.rewriteForHasEdge(base, tenantId, predicate, orgRefName);
var results = datastore.getDatabase().getCollection("orders").find(rewritten).
iterator();

// Example for hasIncomingEdge in code
Bson locationRewritten = rewriter.rewriteForHasIncomingEdge(locationBase, tenantId,
"canAccessLocation", principalId);

```

3) Morphia-typed queries alternative

```

// If you're using Morphia's typed Query API
Set<String> ids = edgeDao.srcIdsByDst(tenantId, "placedInOrg", orgRefName);
if (ids.isEmpty()) {
    return java.util.List.of(); // short-circuit
}
query.filter(dev.morphia.query.filters.Filters.in("_id", ids));

```

Developer requirements and checklist

- Enable ontology (optional feature): set `e2eq.ontology.enabled=true` in your app and add module dependencies.
- Provide a `TBox (OntologyRegistry)` and run `OntologyMaterializer` on entity changes to keep edges up to date.
- Inject `EdgeDao` as a CDI bean (`@Inject`); indexes are ensured automatically at startup.
- Always pass `tenantId` from `RuleContext/DomainContext`; never cross tenants.
- When using variables on the RHS (`dstIdOrVar`), ensure the `RuleContext` exposes them (for example, `principal.orgRefName` or `request.region`).
- Monitor provenance (`edge.prov`) and re-materialize edges when intermediate nodes change.

Notes

- `hasEdge()` is a policy function; it is evaluated before constructing database filters. It is not part of the core BIAPI query grammar.
- If ontology is disabled, skip the rewrite (return base) or configure a no-op implementation so policies that include `hasEdge` are rejected early with a clear error message.

72.13. Label resolution SPI and `hasLabel()` in rules

Labels are a lightweight way to attach semantic markers to principals and resources, and then use them in rule scripts. Quantum provides a pluggable label resolution SPI and a script helper to check labels during policy evaluation.

Why labels? - Decouple policy from rigid fields (e.g., “VIP”, “HARD_DELETE_DISABLED”, “B2B”). - Compute labels from multiple sources (simple tags, `advancedTags`, dynamic attributes, or custom derivations).

Components

- `LabelResolver` (SPI):

```
public interface LabelResolver {
    boolean supports(Class<?> type);
    java.util.Set<String> resolveLabels(Object entity);
}
```

- `DefaultLabelResolver` (built-in): Applies to `UnversionedBaseModel`.
- Collects tags (`String[]`), `advancedTags.name`, and optionally any `dynamicAttributes` whose `isInheritable()==true`, using best-effort reflection.
- `LabelService`: Aggregates all `LabelResolver` beans via CDI and delegates to the first that supports the entity type. Also supports annotation-based extraction (see below).
- Annotations (optional, opt-in at your model):

```
@LabelSource(method = "computeLabels") // class-level method returning
```

```
Collection<String>, String[], or String
public class MyEntity { ... }

public class MyEntity {
    @LabelField                // field can be Collection<String>, String[], or
    String
    private java.util.List<String> policyLabels;
}
```

Availability in rule scripts

During policy evaluation, RuleContext resolves labels for both principal and resource contexts and installs a script helper:

- hasLabel(label: String) → Boolean
- Returns true if the current resource context has the specified label.
- Label resolution uses LabelService (including custom resolvers and annotations) at evaluation time.

Examples (postcondition script)

```
// Allow if the resource is labeled VIP
hasLabel("VIP")

// Combine with other helpers when ontology is enabled
hasLabel("RESTRICTED") && !hasEdge("placedInOrg", "OrgX")
```

Extending labels for your domain

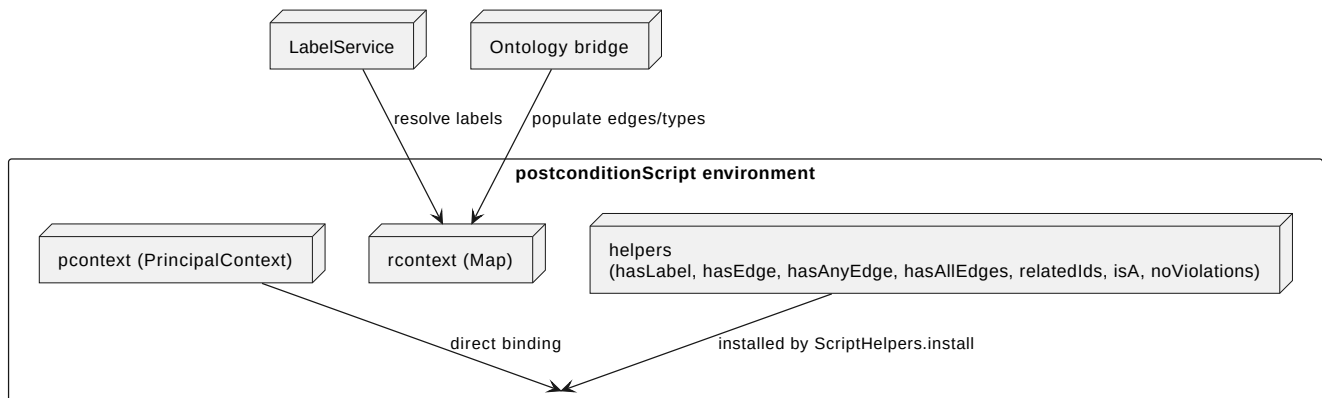
- Add a new resolver:

```
@jakarta.enterprise.context.ApplicationScoped
public class InvoiceLabelResolver implements LabelResolver {
    public boolean supports(Class<?> type) { return type.getName().endsWith("Invoice"); }
    public java.util.Set<String> resolveLabels(Object entity) {
        var out = new java.util.LinkedHashSet<String>();
        var inv = (com.example.Invoice) entity;
        if (inv.isHighValue()) out.add("HIGH_VALUE");
        if (inv.isPastDue()) out.add("PAST_DUE");
        return out;
    }
}
```

- Or annotate your model with @LabelSource/@LabelField to contribute labels without writing a resolver.

Notes - Labels are resolved best-effort; failures are swallowed to keep policy evaluation robust. - If multiple resolvers exist, the first that supports the type wins. Prefer narrow supports() checks. - Keep label names stable; treat them as part of your policy contract.

Chapter 73. Script Helpers reference (when enabled)



Script helpers are small functions injected into the `postconditionScript` environment to make common policy checks concise. They are provided by `com.e2eq.ontology.policy.ScriptHelpers.install(...)` when the optional ontology/label modules are present. In all cases, `postconditionScript` also has direct access to:

- `pcontext`: the Java `PrincipalContext` for the caller
- `rcontext`: a lightweight map for resource-related helper data (labels, types, edges, violations) when populated by your application

Availability notes: - Labels are made available by `RuleContext` via `LabelService`. `hasLabel()` checks the resource's labels only. - Edges, types, and violations must be populated by your application into the `rcontext` map before calling `RuleContext.runScript` (typically via an ontology bridge). If absent, edge/type helpers return `false/empty`.

Helpers and examples:

- `isA(type: String) → Boolean`
- Purpose: Check whether the resource has a semantic type in `rcontext.types`.
- Example:

```
// Allow when the resource is of type "Invoice"
isA("Invoice")
```

- `hasLabel(label: String) → Boolean`
- Purpose: Check whether the resource has a policy label (labels resolved via `LabelService`).
- Example:

```
// Permit view if resource is labeled PUBLIC
hasLabel("PUBLIC")
```

- `hasEdge(predicate: String, dst: String|null) → Boolean`
- Purpose: True if an ontology edge with property `p=predicate` exists from the current resource to `dst`. If `dst` is null, true if any `dst` exists for the predicate.
- Example:

```
// Require that this resource is placed in the caller's org
hasEdge("placedInOrg", pcontext?.dataDomain?.orgRefName)
```

- `hasIncomingEdge(predicate: String, src: String|null) → Boolean`
- Purpose: True if an ontology edge with property `p=predicate` exists from `src` to the current resource. If `src` is null, true if any `src` exists for the predicate.
- Example:

```
// Permit view if the current resource is a location reachable from the caller
hasIncomingEdge("canAccessLocation", principalId)
```

- `hasAnyEdge(predicate: String, dsts: Collection<String>) → Boolean`
- Purpose: True if there is at least one edge to any of the destinations.
- Example:

```
// Allow if the order ships to any of the selected regions
hasAnyEdge("orderShipsToRegion", ["NA", "EU"]) == true
```

- `hasAllEdges(predicate: String, dsts: Collection<String>) → Boolean`
- Purpose: True only if there is an edge for every destination in the list.
- Example:

```
// Require all compliance flags to be present
hasAllEdges("hasComplianceFlag", ["KYC", "AML"]) == true
```

- `relatedIds(predicate: String) → List<String>`
- Purpose: Return the list of `dst` ids for edges with property `p=predicate`.
- Example:

```
// Use in conjunction with a filter list variable (via AccessListResolver)
var projectIds = relatedIds("belongsToProject");
projectIds && projectIds.length > 0
```

- `noViolations() → Boolean`
- Purpose: True if `rcontext.violations` is empty. Helpful when upstream validators populate

violations.

- Example:

```
// Deny action if any violations were detected earlier in the pipeline  
noViolations() === true
```

Best practices: - Prefer simple boolean expressions; keep postconditionScript fast and side-effect free. - When using ontology helpers, always scope your edge materialization by tenantId to avoid cross-tenant leakage. - Combine helpers with SecurityURIBody scoping and repository filters for defense in depth.

Cross-references: - Ontology usage and hasEdge: see [Ontology in Permission Rules](#). - Label SPI and hasLabel: see [Label resolution SPI](#). - AccessListResolver for list variables in filters: see "AccessListResolvers (SPI) for list-based access" above.

Chapter 74. Scoped Action Enablement

Scoped action enablement is the framework-level answer to the question:

"Why is this action unavailable right now?"

Permission rules already answer:

"May this identity perform `area / functionalDomain / action`?"

Action enablement adds the missing operational layer so applications, shells, and setup dashboards can also answer:

- is the action allowed by policy?
- is the action enabled for this rollout or cohort?
- is the action operationally ready, or is some setup still missing?

This guide documents the runtime API, manifest model, SPI contracts, and the framework's built-in dependency types.

74.1. Core principle

The atomic unit is still the same scoped action tuple already used by Quantum security:

```
area / functionalDomain / action
```

Examples:

- `integration / exchange / create`
- `integration / guided_integration / create`
- `integration / workflow / create`
- `system / action-enablement / check`

Quantum intentionally does not introduce a second authorization abstraction. The same tuple defined by `@FunctionalMapping` and `@FunctionalAction` remains canonical for:

- permission checks
- UI action derivation
- action enablement manifests
- operational blocker reporting

Capabilities are still useful, but they are an application-level grouping over one or more scoped actions, not a replacement for the scoped action tuple itself.

74.2. What the framework returns

Each evaluated action produces four booleans:

Field	Meaning
<code>allowed</code>	The permission rule stack allows the action for the evaluated identity and data domain
<code>enabled</code>	Rollout and administrative toggles, especially feature flags, say the action should be exposed
<code>ready</code>	Operational prerequisites such as settings, referenced entities, or external runtime dependencies are satisfied
<code>usable</code>	Convenience result equal to <code>allowed && enabled && ready</code>

This separation is important because it points the user to the right remediation path:

- `allowed=false` usually means a policy or role issue
- `enabled=false` usually means a feature flag or rollout issue
- `ready=false` usually means configuration, provisioning, or runtime setup is incomplete

74.3. Runtime models

The runtime contracts live in `quantum-action-enablement-models`.

```
public class ScopedActionRef {
    String area;
    String functionalDomain;
    String action;
}

public class DependencyCheckRef {
    String type;
    String refName;
    Map<String, Object> config;
}

public class ScopedActionRequirement {
    ScopedActionRef scopedAction;
    String displayName;
    String description;
    List<DependencyCheckRef> dependencies;
}

public class ScopedActionEnablementStatus {
    ScopedActionRef scopedAction;
    boolean allowed;
    boolean enabled;
}
```

```

    boolean ready;
    boolean usable;
    List<EnablementBlocker> blockers;
}

```

Each blocker is machine-readable and user-displayable:

```

public class EnablementBlocker {
    EnablementImpact impact; // ALLOWED, ENABLED, READY
    String type;             // permission, feature-flag, setting-present, ...
    String code;             // permission-denied, setting-missing, ...
    String message;          // human-readable explanation
    String severity;         // warn or error
    Map<String, Object> metadata;
}

```

74.4. REST API

The runtime API lives in `quantum-action-enablement-quarkus` and is exposed by `ActionEnablementResource`.

Endpoints:

- `POST /system/actions/enablement/check`
- `GET /system/actions/enablement/manifest`

The resource itself is mapped as:

- `area=system`
- `functionalDomain=action-enablement`
- `action=check|view`

74.4.1. Check request

```

{
  "identity": "tenant-admin",
  "realm": "acme",
  "roles": ["admin"],
  "orgRefName": "acme-org",
  "accountNumber": "1000000001",
  "tenantId": "acme-tenant",
  "dataSegment": 0,
  "ownerId": "tenant-admin",
  "scope": "api",
  "actions": [
    {
      "area": "integration",

```

```

    "functionalDomain": "guided_integration",
    "action": "create"
  },
  {
    "area": "integration",
    "functionalDomain": "workflow",
    "action": "create"
  }
]
}

```

74.4.2. Check response

```

{
  "results": [
    {
      "scopedAction": {
        "area": "integration",
        "functionalDomain": "guided_integration",
        "action": "create"
      },
      "allowed": true,
      "enabled": false,
      "ready": false,
      "usable": false,
      "blockers": [
        {
          "impact": "ENABLED",
          "type": "feature-flag",
          "code": "feature-flag-disabled",
          "message": "Feature flag 'guided-integrations' is disabled.",
          "severity": "warn"
        },
        {
          "impact": "READY",
          "type": "setting-present",
          "code": "setting-missing",
          "message": "Configuration setting 'quantum.integration.shared-storage.root'
is not set.",
          "severity": "error"
        }
      ]
    }
  ]
}

```

74.4.3. Manifest response

GET `/system/actions/enablement/manifest` returns the registered `ScopedActionRequirement` entries after all CDI contributors have been aggregated.

That endpoint is useful for:

- setup dashboards
- shell tools
- test fixtures
- verifying what requirements are active for a given application build

74.5. Manifest model and contributor SPI

The dependency relationships between settings, feature flags, and scoped actions are expressed as `ScopedActionRequirement` entries.

Applications contribute requirements through CDI:

```
public interface ScopedActionRequirementContributor {  
  
    default int priority() {  
        return 100;  
    }  
  
    Collection<ScopedActionRequirement> requirements();  
}
```

The framework aggregates contributors through `ScopedActionRequirementRegistry`:

```
public interface ScopedActionRequirementRegistry {  
  
    Optional<ScopedActionRequirement> find(ScopedActionRef ref);  
  
    List<ScopedActionRequirement> list();  
}
```

This is where you encode relationships such as:

- `integration / guided_integration / create` depends on feature flag `guided-integrations`
- `integration / exchange / create` depends on tenant filesystem provisioning
- `system / export / view` depends on a specific config property being present

74.5.1. Contributor precedence

Contributors are sorted by ascending `priority()`. Requirements are then indexed by scoped action

key.

Practical implication:

- later contributors for the same scoped action replace earlier entries
- the framework's own contributor can provide a default manifest entry
- applications can override or specialize that entry with their own contributor

74.6. Resolver SPI

Each dependency type is resolved by an `ActionDependencyResolver`:

```
public interface ActionDependencyResolver {  
  
    String supportsType();  
  
    DependencyResolutionResult evaluate(  
        DependencyCheckRef dependency,  
        EnablementEvaluationContext context  
    );  
}
```

`EnablementEvaluationContext` carries the runtime inputs:

- `identity`
- `realm`
- `roles`
- `scope`
- `DataDomain`
- target `ScopedActionRef`

This gives each resolver enough context to evaluate readiness in the correct tenant, realm, and user scope.

74.7. Built-in OSS dependency types

The open-source framework currently ships with these resolver types:

Dependency type	Meaning	Impact
<code>permission</code>	Run the existing permission engine for the scoped action	<code>ALLOWED</code>
<code>feature-flag</code>	Require that a <code>FeatureFlag</code> exists and is enabled in the active realm	<code>ENABLED</code>
<code>setting-present</code>	Require that a named configuration property exists and is non-blank	<code>READY</code>

Dependency type	Meaning	Impact
<code>entity-exists</code>	Require that a referenced entity exists in the current realm	<code>READY</code>

Representative blocker codes include:

- `permission-denied`
- `feature-flag-ref-missing`
- `feature-flag-missing`
- `feature-flag-disabled`
- `setting-name-missing`
- `setting-missing`
- `entity-dependency-invalid`
- `entity-model-unresolved`
- `entity-missing`

74.8. Evaluation behavior

`ScopedActionEnablementService` evaluates dependencies for each requested action and aggregates blockers by impact.

At a high level:

1. Resolve the manifest entry for the scoped action.
2. Build an `EnablementEvaluationContext` from the request plus current security context defaults.
3. Resolve each dependency by type.
4. Collect blockers from all failing dependencies.
5. Derive `allowed`, `enabled`, `ready`, and `usable`.

Important framework behavior:

- If no manifest entry exists, the service creates a synthetic fallback requirement with only a `permission` dependency.
- The response also includes a `manifest-missing` blocker with `impact=READY`.
- If a dependency type has no registered resolver, the response includes `unsupported-dependency-type`.

This means action enablement is designed to fail visibly, not silently.

74.9. Example contributor

The framework itself contributes manifest entries for the action-enablement API. Applications then contribute their own operational requirements.

```

@ApplicationScoped
public class MyActionEnablementManifestContributor
    implements ScopedActionRequirementContributor {

    @Override
    public Collection<ScopedActionRequirement> requirements() {
        return List.of(
            ScopedActionRequirement.builder()
                .scopedAction(ScopedActionRef.builder()
                    .area("integration")
                    .functionalDomain("guided_integration")
                    .action("create")
                    .build())
                .displayName("Generate Guided Integration")
                .description("Requires policy, rollout enablement, and baseline
setup.")
                .dependencies(List.of(
                    DependencyCheckRef.builder().type("permission").build(),
                    DependencyCheckRef.builder()
                        .type("feature-flag")
                        .refName("guided-integrations")
                        .build(),
                    DependencyCheckRef.builder()
                        .type("setting-present")
                        .refName("quantum.integration.shared-storage.root")
                        .build()
                ))
                .build()
        );
    }
}

```

This is the practical link between settings and capabilities:

- the capability is what the product or UI wants to describe
- the scoped action is the framework key
- the manifest records which settings, feature flags, and runtime dependencies attach to that scoped action

74.10. Relationship to permissions and UI actions

Action enablement complements existing security behavior rather than replacing it.

- `@FunctionalMapping` and `@FunctionalAction` define the scoped action tuple.
- Permission rules decide whether the action is authorized.
- `fillUIActions()` still reports which actions are allowed from the permission perspective.
- Action enablement explains why the action should still be disabled or annotated because rollout

or readiness conditions are not met.

This makes it especially useful for:

- onboarding flows
- setup dashboards
- admin consoles
- explainable disabled buttons
- CLI and MCP tooling

74.11. Enterprise extensions

The framework module is intentionally generic. Enterprise modules can contribute deeper operational resolvers through the same `ActionDependencyResolver` SPI.

Current enterprise resolvers include checks such as:

- `managed-secret-configured`
- `tenant-filesystem-provisioned`
- `standard-mounts-present`
- `workflow-runtime-reachable`

Those enterprise resolvers answer questions like:

- has the tenant filesystem been provisioned?
- do required standard mounts exist?
- is the managed secret configured?
- can the workflow runtime be reached?

The framework documentation stops at the extension point and shared response model. Enterprise-specific operational semantics should be documented in the enterprise module.

74.12. Guidance

- Keep authorization canonical in permission rules. Do not re-implement policy logic inside custom readiness resolvers.
- Use `feature-flag` for rollout state, not for durable authorization.
- Use `setting-present` and `entity-exists` for generic readiness checks.
- Put domain-heavy infrastructure checks in application or enterprise resolvers.
- Keep blocker codes stable because UIs and automation may key off them.
- Treat capabilities as UX groupings over scoped actions, not as a replacement for `area / functionalDomain / action`. == Security Annotations: `FunctionalMapping` and `FunctionalAction`

Quantum uses annotations to declare a model or resource's functional area, domain, and actions for security evaluation. This replaces the legacy `bmFunctionalArea()` and `bmFunctionalDomain()` methods.

74.12.1. @FunctionalMapping

Use `@FunctionalMapping` on model classes or resource classes to declare their business placement:

```
import com.e2eq.framework.annotations.FunctionalMapping;

@Entity
@FunctionalMapping(area = "catalog", domain = "product")
public class Product extends BaseModel {
    // No need to override bmFunctionalArea/bmFunctionalDomain
}
```

```
@Path("/products")
@FunctionalMapping(area = "catalog", domain = "product")
public class ProductResource extends BaseResource<Product, ProductRepo> {
    // All methods inherit area/domain from class annotation
}
```

74.12.2. @FunctionalAction

Use `@FunctionalAction` on JAX-RS resource methods when the action differs from the HTTP verb default:

```
@Path("/products")
public class ProductResource {

    @POST
    @FunctionalAction("CREATE") // Explicit, though POST implies CREATE
    public Product create(Product payload) {
        return productRepo.save(payload);
    }

    @GET
    @Path("/{id}")
    // No annotation needed - GET implies VIEW
    public Product get(@PathParam("id") String id) {
        return productRepo.findById(id);
    }

    @PUT
    @Path("/{id}/approve")
    @FunctionalAction("APPROVE") // Custom action beyond standard CRUD
    public Product approve(@PathParam("id") String id) {
```

```

        Product p = productRepo.findById(id);
        p.setStatus("APPROVED");
        return productRepo.save(p);
    }
}

```

74.12.3. Default Action Mapping

When `@FunctionalAction` is not present, actions are inferred from HTTP methods:

HTTP Method	Default Action
GET	VIEW
POST	CREATE
PUT	UPDATE
PATCH	UPDATE
DELETE	DELETE

74.12.4. How the Framework Uses These Annotations

SecurityFilter

- Reads `@FunctionalMapping` from the matched resource class for area/domain
- Reads `@FunctionalAction` from the method, or infers from HTTP method
- Falls back to path-based parsing if annotations are missing

Scoped Action Enablement

- Reuses the same `area / functionalDomain / action` tuple as the dependency-tracking key for runtime action enablement
- Lets the framework answer more than "is this policy allowed?" and also report whether the action is feature-enabled and operationally ready
- Treats capabilities as an application-level composition concept built on top of one or more scoped actions, not as a new core security primitive

MorphiaRepo.fillUIActions

- Uses `@FunctionalMapping` on model classes to resolve allowed UI actions
- Falls back to legacy `bmFunctionalArea()/bmFunctionalDomain()` methods

PermissionResource

- Prefers `@FunctionalMapping` when listing functional domains
- Falls back to legacy methods when annotation is missing

74.12.5. Migration from Legacy Methods

Current (Legacy) Approach

```
@Entity
public class Product extends BaseModel {
    @Override
    public String bmFunctionalArea() {
        return "Catalog";
    }

    @Override
    public String bmFunctionalDomain() {
        return "Product";
    }
}
```

New (Recommended) Approach

```
@Entity
@FunctionalMapping(area = "catalog", domain = "product")
public class Product extends BaseModel {
    // Clean - no method overrides needed
}
```

Transitional Support

You can use both during migration: - If `@FunctionalMapping` is present, it takes precedence - If missing, legacy methods are used as fallback - Plan to remove legacy methods in future releases

74.12.6. Best Practices

Consistent Naming

Use lowercase, kebab-case for areas and domains:

```
@FunctionalMapping(area = "supply-chain", domain = "purchase-order")
@FunctionalMapping(area = "catalog", domain = "product")
@FunctionalMapping(area = "identity", domain = "user-profile")
```

Resource vs Model Annotations

- **Prefer model annotations** for consistency across all usage
- Use resource annotations only when the resource handles multiple model types
- Avoid duplicating annotations on both model and resource for the same entity

- Be consistent with naming because the same tuple may be used by policies, UI action derivation, and scoped action enablement manifests

Custom Actions

Define custom actions for business operations beyond CRUD:

```
@PUT
@Path("/{id}/publish")
@FunctionalAction("PUBLISH")
public Product publish(@PathParam("id") String id) { ... }

@POST
@Path("/{id}/duplicate")
@FunctionalAction("DUPLICATE")
public Product duplicate(@PathParam("id") String id) { ... }

@DELETE
@Path("/{id}/archive")
@FunctionalAction("ARCHIVE") // Soft delete vs hard DELETE
public void archive(@PathParam("id") String id) { ... }
```

74.12.7. Bypassing Data Scoping for System Operations

Some endpoints perform system-level operations that don't operate on tenant-scoped data entities. For example:

- Database migration operations (index creation, schema changes)
- System initialization and setup endpoints
- Administrative operations that don't query/modify tenant-scoped records

For these endpoints, permission rules may return a **SCOPED** decision with data-level constraints like `dataDomain.tenantId:{$pTenantId}`. However, since there's no data entity to scope on, these constraints can't be meaningfully applied.

Use `bypassDataScoping = true` to mark such endpoints:

```
@POST
@Path("/indexes/applyAllIndexes/{realm}")
@RolesAllowed("admin")
@FunctionalMapping(area = "migration", domain = "indexes")
@FunctionalAction(value = "APPLY_ALL_INDEXES", bypassDataScoping = true)
public void applyAllIndexes(@PathParam("realm") String realm) {
    migrationService.applyAllIndexes(realm);
}
```

How It Works

When `bypassDataScoping = true`:

1. **Permission checks still apply** - The user must still be authorized (ALLOW decision) based on their roles and the permission rules
2. **SCOPED constraints are ignored** - Data-level filters (like `dataDomain.tenantId:${pTenantId}`) are bypassed because they're not applicable
3. **Audit logging** - The framework logs when data scoping is bypassed for transparency

Security Considerations



Only use `bypassDataScoping = true` for endpoints that genuinely do not operate on tenant-scoped data. Misuse could expose cross-tenant data access vulnerabilities.

Use Case	bypassDataScoping	Why
Database migrations	true	Index/schema operations don't involve tenant data
System initialization	true	Setup operations are realm-wide, not tenant-scoped
CRUD on tenant data	false (default)	Must respect tenant isolation
Report generation	false	Reports should only include user's accessible data

Example: MigrationResource

The `MigrationResource` class uses `bypassDataScoping` for all index management operations:

```
@Path("/system/migration")
@RolesAllowed({"admin"})
public class MigrationResource {

    @POST
    @Path("/indexes/applyIndexes/{realm}")
    @FunctionalMapping(area = "MIGRATION", domain = "INDEXES")
    @FunctionalAction(value = "APPLY_INDEXES", bypassDataScoping = true)
    public void applyIndexes(@PathParam("realm") String realm) {
        migrationService.applyIndexes(realm);
    }

    @POST
    @Path("/indexes/dropAllIndexes/{realm}")
    @FunctionalMapping(area = "MIGRATION", domain = "INDEXES")
    @FunctionalAction(value = "DROP_ALL_INDEXES", bypassDataScoping = true)
    public void dropIndexes(@PathParam("realm") String realm) {
        migrationService.dropAllIndexes(realm);
    }
}
```

```
}
```

74.12.8. Integration with Permission Rules

Annotations feed into permission rule matching:

```
- name: allow-catalog-reads
  priority: 300
  match:
    method: [GET]
    # Matches area/domain from @FunctionalMapping
    functionalArea: catalog
    functionalDomain: product
  rolesAny: [USER, ADMIN]
  effect: ALLOW
```

```
- name: admin-only-approval
  priority: 100
  match:
    method: [PUT]
    functionalArea: catalog
    functionalDomain: product
    # Matches @FunctionalAction("APPROVE")
    action: APPROVE
  rolesAll: [ADMIN]
  effect: ALLOW
```

74.12.9. See Also

- [Modeling: Functional Mapping](#)
- [Permission Rules](#)
- [DomainContext and RuleContext](#) == Shared Links

Quantum provides a secure link-sharing mechanism for granting time-bound, usage-limited access to resources (typically cloud-stored files). Share links are tracked with full audit logging and can be revoked at any time.

74.12.10. Concepts

Concept	Description
ShareLink	A persistent entity representing a shareable link to cloud storage. Contains the S3 coordinates (bucket, region, object key), expiration rules, and usage tracking.
SharedLink	A lightweight DTO returned to callers after link creation. Contains the shareable URL, token, expiration metadata, and audience information.

Concept	Description
ShareAccessLog	An audit trail entry recording each access attempt against a share link, including requester identity, IP, user agent, and outcome.
LinkSigner	A cryptographic utility that signs and verifies link tokens using HMAC-SHA256 to prevent tampering and enforce time-bound validity.

74.12.11. ShareLink Model

The **ShareLink** entity is stored in MongoDB and tracks the full lifecycle of a shared link.

Field	Type	Description
<code>publicId</code>	String	External-facing identifier used in URLs. Not the MongoDB <code>_id</code> .
<code>bucket</code>	String	S3 bucket name
<code>region</code>	String	AWS region of the bucket
<code>objectKey</code>	String	S3 object key (path within the bucket)
<code>status</code>	Enum	ACTIVE or REVOKED
<code>expiresAt</code>	Date	Optional expiration timestamp. Null means no expiration.
<code>maxUses</code>	Integer	Optional maximum number of accesses. Null or 0 means unlimited.
<code>usedCount</code>	int	Number of times the link has been consumed (starts at 0)
<code>allowAnonymous</code>	boolean	Whether unauthenticated users can access the link
<code>contentDisposition</code>	String	Optional content disposition header (e.g., attachment; filename="report.pdf")
<code>createdBy</code>	String	User ID of the link creator
<code>lastAccessedAt</code>	Date	Timestamp of most recent access
<code>createdAt</code> / <code>lastUpdatedAt</code>	Date	Audit timestamps

74.12.12. Link Lifecycle

```

Create ShareLink (status=ACTIVE)
  |
  v
Generate signed URL with LinkSigner
  |
  v
Return SharedLink DTO to caller
  |
  v
Access: ShareLinkRepo.tryConsumeOneUse()
      +---> Verify ACTIVE status

```



```

+---> Check expiresAt not passed
+---> Check usedCount < maxUses
+---> Atomic increment usedCount
+---> Log ShareAccessLog entry
|
v
Revoke: ShareLinkRepo.revokeByPublicId()
+---> Set status=REVOKED
+---> Further access attempts are rejected

```

74.12.13. Repository Operations

The `ShareLinkRepo` provides three key operations beyond standard CRUD:

- `findByPublicId(String publicId)` — Look up a link by its public identifier
- `revokeByPublicId(String publicId)` — Mark a link as `REVOKED` and update the timestamp
- `tryConsumeOneUse(ShareLink link)` — Atomically consume one use of the link. Returns `true` only if the link is active, not expired, and within its usage limit. Uses a Morphia session to prevent race conditions.

74.12.14. Access Logging

Every access attempt (successful or not) should be recorded in `ShareAccessLog`:

Field	Type	Description
<code>linkId</code>	<code>ObjectId</code>	Reference to the <code>ShareLink</code> document
<code>publicId</code>	<code>String</code>	The public identifier of the link
<code>ts</code>	<code>Date</code>	Access timestamp
<code>requesterIdentity</code>	<code>String</code>	Subject or user ID of the requester (or "anonymous")
<code>ip</code>	<code>String</code>	Source IP address
<code>userAgent</code>	<code>String</code>	Request user agent string
<code>outcome</code>	<code>String</code>	Result of the access attempt (e.g., "OK", "EXPIRED", "REVOKED", "MAX_USES_EXCEEDED")
<code>message</code>	<code>String</code>	Optional detail message

74.12.15. Link Signing with HMAC

The `LinkSigner` utility provides cryptographic signing and verification of share link tokens using HMAC-SHA256.

Configuration

```
# Secret key for HMAC signing (required for link signing to be active)
```

```
share.link.signing.secret=${SHARE_LINK_SECRET}
```

If the secret is not configured, `LinkSigner.isEnabled()` returns `false` and signing/verification is skipped.

Signing

```
@Inject
LinkSigner linkSigner;

if (linkSigner.isEnabled()) {
    long now = Instant.now().getEpochSecond();
    String signature = linkSigner.sign(shareLink.getPublicId(), now);
    // Include signature and timestamp in the shareable URL
}
```

Verification

```
// Verify with default 24-hour clock skew tolerance
boolean valid = linkSigner.verify(publicId, timestamp, signature);

// Verify with custom skew tolerance (e.g., 1 hour)
boolean valid = linkSigner.verify(publicId, timestamp, signature, 3600);
```

The verifier checks:

1. The HMAC signature matches for the given `publicId + timestamp`
2. The timestamp is within the allowed clock skew window (default: 24 hours)
3. Comparison uses constant-time equality to prevent timing attacks

74.12.16. Security Considerations

- Always configure `share.link.signing.secret` in production with a strong random value
- Use `expiresAt` and `maxUses` to limit link exposure
- Set `allowAnonymous=false` unless guest access is explicitly required
- Monitor `ShareAccessLog` for anomalous access patterns
- Revoke links immediately when they are no longer needed

74.12.17. See Also

- [Permissions and Authorization](#)
- [Authentication and Authorization](#)
- [Configuration Reference](#) = Comment Chains and Media References

Quantum comment chains attach an independently persisted, hierarchical discussion to either a Quantum model or an external application model. A chain's subject contains exactly one `quantumEntity` or `externalEntity`. External subjects use the stable tuple `sourceSystem`, `entityType`, and `externalId`; that tuple is unique inside the governed tenant context.

Comments are separate records linked by `chainId` and optional `parentCommentId`. Quantum assigns the authenticated author and creation time, enforces a maximum depth, preserves parent/chain/author identity after creation, and supports request-id idempotency. Applications should append comments instead of embedding a mutable history array in their own model.

The REST surface is rooted at `/comment-chains`. It supports chain creation and lookup, external-subject resolution, chronological comment listing, comment creation, and direct-reply listing. Generate client seams from the published OpenAPI operations rather than hand-writing service HTTP calls.

Chapter 75. Media references

`MediaReference` stores governed metadata and object identity only. It never stores object bytes or signed URLs. The upload flow is:

1. `POST /media-references/uploads` creates metadata and returns a short-lived direct-upload grant.
2. The client uploads bytes to that grant using its required headers.
3. `POST /media-references/{id}/upload-completion` verifies trusted object metadata and advances scan/status state.
4. Comments and application models persist only the `MediaReference` identity.
5. `POST /media-references/{id}/download-grants` returns a fresh short-lived URL only when the object is available.

Deployments must provide exactly one CDI `MediaStorageGateway` bean for the selected object store. Quantum fails closed when no unique gateway is configured, when uploaded length/type/checksum differs from the prepared reference, or when the object is not available for download. Configure `quantum.media.grant-lifetime` and `quantum.media.max-content-length` for deployment policy.

75.1. OAuth Server Module

The `quantum-oauth-server` is an optional module that turns any Quantum application into a standalone OAuth 2.0 / OIDC authorization server. When added, your application exposes standard OAuth endpoints (`/oauth/authorize`, `/oauth/token`, `/oauth/jwks`, `/.well-known/openid-configuration`, `/oauth/userinfo`) so external clients can authenticate using industry-standard OAuth flows — without requiring an external identity provider like Keycloak, Cognito, Auth0, or Okta.

This is the complement to the OIDC/Cognito provider modules: those modules let Quantum **consume** tokens from an external IdP, while this module lets Quantum **issue** tokens itself.

75.1.1. When to Use

Use `quantum-oauth-server` when:

- You want ChatGPT, Gemini, Grok, or other AI platforms to connect to your MCP server via OAuth
- Third-party applications need OAuth-based access to your APIs
- You want a self-contained deployment with no external IdP dependency
- You already manage user identities in Quantum (credentials, user profiles, user groups) and want to leverage them directly

Use an external IdP (Cognito, Keycloak, Auth0) instead when:

- You need advanced IdP features (social login, MFA, federation, SAML)
- Your organization already standardizes on a specific IdP
- You need to share identities across non-Quantum applications

75.1.2. Adding the Dependency

```
<dependency>
  <groupId>com.end2endlogic</groupId>
  <artifactId>quantum-oauth-server</artifactId>
</dependency>
```

The OAuth endpoints activate automatically via CDI discovery. No additional configuration is needed beyond the standard JWT properties that your application already has.

75.1.3. Prerequisites

The module delegates to existing Quantum infrastructure. Ensure your application has:

```
# These properties are already configured if you use quantum-jwt-provider
mp.jwt.verify.issuer=https://your-app.example.com
mp.jwt.verify.publickey.location=publicKey.pem
auth.jwt.secret=${JWT_SECRET}
com.b2bi.jwt.duration=3600
```

75.1.4. Endpoints

Endpoint	Method	Auth	Description
/.well-known/openid-configuration	GET	Public	OIDC discovery document. Returns issuer, endpoint URLs, supported grant types, signing algorithms, and scopes. OAuth clients use this for auto-configuration.
/oauth/jwks	GET	Public	JSON Web Key Set. Serves the RSA public key (from quantum-default-keys or your configured key location) in JWK format so clients and resource servers can verify token signatures.
/oauth/authorize	GET	Public	Authorization endpoint. Validates the client and request parameters, then presents a login form to the user. Supports PKCE with S256 code challenge method.
/oauth/authorize	POST	Public	Processes the login form submission. Authenticates the user via AuthProviderFactory , generates a short-lived authorization code (5-minute TTL), and redirects back to the client with ?code=XXX&state=YYY .

Endpoint	Method	Auth	Description
<code>/oauth/token</code>	POST	Public	Token endpoint. Exchanges credentials or codes for access and refresh tokens. Supports <code>authorization_code</code> , <code>client_credentials</code> , and <code>refresh_token</code> grant types. Accepts client authentication via HTTP Basic or form-encoded <code>client_id/client_secret</code> .
<code>/oauth/userinfo</code>	GET	Bearer	Returns claims about the authenticated user (<code>sub</code> , <code>iss</code> , <code>email</code> , <code>preferred_username</code> , <code>groups</code>) extracted from the access token.

75.1.5. Supported Grant Types

Authorization Code + PKCE

The primary flow for browser-based OAuth clients (ChatGPT, Gemini, Grok, web applications).

1. Client redirects user to:
GET /oauth/authorize?response_type=code
&client_id=my-client
&redirect_uri=https://client.example.com/callback
&scope=openid+profile+email
&state=random-state
&code_challenge=BASE64URL(SHA256(code_verifier))
&code_challenge_method=S256
2. User sees login form, enters credentials
3. Server authenticates user, generates authorization code, redirects to:
https://client.example.com/callback?code=XXXXX&state=random-state
4. Client exchanges code for tokens:
POST /oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=XXXXX
&redirect_uri=https://client.example.com/callback
&client_id=my-client
&client_secret=my-secret
&code_verifier=original-random-verifier
5. Server returns:

```
{
  "access_token": "eyJ...",
  "token_type": "Bearer",
  "expires_in": 3600,
```

```
"refresh_token": "eyJ..."
}
```

Client Credentials

For service-to-service integrations where no user interaction is needed.

```
POST /oauth/token
Authorization: Basic BASE64(client_id:client_secret)
Content-Type: application/x-www-form-urlencoded
```

```
grant_type=client_credentials
```

Response:

```
{
  "access_token": "eyJ...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

For `client_credentials`, the server looks up a `CredentialUserIdPassword` with a `userId` matching the `client_id`. Create this credential when registering the client.

Refresh Token

```
POST /oauth/token
Content-Type: application/x-www-form-urlencoded
```

```
grant_type=refresh_token
&refresh_token=eyJ...
&client_id=my-client
&client_secret=my-secret
```

75.1.6. Registering OAuth Clients

An `OAuthClient` document must exist in the `oauth_clients` MongoDB collection before a client can authenticate. Create one via the Quantum API, seed packs, or directly in MongoDB.

Field	Type	Description
<code>clientId</code>	String	Unique client identifier (e.g., <code>chatgpt-mcp-client</code>)
<code>clientSecretHash</code>	String	Bcrypt hash of the client secret. Use <code>EncryptionUtils.hashPassword(secret)</code> to generate.
<code>publicClient</code>	boolean	If <code>true</code> , no client secret is required (for PKCE-only public clients like SPAs).

Field	Type	Description
<code>redirectUris</code>	List<String>	Allowed redirect URIs. The authorization endpoint rejects requests with unregistered URIs.
<code>allowedGrantTypes</code>	List<String>	Allowed grant types: <code>authorization_code</code> , <code>client_credentials</code> , <code>refresh_token</code> .
<code>allowedScopes</code>	List<String>	Allowed scopes (e.g., <code>openid</code> , <code>profile</code> , <code>email</code>).
<code>realm</code>	String	The realm this client is associated with for tenant scoping.
<code>active</code>	boolean	Set to <code>false</code> to disable the client without deleting it.

For `client_credentials` grant, also create a `CredentialUserIdPassword` with `userId` matching the `clientId` and `credentialType=OAUTH`:

```
CredentialUserIdPassword cred = CredentialUserIdPassword.builder()
    .userId("chatgpt-mcp-client")
    .subject(UUID.randomUUID().toString())
    .credentialType(CredentialType.OAUTH)
    .roles(new String[]{"user"})
    .domainContext(domainContext)
    .lastUpdate(new Date())
    .build();
```

75.1.7. Integration Examples

ChatGPT MCP Integration

Register your Quantum MCP server in ChatGPT with these settings:

ChatGPT Field	Value
Server URL	https://your-app.example.com/mcp
Authentication	OAuth 2.0
Authorization URL	https://your-app.example.com/oauth/authorize
Token URL	https://your-app.example.com/oauth/token
Client ID	Your registered <code>OAuthClient.clientId</code>
Client Secret	The plaintext secret (hashed version stored in <code>clientSecretHash</code>)
Scopes	<code>openid profile email</code>

When a user connects via ChatGPT:

1. ChatGPT opens a browser to `/oauth/authorize` with the registered `client_id`
2. The user sees the Quantum login form and enters their credentials
3. On success, the server redirects back to ChatGPT with an authorization code
4. ChatGPT exchanges the code for tokens at `/oauth/token`

- 5. Subsequent MCP tool calls include the access token as a Bearer header
- 6. Quantum’s `SecurityFilter` validates the token (same issuer, same keys) and establishes the user’s security context

Gemini / Grok / Other AI Platforms

The same pattern applies to any AI platform that supports OAuth 2.0:

- 1. Register an `OAuthClient` with the platform’s callback URL as a redirect URI
- 2. Provide the discovery URL (`/.well-known/openid-configuration`) or the individual endpoint URLs
- 3. The platform handles the OAuth flow; Quantum handles authentication and token issuance

Service-to-Service (Client Credentials)

For backend integrations that don’t involve user interaction:

```
# Obtain an access token
TOKEN=$(curl -s -X POST https://your-app.example.com/oauth/token \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=client_credentials" \
  -d "client_id=my-service" \
  -d "client_secret=my-secret" | jq -r '.access_token')

# Use the token to call APIs
curl -H "Authorization: Bearer $TOKEN" https://your-app.example.com/api/...
```

75.1.8. Architecture

The module is a thin REST layer over existing Quantum infrastructure:

Existing Component	OAuth Use
<code>TokenUtils.generateUserToken()</code>	Issues access tokens at <code>/oauth/token</code>
<code>TokenUtils.generateRefreshToken()</code>	Issues refresh tokens
<code>TokenUtils.readPublicKey()</code>	Serves the <code>/oauth/jwks</code> endpoint
<code>AuthProviderFactory.getAuthProvider().login()</code>	Validates user credentials during authorization code flow
<code>CredentialRepo</code>	Looks up user and client credentials
RSA key pair (<code>quantum-default-keys</code>)	Signs all issued tokens
<code>mp.jwt.verify.issuer</code>	Populates the discovery document <code>issuer</code> field

Existing Component	OAuth Use
<code>SecurityFilter</code>	Validates the issued tokens on subsequent API calls (same issuer, same keys)

Tokens issued by the OAuth server are standard Quantum JWTs — indistinguishable from tokens issued via `/security/login` or `/auth/login`. Any Quantum endpoint that accepts a Bearer token will accept tokens from the OAuth server.

75.1.9. Data Model

Two new MongoDB collections are created:

- `oauth_clients` — registered OAuth clients (persistent, managed by admin)
- `oauth_authorization_codes` — short-lived authorization codes (auto-expire after 5 minutes via MongoDB TTL index)

Both follow standard Quantum patterns (`BaseModel`, realm-scoped, Morphia-managed).

75.1.10. Security Considerations

- Store client secrets securely. The `clientSecretHash` uses bcrypt — never store plaintext secrets.
- Use HTTPS for all OAuth endpoints in production.
- Register specific `redirectUri` per client — never use wildcards.
- PKCE (S256) is supported and recommended for all authorization code flows.
- Authorization codes are single-use and expire after 5 minutes.
- For public clients (SPAs), set `publicClient=true` and rely on PKCE for security.
- The login form is minimal by design. For custom branding, override `OAuthAuthorizeResource` in your application.

75.1.11. See Also

- [Authentication and Authorization](#)
- [Multiple Authentication Providers](#)
- [MCP Server and Client](#)
- [Configuration Reference](#)

Chapter 76. AI Integration

Chapter 77. MCP Server and Client

The `quantum-mcp-server` module exposes the Query Gateway as a set of [Model Context Protocol \(MCP\)](#) tools and resources, and provides an MCP client for calling external MCP tool providers. This enables AI assistants (Claude Desktop, Cursor, ChatGPT, and others) to discover and invoke your application’s CRUDL operations, browse entity schemas, and receive query-building hints—all through the standard MCP JSON-RPC protocol.

The module also includes a REST-based agent layer (`/api/agent/*`) that mirrors the same tool set for non-MCP integrations.

77.1. Module Overview

`quantum-mcp-server` is a standalone Maven module with three key dependencies:

Dependency	Version	Purpose
<code>quantum-framework</code>	<code>\${quantum.version}</code>	Core framework (QueryGatewayResource, security, Morphia)
<code>quarkus-mcp-server-http</code>	1.9.1	Quarkiverse MCP Server — Streamable HTTP + legacy SSE transport
<code>quarkus-langchain4j-mcp</code>	1.6.0	Quarkiverse MCP Client via LangChain4j — connect to external MCP servers

Add the module to your application’s POM:

```
<dependency>
  <groupId>com.end2endlogic</groupId>
  <artifactId>quantum-mcp-server</artifactId>
  <version>${quantum.version}</version>
</dependency>
```

77.2. MCP Server

The MCP server exposes the Query Gateway as six tools and three resources at the `/mcp` endpoint. MCP clients discover these automatically via the `tools/list` and `resources/list` JSON-RPC methods.

77.2.1. Security

The MCP server and the agent REST API require an authorized user. All endpoints and tools are protected by Quarkus Security.

- **REST API (`/api/agent/*`):** Requires a valid Bearer JWT token in the `Authorization` header.
- **MCP Endpoint (`/mcp`):** The MCP protocol transport (Streamable HTTP or SSE) must include the same authentication mechanism (typically a `Authorization: Bearer <token>` header) as other REST APIs.

You can generate a long-lived service token for use with the MCP server using the [Quantum CLI](#).

77.2.2. Tools

Tools are defined in `McpGatewayTools` using the Quarkiverse `@Tool` and `@ToolArg` annotations. Each tool is protected with `@Authenticated` and delegates to `AgentExecuteHandler`, which routes to the `QueryGatewayResource`—reusing the same security, realm resolution, and query execution as the REST API.



Because the tools route through `QueryGatewayResource`, the data-access tools are row- and field-governed. `query_find`, `query_count`, `query_delete`, `query_deleteMany`, and `query_export` apply the caller's `DataDomain` scope plus the `RuleContext` rule `filterConstraints`, and `query_find` additionally excludes deny-wins `excludedFields` at the MongoDB level. The gateway **fails closed** (HTTP 500 `SecurityContextMissing`) if no security context is established for the request, so an agent never executes an ungoverned query. `query_delete` returns "not found" for an id outside the caller's `DataDomain` rather than revealing that the row exists.

`expand(...)` (aggregation) execution is currently disabled: a `query_find` with `expand(...)` returns HTTP 501 because the aggregation path is not yet governed by row- and field-level security. See [Planner and QueryGateway: row- and field-level governance](#).

Tool Name	Description
<code>query_rootTypes</code>	List available entity types (root types) that can be queried, saved, or deleted
<code>query_plan</code>	Return the query execution plan (FILTER vs AGGREGATION) for a <code>rootType</code> and BIAPI query string
<code>query_find</code>	Execute a BIAPI query and return matching entities (supports pagination, realm override)
<code>query_save</code>	Save (insert or update) an entity by <code>rootType</code>
<code>query_delete</code>	Delete a single entity by its <code>ObjectId</code>
<code>query_deleteMany</code>	Delete multiple entities matching a BIAPI query
<code>query_count</code>	Count entities matching a BIAPI query
<code>query_export</code>	Export entities to a stream (CSV) matching a BIAPI query and column selection
<code>query_import_analyze</code>	Analyze a CSV file for import, returning a preview of rows and identifying potential errors
<code>query_import_rows</code>	List analyzed import rows from a session (supports filtering by error)
<code>query_import_commit</code>	Commit an analyzed import session, persisting the valid rows to the database
<code>query_import_cancel</code>	Cancel an import session and clean up temporary data

Tool parameters

`query_find` accepts these arguments:

Parameter	Type	Description
<code>rootType</code>	string	Entity type simple name or FQCN (use <code>query_rootTypes</code> to discover)
<code>query</code>	string	BIAPI query string (e.g. <code>status:ACTIVE && region:West</code>)
<code>realm</code>	string	Optional tenant realm (defaults to caller's realm)
<code>limit</code>	integer	Optional max results (default 50)
<code>skip</code>	integer	Optional offset for pagination (default 0)

`query_save` accepts `rootType`, `entity` (JSON object matching the schema), and optional `realm`. `query_delete` accepts `rootType`, `id` (ObjectId hex), and optional `realm`. `query_deleteMany` accepts `rootType`, `query`, and optional `realm`.

77.2.3. Resources

Resources are defined in `McpSchemaResources` and `McpQueryHintsResource` using the `@Resource` annotation. MCP clients can read these to populate LLM context with schema information and query-building guidance.

Resource URI	Description
<code>quantum://schema</code>	Lists all available root types with class name, simple name, and collection name
<code>quantum://query-hints</code>	BIAPI query grammar summary, example queries by intent, and tips (expand, wildcards, ontology)
<code>quantum://permission-hints</code>	Permission check/evaluate API summary, area/domain/action mapping, and example check requests

77.2.4. Connecting an MCP Client

Any MCP-compatible client can connect to the `/mcp` endpoint. Example configuration for Claude Desktop (`claude_desktop_config.json`):

```
{
  "mcpServers": {
    "quantum": {
      "url": "http://localhost:8080/mcp"
    }
  }
}
```

For Cursor, add the server URL in Settings > MCP Servers.

Once connected, the client can:

1. Call `tools/list` to discover the six gateway tools
2. Call `resources/list` to discover schema and hint resources
3. Call `resources/read` with `quantum://query-hints` to learn the BIAPI query syntax
4. Call `tools/call` with `query_rootTypes` to see what entity types are available
5. Call `tools/call` with `query_find` to query data

77.3. MCP Client

The MCP client side uses the Quarkiverse LangChain4j MCP extension (`quarkus-langchain4j-mcp`) to connect to external MCP servers and consume their tools. Use the MCP client when you need to:

- Integrate with external tool providers (e.g. Helix MCP, Brain, HelixAI, or any MCP-compliant server)
- Let your AI services discover and invoke tools from those servers at runtime
- Access MCP resources (files, API responses, etc.) programmatically or expose them as tools to the LLM

The client implements the [MCP Specification](#) and works with any compatible MCP server (including the Quantum MCP server on another instance).

77.3.1. Configuration

External MCP connections are configured in `application.properties` using the `quarkus.langchain4j.mcp.<client-name>` prefix. Each named block defines one MCP client (e.g. `helix`, `brain`, `filesystem`).

```
# Streamable HTTP (recommended for remote servers)
quarkus.langchain4j.mcp.helix.transport-type=streamable-http
quarkus.langchain4j.mcp.helix.url=http://helix-mcp.example.com/mcp

# STDIO (for local subprocess servers, e.g. npm-based MCP servers)
quarkus.langchain4j.mcp.brain.transport-type=stdio
quarkus.langchain4j.mcp.brain.command=npx,-y,@brain/mcp-server

# Optional: pass environment variables to the STDIO subprocess
quarkus.langchain4j.mcp.brain.environment.API_KEY=${EXTERNAL_API_KEY}
```

Supported transport types:

Transport	Description
<code>streamable-http</code>	Recommended for remote MCP servers; uses HTTP with the MCP streamable transport.
<code>http</code>	Legacy HTTP/SSE transport. URL typically ends with <code>/mcp/sse</code> .

Transport	Description
<code>stdio</code>	Spawns the MCP server as a subprocess; communication via stdin/stdout. Use <code>command</code> (comma-separated parts) and optionally <code>environment.*</code> for env vars.
<code>websocket</code>	Bidirectional WebSocket. Supported by some servers (e.g. those built with <code>quarkus-mcp-server</code>).

77.3.2. Using MCP Tools in AI Services

Enable tools from one or more MCP servers on an AI service using `@McpToolBox`. The LLM can then discover and call those tools during the conversation.

Single MCP server:

```
import io.quarkiverse.langchain4j.mcp.runtime.McpToolBox;
import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;
import io.quarkiverse.langchain4j.RegisterAiService;

@RegisterAiService
public interface MyAiService {

    @SystemMessage("You are a helpful assistant.")
    @McpToolBox("helix")
    String chat(@UserMessage String userMessage);
}
```

Multiple MCP servers: pass an array of client names to use tools from several servers in one service.

```
@McpToolBox({"helix", "brain"})
String chat(@UserMessage String userMessage);
```

All configured MCP servers: omit the value to expose every configured MCP client's tools to the AI service.

```
@McpToolBox
String chat(@UserMessage String userMessage);
```

77.3.3. Programmatic Access to MCP Clients

For fine-grained control (custom tool filtering, non-AI workflows, or resource access), inject an `McpClient` directly with `@McpClientName`:

```
import dev.langchain4j.mcp.client.McpClient;
```



```

import io.quarkiverse.langchain4j.mcp.runtime.McpClientName;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;

@ApplicationScoped
public class McpClientService {

    @Inject
    @McpClientName("helix")
    McpClient helixClient;

    public void useClient() {
        // List tools exposed by the MCP server
        var tools = helixClient.listTools();

        // Execute a tool by name with JSON arguments
        var args = com.fasterxml.jackson.databind.node.JsonNodeFactory.instance
            .objectNode().put("key", "value");
        var request = dev.langchain4j.mcp.client.protocol.McpCallToolRequest.builder()
            .name("some_tool_name")
            .arguments(args)
            .build();
        var result = helixClient.executeTool(request);

        // List and read resources (if the server exposes resources)
        var resources = helixClient.listResources();
        var content = helixClient.readResource("some://resource-uri");
    }
}

```

Use programmatic access when you need to drive tool execution from non-AI code, filter or transform tool results, or integrate MCP resources into your own APIs.

77.3.4. Dev UI: Testing MCP Clients

In development mode (`mvn quarkus:dev`), open the Quarkus Dev UI at <http://localhost:8080/q/dev-ui> and use the **MCP clients** card to:

- View all configured MCP clients
- Browse tools (names, descriptions, parameter schemas)
- Execute tools with custom arguments and inspect results

This is useful for exploring a new MCP server or debugging tool arguments before wiring it into an AI service.

77.3.5. Optional: Resources as Tools

If an MCP server exposes resources, you can expose them as synthetic tools so the LLM can list and read resources:

```
quarkus.langchain4j.mcp.expose-resources-as-tools=true
```

When enabled, the client adds `list_resources` and `get_resource` to the tool set; your AI service can then answer questions like “List available resources” or “Show me the content of resource X” when using `@McpToolBox` for that client.

77.3.6. Optional: Headers and Authorization

- **Static headers:** for HTTP-based clients, set headers via config:

```
quarkus.langchain4j.mcp.helix.header.X-Custom-Header=MyValue
```

- **Dynamic headers:** implement `dev.langchain4j.mcp.client.McpHeadersSupplier` and provide a CDI bean; it will be used for Streamable HTTP clients to add headers per request.
- **Authorization:** implement `io.quarkiverse.langchain4j.mcp.auth.McpClientAuthProvider` to supply a bearer token or API key. Use `@McpClientName` on the bean to restrict it to a specific client. For OIDC, add the dependency `quarkus-langchain4j-oidc-mcp-auth-provider` to propagate the user’s access token to the MCP server.

See the [Quarkus LangChain4j MCP documentation](#) for full details on transports, auth, and the MCP Registry client.

77.4. Agent REST API

The agent layer provides a REST interface at `/api/agent` that mirrors the MCP tools for non-MCP integrations. This is useful for custom agent orchestrators, webhook-based workflows, or testing.

77.4.1. Endpoints

Method	Path	Description
GET	<code>/api/agent/tools</code>	List available gateway tools (optionally filtered by realm)
GET	<code>/api/agent/schema</code>	List all root types (same as <code>query_rootTypes</code>)
GET	<code>/api/agent/schema/{rootType}</code>	JSON Schema-like structure for a single entity type
GET	<code>/api/agent/query-hints</code>	Query grammar summary and example queries
GET	<code>/api/agent/permission-hints</code>	Permission check API summary and examples
POST	<code>/api/agent/execute</code>	Execute a gateway tool by name

77.4.2. Execute Request

The execute endpoint accepts a tool name and arguments:

```
{
  "tool": "query_find",
  "arguments": {
    "rootType": "Location",
    "query": "status:ACTIVE && city:Atlanta",
    "page": { "limit": 10, "skip": 0 }
  }
}
```

```
curl -sS -X POST \
-H 'Content-Type: application/json' \
localhost:8080/api/agent/execute \
-d '{
  "tool": "query_find",
  "arguments": {
    "rootType": "Location",
    "query": "status:ACTIVE && city:Atlanta"
  }
}'
```

The response shape matches the corresponding Query Gateway REST endpoint (e.g. the Collection envelope for `query_find`).

77.5. Per-Tenant Agent Configuration

Agent behavior can be customized per realm using MicroProfile Config properties:

```
# Run agent tools as a specific user in the "acme" realm
quantum.agent.tenant.acme.runAsUserId=agent-user@acme.com

# Only allow find and plan tools for the "acme" realm
quantum.agent.tenant.acme.enabledTools=query_find,query_plan,query_rootTypes

# Cap find results at 100 for this tenant
quantum.agent.tenant.acme.maxFindLimit=100
```

Configuration properties:

Property	Type	Description
<code>quantum.agent.tenant.<realm>.runAsUserId</code>	string	Optional <code>userId</code> whose security context is used for tool execution
<code>quantum.agent.tenant.<realm>.enabledTools</code>	comma-separated	Optional list of tool names to expose (all six enabled when empty)

Property	Type	Description
<code>quantum.agent.tenant.<realm>.maxFindLimit</code>	integer	Optional maximum number of results for <code>query_find</code>

The default implementation (`PropertyTenantAgentConfigResolver`) reads these from `application.properties` or environment variables. You can replace it by providing a CDI bean implementing `TenantAgentConfigResolver`.

77.5.1. Run-As Principal

When `runAsUserId` is configured, tool execution runs under that user's security context. To enable this, provide a CDI bean implementing `RunAsPrincipalResolver`:

```
import com.e2eq.framework.api.agent.RunAsPrincipalResolver;
import com.e2eq.framework.model.securityrules.PrincipalContext;
import jakarta.enterprise.context.ApplicationScoped;
import java.util.Optional;

@ApplicationScoped
public class MyRunAsPrincipalResolver implements RunAsPrincipalResolver {

    @Override
    public Optional<PrincipalContext> resolvePrincipalContext(String realm, String
userId) {
        // Look up user and build PrincipalContext
        // Return Optional.empty() to fall back to caller's context
    }
}
```

77.6. Agent Configuration API

The framework stores agent configurations as realm-scoped `Agent` entities in MongoDB (`agents` collection). Each agent pairs an LLM reference with a system prompt and a tool filter, enabling different personas for different use cases (e.g., a supply-chain analyst vs. a customer-service bot).

77.6.1. Agent Entity

Field	Type	Description
<code>refName</code>	string	Unique reference name within the realm (e.g., <code>supply-chain-assistant</code>)
<code>name</code>	string	Display name
<code>llmConfigRef</code>	string	Reference to an LLM secret or config (e.g., <code>claude-sonnet</code> , <code>gpt-4o</code>)

Field	Type	Description
<code>context</code>	list of PromptStep	Ordered system/user prompt steps that define the agent's persona
<code>enabledTools</code>	list of string	Tool names this agent can use (e.g., <code>["query_rootTypes", "query_find", "query_plan"]</code>)

Each `PromptStep` has `order` (int), `role` (string: "system" or "user"), and `content` (string: prompt text).

77.6.2. CRUD Endpoints

All endpoints are at `/api/agent/config` and require a `realm` query parameter.

Method	Path	Description
GET	<code>/api/agent/config/list?realm=X</code>	List all agents in the realm
GET	<code>/api/agent/config/{refName}?realm=X</code>	Get an agent by reference name
GET	<code>/api/agent/config/id/{id}?realm=X</code>	Get an agent by ObjectId
POST	<code>/api/agent/config?realm=X</code>	Create or update an agent
DELETE	<code>/api/agent/config/{refName}?realm=X</code>	Delete an agent by reference name
DELETE	<code>/api/agent/config/id/{id}?realm=X</code>	Delete an agent by ObjectId

77.6.3. Example: Creating an Agent

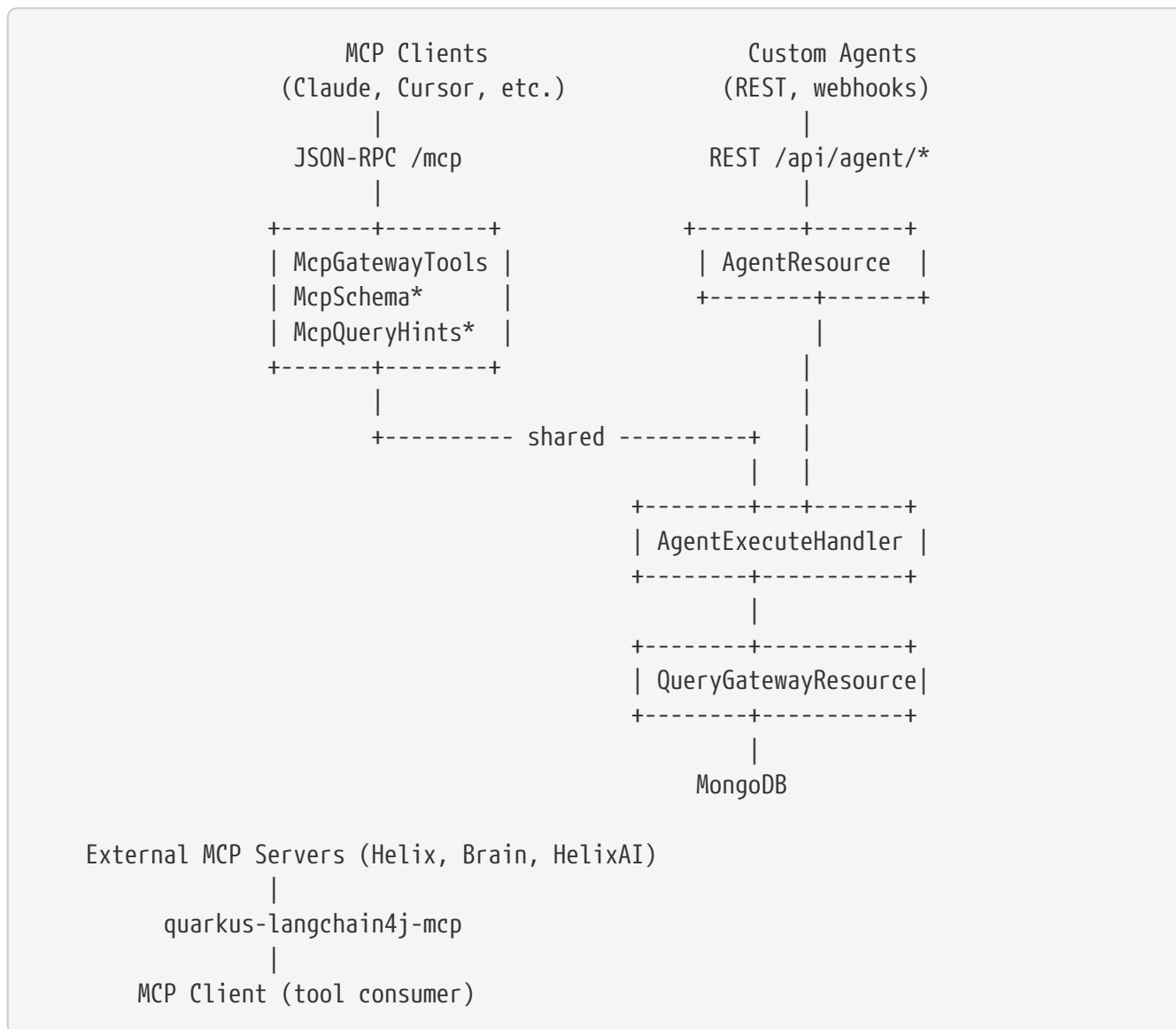
```
curl -sS -X POST \
-H 'Content-Type: application/json' \
'localhost:8080/api/agent/config?realm=acme-corp' \
-d '{
  "refName": "supply-chain-assistant",
  "name": "Supply Chain Assistant",
  "llmConfigRef": "claude-sonnet",
  "context": [
    { "order": 1, "role": "system", "content": "You are a supply-chain analyst. Help
users query orders, track shipments, and monitor inventory." }
  ],
  "enabledTools": ["query_rootTypes", "query_plan", "query_find", "query_count"]
}'
```



Agent configuration complements the property-based tenant configuration ([Per-Tenant Agent Configuration](#)). Properties set operator-level overrides (runAsUserId, maxFindLimit); Agent entities store per-use-case personas and tool filters that can be managed via API.

77.7. Architecture

The following diagram shows how the MCP server, agent REST API, and MCP client relate:



Key design points:

- **Shared execution path:** Both MCP tools and REST agent endpoints delegate to `AgentExecuteHandler`, which routes to `QueryGatewayResource`. Security rules, realm resolution, and query execution are identical regardless of entry point.
- **Zero reverse dependencies:** The `quantum-mcp-server` module depends on `quantum-framework` but the framework has no knowledge of MCP. Applications that do not need MCP simply omit this module.
- **Tenant isolation:** Realm-scoped execution, optional per-tenant tool filtering, and run-as support ensure multi-tenant safety.

77.8. Ontology Tools

In addition to the six gateway tools, the MCP server exposes two ontology discovery tools via

McpOntologyTools:

Tool Name	Description
<code>query_relationships</code>	Find ontology edges (relationships) for an entity — outgoing, incoming, or both, with optional predicate filtering
<code>query_predicates</code>	List ontology predicates (relationship types) defined in the TBox schema, with optional domain/range filtering

These tools allow AI assistants to explore the entity relationship graph. For example, an agent can discover that an `Order` entity has a `placedInOrg` relationship to an `Organization`, then use `query_find` with appropriate filters to retrieve related data.

77.9. Integrating with Claude

Claude supports MCP through Claude Desktop, Claude Code (CLI), and the browser-based Claude.ai (via Custom Connectors). The Quantum MCP server works with all three.

77.9.1. Claude Desktop

Edit `~/Library/Application Support/Claude/claude_desktop_config.json` (macOS) or the equivalent on your platform:

```
{
  "mcpServers": {
    "quantum": {
      "url": "http://localhost:8080/mcp"
    }
  }
}
```

Restart Claude Desktop. The Quantum tools appear in the tool picker and Claude can call them during conversations.

77.9.2. Claude Code (CLI)

Claude Code discovers MCP servers from project-level or user-level configuration.

Project-level — create `.claude/settings.json` in your repository root:

```
{
  "mcpServers": {
    "quantum-mcp": {
      "url": "http://localhost:8080/mcp"
    }
  }
}
```

User-level — edit `~/.claude/settings.local.json` to add the server globally:

```
{
  "mcpServers": {
    "quantum-mcp": {
      "url": "http://localhost:8080/mcp"
    }
  }
}
```

Once configured, Claude Code can use the Quantum tools in any conversation:

> Use `query_rootTypes` to list available entity types

Claude calls `query_rootTypes` and returns:
Location, Order, CodeList, UserProfile, ...

> Find all active locations in the West region

Claude calls `query_find` with:
rootType: "Location"
query: "status:ACTIVE && region:West"
limit: 10

Returns matching Location entities.

77.9.3. Claude.ai (Browser) with Custom Connectors

For cloud-hosted deployments, expose the `/mcp` endpoint over HTTPS and configure a Claude Custom Connector:

1. In Claude.ai, go to Settings > Integrations > Custom Connectors.
2. Add a new connector with your server URL (e.g., <https://api.mycompany.com/mcp>).
3. Configure authentication headers if required by your deployment.

77.9.4. Cursor IDE

Create `.cursor/mcp.json` in your project root:

```
{
  "mcpServers": {
    "quantum": {
      "url": "http://localhost:8080/mcp"
    }
  }
}
```


Or add the server URL in Cursor Settings > MCP Servers.

77.10. How MCP Maps to Framework Agent and Tool Concepts

The framework uses a layered architecture where MCP is one of several access paths to the same underlying tool execution engine. Understanding how these concepts relate helps when extending the framework or building custom integrations.

77.10.1. Concept Mapping

MCP Concept	Framework Concept	Relationship
MCP Tool	<code>@Tool</code> -annotated method	Each MCP tool is a Java method annotated with <code>@Tool</code> (Quarkiverse MCP Server extension) in <code>McpGatewayTools</code> . Tools are code-defined and discovered at startup via classpath scanning — there is no database-backed tool registry.
MCP Tool Name	Method-derived name	The tool name used in MCP <code>tools/call</code> (e.g., <code>query_find</code>) is defined in the <code>@Tool</code> annotation or derived from the method name.
MCP <code>tools/call</code>	<code>AgentExecuteHandler.execute(tool, arguments)</code>	Both the MCP server (<code>McpGatewayTools</code>) and the REST agent endpoint (<code>POST /api/agent/execute</code>) delegate to the same handler.
MCP Resource	<code>SchemaService</code> / <code>QueryHintsProvider</code>	MCP resources (<code>quantum://schema</code> , <code>quantum://query-hints</code> , <code>quantum://permission-hints</code>) are backed by the same provider beans that serve the REST <code>/api/agent/schema</code> and <code>/api/agent/query-hints</code> endpoints.
MCP Client	<code>quarkus-langchain4j-mcp</code>	The framework can also <i>consume</i> tools from external MCP servers. External MCP servers are configured via <code>ToolProviderConfig</code> (connection details) and <code>quarkus.langchain4j.mcp.*</code> properties.
Agent Config	<code>Agent</code> entity	An <code>Agent</code> is a realm-scoped configuration envelope that pairs an LLM reference with a system prompt (<code>PromptSteps</code>) and a tool filter (<code>enabledTools</code>). Managed via <code>/api/agent/config/*</code> CRUD API.

77.10.2. Execution Flow Comparison

Both MCP and REST reach the same execution path:





Security (realm resolution, permission checks, `@FunctionalAction`) is enforced identically at the `QueryGatewayResource` level, regardless of whether the request came through MCP or REST.

77.11. End-to-End Example: Claude Querying a Supply Chain Application

This walkthrough demonstrates a complete flow from starting the application to having Claude interact with live data through MCP.

77.11.1. Prerequisites

- Java 17+, Maven 3.9+
- MongoDB running locally (or via Quarkus Dev Services)
- Claude Desktop or Claude Code installed
- Your Quantum-based application (e.g., a supply chain app with `Location`, `Order`, `Shipment` entities)

77.11.2. Step 1: Add the MCP Server Module

In your application POM, add the `quantum-mcp-server` dependency:

```

<dependency>
  <groupId>com.end2endlogic</groupId>
  <artifactId>quantum-mcp-server</artifactId>
  <version>${quantum.version}</version>
</dependency>

```

77.11.3. Step 2: Configure Agent Properties

In `src/main/resources/application.properties`:

```

# --- Authentication ---
auth.provider=custom
quarkus.smallrye-jwt.enabled=true

```

```

mp.jwt.verify.publickey.location=publicKey.pem
mp.jwt.verify.issuer=https://myapp.example.com/issuer
mp.jwt.verify.audiences=my-api-client
auth.jwt.secret=${JWT_SECRET:dev-secret-change-in-prod}
auth.jwt.expiration=60
auth.jwt.refresh-expiration=120

# --- Per-tenant agent configuration ---
# The "default" realm: agent runs as a service user, limited to read-only tools
quantum.agent.tenant.default.runAsUserId=agent-svc@mycompany.com
quantum.agent.tenant.default.enabledTools=query_rootTypes,query_plan,query_find,query_count
quantum.agent.tenant.default.maxFindLimit=200

# The "acme-corp" realm: full CRUDL access
quantum.agent.tenant.acme-corp.runAsUserId=bot@acme-corp.com
quantum.agent.tenant.acme-corp.enabledTools=query_rootTypes,query_plan,query_find,query_count,query_save,query_delete

```

77.11.4. Step 3: Start the Application

```
mvn quarkus:dev
```

The MCP endpoint is available at <http://localhost:8080/mcp>.

77.11.5. Step 4: Configure Claude

Claude Desktop — edit `claude_desktop_config.json`:

```

{
  "mcpServers": {
    "supply-chain": {
      "url": "http://localhost:8080/mcp"
    }
  }
}

```

Claude Code — create `.claude/settings.json` in the project root:

```

{
  "mcpServers": {
    "supply-chain": {
      "url": "http://localhost:8080/mcp"
    }
  }
}

```

```
}
```

77.11.6. Step 5: Interact with Claude

Below is a realistic conversation showing how Claude uses the MCP tools:

User: What entity types are available in the supply chain system?

Claude calls `query_rootTypes` and responds:

The system has these entity types: Location, Order, Shipment, Product, Customer, CodeList, UserProfile, Organization.

User: Show me all active warehouses in the Southeast region.

Claude calls `query_find`:

- `rootType: "Location"`
- `query: "status:ACTIVE && type:WAREHOUSE && region:Southeast"`
- `limit: 20`

Found 3 active warehouses in the Southeast:

1. ATL-WH-01 — Atlanta Distribution Center (Atlanta, GA)
2. MIA-WH-03 — Miami Fulfillment Hub (Miami, FL)
3. CLT-WH-02 — Charlotte Regional Warehouse (Charlotte, NC)

User: How many open orders are there for the Atlanta warehouse?

Claude calls `query_count`:

- `rootType: "Order"`
- `query: "status:OPEN && warehouseRef:ATL-WH-01"`

There are 47 open orders for the Atlanta Distribution Center (ATL-WH-01).

User: Show me the last 5 orders with their customer details expanded.

Claude calls `query_plan` first to check expand support:

- `rootType: "Order"`
- `query: "expand(customer) && warehouseRef:ATL-WH-01"`

Then calls `query_find`:

- `rootType: "Order"`

- `query: "expand(customer) && warehouseRef:ATL-WH-01"`
- `limit: 5`

Here are the 5 most recent orders for ATL-WH-01, with customer details:

1. ORD-9921 — Acme Corp (acme@example.com), 12 items, \$2,340
2. ORD-9918 — Beta Industries (beta@example.com), 3 items, \$890 ...

User: What relationships does the Atlanta warehouse have?

Claude calls `query_relationships`:

- `entityId: "ATL-WH-01"`
- `direction: "both"`

The Atlanta warehouse has these relationships:

Outgoing: - `locatedIn` → Organization "Southeast Division" - `servesRegion` → Region "Southeast"

Incoming: - `fulfilledAt` ← 47 open Orders - `shipsFrom` ← 12 active Shipments

77.11.7. What Happened Under the Hood

Claude Desktop / Claude Code

```
|
| (1) MCP tools/list -> discovers 9 tools + 3 resources
|
| (2) MCP resources/read "quantum://query-hints"
|     -> learns BIAPI query syntax, expand(), ontology edges
|
| (3) MCP tools/call "query_rootTypes"
|     -> McpGatewayTools.query_rootTypes()
|         -> AgentExecuteHandler.execute("query_rootTypes", {})
|         -> QueryGatewayResource.listRootTypes()
|
| (4) MCP tools/call "query_find" { rootType, query, limit }
|     -> McpGatewayTools.query_find(rootType, query, realm, limit, skip)
|         -> AgentExecuteHandler.execute("query_find", args)
|         -> QueryGatewayResource.find(FindRequest)
|             -> Morphia Query -> MongoDB
|
| (5) MCP tools/call "query_relationships" { entityId, direction }
|     -> McpOntologyTools.query_relationships(entityId, direction, ...)
|         -> OntologyEdgeRepo.findBySrc() / findByDst()
|             -> MongoDB edges collection
|
```

V
JSON response serialized and returned to Claude

At every step, the Quantum security filter validates the caller's identity, resolves the realm, and applies permission rules. The agent never bypasses `@FunctionalAction` checks or data scoping.

77.11.8. Adding Ontology Context for Richer Queries

When your application has an ontology TBox defined (see [Ontologies in Quantum](#)), Claude can use the `query_predicates` tool to discover relationship types and then use `query_relationships` to traverse the graph.

For example:

User: What types of relationships exist between Orders and Organizations?

Claude calls `query_predicates`:

- `domainFilter: "Order"`
- `rangeFilter: "Organization"`

Found 2 predicates:

1. `placedByOrg` — Order → Organization (the org that placed the order)
2. `fulfilledByOrg` — Order → Organization (the org fulfilling the order)

This lets Claude build more targeted queries and provide richer answers about entity relationships.

77.12. MCP Server Configuration Reference

The MCP server requires no additional configuration beyond including the `quantum-mcp-server` module. The `/mcp` endpoint is automatically registered by the Quarkus MCP Server extension.

Optional properties:

Property	Default	Description
<code>quarkus.mcp.server.traffic-logging</code>	<code>NONE</code>	Set to <code>ALL</code> to log MCP JSON-RPC messages for debugging
<code>feature.queryGateway.execution.enabled</code>	<code>false</code>	Set to <code>true</code> to enable <code>expand()</code> (AGGREGATION mode) in <code>query_find</code>
<code>quantum.queryGateway.deleteMany.maxMatches</code>	<code>2000</code>	Maximum entities that <code>query_deleteMany</code> will delete in a single call

See [AI Agent Integration](#) for the full agent API design and additional scenarios.

Chapter 78. Advanced Features

78.1. Email Templates

Quantum provides a built-in email templating system with three important pieces:

- classpath defaults under `templates/email/{templateKey}/`
- realm-scoped overrides stored in MongoDB
- Qute-based rendering plus pluggable delivery

This makes email content part of the framework instead of forcing each product to hand-roll template loading, rendering, preview, and override behavior.

78.1.1. What the framework supports

Email templates are now first-class framework resources:

- `EmailTemplate` documents are stored in the `email_templates` collection.
- `DefaultEmailTemplateResolver` checks the current realm for an active override first, then falls back to the classpath.
- `EmailTemplateRenderService` renders `subject.txt`, `body.html`, and `body.txt` using Qute.
- `EmailTemplateResource` exposes tenant-scoped CRUD and preview APIs at `/settings/email-templates`.
- `TenantOnboardingWorkflow` now carries `defaultInviteEmailTemplateKey`, so onboarding/invite flows can refer to a named template instead of hard-coded content.

78.1.2. Architecture

The email system has three layers:

1. Template resolution
2. Rendering
3. Delivery

```
Application Code
  |
  v
EmailTemplateRenderService.render(realm, templateKey, context)
  |
  v
EmailTemplateResolver.resolve(realm, templateKey)
  |
  +---> Check DB for realm-specific override (active=true)
  |
  |           |
  |           v (not found)
```

```

+---> Load from classpath: templates/email/{templateKey}/
|
v
EmailTemplateDefinition (unrendered Qute templates)
|
v
Qute Engine renders subject, HTML body, text body
|
v
RenderedTemplate --> build RenderedEmail --> EmailDeliveryService.deliver()

```

78.1.3. Template sources and resolution order

The `DefaultEmailTemplateResolver` follows this priority:

1. Realm database override
2. Classpath default
3. Empty result if neither source contains the template

That means products can ship sane defaults in source control, while each tenant can still customize messaging without forking code.

Classpath templates

Place template files on the classpath under `templates/email/{templateKey}/`:

```

src/main/resources/
  templates/
    email/
      welcome/
        subject.txt
        body.html
        body.txt
      password-reset/
        subject.txt
        body.html
        body.txt

```

Each file is optional except that rendering must ultimately produce:

- a non-blank subject
- at least one non-blank body variant (`html` or `text`)

Example:

subject.txt

```
Welcome {name}
```

body.html

```
<p>Hello {name}, welcome to realm <strong>{realm}</strong>.</p>
```

body.txt

```
Hello {name}, welcome to realm {realm}.
```

Realm overrides in MongoDB

Tenants can override any classpath template by storing an `EmailTemplate` document in the `email_templates` collection for their realm.

Field	Description
<code>refName</code>	Persistent reference for the template document. If omitted on create, the API derives it from <code>templateKey</code> by replacing <code>.</code> with <code>-</code> .
<code>templateKey</code>	Logical template id used by application code, such as <code>welcome</code> or <code>tenant-onboarding-invite</code>
<code>subjectTemplate</code>	Qute-formatted subject template
<code>htmlTemplate</code>	Qute-formatted HTML body
<code>textTemplate</code>	Qute-formatted text body
<code>active</code>	Enables or disables the override without deleting it
<code>sourceType</code>	Usually <code>REALM_OVERRIDE</code> ; can also be set to <code>CLASSPATH_DEFAULT</code> for metadata purposes
<code>sampleContextJson</code>	Optional example JSON payload showing the expected render context
<code>schemaVersion</code>	Optional version marker for template lifecycle tracking

78.1.4. Rendering templates

Inject `EmailTemplateRenderService` and call `render()`:

```
@Inject
EmailTemplateRenderService renderService;

RenderedTemplate rendered = renderService.render(
    realmId,
    "password-reset",
    Map.of(
        "name", user.getDisplayName(),
        "realm", realmId,
```

```

        "resetLink", resetUrl
    )
};

```

The render context can be:

- `Map<String, Object>`
- a Java bean
- a Java record
- another Jackson-convertible object

The original object is also exposed to Qute as `{context}`.

Validation rules

After rendering, the framework validates:

- subject must be non-blank
- at least one of HTML or text body must be non-blank

If either rule fails, rendering throws `IllegalStateException`.

78.1.5. Tenant-scoped REST API

The framework exposes authenticated tenant-scoped endpoints at `/settings/email-templates`:

Endpoint	Purpose
GET <code>/settings/email-templates</code>	List templates stored for the current realm
GET <code>/settings/email-templates/{refName}</code>	Fetch one template by refName
POST <code>/settings/email-templates</code>	Create a new realm-scoped template
PUT <code>/settings/email-templates/{refName}</code>	Update an existing realm-scoped template
DELETE <code>/settings/email-templates/{refName}</code>	Delete a realm-scoped template
POST <code>/settings/email-templates/preview</code>	Render either an existing template by <code>templateKey</code> or inline template content

Example create request:

```

{
  "templateKey": "tenant-onboarding-invite",
  "displayName": "Tenant onboarding invite",
  "description": "Invite email shown to tenant users during onboarding.",

```

```

"active": true,
"subjectTemplate": "You're invited to {tenantDisplayName}",
"htmlTemplate": "<p>Hello {inviteeName}, use <a href=\"{inviteLink}\">this link</a>
to get started.</p>",
"textTemplate": "Hello {inviteeName}, use {inviteLink} to get started.",
"sampleContextJson": "{\"inviteeName\":\"Sam\",\"tenantDisplayName\":\"Acme\",
\"inviteLink\":\"https://example.test/invite/abc\"}"
}

```

Example preview request against a stored template:

```

{
  "templateKey": "tenant-onboarding-invite",
  "context": {
    "inviteeName": "Sam",
    "tenantDisplayName": "Acme Logistics",
    "inviteLink": "https://app.example.com/invite/abc123"
  }
}

```

Example preview request using inline content instead of a stored template:

```

{
  "templateKey": "preview-only",
  "subjectTemplate": "Welcome {firstName}",
  "textTemplate": "Hello {firstName}",
  "context": {
    "firstName": "Sam"
  }
}

```

78.1.6. Onboarding workflow integration

`TenantOnboardingWorkflow` now includes `defaultInviteEmailTemplateKey`.

This is the template key the tenant onboarding flow should use when it needs to render an invite email. The framework default key is:

```
tenant-onboarding-invite
```

That value is configuration, not a guaranteed built-in classpath template. In practice:

- your application can ship a classpath template with that key
- a tenant can override that key in the database
- the workflow can be updated to point at a different key per realm

Representative workflow update payload:

```
{
  "displayName": "Tenant User Onboarding",
  "inviteRequired": true,
  "registrationRequired": true,
  "defaultInviteEmailTemplateKey": "tenant-onboarding-invite"
}
```

This separation is useful because the onboarding workflow owns *which template key to use*, while the email-template subsystem owns *how that template is resolved and rendered*.

78.1.7. Delivering email

Build a `RenderedEmail` from the rendered output and pass it to an `EmailDeliveryService`:

```
@Inject
EmailDeliveryService deliveryService;

RenderedEmail email = RenderedEmail.builder()
    .from("noreply@example.com")
    .to(List.of(user.getEmail()))
    .subject(rendered.subject())
    .htmlBody(rendered.htmlBody())
    .textBody(rendered.textBody())
    .messageStream("outbound")
    .build();

deliveryService.deliver(email);
```

Postmark integration

The framework ships with `PostmarkEmailSender`.

```
postmark.api-key=${POSTMARK_API_KEY}
postmark.default-from-email-address=noreply@yourdomain.com
```

If either property is missing, the sender logs and returns without sending. That keeps local development and tests safe when no real delivery credentials are configured.

Custom delivery services

You can replace delivery with your own implementation:

```
@ApplicationScoped
@Alternative
@Priority(1)
```

```
public class SesEmailSender implements EmailDeliveryService {
    @Override
    public void deliver(RenderedEmail email) {
        // send via AWS SES, SendGrid, etc.
    }
}
```

Custom template resolvers

You can also replace template resolution entirely:

```
@ApplicationScoped
@Alternative
@Priority(1)
public class CmsBackedTemplateResolver implements EmailTemplateResolver {
    @Override
    public Optional<EmailTemplateDefinition> resolve(String realm, String templateKey)
    {
        // load from S3, CMS, Git, etc.
    }
}
```

78.1.8. When to use email templates

Use the framework email-template subsystem when:

- the output is user-facing email content
- tenants may need per-realm content overrides
- you want previewable Qute templates instead of hard-coded strings

Do not use email templates for:

- provisioning reference data, which belongs in [seed packs](#)
- multi-step operational setup, which belongs in [BootstrapPacks](#)

78.1.9. See also

- [Tenant onboarding and access invites](#)
- [Seed packs and declarative tenant seeding](#)
- [Bootstrap Packs](#)
- [Configuration reference](#) === Bootstrap Packs

BootstrapPacks are the framework mechanism for idempotent runtime setup work that is bigger than declarative seeding.

They are intentionally distinct from migrations and seed packs:

- migrations change database structure and initialization state
- seed packs materialize deterministic datasets
- BootstrapPacks orchestrate operational setup steps across one or more systems

`quantum-bootstrap-core` provides the BootstrapPack runtime, startup runner, models, and SPI hooks. Product modules contribute actual packs and step handlers by depending on that endpoint-free artifact.

`quantum-bootstrap-rest` is the opt-in system-management adapter that exposes the admin API. It is selected by `quantum-system-service`; ordinary application backends should omit it and consume the generated system-service SDK instead.

78.1.10. What a BootstrapPack is

A BootstrapPack is a named, versioned sequence of steps:

- `packRef`
- `packVersion`
- `productRef`
- `profileRef`
- ordered `steps`

Each step has:

- `stepRef`
- `description`
- `kind`
- `applyPolicy`
- `dependsOn`
- `config`

At runtime, `BootstrapPackService` resolves a pack from the registry, builds a scope, executes each step through the matching `BootstrapPackStepHandler`, and records a `BootstrapPackRun`.

78.1.11. Why BootstrapPacks exist

Seed packs are excellent for baseline data, but some startup and tenant-activation work is not just "load these JSON records":

- calling a product service to ensure a resource exists
- triggering an HTTP integration to an external platform
- validating that a prerequisite already exists
- publishing workflow or solution metadata
- sequencing several actions with dependencies and run history

That is the gap BootstrapPacks fill.

78.1.12. BootstrapPacks vs seed packs

Question	Seed packs	BootstrapPacks
Primary job	Load deterministic baseline data	Coordinate idempotent operational setup steps
Input format	Manifest plus JSON/NDJSON datasets	Java <code>BootstrapPackDefinition</code> plus step configs
Best for	Code lists, policies, defaults, reference rows, baseline records	Calling services, verifying prerequisites, external side effects, multi-step orchestration
Idempotency model	Dataset checksums plus natural keys	Step handler logic plus apply mode and apply policy
Cross-system work	Poor fit	First-class use case
Dependencies	Includes/archetypes inside manifests	<code>dependsOn</code> between steps inside one pack
Typical owner	Framework or product module shipping data	Product/application module shipping setup logic

78.1.13. When to use one vs the other

Use a seed pack when the desired result is "these records should exist in the tenant realm."

Examples:

- install code lists
- create default policies
- load default feature metadata
- establish baseline survey definitions

Use a BootstrapPack when the desired result is "run these steps to make the environment ready."

Examples:

- ensure an app service has provisioned a workspace
- call an external API to register a tenant
- run assertions before enabling a solution
- publish a workflow package after required seed data exists

Use both together when needed:

- a BootstrapPack step can depend on a `SEED_PACK` step to materialize baseline data first
- later steps can then call services or external APIs that rely on that seeded data

78.1.14. Rule of thumb

If you can review the change entirely as data files, prefer a seed pack.

If the change needs code, orchestration, side effects, or conditional runtime checks, prefer a `BootstrapPack`.

78.1.15. Step kinds

The framework model currently defines these step kinds:

- `SEED_PACK`
- `APP_SERVICE`
- `HTTP_CALL`
- `ASSERTION`
- `WORKFLOW_PLATFORM_PUBLICATION`

The enum is shared infrastructure. Actual behavior depends on the `BootstrapPackStepHandler` implementations contributed by the application or product modules on the classpath.

78.1.16. Apply modes and policies

`BootstrapPack` apply mode controls how the pack is being run:

- `APPLY_MISSING`
- `REAPPLY`
- `VALIDATE_ONLY`

Step apply policy describes the intended handler behavior:

- `ONCE_PER_SCOPE`
- `WHEN_MISSING`
- `ALWAYS_IDEMPOTENT`

The framework passes both values into handlers through `BootstrapStepRequest`. Handler authors decide what those semantics mean for a specific step kind.

78.1.17. SPI model

Applications contribute packs with `BootstrapPackContributor`:

```
@ApplicationScoped
public class DemoBootstrapContributor implements BootstrapPackContributor {
    @Override
    public Collection<BootstrapPackDefinition> bootstrapPacks() {
        return List.of(new BootstrapPackDefinition(
            "demo-pack",
```



```

        "2026.04.0",
        "b2bi",
        "local-demo",
        List.of(
            new BootstrapPackStepDefinition(
                "ensure-a",
                "Ensure resource A exists",
                BootstrapStepKind.APP_SERVICE,
                BootstrapStepApplyPolicy.WHEN_MISSING,
                List.of(),
                Map.of("resource", "A")
            ),
            new BootstrapPackStepDefinition(
                "ensure-b",
                "Ensure resource B exists",
                BootstrapStepKind.APP_SERVICE,
                BootstrapStepApplyPolicy.WHEN_MISSING,
                List.of("ensure-a"),
                Map.of("resource", "B")
            )
        ));
    }
}

```

Handlers implement the work for one or more step kinds:

```

@ApplicationScoped
public class DemoAppServiceHandler implements BootstrapPackStepHandler {
    @Override
    public boolean supports(BootstrapStepKind kind) {
        return kind == BootstrapStepKind.APP_SERVICE;
    }

    @Override
    public BootstrapStepResult execute(BootstrapStepRequest request) {
        return new BootstrapStepResult(
            BootstrapStepStatus.COMPLETED,
            BootstrapStepOutcome.CREATED,
            Map.of("handledStep", request.stepRef())
        );
    }
}

```

78.1.18. Admin REST API

The system-management service exposes the `quantum-bootstrap-rest` admin endpoints at `/admin/bootstrap-packs`:

Endpoint	Purpose
GET /admin/bootstrap-packs	List available pack definitions
GET /admin/bootstrap-packs/pending/{realm}	List packs with no successful completed run for the realm
GET /admin/bootstrap-packs/history/{realm}	Show recorded runs for the realm
POST /admin/bootstrap-packs/validate/{realm}	Validate all selected packs without applying changes
POST /admin/bootstrap-packs/{realm}/{packRef}/validate	Validate one pack
POST /admin/bootstrap-packs/apply/{realm}	Apply all selected packs
POST /admin/bootstrap-packs/{realm}/{packRef}/apply	Apply one pack

Useful query parameters:

- filter=pack-a,pack-b
- mode=APPLY_MISSING
- mode=REAPPLY
- mode=VALIDATE_ONLY

These endpoints are protected for **admin** and **system** roles.

78.1.19. Startup application

BootstrapPacks can also be auto-applied at startup by **BootstrapPackStartupRunner**.

Relevant properties:

```
quantum.bootstrap-pack.enabled=true
quantum.bootstrap-pack.apply.on-startup=false
quantum.bootstrap-pack.apply.realms=system-com,mycompany-com,test-com
quantum.bootstrap-pack.apply.filter=pack-a,pack-b
```

If startup application is enabled, the runner:

1. resolves the target realms
2. resolves the selected packs
3. runs each pack as **APPLY_MISSING**
4. logs failures and pending packs

78.1.20. Run history

The framework records `BootstrapPackRun` objects, including:

- pack ref and version
- execution scope such as realm and environment
- per-step status and outcome
- start and finish timestamps

By default, the framework uses an in-memory run repository. That is useful for local development and tests, but products that need durable operational history should plug in a persistent `BootstrapPackRunRepository`.

78.1.21. Examples

Example 1: Use a seed pack only

Requirement: Add baseline code lists, policies, and default records for every new tenant.

Best choice: Use a seed pack.

Reason: The desired result is purely deterministic data in MongoDB.

Example 2: Use a `BootstrapPack` only

Requirement: After a tenant is created, call a product service to create a workspace and then verify it exists.

Best choice: Use a `BootstrapPack` with `APP_SERVICE` and `ASSERTION` steps.

Reason: The work is procedural and depends on service logic, not just document inserts.

Example 3: Use both together

Requirement: Seed workflow definitions first, then publish them to an external workflow platform.

Best choice:

- Use a seed pack for the workflow and policy records
- Use a `BootstrapPack` with a `SEED_PACK` step followed by `WORKFLOW_PLATFORM_PUBLICATION`

Reason: The first half is declarative data, while the second half is an operational publication step.

78.1.22. Choosing the right abstraction

Prefer seed packs by default for tenant baseline data.

Move to `BootstrapPacks` when:

- the setup spans more than one system

- steps must run in order with dependencies
- you need dry-run validation semantics
- the setup is product-owned logic rather than framework-owned baseline data

78.1.23. See also

- [Seed packs and declarative tenant seeding](#)
- [Email Templates](#)
- [Tenant onboarding and access invites](#) = Database Migrations and Index Management

This guide explains Quantum's MongoDB migration subsystem (quantum-morphia-repos), how migrations are authored and executed, and how to manage indexes. It also documents the REST APIs that trigger migrations and index operations.

Chapter 79. Overview

Quantum uses a simple, versioned change-set mechanism to evolve MongoDB schemas and seed data safely across realms (databases). Key building blocks:

- `ChangeSetBean`: a CDI bean describing one migration step with metadata (from/to version, priority, etc.) and an `execute` method.
- `ChangeSetBase`: convenience base class you can extend; provides logging helpers and optional targeting controls.
- `MigrationService`: discovers pending change sets, applies them in order within a transaction, records execution, and bumps the `DatabaseVersion`.
- `DatabaseVersion` and `ChangeSetRecord`: stored in Mongo to track current schema version and previously executed change sets.

Chapter 80. Semantic Versioning

Semantic Versioning (SemVer) expresses versions in the form MAJOR.MINOR.PATCH (for example, 1.4.2):

- MAJOR: increment for incompatible/breaking schema changes.
- MINOR: increment for backward-compatible additions (new collections/fields that don't break existing code).
- PATCH: increment for backward-compatible fixes or small adjustments.

Why this matters for migrations: - Ordering: migrations must apply in a deterministic order that reflects real compatibility. SemVer provides a natural ordering and clear intent for authors and reviewers. - Compatibility checks: the application can assert that the current database is “new enough” to run the code safely.

How semver4j is used: - Parsing and validation: version strings are parsed into a SemVer object. Invalid strings fail fast during parsing, ensuring only compliant versions are stored and compared. - Introspection and comparison: the parsed object exposes major/minor/patch components and supports comparisons, enabling safe ordering and “greater than / less than” checks. - Consistent string form: the canonical string is retained for display, logs, and API responses.

How DatabaseVersion leverages SemVer: - Single source of truth: DatabaseVersion stores the canonical SemVer string alongside a parsed SemVer object for logic and comparisons. - Efficient ordering: for fast sorting and tie-breaking, DatabaseVersion also keeps a compact integer encoding of MAJOR.MINOR.PATCH as $(\text{major} \times 100) + (\text{minor} \times 10) + \text{patch}$ (e.g., 1.0.3 $\rightarrow$ 103). This makes numeric comparisons straightforward while still recording the exact SemVer string. - Migration flow: when migrations run, successful execution records the new database version in DatabaseVersion. Startup checks compare the stored version to the required quantum.database.version to prevent the app from running against an older, incompatible schema.

Recommendations: - Always bump MAJOR for breaking data changes, MINOR for additive changes, and PATCH for backward-compatible fixes. - Keep change sets small and target a single to-version per change set to make intent clear. - Use SemVer consistently in getDbFromVersion/getDbToVersion across all change sets so ordering and compatibility checks remain reliable.

Chapter 81. Configuration

The following MicroProfile config properties influence migrations:

- `quantum.database.version`: target version the application requires (SemVer, e.g., 1.0.3). `MigrationService.checkDataBaseVersion` compares this to the stored version.
- `quantum.database.migration.enabled`: feature flag checked by resources/services when running migrations. Default: true.
- `quantum.database.migration.changeset.package`: package containing change sets (CDI still discovers beans via type, but this property documents the intended package).
- `quantum.migration.apply.realms`: optional comma-separated list of additional realms to initialize during application startup. These startup migrations are evaluated before `SeedStartupRunner` runs, so realms listed here can be safely included in `quantum.seed-pack.apply.realms`.
- `quantum.realmConfig.systemRealm`, `quantum.realmConfig.defaultRealm`, `quantum.realmConfig.testRealm`: well-known realms used by `MigrationResource` when running migrations across environments. On startup, `MigrationService` checks whether the *system realm* database exists and is initialized; if missing or uninitialized, it runs all pending change sets and applies indexes (creating all entity collections, including credentials). Set `quantum.realmConfig.systemRealm` to your application's system database name (e.g. `system-com`) in `application.properties` so the correct database is created and initialized when dropped or empty.
- `quantum.defaultSystemPassword`: initial password for the system user credential when created by the `AddSystemUserCredential` change set (default: `test123456`). Set in `application.properties`; the system user (`quantum.realmConfig.systemUserId`) is created in the credentials collection so login with that user works.
- `quantum.database.version`: target schema version (e.g. 1.0.4). Must be at least 1.0.4 for the system user credential to be created during migration.

Chapter 82. How change sets are discovered and executed

- **Discovery:** `MigrationService#getAllChangeSetBeans` locates all CDI beans implementing `ChangeSetBean`.
- **Ordering:** change sets are sorted by `dbToVersionInt`, then by priority (ascending). That ensures lower target versions apply before higher ones; priority resolves ties.
- **Pending selection:** For the target realm, `MigrationService#getAllPendingChangeSetBeans` considers a change set pending if either (a) it has never run, (b) the bean's `changeSetVersion` is greater than the last recorded one, or (c) the bean's checksum is non-null and differs from the last recorded checksum in `ChangeSetRecord`. It also compares each change set's `dbToVersion` against the stored `DatabaseVersion`.
- **Locking:** A distributed lock (Mongo-backed Sherlock) is acquired per realm before applying change sets to prevent concurrent execution.
- **Transactions:** Each change set runs within a `MorphiaSession` transaction; on success the change is recorded in `ChangeSetRecord` and `DatabaseVersion` is advanced (if higher). On failure the transaction is aborted and the error returned.
- **Realms:** Migrations run per realm (Mongo database). A change set can optionally be restricted to certain database names or even override the realm it executes against (see below).

Chapter 83. Authoring a change set

Implement `ChangeSetBean`; most change sets extend `ChangeSetBase`.

Required metadata methods:

- `getId()`: a string id for human tracking (e.g., 00003)
- `getDbFromVersion()` / `getDbFromVersionInt()`: previous version you are migrating from (SemVer and an int like 102 for 1.0.2)
- `getDbToVersion()` / `getDbToVersionInt()`: target version after running this change (SemVer and int)
- `getPriority()`: integer priority when multiple change sets have same `toVersion`
- `getAuthor()`, `getName()`, `getDescription()`, `getScope()`: informational fields recorded in `ChangeSetRecord`
- `getChangeSetVersion()` [optional]: an integer you bump when the definition/logic of this change set evolves but its target database version does not. Defaults to 1.
- `getChecksum()` [optional]: a stable checksum string representing the content/logic of the change set. If provided and it changes, the change set will re-run even if versions did not change.

Execution method:

- `void execute(MorphiaSession session, MongoClient mongoClient, MultiEmitter<? super String> emitter)`
- Perform your data/index changes using the provided session (transaction).
- Use `emitter.emit("message")` to stream log lines back to SSE clients.

Optional targeting controls (provided by `ChangeSetBase`):

- `boolean isOverrideDatabase()`: return true to execute against a specific database instead of the requested realm.
- `String getOverrideDatabaseName()`: the concrete database name to use when overriding.
- `Set<String> getApplicableDatabases()`: return a set of database names to which this change set should apply. Return null or an empty set to allow all.

Logging helper:

- `ChangeSetBase.log(String, MultiEmitter)` emits to both Quarkus log and the SSE stream.

Chapter 84. Example change sets in the framework

Package: `com.e2eq.framework.model.persistent.morphia.changesets`

- `InitializeDatabase`
- Seeds foundational data in a new realm: counters (e.g., `accountNumber`), system `Organization` and `Account`, initial `Rule` and `Policy` scaffolding, default user profiles and security model. Uses `EnvConfigUtils` and `SecurityUtils` to derive system `DataDomain` and defaults.
- `AddAnonymousSecurityRules`
- Adds a `defaultAnonymousPolicy` with an allow rule for unauthenticated actions such as registration and contact-us in the website area.
- `AddRealms`
- Creates the system and default `Realm` records based on configuration, if missing.

These are typical examples of idempotent change sets that can be safely re-evaluated.

Chapter 85. REST APIs to trigger migrations (MigrationResource)

Base path: /system/migration

The deployable `quantum-system-service` owns this API by selecting the `com.end2endlogic:quantum-migration-rest` adapter. Application backends and SDK-only consumers should omit that artifact so they do not expose database administration endpoints.

Security: Most endpoints require admin role; dbversion is PermitAll for introspection.

- GET /system/migration/dbversion/{realm}
- Returns the current DatabaseVersion document for the given realm, or 404 if not found.
- Example: curl -s <http://localhost:8080/system/migration/dbversion/system-com>
- POST /system/migration/indexes/applyIndexes/{realm}
- Admin only. Calls MigrationService.applyIndexes(realm) which invokes Morphia Datastore.applyIndexes() for all mapped entities. Use this after adding @Indexed annotations.
- Example: curl -X POST -H "Authorization: Bearer \$TOKEN" <http://localhost:8080/system/migration/indexes/applyIndexes/system-com>
- POST /system/migration/indexes/applyIndexes/{realm}/{collection}
- Admin only. Applies indexes for one mapped collection in the realm.
- POST /system/migration/indexes/applyAllIndexes/{realm}
- Admin only. Applies all declared indexes across mapped collections, bypassing data scoping for this system-level operation.
- POST /system/migration/indexes/dropAllIndexes/{realm}
- Admin only. Drops all indexes on all mapped collections in the realm. Useful before re-creation or when changing index definitions.
- Example: curl -X POST -H "Authorization: Bearer \$TOKEN" <http://localhost:8080/system/migration/indexes/dropAllIndexes/system-com>
- POST /system/migration/indexes/drop/{realm}/{collection}
- Admin only. Drops indexes for one collection in the realm.
- POST /system/migration/changeSet/execute/{realm}/{beanRefName}
- Admin only. SSE stream that executes the named change-set bean for the realm.
- POST /system/migration/initialize/{realm}
- Admin only. Server-Sent Events (SSE) stream that executes all pending change sets for the specific realm.
- Example (note -N to keep connection open): curl -N -X POST -H "Authorization: Bearer \$TOKEN" <http://localhost:8080/system/migration/initialize/system-com>
- GET /system/migration/start

- Admin only. SSE stream that runs pending change sets across test, system, and default realms from configuration.
- Example: `curl -N -H "Authorization: Bearer $TOKEN" http://localhost:8080/system/migration/start`
- `GET /system/migration/start/{realm}`
- Admin only. SSE for a specific realm.
- Example: `curl -N -H "Authorization: Bearer $TOKEN" http://localhost:8080/system/migration/start/my-realm`

SSE responses stream human-readable messages produced by MigrationService and your change sets. The connection ends with "Task completed" or an error message.

Chapter 86. Per-entity index management (BaseResource)

Every entity resource that extends `BaseResource<T, R extends BaseMorphiaRepo<T>>` exposes a convenience endpoint to (re)create indexes for a single collection in a realm.

- `POST <entity-resource-base-path>/indexes/ensureIndexes/{realm}?collectionName=<collection>`
- Admin only. Invokes `R.ensureIndexes(realm, collectionName)`.
- Use this when you want to reapply indexes for one collection without touching others.
- Example (assuming a `ProductResource` at `/products`): `curl -X POST -H "Authorization: Bearer $TOKEN" \ "http://localhost:8080/products/indexes/ensureIndexes/system-com?collectionName=product"`

Chapter 87. Global index management (MigrationService)

MigrationService also exposes programmatic index utilities used by the MigrationResource endpoints:

- `applyIndexes(realm)`: calls Morphia Datastore.`applyIndexes()` for the realm.
- `dropAllIndexes(realm)`: iterates mapped entities and drops indexes on each underlying collection.

Chapter 88. Validating versions at startup

- `MigrationService.checkDataBaseVersion()` compares the stored `DatabaseVersion` in each well-known realm to `quantum.database.version` and throws a `DatabaseMigrationException` when lower than required. This prevents the app from running against an incompatible schema.
- `MigrationService.checkInitialized(realm)` is a convenience that asserts `DatabaseVersion` exists and is $\geq$ required version; helpful for preflight checks.

Chapter 89. Notes and best practices

- Make change sets idempotent: Always check for existing records before creating/updating indexes or documents.
- Use SemVer consistently for from/to versions. The framework computes an integer form (e.g., 1.0.3 → 103) for ordering.
- Prefer small, focused change sets with clear descriptions and authorship.
- Use the MultiEmitter in execute(...) to provide progress to operators consuming the SSE endpoint.
- Apply new indexes with applyIndexes after deploying models with new @Indexed annotations; optionally dropAllIndexes then applyIndexes when changing index definitions across the board.
- Limit scope: use getApplicableDatabases() to constrain execution to specific databases, or isOverrideDatabase/getOverrideDatabaseName to target a different database when appropriate.

89.1. Checksum vs. from/to versions

ChangeSetBean introduces two optional mechanisms that influence whether a change set runs again after it has already executed:

- `changeSetVersion` (int): A manual counter you increment when you intentionally want the same change set (same to-version) to run again because its logic changed or needs to reapply. Defaults to 1.
- `checksum` (String): A content-derived fingerprint of the change set's implementation. If provided, the framework will re-run the change set whenever the checksum differs from the last recorded value, even if versions and `changeSetVersion` are unchanged.

How they complement `getDbFromVersion/getDbToVersion`:

- `getDbFromVersion/getDbToVersion` define the database version boundary the change set moves the realm across. They control ordering and whether the database version should advance on success.
- `changeSetVersion/checksum` control repeatability of a change set at a fixed to-version. They do not change ordering; they only indicate that the change set should be applied again.

Typical usage patterns:

- Minor logic refactor that should re-apply: bump `changeSetVersion`, or update checksum if you compute it from code/resources.
- Data or index definition tweaked without a version bump: provide `getChecksum()` that reflects the relevant definitions (e.g., hash of DDL/index specs or embedded dataset), so the system auto-detects changes and re-runs.
- Actual schema version change: bump `getDbToVersion` (and possibly from) as usual; you may leave `changeSetVersion` and checksum alone.

Recording and behavior in `ChangeSetRecord`:

- On each execution, MigrationService stores changeSetVersion and checksum alongside from/to versions.
- A change set is considered pending if no record exists, or the bean's changeSetVersion is greater than the recorded one, or the bean's checksum is non-null and differs from the recorded checksum.
- After a successful run, DatabaseVersion is updated to dbToVersion if it is higher than current.

Caveats:

- If you return null from getChecksum(), only first-run and changeSetVersion increases can trigger re-execution.
- Ensure your checksum is stable across processes and deterministic for the same logic; do not include timestamps or environment-specific data.
- When migrating large datasets, prefer idempotent logic so repeated runs are safe even if checksum forces a re-run. == Testing in Quantum: Security Contexts, Repos, and REST APIs

89.2. Testing Framework

This guide explains how to write effective tests in the Quantum framework, with a focus on setting the security context and choosing the right Quarkus testing patterns for repository logic vs. REST APIs.

It covers four practical patterns you can use to establish the security context in tests: - Extending BaseRepoTest - Using a try-with-resources SecuritySession - Using a scoped-call pattern to wrap work under a SecuritySession - Using the @TestSecurity annotation

In addition, it outlines how to test REST endpoints vs. repository logic and highlights useful features of the Quarkus Test Framework.

89.3. Prerequisites and glossary

- RuleContext: the in-memory rule engine configuration used by authorization checks.
- PrincipalContext (pContext): who is performing the action (userId, roles, realms, data domain).
- ResourceContext (rContext): what is being acted upon (area/domain, resource, action).
- SecuritySession: a small utility that binds PrincipalContext and ResourceContext to SecurityContext for the current thread, and clears them on close.
- SecurityIdentity: Quarkus' current identity (used by @TestSecurity and HTTP request security).

89.4. Pattern 1 — Extend BaseRepoTest

For repository tests (no HTTP), the simplest approach is to extend BaseRepoTest.

What BaseRepoTest does for you: - Initializes RuleContext with default rules. - Builds default pContext and rContext for a system/test user. - Ensures test database migrations are applied. - Gives you protected fields pContext and rContext you can use in your test.

Example:

```
@QuarkusTest
class MyRepoTest extends com.e2eq.framework.persistent.BaseRepoTest {

    @Inject
    com.e2eq.framework.model.persistent.morphia.UserProfileRepo userProfileRepo;

    @Test
    void canReadUnderTestUser() {
        // Activate the security context for this block
        try (final com.e2eq.framework.securityrules.SecuritySession ignored =
            new com.e2eq.framework.securityrules.SecuritySession(pContext, rContext))
        {
            var list = userProfileRepo.list(testUtils.getTestRealm());
            org.junit.jupiter.api.Assertions.assertNotNull(list);
        }
    }
}
```

Notes: - BaseRepoTest prepares contexts but does not keep them permanently active. Wrap repo calls that require authorization in a SecuritySession (see Pattern 2), or activate it in @BeforeEach if many test methods use it. - BaseRepoTest also runs migrations once using your test principal, which avoids authorization failures during initialization.

89.5. Pattern 2 — Try-with-resources SecuritySession

Use SecuritySession explicitly to scope work that should run under a specific PrincipalContext and ResourceContext. This is the most explicit pattern and works in both repository and service-level tests.

Example:

```
@QuarkusTest
class CredentialRepoTest extends com.e2eq.framework.persistent.BaseRepoTest {

    @Inject
    com.e2eq.framework.model.persistent.morphia.CredentialRepo credRepo;

    @Test
    void findByUserId_asTestUser() {
        try (final com.e2eq.framework.securityrules.SecuritySession s =
            new com.e2eq.framework.securityrules.SecuritySession(pContext, rContext))
        {
            var op = credRepo.findByUserId(testUtils.getTestUserId(), testUtils
                .getSystemRealm());
            org.junit.jupiter.api.Assertions.assertTrue(op.isPresent());
        }
    }
}
```

```
}  
}
```

Tips: - Prefer try-with-resources so contexts are always cleared, even when assertions fail. - You can construct custom `PrincipalContext/ResourceContext` for specific scenarios (e.g., different roles or realms) and pass them to `SecuritySession`.

89.6. Pattern 3 — Scoped-call wrapper (ScopedCallScope)

If you prefer not to repeat try-with-resources blocks, wrap your work in a helper that creates a `SecuritySession`, runs your logic, and ensures cleanup. This is sometimes called a “scoped call” pattern, often referred to as a `ScopedCallScope`.

Example helper (placed in test sources):

```
public final class SecurityScopes {  
    private SecurityScopes() {}  
  
    public static <T> T call(  
        com.e2eq.framework.model.securityrules.PrincipalContext p,  
        com.e2eq.framework.model.securityrules.ResourceContext r,  
        java.util.concurrent.Callable<T> work) {  
        try (final com.e2eq.framework.securityrules.SecuritySession s =  
            new com.e2eq.framework.securityrules.SecuritySession(p, r)) {  
            try { return work.call(); }  
            catch (Exception e) { throw new RuntimeException(e); }  
        }  
    }  
  
    public static void run(  
        com.e2eq.framework.model.securityrules.PrincipalContext p,  
        com.e2eq.framework.model.securityrules.ResourceContext r,  
        Runnable work) {  
        try (final com.e2eq.framework.securityrules.SecuritySession s =  
            new com.e2eq.framework.securityrules.SecuritySession(p, r)) {  
            work.run();  
        }  
    }  
}
```

Usage:

```
var result = SecurityScopes.call(pContext, rContext, () -> repo.getByUserId(realms,  
    userId));  
SecurityScopes.run(pContext, rContext, () -> repo.save(realms, entity));
```

This achieves the same effect as try-with-resources but centralizes the pattern.

89.7. Pattern 4 — @TestSecurity annotation (Quarkus)

For tests that run through HTTP (and in some repo tests), you can use Quarkus' `@io.quarkus.test.security.TestSecurity` to set the `SecurityIdentity` without creating a `SecuritySession`.

Quantum integrates with this in two places: - `SecurityFilter`: when a request has a `SecurityIdentity` but no JWT, it builds a `PrincipalContext` from the identity (user and roles). You can also pass `X-Realm` to control the realm. - `MorphiaRepo`: when repo methods are invoked under `@TestSecurity` with no active `SecuritySession`, `MorphiaRepo` lazily builds `PrincipalContext` from `SecurityIdentity` and sets a safe default `ResourceContext` to enable rule evaluation.

Example (HTTP):

```
@QuarkusTest
class SecureResourceTest {

    @Inject com.e2eq.framework.util.TestUtils testUtils;

    @Test
    @io.quarkus.test.security.TestSecurity(user = "test@system.com", roles = {"user"})
    void listProfiles_asUser() {
        io.restassured.RestAssured.given()
            .header("X-Realm", testUtils.getTestRealm())
            .when().get("/user/userProfile/list")
            .then().statusCode(200);
    }
}
```

Example (repo call under `@TestSecurity` fallback, no `SecuritySession`):

```
@QuarkusTest
class RepoFallbackTest {

    @Inject com.e2eq.framework.util.TestUtils testUtils;
    @Inject com.e2eq.framework.model.persistent.morphia.CredentialRepo credentialRepo;

    @Test
    @io.quarkus.test.security.TestSecurity(user = "test@system.com", roles = {"user"})
    void repoUsesIdentityWhenNoSecuritySession() {
        // Internally, MorphiaRepo will ensure PrincipalContext exists using
        // SecurityIdentity
        credentialRepo.findById("nonexistent@end2endlogic.com", testUtils.
getTestRealm(), false);
        // Optionally assert that SecurityContext has been initialized
        org.junit.jupiter.api.Assertions.assertTrue(
            com.e2eq.framework.model.securityrules.SecurityContext.getPrincipalContext()
        );
    }
}
```

```
.isPresent());  
    }  
}
```

Notes: - `@TestSecurity` is perfect for authorizing requests in HTTP tests without generating JWTs. - For repo tests that require precise `ResourceContext` (area/domain/action), prefer `SecuritySession`; `MorphiaRepo` sets a generic default `ResourceContext` when needed.

89.8. Testing REST APIs vs. Repository Logic

When to prefer REST (HTTP) tests: - End-to-end authorization: validate request filters, identity mapping, realm headers, and JWT handling. - Request/response shape and status codes. - Role-based access checks via `@TestSecurity`.

How to test REST APIs: - Use `@QuarkusTest` and `RestAssured`: [source,java] ---- `var resp = io.restassured.RestAssured.given() .header("Content-Type", "application/json") .header("X-Realm", testUtils.getTestRealm()) .when().get("/user/userProfile/list") .then().statusCode(200).extract().response();` ---- - To test JWT-protected endpoints end-to-end, first call the login API to obtain a token, then pass `Authorization: Bearer <token>`. See `SecurityTest.testGetUserProfileRESTAPI` for a complete example.

When to prefer repository/service tests: - You want precise control over `PrincipalContext/ResourceContext` and rule evaluation without HTTP overhead. - You are asserting persistence logic, query filters, or domain rules.

How to test repository logic: - Extend `BaseRepoTest` (Pattern 1) for ready-to-use `pContext/rContext` and migrations. - Wrap calls with `SecuritySession` (Pattern 2) or use a scoped-call helper (Pattern 3).

89.9. Useful Quarkus Test features

- `@QuarkusTest`: boots the app for integration tests with CDI, config, and persistence.
- `RestAssured`: fluent HTTP client baked into Quarkus tests; supports JSON assertions and extraction.
- `@TestSecurity`: set `SecurityIdentity` (user, roles) for tests.
- `@InjectMock/@InjectSpy` (quarkus-junit5-mockito): replace beans with mocks/spies for isolation.
- `@QuarkusTestResource`: manage external resources (e.g., starting/stopping containers) for a test class or suite.
- `@TestHTTPEndpoint` and `@TestHTTPResource`: convenient endpoint URI injection.

89.10. Real-world tips

- Clearing thread locals: If you manipulate `SecurityContext` directly in advanced tests, clear it in `@AfterEach` to avoid cross-test leakage: [source,java] ---- `@AfterEach void cleanup() { com.e2eq.framework.model.securityrules.SecurityContext.clear(); }` ----
- Realm routing: pass `X-Realm` in REST tests to select the target realm. `SecurityFilter` also validates

realm access against user credentials when present.

- Data prep: If your test needs specific users/roles, create them under a `SecuritySession` beforehand (see `SecurityTest.ensureTestUserExists()`).
- Logging: enable `DEBUG` for `com.e2eq` to inspect rule evaluation and identity resolution during tests.

89.11. Summary

- Use `BaseRepoTest` for repository tests and migrations, and wrap work in `SecuritySession`.
- For less ceremony, create a simple scoped-call helper to run code under a `SecuritySession`.
- For REST/API tests and quick identity setup, use `@TestSecurity`, realm headers, and `RestAssured`.
- For full e2e security, obtain a JWT via the login API and include it in requests.

REST Usage Governance and Metering Boundaries

Quantum REST usage governance is opt-in and defaults to **OFF**. It provides automatic post-authentication endpoint admission, a bounded per-JVM token bucket, and an OSS observation SPI. It does not use a MongoDB request counter, does not poll, and does not enable a pre-authentication network limiter.

Chapter 90. Runtime modes

OFF

The default. The filter performs no policy lookup, bucket allocation, or observation.

OBSERVE

The filter resolves the same trusted endpoint and principal identity used by enforcement, consumes the local bucket, and emits an observation. Exhaustion is recorded as `WOULD_REJECT`; the request continues. Unavailable enforcement state is recorded as `ENFORCEMENT_ERROR`; the request continues. Registry capacity pressure is recorded as `CAPACITY_BYPASSED`; the request continues without allocating an unbounded key.

ENFORCE

Exhaustion returns HTTP 429 with the typed `QuantumUsageGovernanceError` body and a whole-second `Retry-After` header. Missing endpoint identity, trusted principal identity, or policy state returns a typed HTTP 503 and does not invoke the endpoint. Bucket-registry capacity is never allowed to become a cross-tenant 503 primitive.

Chapter 91. Stable identity

The policy key is:

```
policy id/version + functional area/domain/OpenAPI operationId + tenant + subject
```

The area and domain come from `@FunctionalMapping`, including model mappings made available by the security runtime. The operation identity comes from an explicit `@Operation(operationId = "...")`; the stable JAX-RS method name is the runtime fallback and requires the matching `SmallRye METHOD` OpenAPI operation-ID strategy. Mutable URL paths are not policy identity. When governance is active, OpenAPI generation validates that operation IDs are present and unique and that exact configured selectors refer to generated operations.

Tenant and subject are read only from the effective framework `PrincipalContext` established by `SecurityFilter`. The tenant is the effective data-domain tenant after validated realm override; the subject is the provider subject with framework user ID as fallback. Raw identity attributes, headers, query parameters, and payload fields are not identity authorities. Endpoints explicitly annotated `@PermitAll` are outside this post-auth layer and need separately managed edge/network controls where appropriate. A class-level `@PermitAll` does not exempt a method that declares `@Authenticated`, `@RolesAllowed`, or `@DenyAll`.

Chapter 92. Property manifest

Property	Default	Description
<code>quantum.rest.usage.mode</code>	<code>OFF</code>	<code>OFF</code> , <code>OBSERVE</code> , or <code>ENFORCE</code> .
<code>quantum.rest.usage.policy.id</code>	<code>default-rest-usage</code>	Stable policy identifier.
<code>quantum.rest.usage.policy.version</code>	<code>1</code>	Policy version included in bucket identity.
<code>quantum.rest.usage.policy.request-limit</code>	<code>1000</code>	Bucket capacity and refill token count.
<code>quantum.rest.usage.policy.refill-period</code>	<code>PT5S</code>	Positive ISO-8601 refill period.
<code>quantum.rest.usage.policy.endpoints</code>	<code>*</code>	Comma-separated <code>area:domain:operationId</code> selectors or <code>*</code> .
<code>quantum.rest.usage.allow-unmatched-endpoints</code>	<code>true</code>	When <code>ENFORCE</code> resolves a trusted endpoint but the policy source returns no matching policy, admit it as <code>BYPASSED</code> . Set to <code>false</code> to fail closed with the typed <code>USAGE_POLICY_NOT_FOUND</code> 503. This flag never covers indeterminate endpoint, identity, source, or bucket state; those always fail closed.
<code>quantum.rest.usage.bucket.max-tracked-keys</code>	<code>10000</code>	Strict per-JVM bound for policy/endpoint/tenant/subject keys.
<code>quantum.rest.usage.bucket.idle-timeout</code>	<code>PT10M</code>	Positive ISO-8601 idle period before request-triggered eviction.
<code>mp.openapi.extensions.smallrye.operationIdStrategy</code>	<code>none</code>	Must be explicitly set to <code>METHOD</code> whenever governance is active so generated OpenAPI and runtime operation identities cannot diverge.

Invalid active configuration stops startup. `OFF` deliberately leaves dormant policy values unparsed so an upgrade remains inert. A typical observation-first rollout is:

```
quantum.rest.usage.mode=OBSERVE
mp.openapi.extensions.smallrye.operationIdStrategy=METHOD
quantum.rest.usage.policy.id=standard-api
quantum.rest.usage.policy.version=2026-08-15
quantum.rest.usage.policy.request-limit=100
quantum.rest.usage.policy.refill-period=PT1M
quantum.rest.usage.policy.endpoints=sales:order:listOrders,sales:order:createOrder
quantum.rest.usage.allow-unmatched-endpoints=true
```

Chapter 93. Extension contracts

The reusable policy, admission, observation, identity, and error contracts live in `quantum-contract-core`. This is the acyclic leaf: `quantum-rest-core` was rejected because it depends on `quantum-morphia-repos`, and `quantum-models` was rejected because it directly carries Morphia persistence types.

`UsagePolicySource` is the policy lookup SPI. Its default CDI implementation is `PropertyUsagePolicySource`; applications can provide another implementation without changing the filter. A source receives `UsageRequest` and returns `Optional<UsagePolicy>`. Returning an inapplicable policy, returning `null`, or throwing is an explicit policy-state failure.

`UsageObserver` is the downstream observation SPI. `UsageObservation` contains:

- endpoint, tenant, subject, and HTTP method;
- response status and latency;
- request and response byte counts (`-1` when unknown); and
- `ADMITTED`, `WOULD_REJECT`, `REJECTED`, `BYPASSED`, `CAPACITY_BYPASSED`, or `ENFORCEMENT_ERROR`, plus policy/retry/error details in the admission result.

Observers run after a response exists. Tenant and subject identifiers are sensitive; trusted observer implementations must not log them unredacted or export them to untrusted sinks. An observer can bridge to metrics, events, or durable billing, but its failure cannot retroactively change admission. Keep exporters bounded and nonblocking.

Chapter 94. Storage and scaling boundary

The admission bucket is a framework-owned atomic in-memory token bucket with an injectable monotonic clock. The registry is strictly bounded and evicts idle entries opportunistically during requests; there is no polling, scheduler, or third-party bucket runtime. When the bound is reached, a new key is admitted without a bucket and observed as `CAPACITY_BYPASSED`; this preserves the memory bound without allowing one tenant's cardinality to deny another tenant.

Each application replica has its own allowance. Therefore a deployment with three replicas has approximately three times the configured aggregate request capacity. Phase 1 intentionally prefers a deterministic, bounded OSS request path over a MongoDB read/modify/write counter. Cluster-wide entitlement and durable billing can consume the OSS contracts in a later, separately reviewed integration.

The existing `UsageMeteringService` and persistent token-allocation models are manual durable metering APIs. They are not called by `UsageGovernanceFilter` and must not be inserted into its admission path. If used, connect them downstream through an observer or event boundary with explicit delivery and backpressure semantics.

Chapter 95. Network guard boundary

Post-auth usage governance does not protect public authentication work. Deployments that need source-network denial-of-service protection should configure an edge or HTTP-layer guard separately. Such a guard uses a network identity and must never be described as tenant, subject, quota, or billing enforcement.

Chapter 96. Tutorials

Supply Chain Collaboration SaaS: A Business-First Guide

This guide explains, in plain language, how the Quantum framework helps you build a Supply Chain Collaboration Software-as-a-Service (SaaS). We focus on real business problems—secure data sharing, multi-party workflows, and tenant isolation—and show how the framework’s building blocks solve them without requiring you to stitch together dozens of bespoke APIs.

Chapter 97. What a supply-chain SaaS needs (and how Quantum helps)

Common needs when launching a collaboration platform:

- Secure sharing across companies: Buyers, suppliers, carriers, and 3PLs must see the same truth, but only what they're allowed to see.
- Role-appropriate views: Planners, operations, and analysts look at the same orders/shipments but need different fields, filters, and actions.
- Auditability and control: You need a clear explanation of who saw or changed what, and why that was allowed.
- Fast onboarding: Each partner authenticates differently; you can't force everyone into one identity provider.
- API consistency: Your UI and integrations shouldn't learn a different API for every screen or entity.
- Data lifecycle and stewardship: Easy data imports/exports, clear status tracking, and repeatable completion steps.

How Quantum maps to these needs:

- Multi-tenancy by design: Each organization (tenant) is isolated by default; sharing is added deliberately and safely.
- Policy-driven permissions: Human-readable rules answer "who can do what" and add scoping filters so only the right data shows up.
- Consistent REST and Query: One List API and a simple query language cover most searches and reports—no explosion of bespoke endpoints.
- Flexible identity: Support for different authentication methods per organization, plus delegated admin so each tenant manages its own users.
- State + tasks: Built-in patterns for long-running, stateful processes and "completion tasks" to move work forward predictably.

Chapter 98. Why multi-tenancy is a natural fit for supply chains

Supply chains are networks. Everyone shares a process, but not a database. Quantum isolates each tenant's data by default and lets you selectively share records with partners:

- Private by default: A supplier's purchase orders are not visible to other suppliers.
- Share on purpose: Create a "collaboration bubble" around a purchase order or shipment so a buyer and a specific carrier can see the same milestones and documents.
- Central where it helps: Keep a global partner directory or product catalog in a shared domain if that reduces duplication.
- Regional and regulatory needs: Pin certain data (e.g., EU shipments) to the correct region with simple policies.

Chapter 99. Who uses the system (organizations and roles)

Organizations (tenants) - Shippers and customers: create orders, monitor shipments, approve changes. - Carriers and 3PLs: accept tenders, provide status, confirm delivery. - Suppliers: acknowledge POs, provide ASN/invoice data.

Common user types within each organization - Planners: need forward-looking visibility—capacity, forecasts, purchase orders. - Operations: need day-to-day details—stops, ETAs, exceptions. - Business analysts: need trends and history—on-time performance, cost, root causes.

Data visibility examples - Private notes: an operations note on a shipment visible only inside the shipper's tenant. - Shared context: a delivery appointment visible to both the shipper and the carrier for a specific shipment. - Role-filtered: analysts see aggregated KPIs, planners see open exceptions, operators see actionable tasks.

Chapter 100. Identity and access: meet partners where they are

Every organization may authenticate differently: - Enterprise SSO (OIDC/SAML) for shippers and large suppliers. - Username/password for smaller partners. - Service accounts and tokens for system-to-system integrations.

Quantum supports these patterns and lets each tenant manage its own users (delegated administration). Permissions can differ by user type and tenant while staying auditable and predictable.

Chapter 101. Modeling without jargon:

Areas, Domains, and Actions

To keep your platform organized, Quantum groups things into: - Functional Areas: broad business spaces like Collaboration, Finance, or Catalog. - Functional Domains: specific entity types within an area—Partner, Shipment, PurchaseOrder, Invoice. - Functional Actions: what users do—VIEW, CREATE, UPDATE, DELETE, APPROVE, EXPORT, etc.

Why this matters: - Clear menus for the UI (group screens by area and domain). - Clear policy rules (easier to say “Carriers can VIEW Shipments” or “Only Finance can APPROVE Invoices”).

Chapter 102. Policies that say “who can do what” (Rule Language)

Policies are simple, readable rules that match: - Who is calling (identity and roles) - What they’re trying to access (area, domain, action) - Request details (headers/body, like a shipment id or tenant)

Rules then allow or deny the action and can add filters so the user only sees data they’re permitted to see. See the Permissions guide for authoring details ([Permissions Guide](#)).

Write-time data placement (where a new record belongs) - By default, new records use the creator’s organization. - You can override per area/domain with a small policy—for example, keep Partner records in a shared “directory” domain while Shipments stay tenant-local. See “Data domain assignment on create” ([../user-guide/permissions.pdf](#)).

Chapter 103. Domain contexts and domain policies (placing and sharing data)

In supply chains, the same entity often lives in different business contexts (e.g., a Shipment inside a Supplier's operations vs. the Buyer's view). Domain contexts let you keep data where it naturally belongs while still collaborating safely:

- Keep most data tenant-local: Each organization owns its operational truth (orders, shipments, tasks).
- Create shared contexts deliberately: Use a shared “Directory” context for master data like Partners and Facilities so everyone references the same records.
- Policy-driven sharing: Collaboration bubbles share just the relevant records (a specific PO, shipment, or appointment) with the right partners. Policies ensure each party only sees the fields and actions appropriate to them.
- Regional and compliance contexts: Pin data to a geography (e.g., EU) when regulations require it; policies keep reads and writes aligned.

Business impact - Lower friction: Organize data the way teams actually work (local operations + shared directories). - Safer collaboration: Sharing is explicit and explainable—no surprise data leaks. - Compliance by construction: Residency and scoping rules are enforced by policy, not by custom code.

Chapter 104. Bootstrap tenants with Seed Packs (zero-touch baseline data)

Onboarding partners is a bottleneck in many SaaS rollouts. Seed packs provide a fast, repeatable way to initialize each tenant with the right starting data and settings.

What seed packs give you - Versioned, reviewable baseline: Curate CSV/JSON datasets and settings (code lists, roles, sample workflows) under version control. - Archetypes for editions: Bundle multiple packs into a named edition (e.g., “Logistics Core” vs. “Supplier Lite”) to match your product tiers. - Safe, idempotent updates: Re-apply packs any time; only changes are applied. You can inspect what’s pending before you apply. - Admin APIs and startup automation: New tenants can be provisioned with the right archetype automatically; admins can check pending updates and apply them when ready.

Why this matters to a supply-chain SaaS - Faster go-live: New buyers, suppliers, and carriers start with consistent roles, code lists, and sample workflows. - Less manual setup: Reduce back-and-forth with partners; eliminate error-prone spreadsheets and one-off scripts. - Controlled change: Roll out new catalogs or policy tweaks as a versioned pack; apply to selected tenants when they opt in.

Learn more: [Seed packs and declarative tenant seeding](#).

Chapter 105. Security policies: practical protections and proofs

Security isn't just login. The framework's policy layer turns business rules into enforceable, auditable protections across UI, APIs, and imports/exports.

What's protected - Least privilege by role: Planners, Operators, and Analysts only get the actions and fields they need. - Field-level and record-level controls: Share a shipment with a carrier without exposing private buyer notes or margins. - Tamper-evident changes: Important state transitions and task completions are tracked and explainable. - Delegated administration: Each organization manages its users and roles; platform admins retain oversight with audit trails.

Why it's better than ad-hoc controls - One policy engine, everywhere: The same rules guard screens, APIs, and bulk operations—reducing gaps and regressions. - Explainable decisions: When an action is denied or filtered, admins can see why. - Easier audits: Policies and seed packs create a repeatable, documented configuration per tenant.

Chapter 106. A small, powerful API surface:

List + Query

Instead of building a unique search endpoint for every screen, Quantum gives you: - List API: a single, consistent endpoint per domain to list, filter, sort, page, and project fields. - Query Language: a simple filter syntax so UIs and reports can ask precise questions. - Automatic enforcement: policies and data-domain rules are always applied server-side, so callers only receive allowed data.

Business outcomes - Faster delivery: new screens reuse the same list API. - Fewer mistakes: less custom code, more consistent results. - Safer by default: even power users can't bypass policy enforcement.

Chapter 107. Delegated Administration (tenant-level user management)

Empower each organization to run their own house while you keep platform-wide safety: - Tenant admins invite users, reset passwords, and assign roles. - Role templates per tenant align with their org structure (Planner, Ops, Analyst, Carrier Dispatcher, etc.). - Cross-tenant boundaries are respected; global administrators can still support, audit, and troubleshoot with impersonation/acting-on-behalf-of where permitted and logged.

Chapter 108. Integrations and data management

Supply chains depend on clean, timely data. Quantum provides:

- CSV imports/exports: onboard master data quickly, rerun safely, and let business users fix and re-upload.
- Stateful objects: model processes (e.g., Shipment lifecycle) with clear states—Created → In-Transit → Delivered → Closed.
- Completion Tasks: checklist-like steps (confirm pickup, upload POD, reconcile invoice) that drive work to done and provide accountability.
- Consistent access: the same policies that protect your UI protect imports/exports and API calls.

Example uses

- Bulk load a new supplier catalog with CSV import; analysts export exceptions weekly for review.
- Track shipment exceptions as tasks; operations completes them with evidence (attachments/notes), all audited.

Chapter 109. End-to-end examples

1) Buyer–supplier collaboration on a Purchase Order - Create a collaboration bubble around a PO so both parties see schedule, holds, and documents. - Supplier can UPDATE promised dates; buyer can APPROVE changes. Private buyer notes remain private.

2) Shared partner directory, curated centrally - Keep one shared Partner domain so everyone finds the same carrier and facility records. - Only directory curators can CREATE/UPDATE; all tenants can VIEW.

3) EU shipment residency - Shipments created by anyone in Europe are written to an EU partition by policy. Reads remain role- and tenant-scoped.

Chapter 110. What you don't have to build from scratch

- Data isolation and safe sharing across tenants
- A consistent CRUD and search API for every domain
- A policy engine that explains its decisions and applies filters
- A write-time placement policy (so data lands in the right partition)
- Patterns for long-running, stateful business processes and task completion

The framework gives you these foundations so your teams focus on business value—on-time deliveries, lower cost, happier customers.

Chapter 111. Other helpful capabilities you get out of the box

- Pending seed visibility: Admins can see which seed packs have updates pending for a tenant before applying them, reducing surprise changes.
- Version selection and rollback strategy: The platform automatically selects the latest compatible seed pack version; because packs are versioned and reviewable, you can test in a sandbox realm before rolling out.
- Archetypes (editions): Model product tiers by composing packs; useful for offering lighter supplier/carrier editions versus full buyer editions.
- Explainable access decisions: When access is denied or results are filtered, admins can inspect the reasoning—useful during partner onboarding and audits.
- Idempotent imports/exports: Bulk data operations can be retried safely, which is critical when partners exchange large files over unreliable networks.
- Observability hooks: Standardized endpoints and logs make it easier to monitor SLAs (e.g., late tasks, delayed updates) without custom plumbing.

Chapter 112. Next steps

- Start with siloed defaults; prove value quickly using the List API.
- Add small, targeted policies to enable collaboration bubbles and shared directories.
- Introduce delegated administration so partners self-serve.
- Use CSV imports and Completion Tasks to operationalize data stewardship.
- Deep dive: Permissions and Rule Language ([Permissions Guide](#)), and Data domain assignment on create ([Data domain assignment on create](#)).

Chapter 113. A day in the life: From Purchase Order to Delivery

This story ties the pieces together in a realistic sequence. We follow a Purchase Order (PO) from creation to delivery, with shared visibility for suppliers and carriers, a clear state graph, and checklist-like Completion Tasks guiding the work.

1) Purchase Order is created (by the Buyer) - Action: A buyer creates a PO in the Collaboration area under the PurchaseOrder domain. - Data placement: By default, the PO is written to the buyer's organization (their data domain). If you prefer a shared domain for POs, configure a small policy; otherwise, the default works well. - Programmatic sharing: A rule shares the specific PO with the chosen Supplier (or Suppliers). The Supplier can view the PO and update the fields you allow (e.g., promised date), but cannot see private buyer-only fields.

State graph (illustrative) - Draft → Open → SupplierAcknowledged → ReadyToShip → PartiallyShipped → FullyShipped → Received → Closed

Completion Tasks (examples attached to the PO) - Buyer: Provide required documents (commercial terms, incoterms) - Supplier: Acknowledge PO (due in 24 hours) - Supplier: Provide ASN (advanced shipping notice) for each shipment - Supplier: Confirm pickup window - Buyer: Approve any date changes

2) The Supplier prepares shipments (shared onward to Carriers) - Action: The Supplier creates one or more Shipments linked to the PO (and optionally to specific lines). - Data placement: Shipments are written to the Supplier's domain by default (their own organization), but are shared with the Buyer so both parties see the same timeline. - Sharing to Carriers: When the Supplier tenders a shipment, the shipment is shared with the selected Carrier so they can update movement and milestones.

Shipment state graph (illustrative) - Planned → Tendered → Accepted → InTransit → Delivered → ProofVerified → Closed

Shipment Completion Tasks (examples) - Carrier: Confirm pickup - Carrier: Update in-transit location/ETA - Carrier: Upload POD (proof of delivery) - Supplier: Reconcile quantities shipped vs. ordered

3) Status updates complete tasks and move states forward - When the Supplier marks "SupplierAcknowledged," the PO's acknowledgement task completes and the PO moves to SupplierAcknowledged. - When all lines are ready and at least one shipment is created, the PO advances to ReadyToShip. If some but not all lines ship, it enters PartiallyShipped; once all lines ship, it becomes FullyShipped. - Carrier updates (e.g., Delivered with POD uploaded) complete shipment tasks. Those completion events can also advance the PO state (e.g., all shipments Delivered → PO moves to Received). Final checks (invoices matched, discrepancies resolved) move the PO to Closed.

Why this is safe and predictable - Roles and policies ensure each party sees only what they should: the Buyer sees everything; the Supplier sees the shared PO and its related shipments; the Carrier sees only the shipments they handle. - Completion Tasks remove ambiguity: everyone knows the

next step and who owns it. Each task completion is audited. - The state graph makes lifecycle transitions explicit. Policies can require certain tasks to be completed before a state transition is allowed.

4) Business visibility and reporting (List API + Query) - Operations view: “Purchase Orders in progress” shows all POs in Open, SupplierAcknowledged, ReadyToShip, or PartiallyShipped, including late tasks and upcoming milestones. - Buyer/supplier view: Both parties see the same PO status and related Shipments, with role-appropriate fields. - Simple reporting example (illustrative):
GET /collaboration/purchaseorder/list?filter=status IN
("Open","SupplierAcknowledged","ReadyToShip","PartiallyShipped")&sort=dueDate:asc&limit=50 -
Add projections to include key fields and roll-ups (e.g., shipped vs. ordered quantities). Related Shipment info can be retrieved similarly via the List API on the Shipment domain, filtered by the PO id.

What made this easy (and repeatable) - Multi-tenancy by default: Each org’s data is isolated; sharing is explicit and safe. - Policies (Rule Language): Define who can see or update which fields and when. The same rules apply to UI, API, and imports/exports. - Data domain assignment on create: Defaults keep data in the creator’s org; you can configure exceptions (e.g., shared directories) with a tiny policy. - Stateful objects + Completion Tasks: Clear states and checklists turn complex collaboration into a predictable flow. - List API + Query Language: One consistent way to fetch work lists, timelines, and reports without proliferating custom endpoints.

Chapter 114. Appendix

114.1. Configuration Reference

This appendix explains key configuration properties used in Quantum applications, organized by functional area.

114.1.1. Configuration Basics

File Locations

- `src/main/resources/application.properties` - main configuration
- `src/test/resources/application.properties` - test overrides
- `.env` file - local development secrets (not committed)

Profiles and Environment Variables

```
# Profile-specific properties
%dev.auth.jwt.duration=7200
%test.quarkus.mongodb.database=test-db
%prod.quarkus.log.level=WARN

# Environment variable substitution
quarkus.mongodb.connection-
string=${MONGODB_CONNECTION_STRING:mongodb://localhost:27017}
auth.jwt.secret=${JWT_SECRET:change-me-in-production}
```

Precedence

System properties > Environment variables > `application.properties`

114.1.2. Database Configuration

MongoDB Connection

```
# Basic connection
quarkus.mongodb.connection-string=mongodb://localhost:27017
quarkus.mongodb.database=my-app

# With authentication
quarkus.mongodb.connection-
string=mongodb://user:pass@host:27017/database?authSource=admin

# Disable dev services (use real MongoDB)
quarkus.mongodb.devservices.enabled=false
```

Morphia Configuration

```
# Database and package scanning
quarkus.morphia.database=my-app-db
quarkus.morphia.packages=com.example.model,com.example.entities

# Schema management
quarkus.morphia.create-caps=true
quarkus.morphia.create-indexes=true
quarkus.morphia.create-validators=true
```

114.1.3. Authentication Configuration

JWT Provider

```
# Enable JWT authentication
quarkus.smallrye-jwt.enabled=true
auth.provider=custom

# JWT validation
mp.jwt.verify.publickey.location=publicKey.pem
mp.jwt.verify.issuer=my-app
mp.jwt.verify.audiences=my-app-users

# Custom JWT settings
auth.jwt.secret=${JWT_SECRET:your-secret-here}
auth.jwt.expiration=60
auth.jwt.refresh-expiration=1440
```

OIDC Provider

```
# Enable OIDC (alongside or instead of built-in JWT)
quarkus.oidc.enabled=true
auth.provider=custom,oidc          # or just: auth.provider=oidc

# OIDC configuration
quarkus.oidc.auth-server-url=https://your-provider.com/auth/realms/your-realm
quarkus.oidc.client-id=your-client-id
quarkus.oidc.credentials.secret=your-client-secret
quarkus.oidc.token.issuer=https://your-provider.com/auth/realms/your-realm

# Quantum OIDC provider settings
auth.oidc.provider-name=oidc          # Provider name (default: oidc)
auth.oidc.role-claim-path=groups      # Claim path for roles (default:
groups)
auth.oidc.username-claim=preferred_username # Claim for username (default:
preferred_username)
```

Dedicated Cognito Provider

```
# Enable Cognito provider (full AWS SDK integration)
auth.provider=cognito

# AWS Cognito settings
aws.cognito.user-pool-id=${USER_POOL_ID}
aws.cognito.client-id=${APPLICATION_CLIENT_ID}
aws.cognito.region=us-east-1
aws.cognito.jwks.url=https://cognito-
idp.${aws.cognito.region}.amazonaws.com/${aws.cognito.user-pool-id}/.well-
known/jwks.json
```

See [Multiple Authentication Providers](#) for details on configuring external identity providers.

Application Registration (optional module)

The `quantum-application-registration` module lets an application announce itself to its environment at startup, so first arrival can install, seed and migrate it. It is inert unless enabled, and requires an `ApplicationRegistrar` implementation on the classpath.

```
# Off by default; the framework starts identically without this.
quantum.application-registration.enabled=false

# Selects the registrar: matches its getName() OR its fully-qualified class name.
quantum.application-registration.provider=system-manager

# Required when enabled. Never inferred -- a service that guesses its
# environment can register into the wrong one.
quantum.application-registration.application-id=my-app
quantum.application-registration.environment=production

# Realms this application reads and writes, comma separated.
quantum.application-registration.realms=my-app-P1

# Build identity; participates in the already-registered fingerprint.
quantum.application-registration.application-version=1.4.2

# Whether a failed registration stops the service. Defaults to true.
quantum.application-registration.fail-on-error=true
```

Registration is attempted once per fingerprint (application + version + environment + realms). On every later boot the registrar's cheap local check short-circuits it, so a restart costs one indexed read rather than a call to the control plane.

See [Application Registration](#) for the SPI and implementation rules.

OAuth Server (optional module)

The `quantum-oauth-server` module uses the same JWT configuration properties listed above (`mp.jwt.verify.issuer`, `mp.jwt.verify.publickey.location`, `com.b2bi.jwt.duration`). No additional properties are required. The OAuth endpoints (`/oauth/authorize`, `/oauth/token`, `/oauth/jwks`, `/.well-known/openid-configuration`, `/oauth/userinfo`) activate automatically when the module is on the classpath.

OAuth clients are registered in the `oauth_clients` MongoDB collection. Authorization codes are stored in `oauth_authorization_codes` with a 5-minute TTL index for automatic cleanup.

See [OAuth Server Module](#) for client registration and integration examples.

Email Templates (Postmark)

```
# Postmark API key (required for email delivery)
postmark.api-key=${POSTMARK_API_KEY}

# Default sender address (required for email delivery)
postmark.default-from-email-address=noreply@yourdomain.com
```

If either property is missing, the `PostmarkEmailSender` logs an info message and skips delivery, allowing development environments to run without Postmark credentials.

Classpath templates are loaded from `templates/email/{templateKey}/` with files `subject.txt`, `body.html`, and `body.txt`. See [Email Templates](#) for details.

Shared Link Signing

```
# HMAC-SHA256 secret for signing share link tokens (required for link signing)
share.link.signing.secret=${SHARE_LINK_SECRET}
```

If the secret is not configured, `LinkSigner.isEnabled()` returns `false` and link signing/verification is disabled. See [Shared Links](#) for details.

114.1.4. Quantum Framework Configuration

Database Migration

```
# Migration settings
quantum.database.version=1.0.0
quantum.database.scope=DEV
quantum.database.migration.enabled=true
quantum.database.migration.changeset.package=com.example.migrations

# Optional CSV list of additional realms to initialize on startup before seed packs
run
```

```
quantum.migration.apply.realms=system-com,mycompany-com,test-com
```

Seed packs (filesystem + classpath)

```
# Enable seed pack discovery and application on startup
quantum.seed-pack.enabled=true
quantum.seed-pack.apply.on-startup=true

# Filesystem root for seed packs when present in the app repo
# If the path does not exist, the framework still loads classpath seed packs
# contributed by dependencies (e.g., quantum-framework) from resources under seed-
# packs/
quantum.seed-pack.root=src/main/resources/seed-packs

# Optional CSV filter to apply only specific seed pack names
# Example: quantum.seed.apply.filter=system-initialize,my-tenant
quantum.seed.apply.filter=
```



Classpath seed loading: The framework automatically discovers seed packs packaged under **seed-packs/** on the application classpath (including JAR dependencies). Apps depending on the framework no longer need to copy framework seeds locally or reference absolute file paths. If both filesystem and classpath contain seed packs with the same name, the latest semantic version is applied.

Bootstrap packs

```
# Enable BootstrapPack infrastructure
quantum.bootstrap-pack.enabled=true

# Apply discovered bootstrap packs on startup
quantum.bootstrap-pack.apply.on-startup=false

# Optional CSV list of realms to target on startup
# If omitted, the framework uses the system/default/test realms when available
quantum.bootstrap-pack.apply.realms=system-com,mycompany-com,test-com

# Optional CSV filter to apply only selected pack refs
quantum.bootstrap-pack.apply.filter=pack-a,pack-b
```



Bootstrap packs are not the same thing as seed packs.

- Use seed packs for deterministic baseline data.
- Use bootstrap packs for idempotent operational setup steps.

See [Bootstrap Packs](#) for details.

Security rules diagnostics

```
# Elevate RuleContext auto-reload diagnostics from DEBUG to INFO when troubleshooting
quantum.securityrules.ruleContext.debugAutoReload=false
```

Tips:

- In unit tests that run inside Quarkus, prefer using a CDI-managed `RuleContext`. A helper is provided in tests: `RuleContextTestUtils.reload(<realm>)` to force a refresh of policies from the repository for a given realm.
- If you construct `RuleContext` manually (outside CDI), repository injection is not available, and auto-reload is skipped to avoid wiping programmatically added rules.

Realm and Tenant Configuration

```
# System realm configuration
quantum.realmConfig.systemTenantId=system
quantum.realmConfig.systemAccountId=system-account
quantum.realmConfig.systemOrgRefName=SYSTEM
quantum.realmConfig.systemUserId=system-user

# Default tenant settings
quantum.realmConfig.defaultTenantId=default
quantum.realmConfig.defaultAccountId=default-account
quantum.realmConfig.defaultOrgRefName=DEFAULT

# Anonymous user (use carefully)
quantum.anonymousUserId=anonymous
```

114.1.5. See Also

- [Quarkus Configuration Guide](#)
- [Authentication Configuration](#)

114.1.6. Ontology Configuration (optional)

Enablement

```
e2eq.ontology.enabled=false # default; set true to enable ontology wiring in your app
```

TBox Persistence

```
# Enable TBox persistence to MongoDB (default: true)
# When enabled, the ontology TBox (schema) is persisted to MongoDB for faster startup
```

```
# and version history. The TBox is automatically rebuilt if the YAML source changes.
quantum.ontology.tbox.persist=true

# Force rebuild of TBox even if persisted version exists (default: false)
# Useful for development or when you want to ensure fresh rebuild from sources
quantum.ontology.tbox.force-rebuild=false

# Package scanning for @OntologyClass and @OntologyProperty annotations
quantum.ontology.scan.packages=com.example.model,com.example.entities
```

Registry modes

- In-memory (recommended to start): build a TBox in code/resources and instantiate `OntologyRegistry.inMemory(tbox)`.
- Mongo-backed (advanced): load a TBox from a Mongo collection and cache it in memory for hot reload/versioning.
- Persisted TBox (recommended for production): TBox is persisted to MongoDB and loaded on startup for faster initialization.

Edges collection and indexes

- The MongoDB `edges` collection (default name configured via `ontology.mongo.collection.edges`) supports open edge qualification via compound indexes:
- `idx_domain_p_dst_props: { "dataDomain": 1, "p": 1, "dst": 1, "props": 1 }`—optimizes `hasEdge / srcIdsByDst` lookups.
- `idx_domain_src_p_props: { "dataDomain": 1, "src": 1, "p": 1, "props": 1 }`—optimizes `hasIncomingEdge / dstIdsBySrc` lookups.
- Inferred edges carry `inferred: true` with provenance and timestamp metadata for maintenance and TTL eviction.

Materialization hooks

- Use `OntologyMaterializer` to recompute inferred edges when entities change (sources and intermediates for your property chains).

114.1.7. Query Gateway Configuration

Property	Default	Description
<code>feature.queryGateway.execution.enabled</code>	<code>false</code>	Guards execution of <code>POST /api/query/find</code> for <code>AGGREGATION</code> -mode queries. When <code>false</code> , aggregation queries return HTTP 501 (<code>ExecutionDisabled</code>). Must be set to <code>true</code> to allow the gateway to execute governed MongoDB aggregation pipelines (with inner-pipeline <code>\$lookup</code> pushdown and multi-hop traversals).

Property	Default	Description
<code>feature.queryGateway.maxPageSize</code>	<code>1000</code>	Maximum page size allowed for <code>/api/query/find</code> requests.
<code>ontology.mongo.database</code>	<code>quantum</code>	Default database name for the ontology edges collection when not overridden by realm context.
<code>ontology.mongo.collection.edges</code>	<code>edges</code>	Collection name storing ontology edge documents.

See also

- [Ontologies in Quantum](#)
- [Dynamic Edge Properties & Facet Segmentation](#)
- [Planner and Query Gateway](#)
- [User-Definable Facet Segmentation & Coordinate SPI](#)

114.1.8. Control-plane split (deployment mode)

Property	Default	Description
<code>quantum.mode</code>	<code>embedded</code>	Deployment mode of the control-plane split. <code>embedded</code> : the app owns its system realm — system-realm migrations, baseline identity, seeds, and bootstrap packs run locally (today's behavior, wp3 tiers 0/1). <code>remote</code> : the control plane owns the system realm; this app skips system-realm startup work while app-realm work runs locally (tier 2). Unknown values fail loud at startup.
<code>quantum.system-service.base-url</code>	—	Control-plane service endpoint. Required when <code>quantum.mode=remote</code> (startup fails loud without it); unused in embedded mode.
<code>quantum.system.directory.mode</code>	follows <code>quantum.mode</code>	Optional override for the SystemDirectory implementation alone (<code>local/remote</code>) — for migration windows where an app runs remote but still resolves the directory locally. Remote directory resolution fails loud until the Phase C control-plane client ships.

Property	Default	Description
<code>quantum.system-rest.enabled</code>	<code>true</code>	Build-time switch over the control-plane admin REST surface (<code>/admin/tenants*</code> , <code>/security/realm*</code> , <code>/security/realm-memberships</code> , <code>/admin/seeds</code> , <code>/system/migration</code> , <code>/admin/bootstrap-packs</code> , <code>/onboarding/*</code>). <code>false</code> removes all nine resources at build — the tier-1 hardening that replaces per-type <code>quarkus.arc.exclude-types</code> lists, and what a tier-2 tenant-plane app sets.

114.2. Quarkus Foundation

Quantum builds on Quarkus, a Kubernetes-native Java stack optimized for fast startup, low memory footprint, and developer productivity.

114.2.1. Key Quarkus Capabilities

Developer Experience

- **Live reload:** Code changes are reflected immediately in dev mode
- **Unified config:** Single `application.properties` for all configuration
- **Dev UI:** Rich development interface at <http://localhost:8080/q/dev/>
- **Test-first ergonomics:** Built-in test support with `@QuarkusTest`

Build-time Optimizations

- **Classpath indexing:** Aggressive build-time analysis reduces runtime scanning
- **Ahead-of-time processing:** Framework initialization moved to build time
- **Jandex indexes:** Annotation discovery without runtime reflection

Cloud Native

- **Container integration:** Seamless Docker and Kubernetes deployment
- **Health checks:** Built-in health and readiness endpoints
- **Metrics:** Micrometer and Prometheus integration
- **Configuration:** Environment-aware config with profiles

114.2.2. Arc (Quarkus CDI)

Arc is Quarkus' CDI implementation, focused on build-time analysis and small runtime overhead.

Common Scopes

Scope	Usage
<code>@ApplicationScoped</code>	Stateless services, repositories (recommended default)
<code>@RequestScoped</code>	Per-request context holders
<code>@Singleton</code>	Single instance without CDI proxies
<code>@Dependent</code>	Short-lived helpers (default if no scope declared)

Bean Discovery

- Classes become beans with scope annotations or producer methods
- Use `@Qualifier` to disambiguate multiple implementations
- `Instance<T>` for programmatic bean selection

Example

```
@ApplicationScoped
public class ProductService {
    @Inject ProductRepo repo;

    public List<Product> findActive() {
        return repo.findByActive(true);
    }
}
```

114.2.3. Native Compilation with GraalVM

Benefits

- Millisecond startup times
- Drastically reduced memory footprint
- Ideal for serverless and microservices

Constraints

- Reflection needs explicit configuration
- Dynamic proxies require substitution
- Some dynamic classloading limitations

Configuration

Use `@RegisterForReflection` for classes accessed via reflection:

```
@RegisterForReflection
@Entity
public class Product extends BaseModel {
```

```
// Ensures reflection metadata in native image  
}
```

114.2.4. Extension Ecosystem

Quarkus provides extensions for: - **Data**: MongoDB, PostgreSQL, Redis, Elasticsearch - **Security**: JWT, OIDC, OAuth2, RBAC - **Messaging**: Kafka, AMQP, JMS - **Observability**: Metrics, tracing, health checks - **Cloud**: AWS, Azure, Google Cloud integrations

114.2.5. Quantum's Quarkus Integration

Quantum leverages Quarkus for: - **CDI**: Dependency injection for repositories and services - **JAX-RS**: REST endpoint framework via BaseResource - **Security**: JWT/OIDC integration for authentication - **MongoDB**: Morphia integration for persistence - **Configuration**: MicroProfile Config for settings - **OpenAPI**: Automatic API documentation

114.2.6. Getting Started with Quarkus

Create Project

```
mvn io.quarkus:quarkus-maven-plugin:create \
  -DprojectGroupId=com.example \
  -DprojectArtifactId=my-app \
  -DclassName="com.example.MyResource" \
  -Dpath="/hello"
```

Development Mode

```
./mvnw quarkus:dev
```

Native Build

```
./mvnw package -Pnative
```

114.2.7. See Also

- [Quarkus Guides](#)
- [CDI Reference](#)
- [Native Image Guide](#) == Glossary

114.2.8. Core Concepts

BaseModel

Abstract base class for all Quantum entities, providing DataDomain, audit fields, ID

management, and validation hooks.

DataDomain

Scoping information attached to every model (tenantId, orgRefName, ownerId, etc.) that enables multi-tenant isolation and controlled sharing.

DomainContext

Runtime context representing the current execution scope (tenant, org, user, functional area/domain, action) used by repositories and security rules.

Functional Area

Broad business capability grouping (e.g., Catalog, Collaboration, Identity) used for organizing models and security policies.

Functional Domain

Specific entity type within a functional area (e.g., Product within Catalog, Shipment within Collaboration) used for fine-grained permissions.

RuleContext

Policy evaluation engine that determines allowed actions and contributes data filters based on identity, functional area/domain, and business rules.

114.2.9. Multi-Tenancy

Tenant

Logical partition representing a customer or organization in a multi-tenant system, identified by tenantId.

Organization (Org)

Business unit within a tenant, identified by orgRefName, enabling sub-tenant grouping and sharing.

Realm

Database or data partition, often corresponding to a MongoDB database, that can be selected per request via X-Realm header.

Shared Domain

Data domain accessible across multiple tenants (e.g., public catalogs, partner directories) as opposed to tenant-isolated data.

114.2.10. Security

Principal

Authenticated user identity with associated roles, tenant membership, and permissions.

Permission Rule

Declarative policy that matches requests (URL, method, headers, body) and decides ALLOW/DENY with optional data filters.

Access Resolver

Plugin that computes dynamic access lists (e.g., customer IDs a user can see) for use in permission filters.

Impersonation

Acting as another user identity for troubleshooting or administrative purposes, controlled by scripts and realm restrictions.

114.2.11. Data and Persistence

MorphiaRepo

Repository interface extending MongoDB operations with Quantum's DataDomain filtering, validation, and audit capabilities.

EntityReference

Lightweight reference object containing ID, type, refName, and displayName, used for foreign keys without full object loading.

StateGraph

Finite state machine definition for model fields, enforcing valid states and transitions (e.g., Order: Draft → Processing → Shipped).

Completion Task

Persistent work item with status tracking, used for checklists and long-running processes with audit trails.

114.2.12. REST and APIs

BaseResource

Abstract REST resource class providing consistent CRUD endpoints (find, get, list, save, update, delete) with automatic security and validation.

Query Language

ANTLR-based filter syntax used across all list endpoints, permission rules, and access resolvers for consistent data querying.

UIActions

List of actions a user can perform on a specific entity instance, computed based on entity state and user permissions.

CSV Import/Export

Built-in endpoints for bulk data operations with validation, preview sessions, and error handling.

114.2.13. Framework Components

ValidationInterceptor

Morphia interceptor that runs Jakarta Bean Validation and defaults DataDomain before persistence.

SecurityFilter

JAX-RS filter that builds security context (PrincipalContext, ResourceContext) and evaluates permissions for each request.

DataDomainResolver

Service that determines which DataDomain to assign to new entities based on functional area/domain policies.

Migration

Versioned database schema and data changes managed by Quantum's migration framework with changeset tracking.

114.2.14. Annotations

@FunctionalMapping

Class-level annotation declaring a model's functional area and domain, replacing legacy bmFunctionalArea() methods.

@FunctionalAction

Method-level annotation specifying the action performed by a REST endpoint when it differs from HTTP verb defaults.

@TrackReferences

Field annotation on @Reference fields that maintains back-reference sets for referential integrity checking.

@RegisterForReflection

Quarkus annotation ensuring classes are available for reflection in native images.

114.2.15. External Integrations

JWT Provider

Authentication module that validates JSON Web Tokens and populates security context with user identity and roles.

OIDC Integration

OpenID Connect support for enterprise identity providers like Keycloak, Auth0, and AWS Cognito.

Feature Flags

Configuration-driven capability toggles with targeting rules for gradual rollouts and A/B testing.

Postmark Integration

Email service integration for transactional messaging with template support.

114.2.16. Ontology (optional feature)

Ontology

A formal description of concepts (classes) and their relationships (properties/predicates) with rules (e.g., property chains) that allow inferring implied facts. See [Ontologies in Quantum](#).

Predicate / Property

A named relationship (e.g., placedBy, memberOf, orderShipsToRegion). Predicates connect source and destination IDs and can be declared inverse or transitive.

Property Chain

A rule of the form $p \sqcap q \Rightarrow r$ meaning if $(A \dashv p \dashv B)$ and $(B \dashv q \dashv C)$, infer $(A \dashv r \dashv C)$. Chains make common traversals first-class and fast.

Edge (materialized)

A persisted tuple (tenantId, src, p, dst) representing a relationship, often inferred from rules. Edges live in a dedicated collection and power single-hop queries and policies.

Ontology Materializer

A component that computes inferred edges for an entity snapshot and upserts them to the edges collection. Keeps edges fresh as data changes.

ListQueryRewriter

A helper that turns semantic constraints into efficient Mongo queries (e.g., id IN srcIdsByDst). Integrates with permission rules and Morphia repos.

OntologyAwareResource

An abstract REST base class (com.e2eq.ontology.policy.rest, quantum-ontology-policy-bridge) that extends BaseResource to add ontology list and detail context endpoints. Supports ontology constraints for filtering and optional aggregation/expand for combined attributes spanning collections.